



BCSE332L-Deep Learning

Dr. Rohith G

Assistant Professor Senior, CAMIT

School of Electronics Engineering (SENSE)

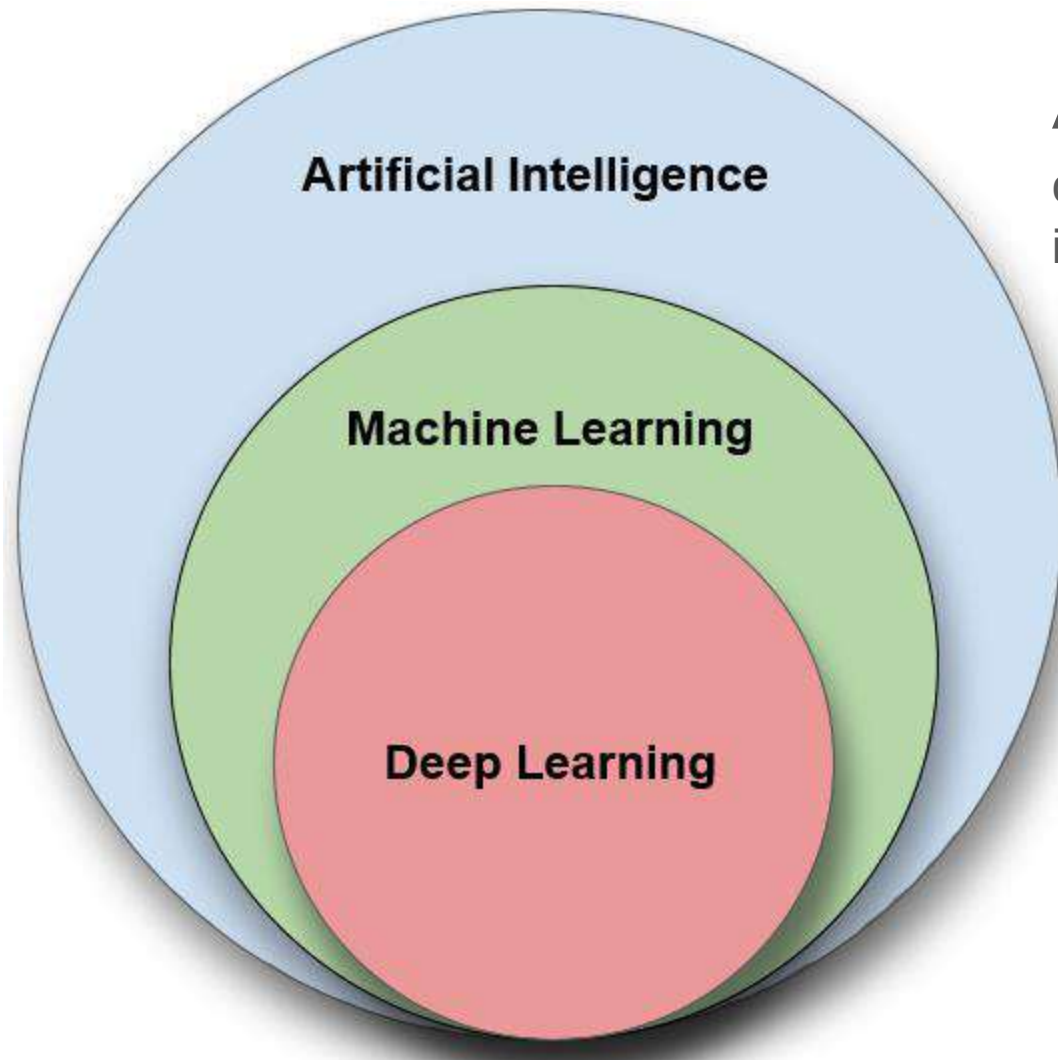
Contents

- **Module-1: Introduction to neural networks and deep neural networks**
- **Module-2: Convolution neural networks**
- **Module-3: Improving deep neural networks**
- **Module-4: Recurrent networks**
- **Module-5: Recursive neural networks**
- **Module-6: Advanced Neural networks**
- **Module-7: Deep reinforcement learning**

Introduction to Deep Learning

- Deep learning is based on the branch of machine learning, which is a subset of artificial intelligence which uses neural networks imitate the human brain.
- Deep learning is a class of machine learning algorithms that:
 - use a cascade of multiple layers of nonlinear processing units for feature extraction and transformation. Each successive layer uses the output from the previous layer as input.
 - learn in supervised (e.g., classification) and/or unsupervised (e.g., pattern analysis) manners.
 - learn multiple levels of representations that correspond to different levels of abstraction; the levels form a hierarchy of concepts.
- In deep learning is a machine learning class that makes use of numerous nonlinear processing units so as to perform feature extraction as well as transformation.
- It is extremely beneficial to data scientists who are tasked with collecting, analyzing and interpreting large amounts of data; deep learning makes this process faster and easier.

Introduction to Deep Learning



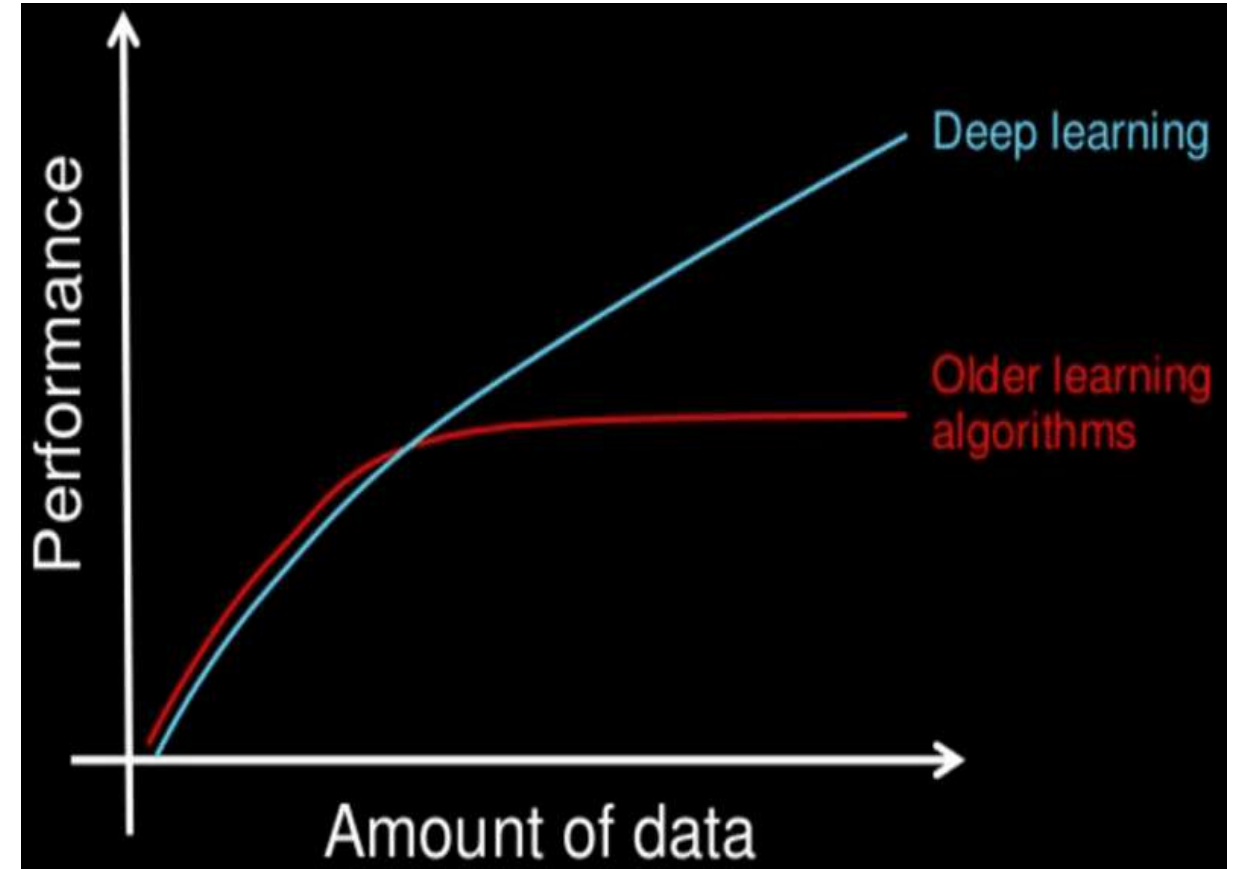
AI is man-made intelligence exhibited by machines as opposed to natural intelligence displayed by animals including humans.

ML is an approach to achieve AI and the study of computer algorithms that can improve automatically through experience and using data. It is seen as a part of artificial intelligence.

DL is part of a broader family of machine learning methods based on artificial neural networks and technique to implement ML.

Why Deep Learning

- Limitations of traditional machine learning algorithms
 - not good at handling high dimensional data.
 - difficult to do feature extraction and object recognition.
- Advantages of deep learning
 - DL is computationally expensive, but it is capable of handling high dimensional data.
 - feature extraction is done automatically.



Deep Learning-Advantages and Limitations

Advantages

- It lessens the need for feature engineering.
- It eradicates all those costs that are needless.
- It easily identifies difficult defects.
- It results in the best-in-class performance on problems.

Limitations

- It requires an ample amount of data.
- It is quite expensive to train.
- It does not have strong theoretical groundwork.
- Data labeling
- Obtain huge training datasets
- Explain a problem

Deep Learning-Applications

Self-Driving Cars

In self-driven cars, it is able to capture the images around it by processing a huge amount of data, and then it will decide which actions should be incorporated to take a left or right or should it stop. So, accordingly, it will decide what actions it should take, which will further reduce the accidents that happen every year.



Voice Controlled Assistance

When we talk about voice control assistance, then **Siri** is the one thing that comes into our mind. So, you can tell Siri whatever you want it to do it for you, and it will search it for you and display it for you.

Automatic Image Caption Generation

Whatever image that you upload, the algorithm will work in such a way that it will generate caption accordingly. If you say blue colored eye, it will display a blue-colored eye with a caption at the bottom of the image.

Automatic Machine Translation

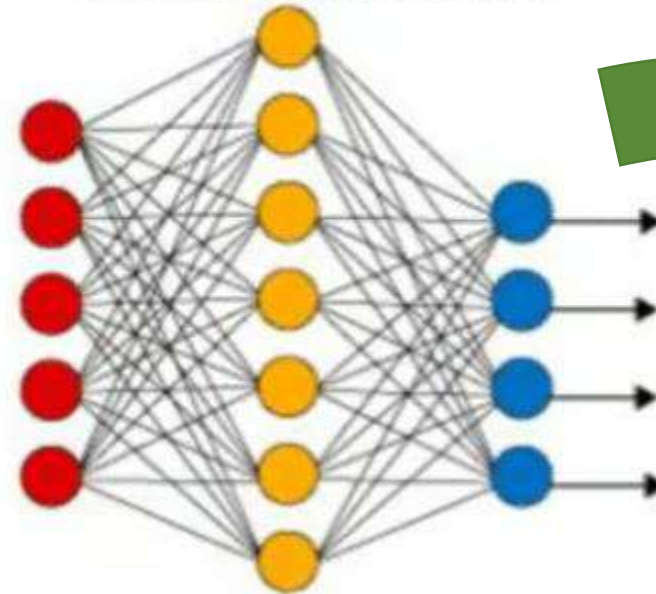
With the help of automatic machine translation, we are able to convert one language into another with the help of deep learning.



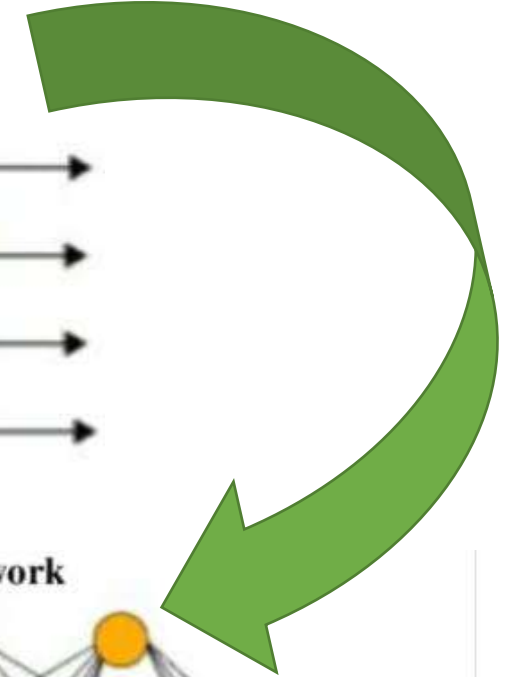
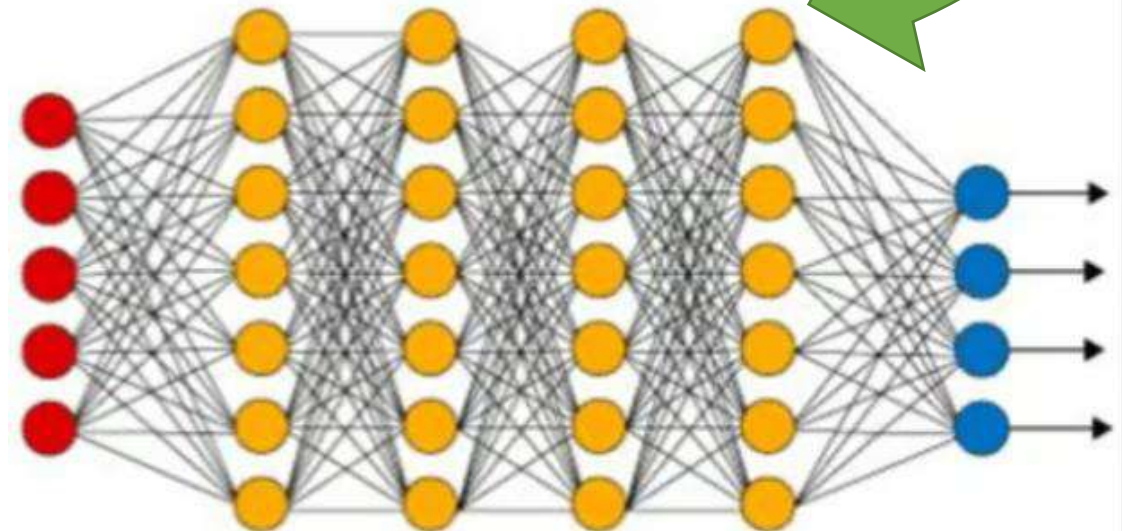
Module-1-Introduction to neural networks and deep neural networks

- Neural Networks Basics
- Functions in Neural networks- Activation function, Loss function, Function approximation
- Classification and Clustering problems
- Deep networks basics -Shallow neural networks
- Gradient Descent
- Deep Neural Networks – Forward and Back Propagation
- Parameters - Hyperparameters

Artificial Neural Network



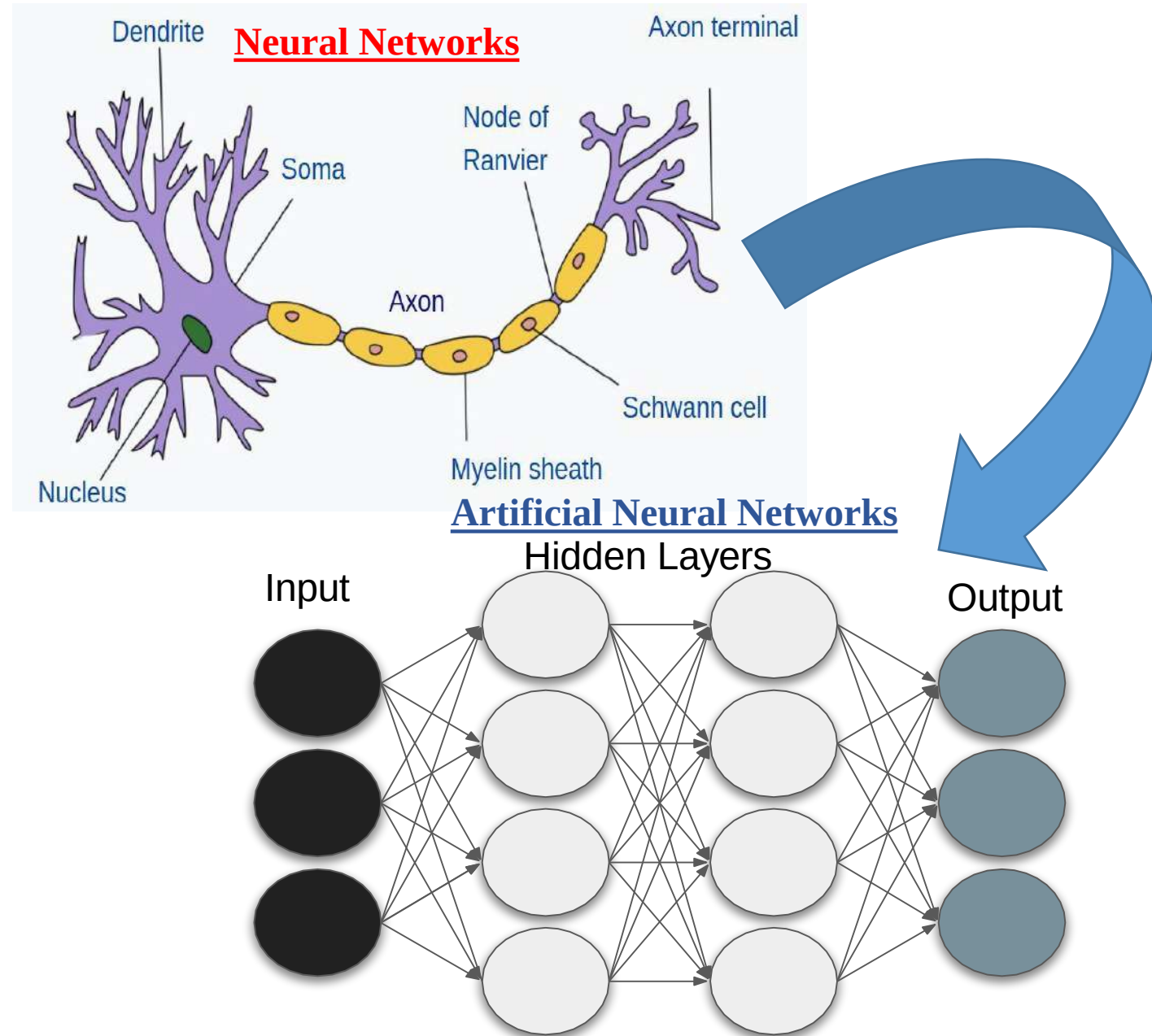
Deep Neural Network



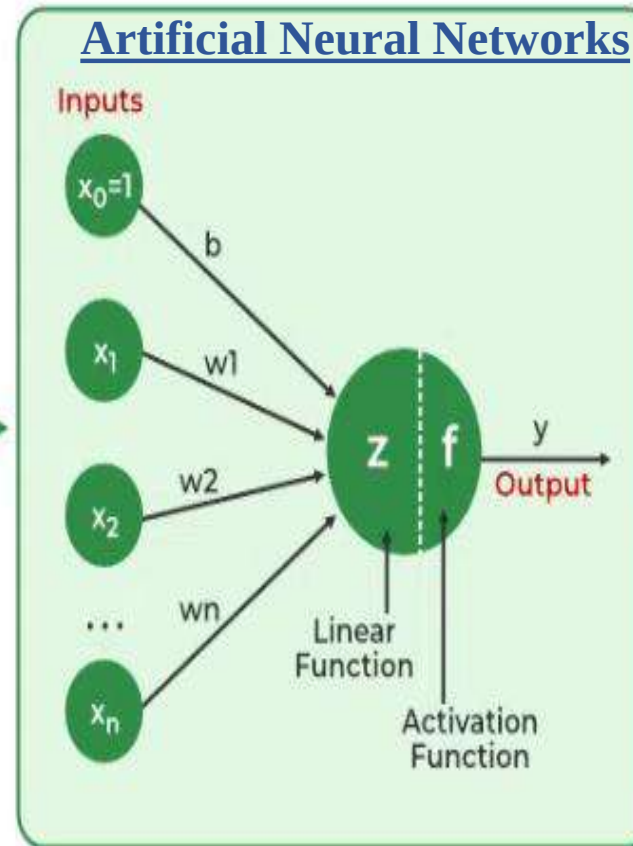
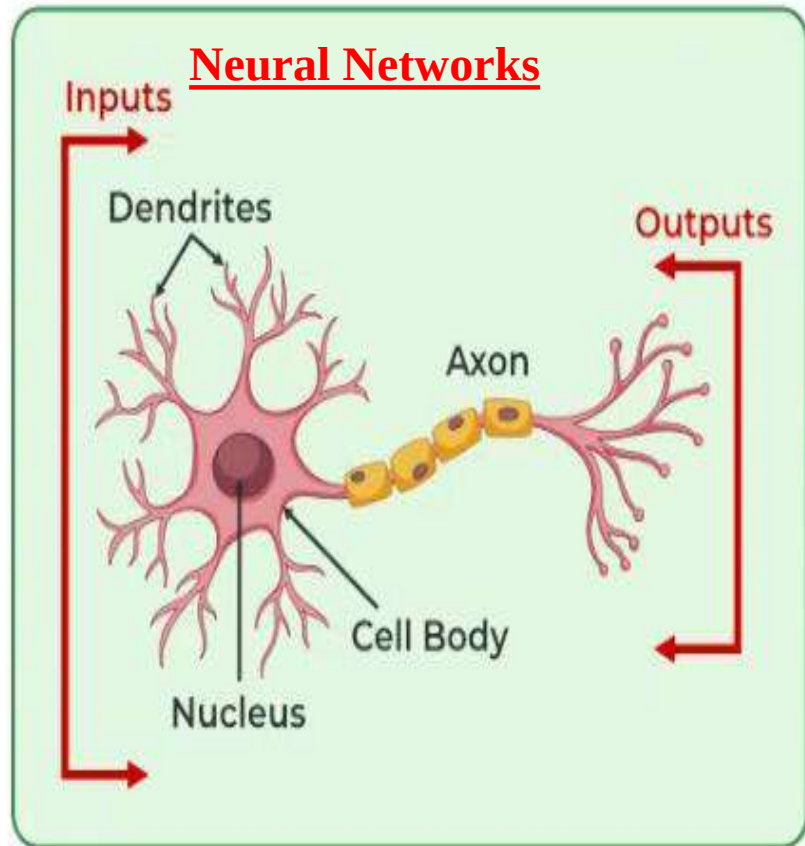
Neural Networks

- **Neural Networks** are networks of interconnected neurons for example in human brains
- **Artificial Neural Networks** are highly connected to other neurons, and performs computations by combining signals from other neurons.
- Outputs of these computations may be transmitted to one or more other neurons.
- The neurons are connected together in a specific way to perform particular task.

Module-1; Topic-1: Neural Networks Basics



Artificial Neural Networks



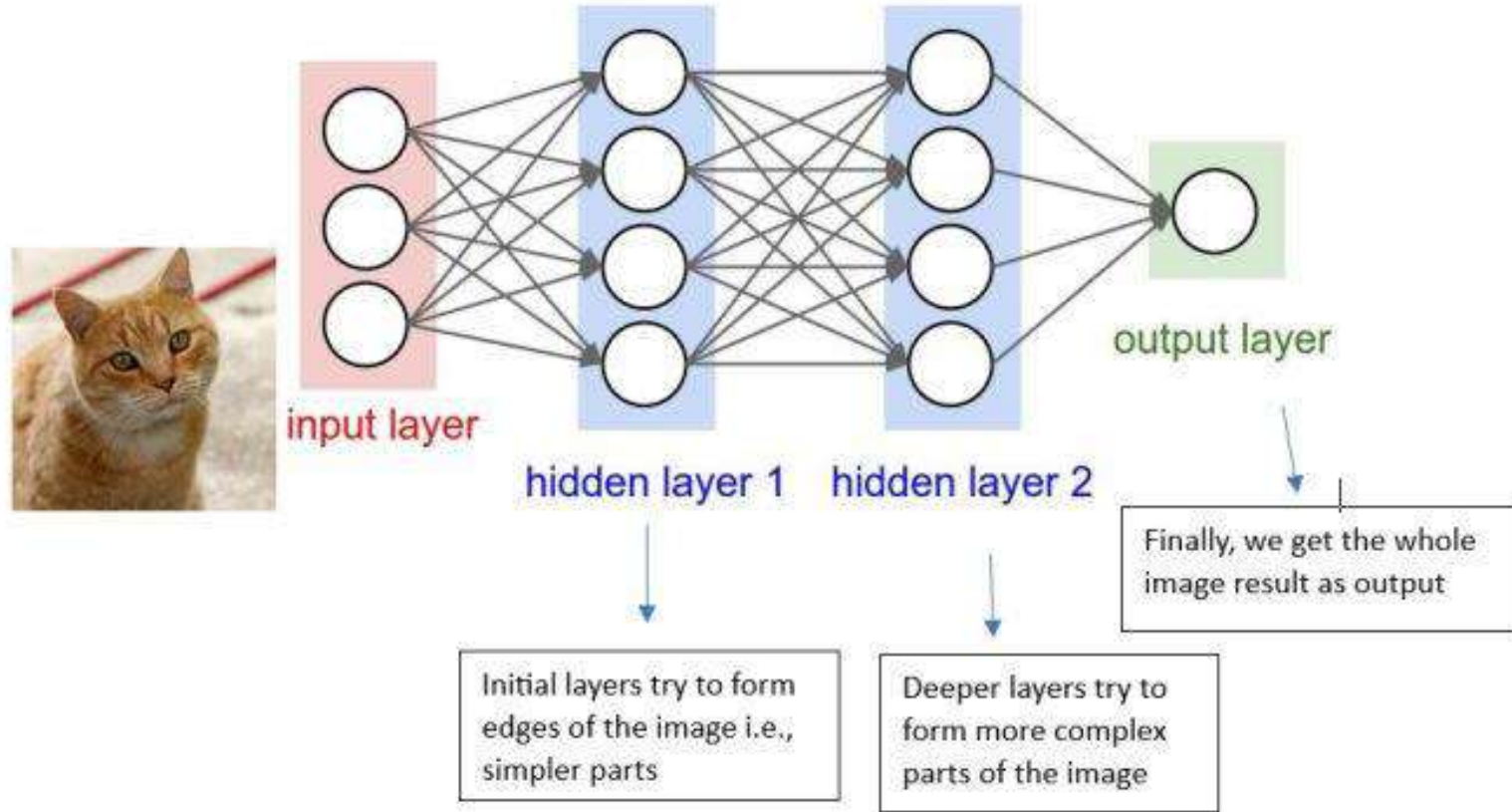
Biological Neuron	Artificial Neuron
Dendrite	Inputs
Cell nucleus or Soma	Nodes
Synapses	Weights
Axon	Output

- A neural network is a function which consists of :
 - a. **Neurons**: which pass input values through functions and output the result.
 - b. **Weights**: which carry values (real-number) between neurons.
- Neurons can be categorized into layers: a. Input Layer, b. Hidden Layer, and c. Output Layer

How Artificial Neural Networks learn

Module-1; Topic-1: Neural Networks Basics

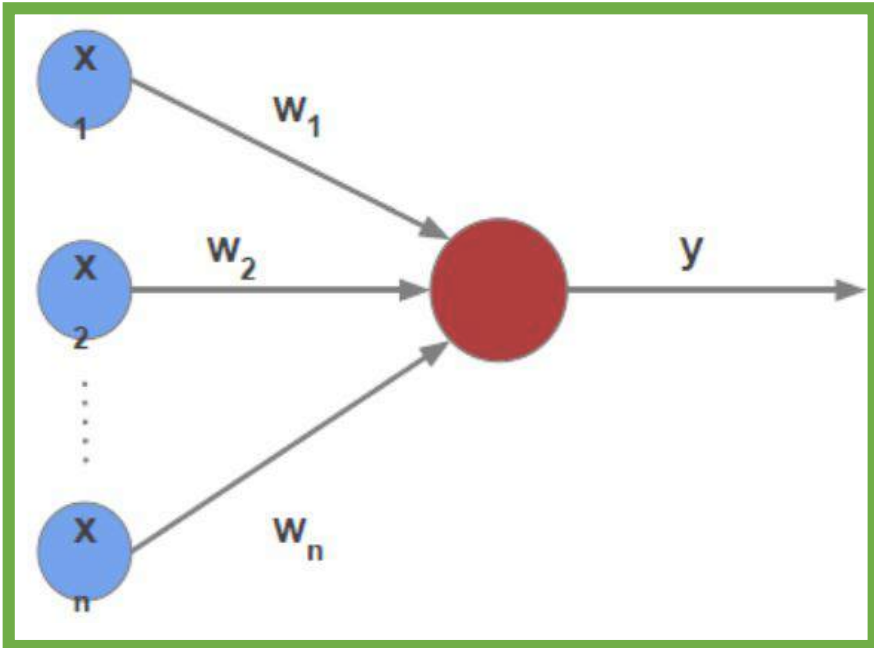
- Artificial neural networks are trained using a training set.
- For example, suppose you want to teach an ANN to recognize a cat. An ANN is trained to recognize cats by analyzing thousands of images, and then tested for correct identification by classifying the provided images.
- The output obtained by the ANN is corroborated by a human-provided description of whether the image is a cat image or not. If the ANN identifies incorrectly then [backpropagation](#) is used to adjust whatever it has learned during training.



- [Backpropagation](#) is done by fine-tuning the weights of the connections in ANN units based on the error rate obtained. This process continues until the artificial neural network can correctly recognize a cat in an image with minimal possible error rates.

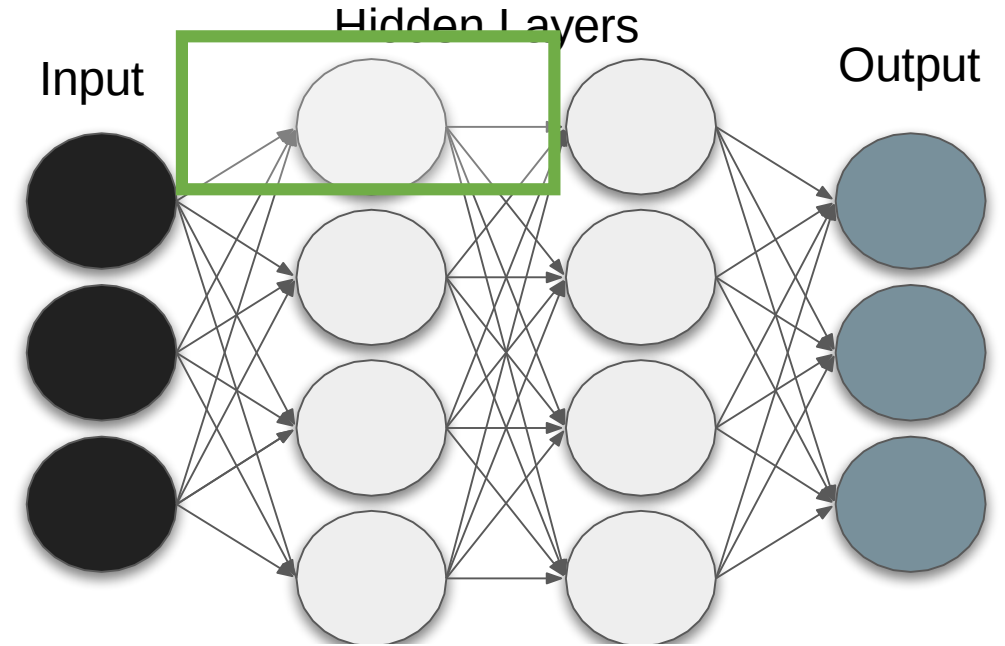
The Perceptron Model

- A perceptron is a computing element where inputs are associated with the weights and the cell having a threshold value.



Rewrite $\sum w_i x_i$ as $w \cdot x$

- Replace threshold = -b
- b**: Bias, a prior inclination towards some decision.



$$y = \begin{cases} 1, & \text{if } \sum w_i x_i > \text{threshold} \\ 0 & \text{otherwise} \end{cases} \quad \dots(1)$$

$$y = \begin{cases} 1, & \text{if } w \cdot x + b > 0 \\ 0 & \text{otherwise} \end{cases} \quad \dots(2)$$

The Perceptron Model-Training

Module-1; Topic-1: Neural Networks Basics

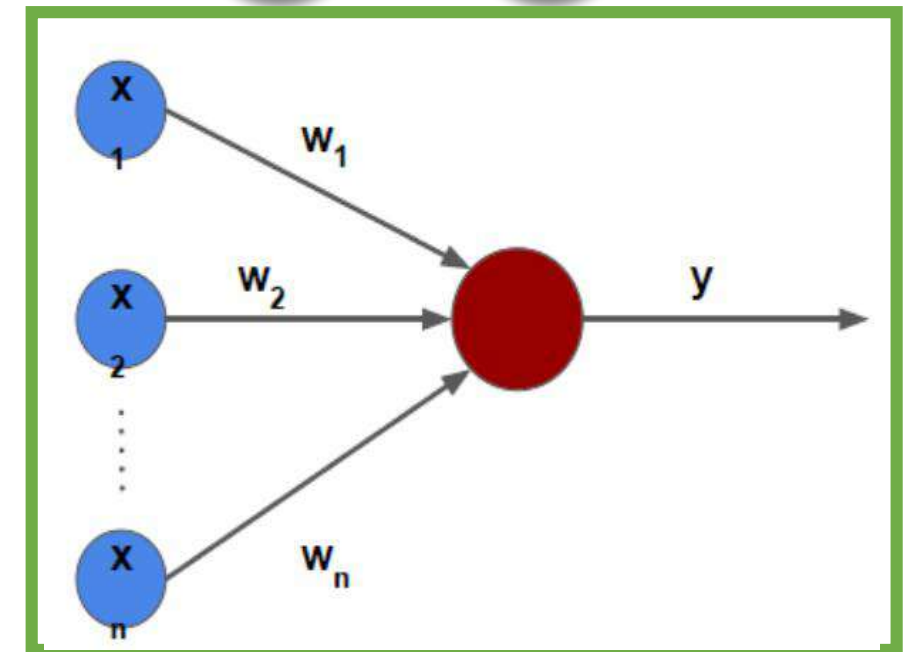
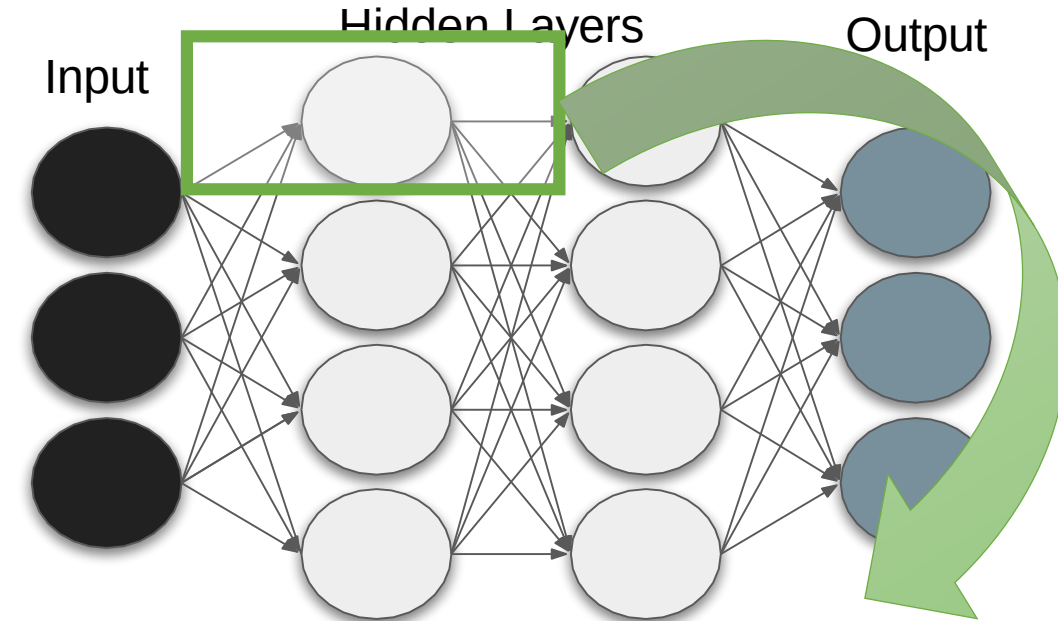
Step-1: Absorb bias b as weight.

Step-2: Start with a random value of weight w_j

Step-3: Predict for each input x_i :
If the prediction is correct $\forall x$, then
Return w

Step-4: On a mistake for given input x ,
update as follows:

- Mistake on positive ($y=1$), update $w_{j+1} \leftarrow w_j + x$
- Mistake on negative ($y=0$), update $w_{j+1} \leftarrow w_j - x$



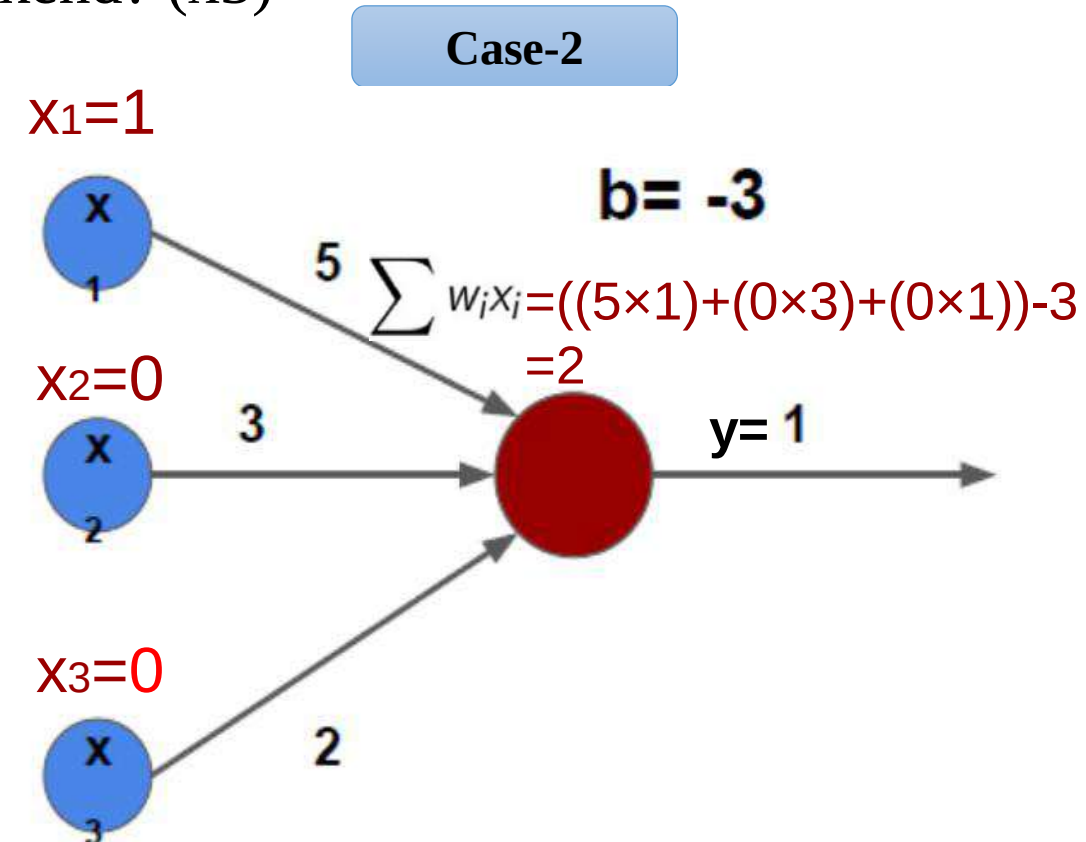
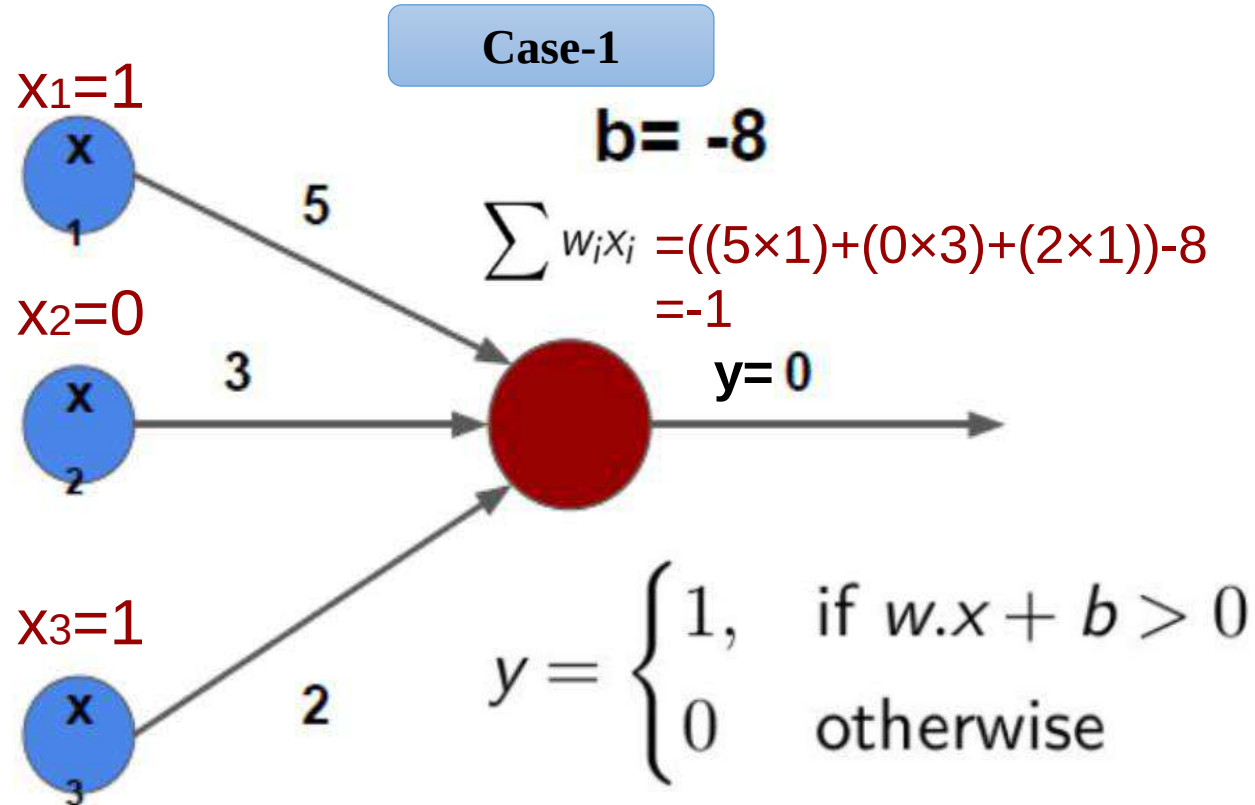
The Perceptron Model-A Simple Decision making

Task: Whether should you go to watch movie this weekend?

The decision variables are: ○ Is there any extra lecture this weekend? (x1)

○ Does your friend want to go with you? (x2)

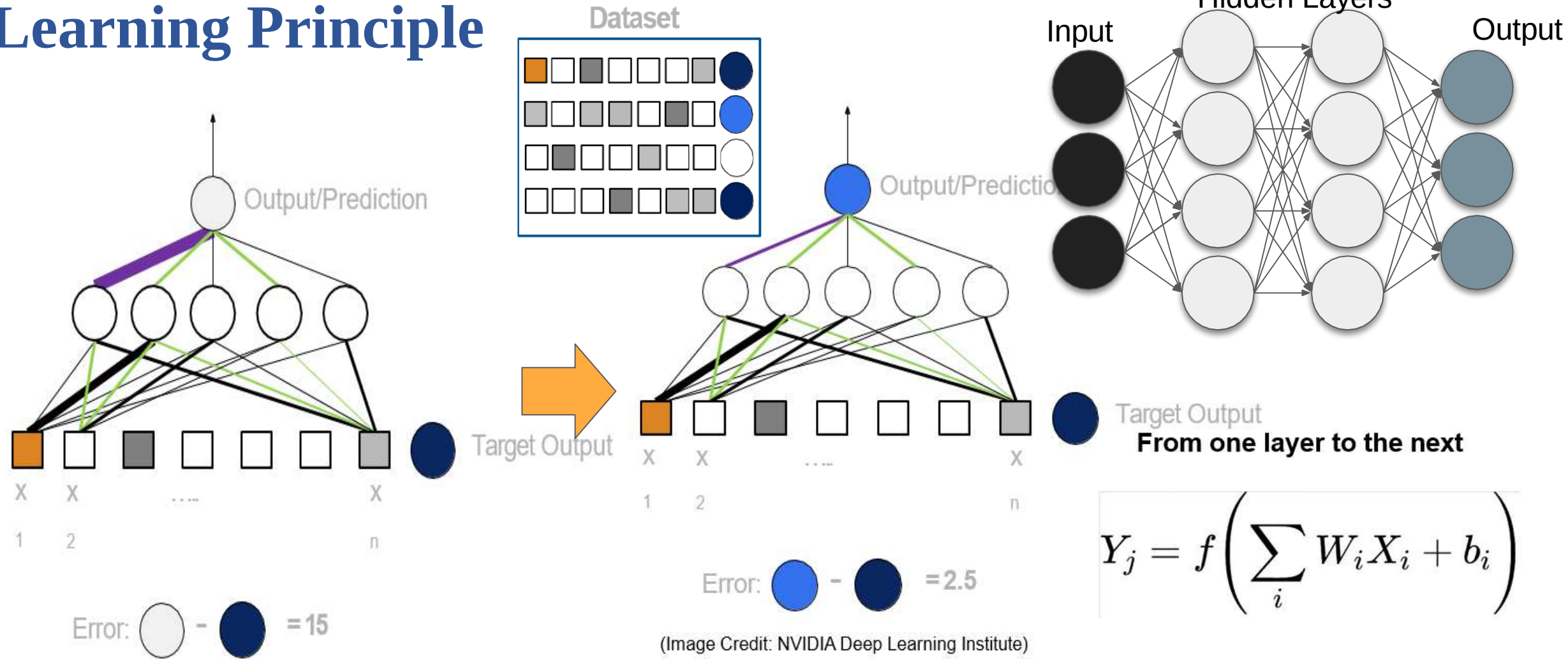
○ Do you have pending assignments due on the weekend? (x3)



Artificial Neural Networks

Learning Principle

Module-1; Topic-1: Neural Networks Basics



(Image Credit: NVIDIA Deep Learning Institute)

f is the activation function, W_i is the weight, and b_i is the bias.

Artificial Neural Networks Learning Principle-Training

Training-To minimize the loss function

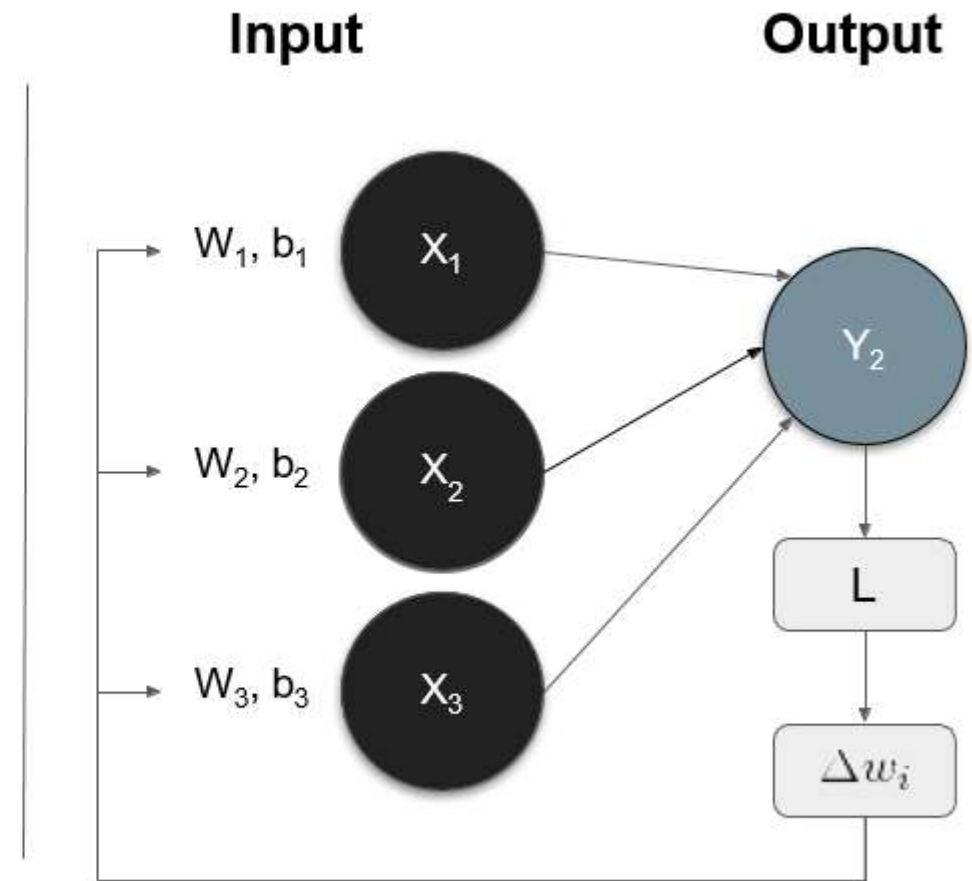
The loss function with regard to weights and biases can be defined as

$$L(\mathbf{w}, \mathbf{b}) = \frac{1}{2} \sum_i (\mathbf{Y}(\mathbf{X}, \mathbf{w}, \mathbf{b}) - \mathbf{Y}'(\mathbf{X}, \mathbf{w}, \mathbf{b}))^2$$

The weight update is computed by moving a step to the opposite direction of the cost gradient.

$$\Delta w_i = -\alpha \frac{\partial L}{\partial w_i}$$

Iterate until L stops decreasing.



Perceptron training algorithm

- Step 1:--Initialize the weights, biases and learning rate suitably. ($\alpha \rightarrow 0$ to 1)
- Step 2: while stopping condition is false do steps 3-7
- Step 3: For each training pair s:t do steps 4-6
- Step 4, Set activation of each input unit $i = 1$ to n

$$x_i = s_i$$

- Step 5: Calculate output response of each output unit; First, the net input is calculated as

$$y_{in} = b + \sum_{i=1}^n x_i w_i$$

- Then activations are applied over the net input to calculate the output response: $y = f(y_{in})$
- Step 6: Make adjustment in weights and bias for $j = 1$ to m and $i = 1$ to n .
- Step-7: Test for the stopping condition, i.e., if there is no change in weights then stop the training process, else start again from Step 2.

$$y_j = \begin{cases} 1 & \text{if } y_{inj} > 0 \\ 0 & \text{if } y_{inj} = 0 \\ -1 & \text{if } y_{inj} \leq 0 \end{cases}$$

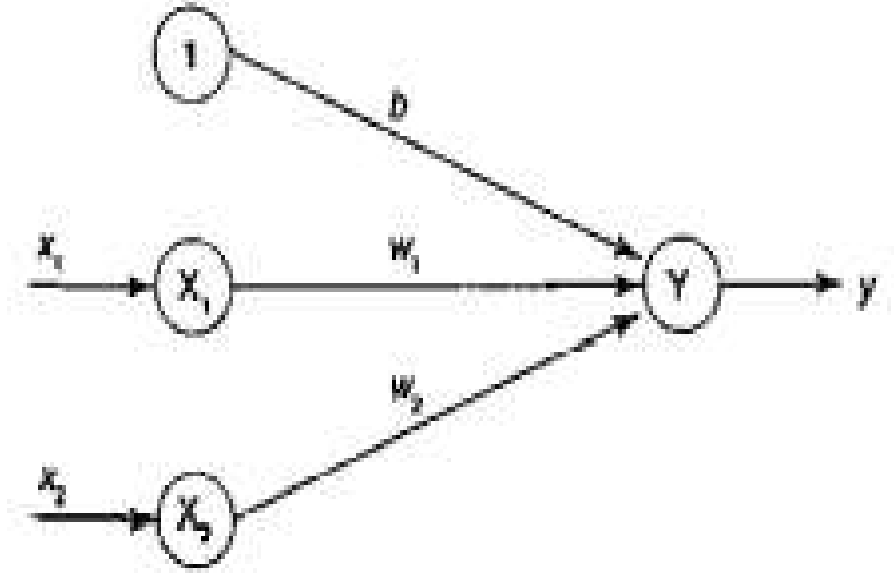
If $t \neq y$

$w_i(\text{new}) = w_i(\text{old})$
 $+ \alpha t x_i$
 $b(\text{new}) = b(\text{old}) + \alpha t$
else
 $w_i(\text{new}) = w_i(\text{old})$
 $b(\text{new}) = b(\text{old})$

Solved problem-1

- Realize a Perceptron net for the AND function with bipolar inputs and targets.

x1	x2	y(target)
1	1	1
1	-1	-1
-1	1	-1
-1	-1	-1



- Step 1: initialize the weights:

$w_1=w_2=0$ and $b=0$, $\alpha=1$, threshold=0

Step: 2 Begin computation

Step3: For the input pair (1,1): 1 do steps 4-6

Step 4: set activations of input units: $x_i=(1,1)$

Step 5: calculate net input

$$y_{\text{in}} = b + \sum x_i w_i$$

Epoch/iteration-1

input		target	Net	output	Weight change			New weights		
x_1	x_2	t	y_{in}	Y	$\Delta w_1 = \alpha t x_1$	$\Delta w_2 = \alpha t x_2$	$\Delta b = \alpha t$	w_1	w_2	b
								0	0	0
1	1	1	0	0(y≠t)	1	1	1	1	1	1
1	-1	-1	1	1(y≠t)	-1	1	-1	0	2	0
-1	1	-1	2	1(y≠t)	1	-1	-1	1	1	-1
-1	-1	-1	-3	-1(y=t)	0	0	0	1	1	-1

Epoch/iteration-1(calculations)

- first input pair(1 1):1

$$w1=w2=b=0$$

$$\begin{aligned}y_{in} &= b + \sum x_i w_i \\ &= b + x1w1 + x2w2 \\ &= 0\end{aligned}$$

$$Y=f(yin) = 0$$

$$y \neq t$$

Update weight

- second input pair(1 -1):-1

$$w1=w2=b=1$$

$$\begin{aligned}y_{in} &= b + \sum x_i w_i \\ &= b + x1w1 + x2w2 \\ &= 1 + 1 - 1 = 1\end{aligned}$$

$$y \neq t$$

Update weight

- Third input pair(-1 1):-1

$$w1=0, w2=2, b=0$$

$$\begin{aligned}y_{in} &= b + \sum x_i w_i \\ &= b + x1w1 + x2w2 \\ &= 0 + 0 + 2 = 2\end{aligned}$$

$$y=f(yin) = 1$$

$$y \neq t$$

Update weight

- fourth input pair(-1 -1):-1

$$w1=w2=1, b=-1$$

$$\begin{aligned}y_{in} &= b + \sum x_i w_i \\ &= b + x1w1 + x2w2 \\ &= -1 + (-1 \times 1) + (-1 \times 1) \\ &= -3\end{aligned}$$

$$y = t$$

No update of weights(no change in weights)

New weights:

$$w1=w2=1, b=-1$$

Epoch/iteration-2

- first input pair(1 1):1

$$w1=w2=1, b=-1$$

$$\begin{aligned} y_{\text{in}} &= b + \sum x_i w_i \\ &= b + x1w1 + x2w2 \\ &= -1 + 1 + 1 = 1 \end{aligned}$$

$$y = t$$

No update of weights(no change in weights)

- second input pair(1 -1):-1

$$w1=w2=1, b=-1$$

$$\begin{aligned} y_{\text{in}} &= b + \sum x_i w_i \\ &= b + x1w1 + x2w2 \\ &= -1 + 1 - 1 = -1 \end{aligned}$$

$$y = t$$

No update of weights(no change in weights)

- Third input pair(-1 1):-1

$$w1=w2=1, b=-1$$

$$\begin{aligned} y_{\text{in}} &= b + \sum x_i w_i \\ &= b + x1w1 + x2w2 \\ &= -1 - 1 + 1 = -1 \end{aligned}$$

$$y = t$$

No update of weights(no change in weights)

- fourth input pair(-1 -1):-1

$$w1=w2=1, b=-1$$

$$\begin{aligned} y_{\text{in}} &= b + \sum x_i w_i \\ &= b + x1w1 + x2w2 \\ &= -1 + (-1 \times 1) + (-1 \times 1) \\ &= -3 \end{aligned}$$

$$Y = f(y_{\text{in}}) = -1$$

$$y = t$$

No update of weights(no change in weights)

New weights:

$$w1=w2=1, b=-1$$

Epoch/iteration-2

input		target	Net	output	Weight change			New weights		
x_1	x_2	t	y_{in}	y	$\Delta w_1 = \alpha t x_1$	$\Delta w_2 = \alpha t x_2$	$\Delta b = \alpha t$	w_1	w_2	b
1	1	1	1	1(y=t)	0	0	0	1	1	-1
1	-1	-1	-1	-1(y=t)	0	0	0	1	1	-1
-1	1	-1	-1	-1(y=t)	0	0	0	1	1	-1
-1	-1	-1	-3	-1(y=t)	0	0	0	1	1	-1

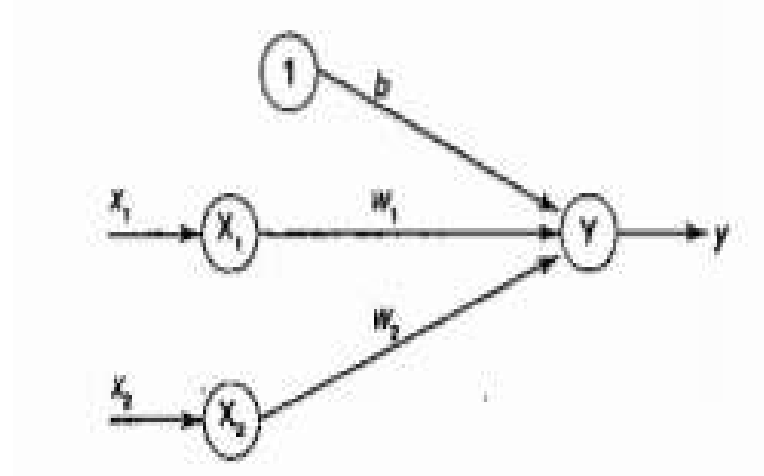
The final weights after second epoch is

$w_1=1, w_2=1, b=-1$

Solved problem-2

- Implement the OR function with binary inputs and bipolar targets using perceptron training algorithm up to 3 epochs.

x1	x2	y(target)
1	1	1
1	0	1
0	1	1
0	0	-1



Contd..

Input			Target (t)	Net input (y_{in})	Calculated output (y)	Weight changes			Weights		
x_1	x_2	1				Δw_1	Δw_2	Δb	w_1 (0	w_2 0	b 0)
EPOCH-1											
1	1	1	1	0	0	1	1	1	1	1	1
1	0	1	1	2	1	0	0	0	1	1	1
0	1	1	1	2	1	0	0	0	1	1	0
0	0	1	-1	1	1	0	0	-1	1	1	0
EPOCH-2											
1	1	1	1	2	1	0	0	0	1	1	0
1	0	1	1	1	1	0	0	0	1	1	0
0	1	1	1	1	1	0	0	0	1	1	0
0	0	1	-1	0	0	0	0	0	1	1	-1
EPOCH-3											
1	1	1	1	1	1	0	0	0	1	1	-1
1	0	1	1	0	0	1	0	1	2	1	0
0	1	1	1	1	1	0	0	0	2	1	0
0	0	1	-1	0	0	0	0	-1	2	1	-1

$w_1 = 2, w_2 = 1, b = -1$

Solved problem -3

Using the perceptron learning rule, find the weights required to perform the following classifications. Vectors $(1 \ 1 \ 1 \ 1)$, $(-1 \ 1 \ -1 \ -1)$ are members of class (having target value 1); Vectors $(1 \ 1 \ 1 \ -1)$, $(1 \ -1 \ -1 \ 1)$ are not members of class (having target value -1); use learning rate 1 and starting weights 0. Using each of the training vectors as input, test the response of the net.

Epoch/iteration-1

input				target	Net	out put	Weight change					New weights				
x_1	x_2	x_3	x_4	t	y_{in}	y	$\Delta w_1 = \alpha t x_1$	$\Delta w_2 = \alpha t x_2$	$\Delta w_3 = \alpha t x_3$	$\Delta w_4 = \alpha t x_4$	$\Delta b = \alpha t$	w_1	w_2	w_3	w_4	b
												0	0	0	0	0
1	1	1	1	1	0	0	1	1	1	1	1	1	1	1	1	1
-1	1	-1	-1	1	-1	-1	-1	1	-1	-1	1	0	2	0	0	2
1	1	1	-1	-1	1	1	-1	-1	-1	1	-1	-1	1	-1	1	1
1	-1	-1	1	-1	0	0	-1	1	1	-1	-1	-2	2	0	0	0

Epoch/iteration-2

input				target	Net	output	Weight change					New weights				
x_1	x_2	x_3	x_4	t	y_{in}	y	$\Delta w_1 = \alpha t x_1$	$\Delta w_2 = \alpha t x_2$	$\Delta w_2 = \alpha t x_3$	$\Delta w_2 = \alpha t x_4$	$\Delta b = \alpha t$	w_1	w_2	w_3	w_4	b
												-2	2	0	0	0
1	1	1	1	1			1	1	1	1	1	-1	3	1	1	1
-1	1	-1	-1	1			0	0	0	0	0	-1	3	1	1	1
1	1	1	-1	-1			-1	-1	-1	1	1	-2	2	0	2	2
1	-1	-1	1	-1			0	0	0	0	0	-2	2	0	2	2

Epoch/iteration-3

input				target	Net	output	Weight change					New weights				
x_1	x_2	x_3	x_4	t	y_{in}	y	$\Delta w_1 = \alpha t x_1$	$\Delta w_2 = \alpha t x_2$	$\Delta w_3 = \alpha t x_3$	$\Delta w_4 = \alpha t x_4$	$\Delta b = \alpha t$	w_1	w_2	w_3	w_4	b
1	1	1	1	1			0	0	0	0	0	-2	2	0	2	2
-1	1	-1	-1	1			0	0	0	0	0	-2	2	0	2	2
1	1	1	-1	-1			0	0	0	0	0	-2	2	0	2	2
1	-1	-1	1	-1			0	0	0	0	0	-2	2	0	2	2



Network is learned

Testing

- Final weights after learning:
 $w_1=-2, w_2=2, w_3=0, w_4=2, b=2$

- For the first input, $(1 \ 1 \ 1 \ 1):1$

$$y_{in} = b + \sum x_i w_i$$

$$y = f(y_{in})$$

$$y_{in} = 2 - 2 + 2 + 0 + 2 = 4 > 0$$

$$y = 1 \quad (y=t)$$

Correct response

- For the second input,
 $(-1 \ -1 \ -1 \ 1):1$

$$y_{in} = b + \sum x_i w_i$$

$$y = f(y_{in})$$

$$y_{in} = 2 + 2 - 2 + 0 + 2 = 4$$

$$y = 1 \quad (y=t)$$

Correct response

Testing

- For the second input,

(1 1 1 -1):-1

$$y_{in} = b + \sum x_i w_i$$

$$y = f(y_{in})$$

$$y_{in} = 2 - 2 + 2 + 0 - 2 = 0$$

Correct response

- $w_1 = -2, w_2 = 2, w_3 = 0, w_4 = 2, b = 2$

- For the second input,

(1 -1 -1 1):-1

$$y_{in} = b + \sum x_i w_i$$

$$y = f(y_{in})$$

$$y_{in} = 2 - 2 - 2 + 0 + 2 = 0$$

Correct response

TWO CLASS: CLASS 1, CLASS 0

Solved problem -4

Using the perceptron learning rule, find the weights required to perform the following classifications. Vectors $(1 \ 1 \ 1 \ 1)$, $(-1 \ 1 \ -1 \ -1)$ are members of class (having target value - 1); Vectors $(1 \ 1 \ 1 \ -1)$, $(1 \ -1 \ -1 \ 1)$ are not members of class (having target value 1); use learning rate 1 and starting weights 0. Using each of the training vectors as input, test the response of the net.

epoch/iteration-1

[illegible]

Solved problem-5

For the following noisy versions of training patterns, identify the response of network by segregating it into correct , incorrect or indefinite.

(0-1 1), (0 1-1), (0 0 1), (0 0 -1), (0 1 0), (1 0 1),
(1 0 -1), (1 -1 0), (1 0 0), (1 1 0), (0 -1 0), (1 1 1)

Contd..

Solution:

if $x_1w_1+x_2w_2+x_3w_3 > 0$, response is correct

if $x_1w_1+x_2w_2+x_3w_3 < 0$, response is incorrect

if $x_1w_1+x_2w_2+x_3w_3 = 0$, response is indefinite or undetermined.

If weights are taken as $w_1=0, w_2=-2, w_3=2$

For $(0 \ -1 \ 1)$, $x_1=0, x_2=-1, x_3=1$

So, $x_1w_1+x_2w_2+x_3w_3 = 0+2+2 = 4$

> 0

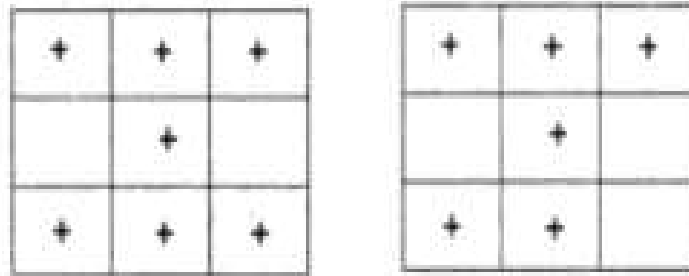
CORRECT

Contd..

x1	x2	x3	$x_1w_1+x_2w_2+x_3w_3$ w1=0,w2=-2,w3=2	Response
0	-1	0	2	CORRECT
0	-1	1	4	CORRECT
0	1	-1	-4	INCORRECT
0	0	1	2	CORRECT
0	0	-1	-2	INCORRECT
0	1	0	-2	INCORRECT
1	0	1	2	CORRECT
1	0	-1	-2	INCORRECT
1	-1	0	2	CORRECT
1	0	0	-0	UNDETERMINED
1	1	0	-2	INCORRECT
1	1	1	0	UNDETERMINED

Application based question

- Classify the two-dimensional input pattern using perceptron neural network (letters I-J pattern)



Functions in Neural Networks

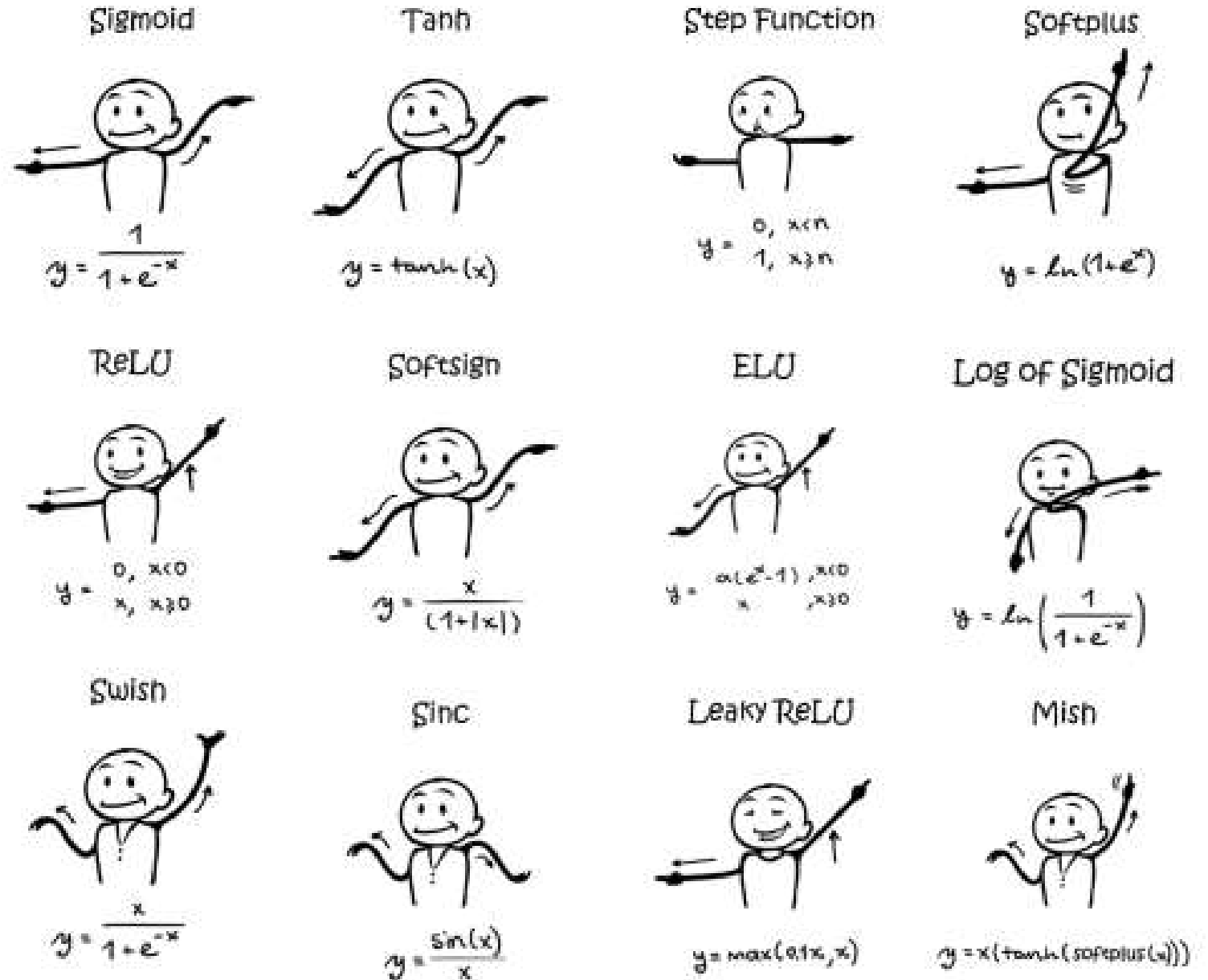
- Several types of functions are employed to transform inputs, compute outputs, and adjust the weights during training.
- These functions are fundamental to how a neural network learns, makes predictions, and adjusts to data.
- Each of the following functions plays a distinct but interconnected role in the training, generalization, and overall performance of deep neural networks.
 - **Activation Functions:** Sigmoid, Tanh, ReLU, Leaky ReLU, Softmax
 - **Loss Functions:** MSE, Cross-Entropy Loss, Hinge Loss, Huber Loss
 - **Optimization Functions:** Gradient Descent, Adam, RMSprop
 - **SGDNormalization:** Batch Normalization, Layer Normalization
 - **Regularization:** L1, L2, Dropout

Activation Functions

- Activation functions introduce non-linearity into the neural network, allowing it to learn complex patterns.
- Without them, the network would only be able to model linear relationships, regardless of how many layers it had.

Source:

<https://sefiks.com/2020/02/02/dance-moves-of-deep-learning-activation-functions/>



Activation Functions

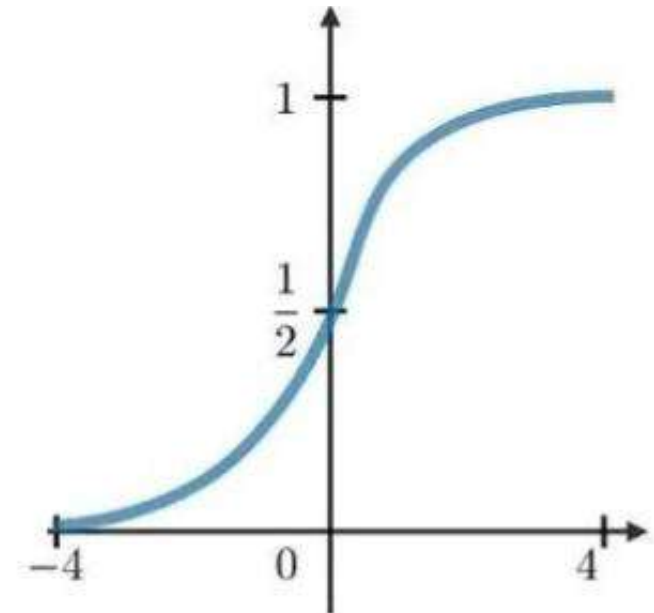
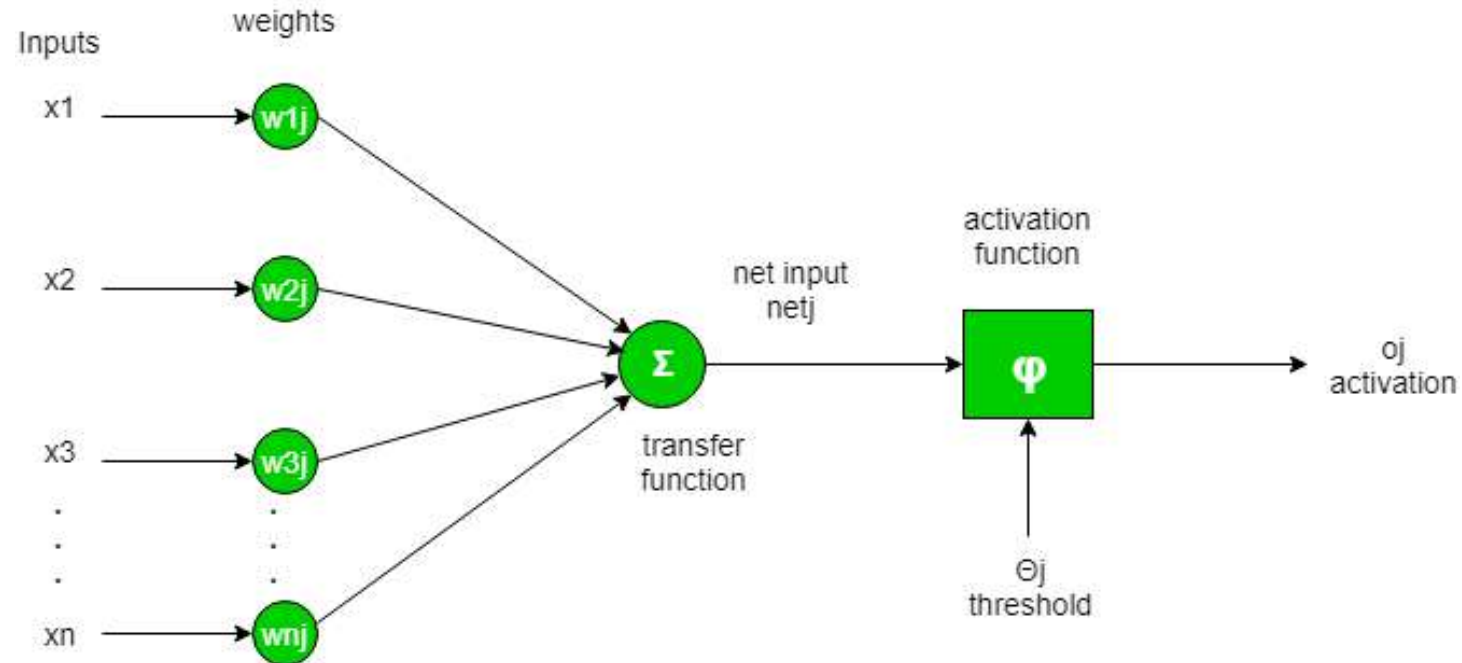
- An activation function is a mathematical equation that determines the output of a neural network model.
- The function is attached to each neuron in the network, and it decides whether it should be activated ("fired") or not, based on the input it receives, effectively introducing non-linear properties to the network.

Common Activation Functions:

- Sigmoid (Logistic):

$$f(x) = \frac{1}{1 + e^{-x}}$$

- Output range: (0, 1)
- Commonly used in binary classification problems.
- Issues: Vanishing gradients for large positive or negative inputs.



Activation Functions

Softmax:

$$f(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

- Output range: (0, 1), but sum of all outputs = 1.
- Used in the final layer for multi-class classification problems, converting logits into probabilities.

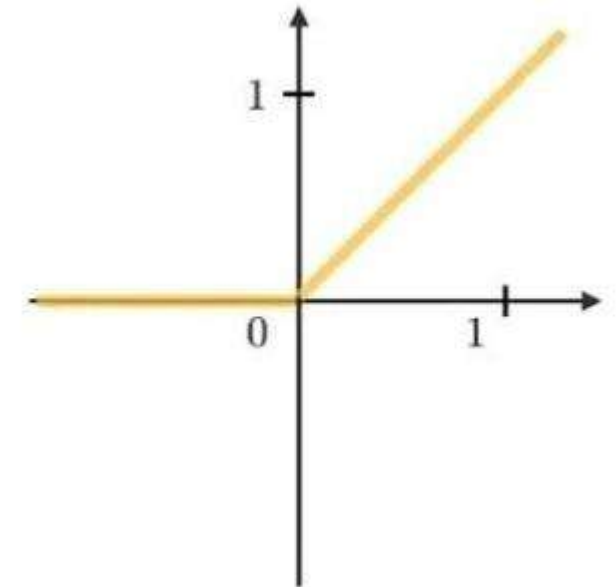
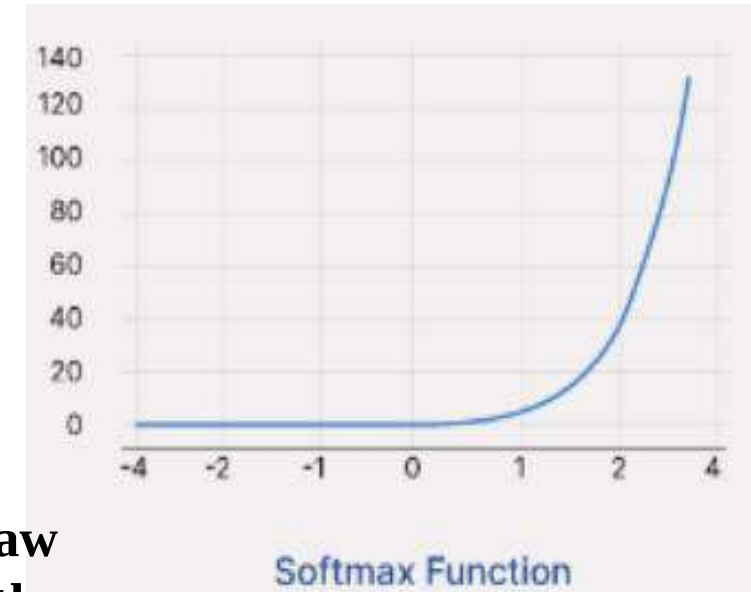
Softmax is a classification network output layer technique that converts raw scores into probabilities by exponentiating each logit and normalizing by the sum of exponentiated logits.

Rectified Linear Unit (ReLU):

$$f(x) = \max(0, x)$$

- Output range: $[0, \infty)$
- Widely used in modern deep learning architectures because it is computationally efficient and helps mitigate the vanishing gradient problem.
- Issues: "Dying ReLU" problem, where neurons can become inactive and stop learning.

Module-1; Topic-2: Functions in Neural networks

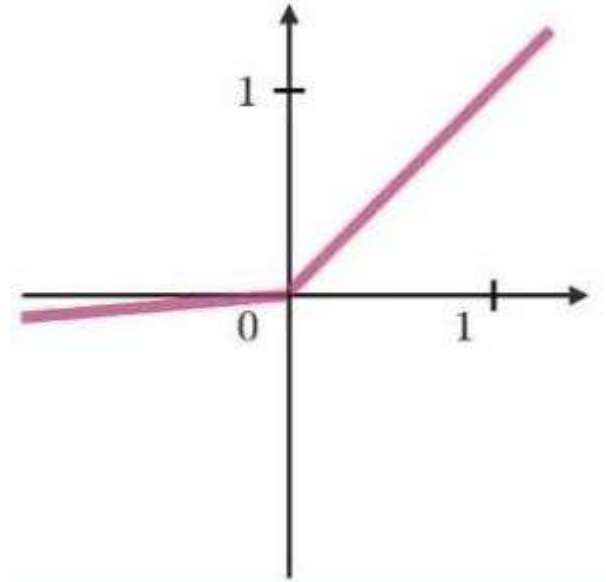


Activation Functions

Leaky ReLU:

$$f(x) = \max(\alpha x, x)$$

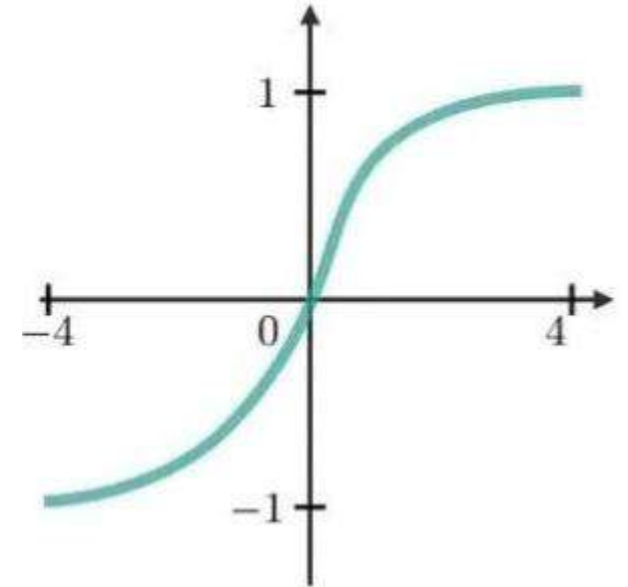
- Output range: $(-\infty, \infty)$ (with small slope for negative inputs)
- A variation of ReLU that allows a small gradient for negative values, helping to avoid the "dying ReLU" problem.



Hyperbolic Tangent (tanh):

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

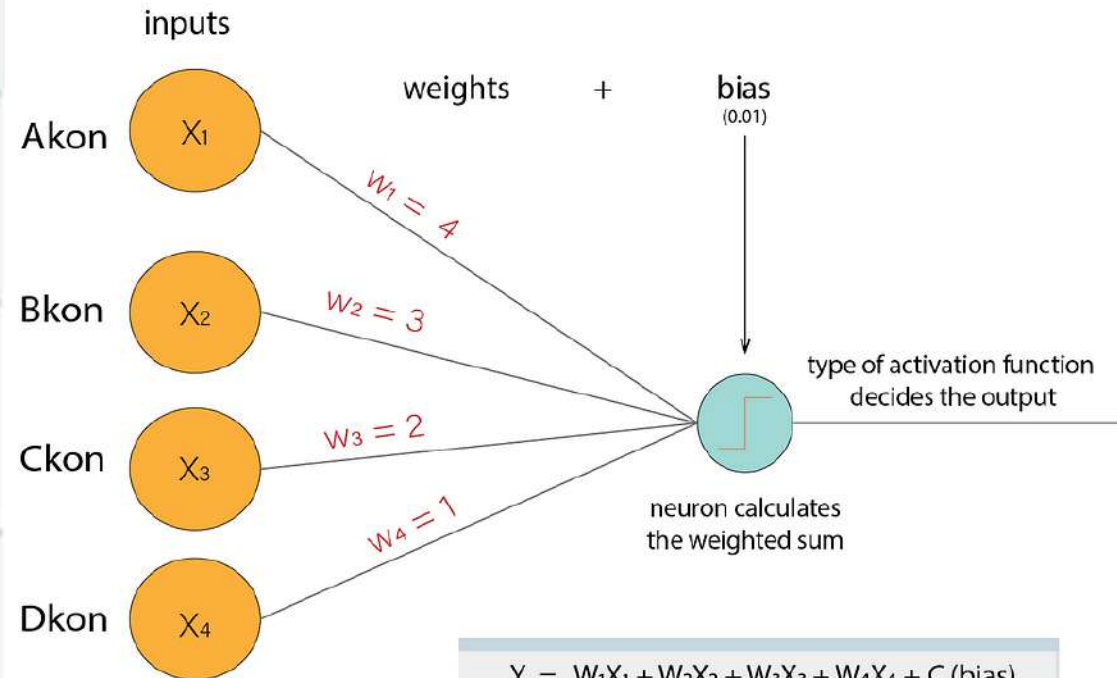
- Output range: $(-1, 1)$
- Like the sigmoid but with a wider output range. However, it also suffers from vanishing gradients.



Activation Functions-An Example

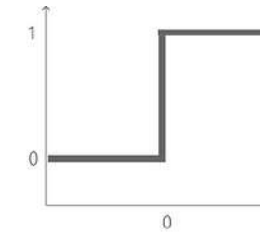
Votes for vacation destinations

	Akon	Bkon	Ckon	Dkon
Thailand	-9	10	3	-2
Jamaica	-4	5	-1	8
Dubai	-3	6	-2	6



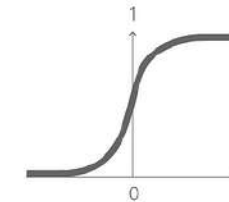
$$Y = W_1X_1 + W_2X_2 + W_3X_3 + W_4X_4 + C \text{ (bias)}$$

Thailand	$(4 \times -9) + (3 \times 10) + (2 \times 3) + (1 \times -2)$	=	-2
Jamaica	$(4 \times -4) + (3 \times 5) + (2 \times -1) + (1 \times 8)$	=	5
Dubai	$(4 \times -3) + (3 \times 6) + (2 \times -2) + (1 \times 6)$	=	8



Step

Thailand: 0
Jamaica: 1
Dubai: 1



Sigmoid
(Probabilities)

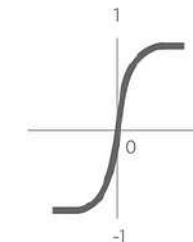
Thailand: 0.002
Jamaica: 0.85
Dubai: 0.94

*probabilities do not add to 1

Softmax
(Probabilities)

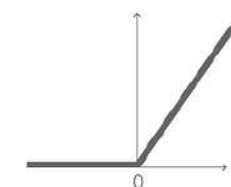
→ Thailand: 0.02
Jamaica: 0.22
Dubai: 0.76

*probabilities add to 1



tanh

Thailand: -0.2
Jamaica: 0.5
Dubai: 0.8



ReLU

Thailand: 0
Jamaica: 5
Dubai: 8

Hebbian Rule

The Hebbian learning rule is a principle used in neural networks where the connection strength between two neurons is increased if they are activated together. It is often summarized as:

$$\Delta w_{ij} = \eta \cdot x_i \cdot y_j$$

Where:

- Δw_{ij} = change in the synaptic weight between neuron i and neuron j
- η = learning rate (a small positive constant)
- x_i = input from neuron i
- y_j = output from neuron j

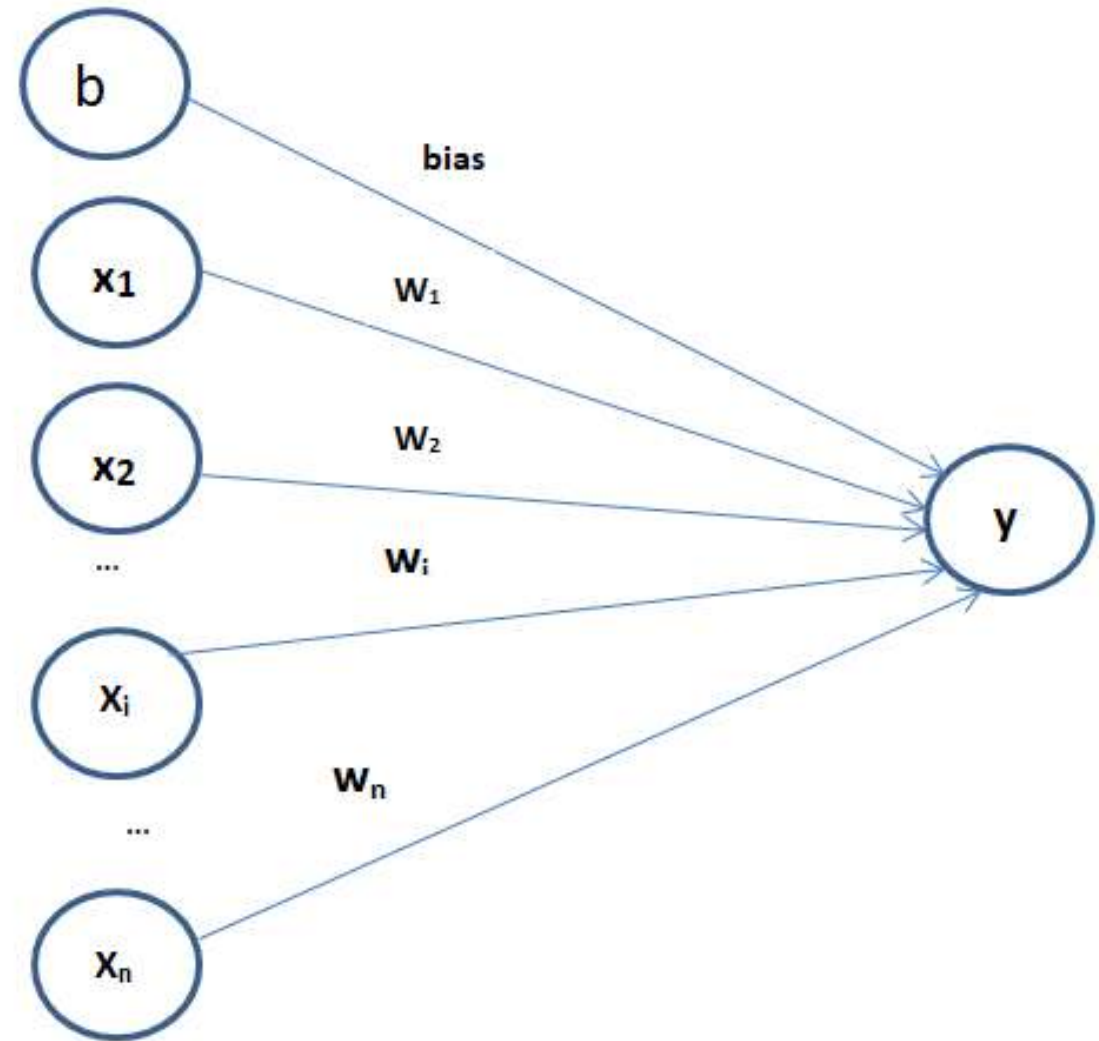
The rule states that if both x_i and y_j are high (i.e., the pre- and post-synaptic neurons are active together), the synaptic weight increases, strengthening the connection between them.

Hebb network

- Hebb rule is more suited for bipolar data
- limitations : Hebb rule not applicable for binary data
- weight update in Hebb rule:

$$w_i(\text{new}) = w_i(\text{old}) + \Delta w$$

$\Delta w \rightarrow$ weight change



Hebbian learning-training algorithm

- Step 0: First initialize the weights. Basically in this network they may be zero, i.e., $w = 0$ for $i = 1$ to n
where "n" may be the total number of input neurons.
- Step 1: Steps 2-4 have to be performed for each input training vector and target output pair, $s: t$.
- Step 2: Input units activations are set.
$$x_i = s_i \text{ for } i = 1 \text{ to } n$$
- Step 3: Output units activations are set: $y = t$
- Step 4: Weight & bias calculations:
$$w_i(\text{new}) = w_i(\text{old}) + x_i y$$
$$b(\text{new}) = b(\text{old}) + y$$

Solved problem-1

- Realize a Hebb net for the AND function with bipolar inputs and targets.

x_1	x_2	$y(\text{target})$
1	1	1
1	-1	-1
-1	1	-1
-1	-1	-1

Initialize : $w_1=w_2=0$ and bias, $b=0$

Change in weight and bias $\Delta w_i = x_i y$, $\Delta b = y$

Solved problem-1

inputs		target	Change in weights		Change in bias	New weight	New weight	New bias
x_1	x_2	y	$\Delta w_1 = x_1 y$	$\Delta w_2 = x_2 y$	$\Delta b = y$	w_1	w_2	b
1	1	1	1	1	1	1	1	1
1	-1	-1	-1	1	-1	0	2	0
-1	1	-1	1	-1	-1	1	1	-1
-1	-1	-1	1	1	-1	2	2	-2

testing $W1=2, w2=2, b=-2 \rightarrow (1, 1)$
 $X1w1+x2w2+b=2 \rightarrow y=f(yin) = 1 \rightarrow y=t$

$W1=2, w2=2, b=-2 \rightarrow (1, -1)$
 $X1w1+x2w2+b= -2 \rightarrow y=f(yin) = -1 \rightarrow y=t$

Solved problem-2

- Realize a Hebb net for the XOR function with bipolar inputs and targets.

x_1	x_2	y (target)
1	1	-1
1	-1	1
-1	1	1
-1	-1	-1

Solved problem-2

x_1	x_2	y	$\Delta w_1 = x_1 y$	$\Delta w_2 = x_2 y$	$\Delta b = y$	$W_1 = 0$	$w_2 = 0$	$b = 0$
1	1	-1	-1	-1	-1	-1	-1	-1
1	-1	1	1	-1	1	0	-2	0
-1	1	1	-1	1	1	-1	-1	1
-1	-1	-1	1	1	-1	0	0	0

Exercise problems

1. Design a Hebb net to implement OR function (consider bipolar inputs and targets).

Exercise problems

1. Using the hebb rule, find the weights required to perform the following classifications:

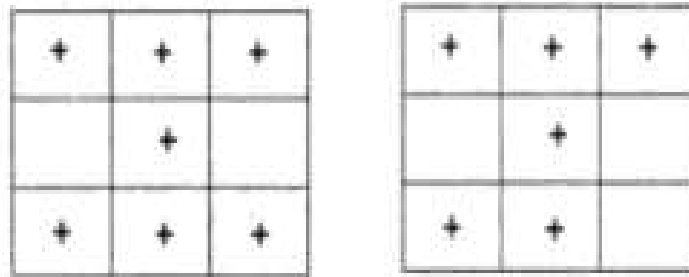
vectors (1 1 1 1) and (-1 1 -1 -1) are the members of class (with target value 1) ;

vectors (1 1 1 -1) and (1 -1 -1 1) are the members of class (with target value

x1	x2	x3	x4	t		
1	1	1	1	1		
-1	1	-1	-1	1		
1	1	1	-1	-1		
1	-1	-1	1	-1		

Application based question

- Classify the two-dimensional input pattern using Hebb neural network (letters I-J pattern)

[illegible]

Hebbian Rule

Numerical Example of Hebbian Learning:

Consider the following scenario:

- You have two neurons, i and j .
- The input from neuron i , $x_i = 0.5$
- The output from neuron j , $y_j = 1$
- The initial synaptic weight $w_{ij} = 0.2$
- Learning rate $\eta = 0.1$

Updated Weight:

The new weight after applying the Hebbian rule would be the old weight plus the change:

$$w_{ij}^{new} = w_{ij} + \Delta w_{ij}$$

$$w_{ij}^{new} = 0.2 + 0.05 = 0.25$$

Thus, the new synaptic weight between neuron i and neuron j after applying the Hebbian rule is 0.25.

We can calculate the change in the synaptic weight using the Hebbian rule.

$$\Delta w_{ij} = \eta \cdot x_i \cdot y_j$$

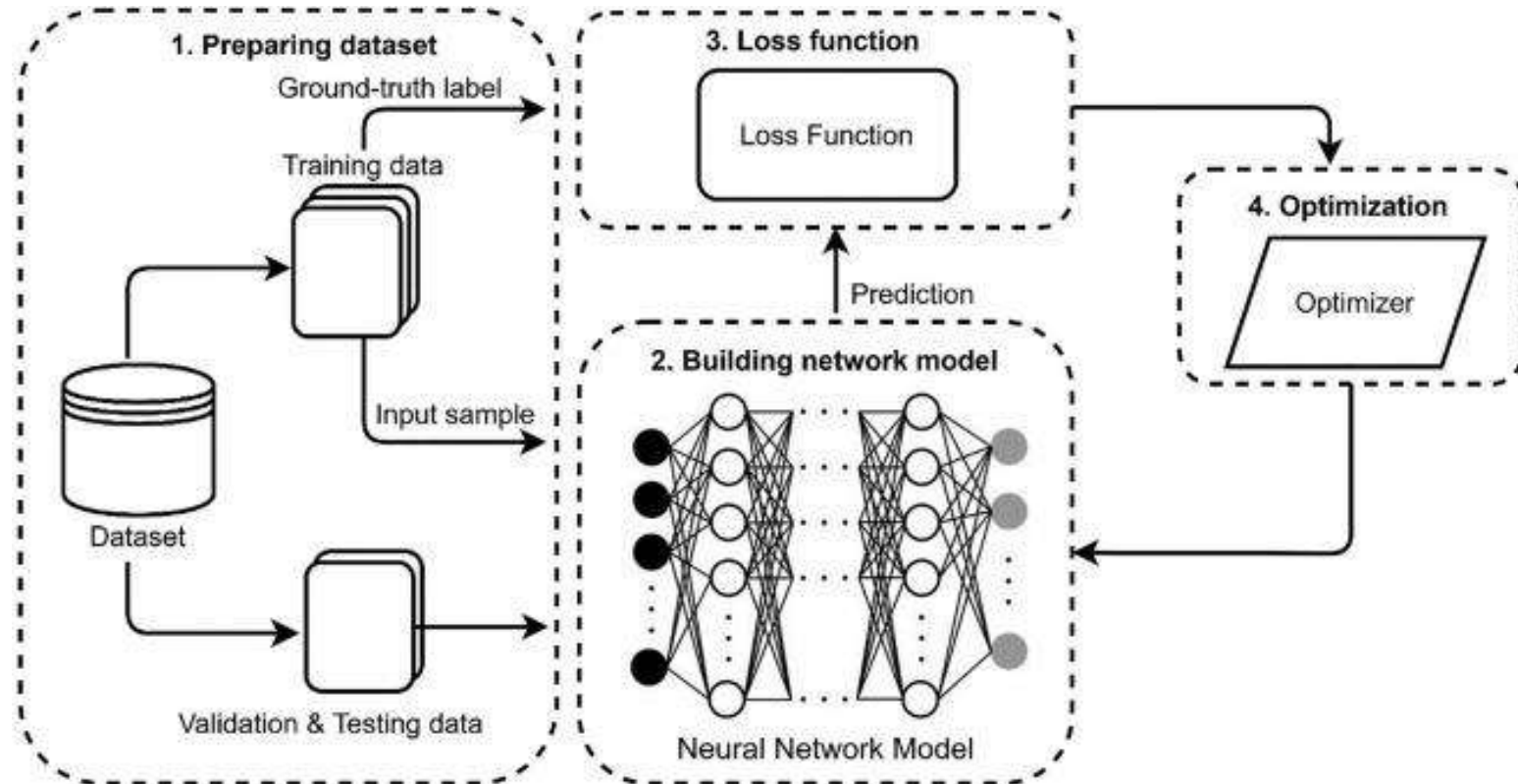
Substituting the values:

$$\Delta w_{ij} = 0.1 \cdot 0.5 \cdot 1 = 0.05$$

So, the change in the synaptic weight is 0.05.

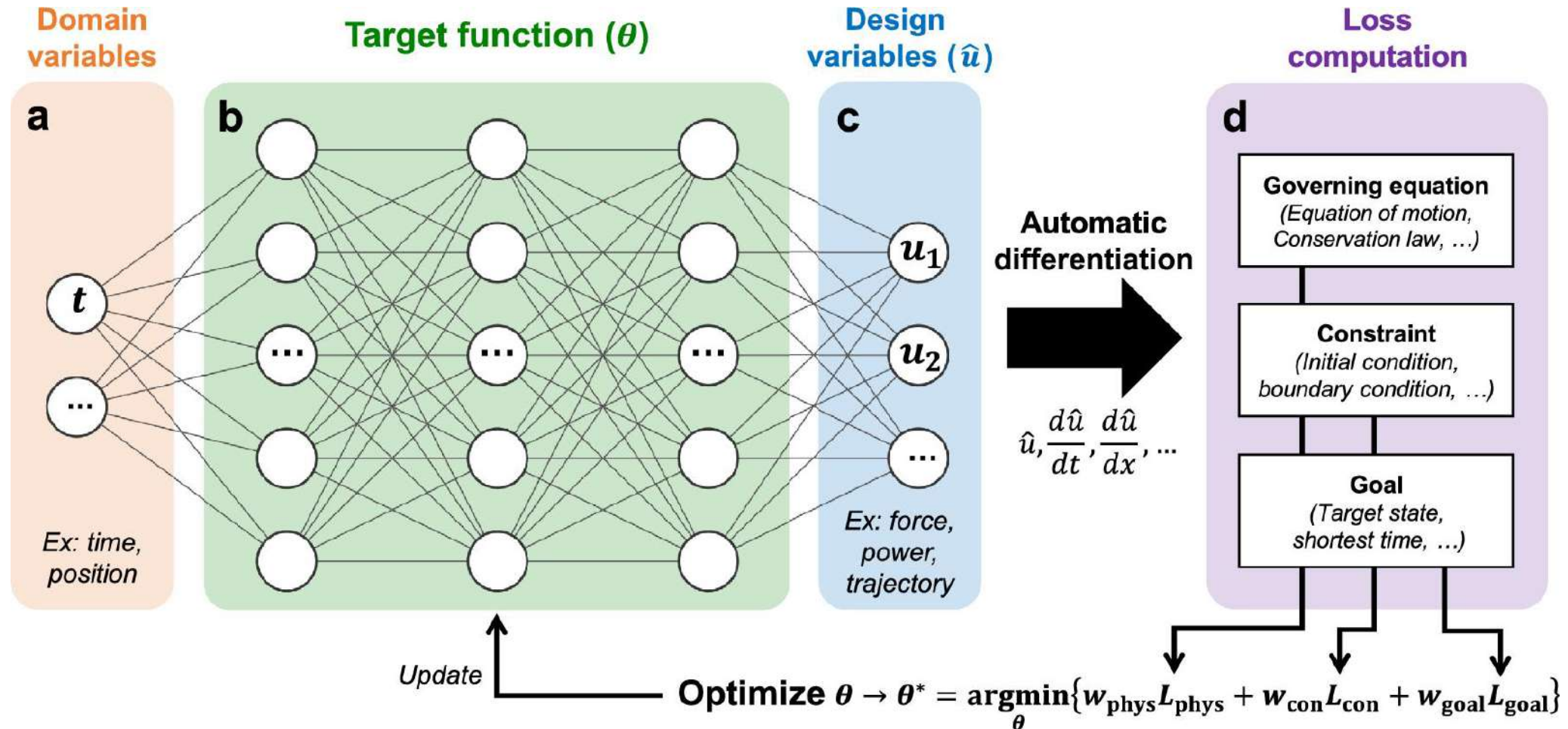
Optimizing Functions

- Optimizers and loss functions are two components that help improve the performance of the model.
- By calculating the difference between the expected and actual outputs of a model, a loss function evaluates the effectiveness of a model.
- Among the loss functions are log loss, hinge loss, and mean square loss.
- By modifying the model's parameters to reduce the loss function value, the optimizer contributes to its improvement. RMSProp, ADAM, and SGD are a few examples of optimizers.



- **The optimizer's job is to determine which combination of the neural network's weights and biases will give it the best chance to generate accurate predictions.**

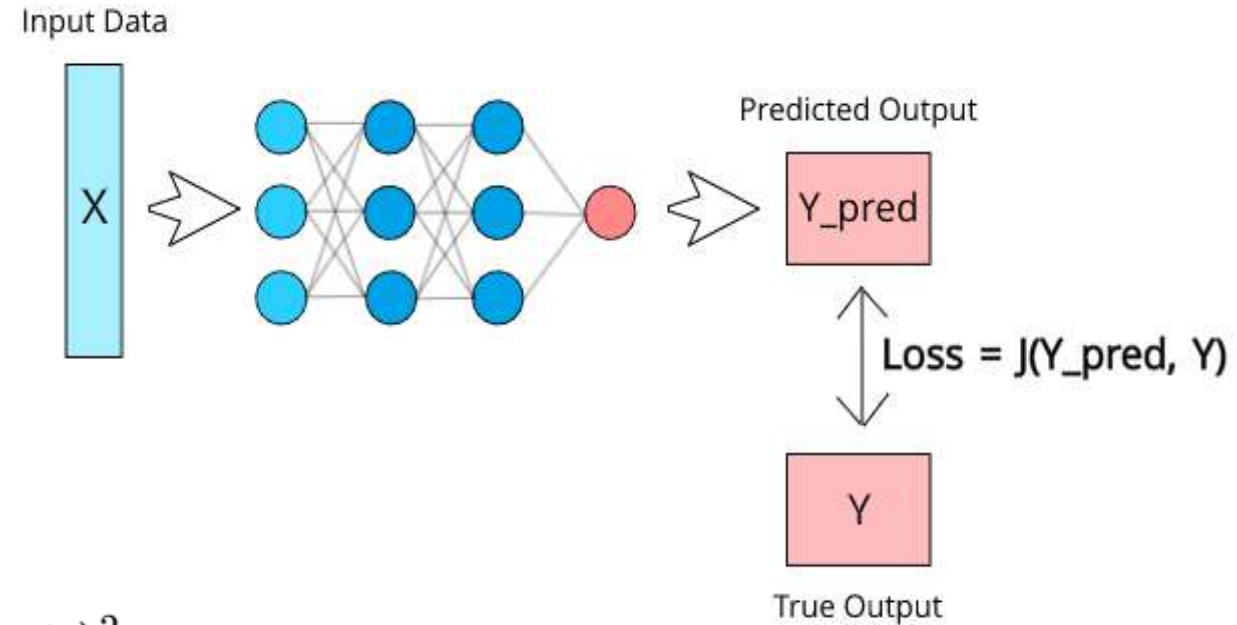
Optimizing Functions



- Gradient Descent:** An iterative optimization algorithm used to minimize the loss function.
- Involves computing the gradient of the loss function with respect to the model parameters (weights), then updating the weights in the opposite direction of the gradient.

Loss Functions(Objective Functions)

- Loss functions measure how well the neural network's predictions align with the actual values. The network aims to minimize the loss during training.



Mean Squared Error (MSE):

$$L = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- Commonly used for regression tasks.
- Measures the average squared difference between the predicted and actual values.

Loss Functions

Cross-Entropy Loss (Log Loss):

$$L = - \sum_{i=1}^n y_i \log(\hat{y}_i)$$

- Used for classification tasks.
- Measures the difference between the true labels and the predicted probabilities, penalizing incorrect predictions more heavily.

Hinge Loss:

$$L = \max(0, 1 - y \cdot \hat{y})$$

- Used primarily in Support Vector Machines (SVMs) but can be used in neural networks for binary classification.

Loss Functions

Huber Loss:

$$L = \sum_{i=1}^n \text{Huber}(y_i - \hat{y}_i)$$

- A combination of MSE and Mean Absolute Error (MAE) that is less sensitive to outliers than MSE.

Summary:

Task	Error type	Loss function	Note
Regression	Mean-squared error	$\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$	Easy to learn but sensitive to outliers (MSE, L2 loss)
	Mean absolute error	$\frac{1}{n} \sum_{i=1}^n y_i - \hat{y}_i $	Robust to outliers but not differentiable (MAE, L1 loss)
Classification	Cross entropy = Log loss	$-\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] =$	Quantify the difference between two probability

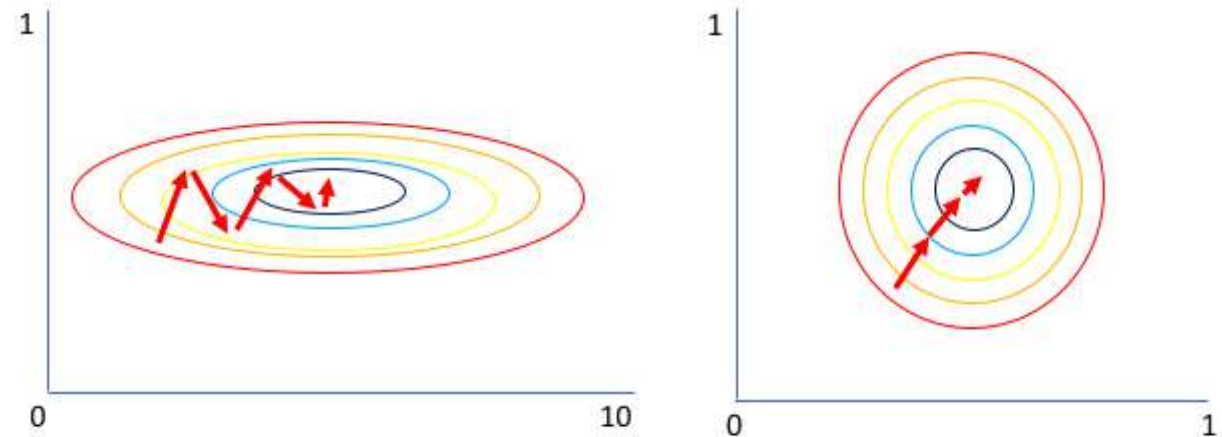
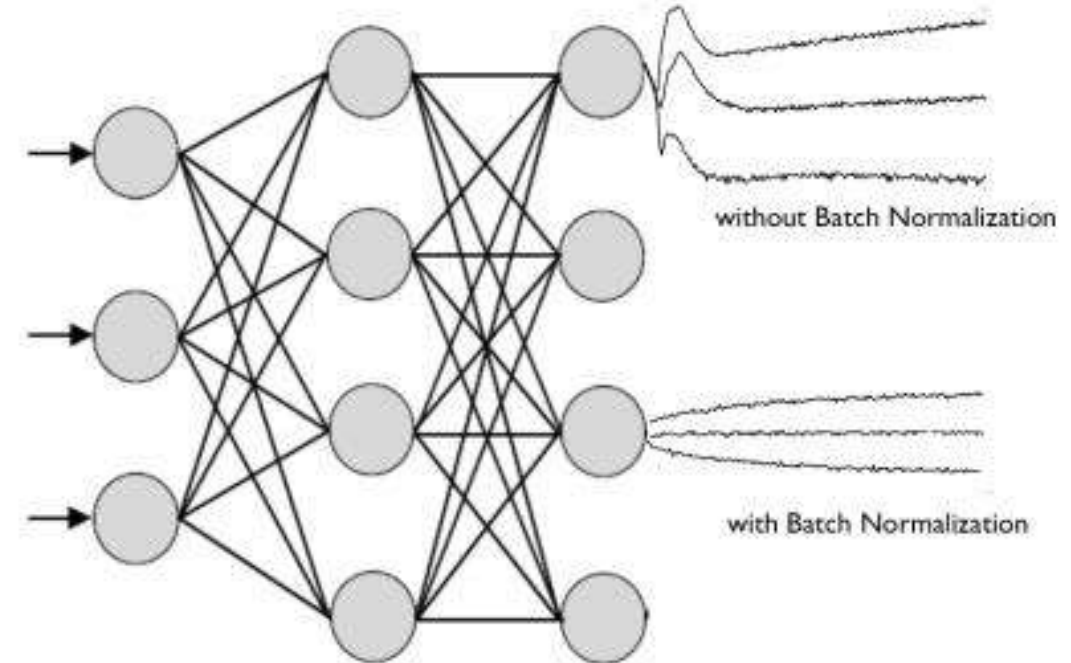
Normalization Functions

Normalization functions adjust data or layer outputs to improve training stability and performance.

•Batch Normalization:

- Normalizes the input to each layer to have a mean of 0 and variance of 1, which helps prevent issues like vanishing/exploding gradients and speeds up convergence.
- This technique is often applied after the activation function in a neural network layer.

•**Layer Normalization:** Similar to batch normalization but normalizes across the features rather than the batch dimension. Used primarily in sequence models like RNNs and Transformers.



Gradient of larger parameter dominates the update

Both parameters can be updated in equal proportions

Regularization Functions

Regularization is a technique used to prevent overfitting by adding a penalty term to the loss function, discouraging the model from assigning too much importance to individual features or coefficients.

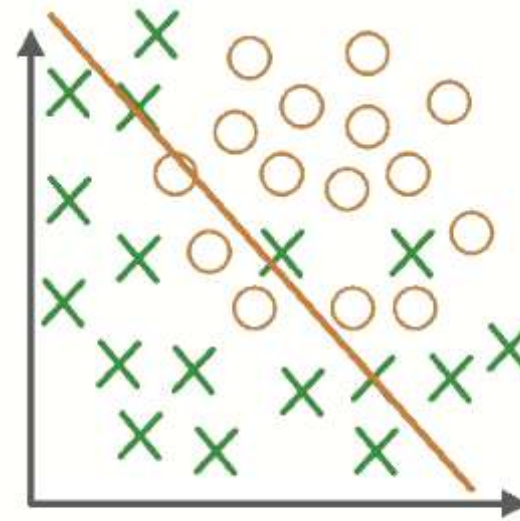
- 1. Complexity Control:** Regularization helps control model complexity by preventing overfitting to training data, resulting in better generalization to new data.
- 2.Preventing Overfitting:** One way to prevent overfitting is to use regularization, which penalizes large coefficients and constrains their magnitudes, thereby preventing a model from becoming overly complex and memorizing the training data instead of learning its underlying patterns.
- 3.Balancing Bias and Variance:** Regularization can help balance the trade-off between model bias (underfitting) and model variance (overfitting) in machine learning, which leads to improved performance.
- 4.Feature Selection:** Some regularization methods, such as L1 regularization (Lasso), promote sparse solutions that drive some feature coefficients to zero. This automatically selects important features while excluding less important ones.
- 5.Handling Multicollinearity:** When features are highly correlated (multicollinearity), regularization can stabilize the model by reducing coefficient sensitivity to small data changes.
- 6.Generalization:** Regularized models learn underlying patterns of data for better generalization to new data, instead of memorizing specific examples.

Regularization Functions

What are Overfitting and Underfitting?

- **Overfitting** is a phenomenon that occurs when a model is constrained to the training set and not able to perform well on unseen data.
- That is when our model learns the noise in the training data as well. This is the case when our model memorizes the training data instead of learning the patterns in it.
- **Underfitting** on the other hand is the case when our model is not able to learn even the basic patterns available in the dataset.
- In the case of the underfitting model is unable to perform well even on the training data hence we cannot expect it to perform well on the validation data.
- This is the case when we are supposed to increase the complexity of the model or add more features to the feature set.

Module-1; Topic-2: Functions in Neural networks

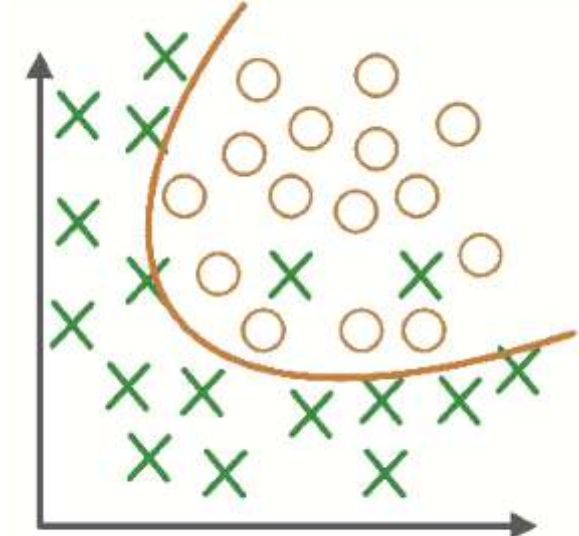


Under-fitting

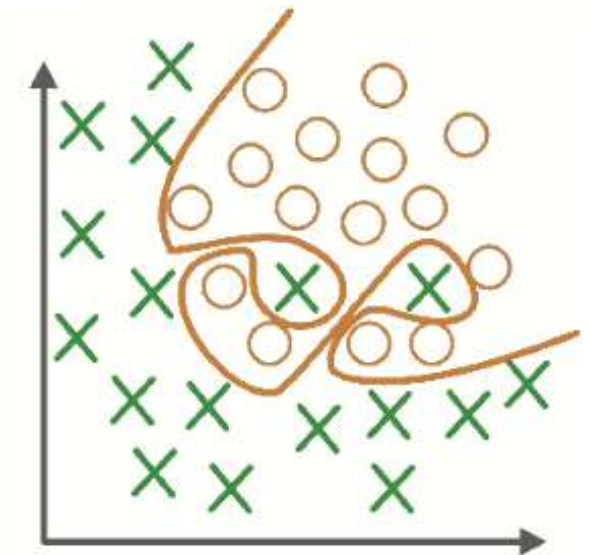
(too simple to explain the variance)

Source:

<https://www.geeksforgeeks.org/regularization-in-machine-learning/>



Appropriate-fitting

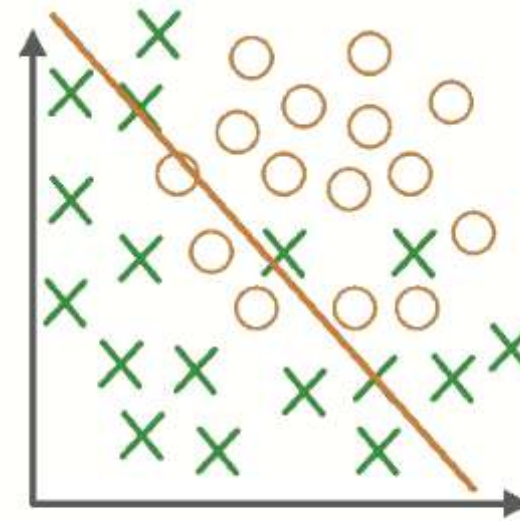


Over-fitting

Regularization Functions

What are Overfitting and Underfitting?

- **Overfitting** is a phenomenon that occurs when a model is constrained to the training set and not able to perform well on unseen data.
- That is when our model learns the noise in the training data as well. This is the case when our model memorizes the training data instead of learning the patterns in it.
- **Underfitting** on the other hand is the case when our model is not able to learn even the basic patterns available in the dataset.
- In the case of the underfitting model is unable to perform well even on the training data hence we cannot expect it to perform well on the validation data.
- This is the case when we are supposed to increase the complexity of the model or add more features to the feature set.

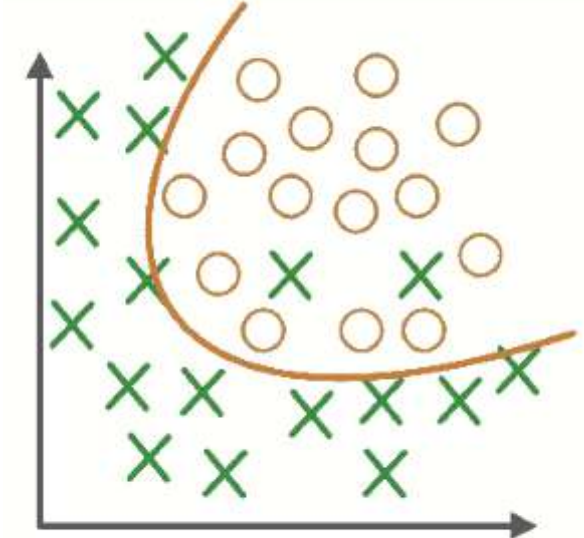


Under-fitting

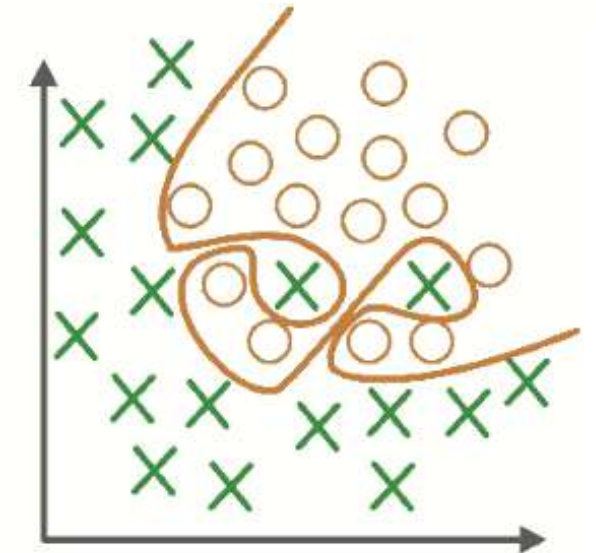
(too simple to explain the variance)

Source:

<https://www.geeksforgeeks.org/regularization-in-machine-learning/>



Appropriate-fitting

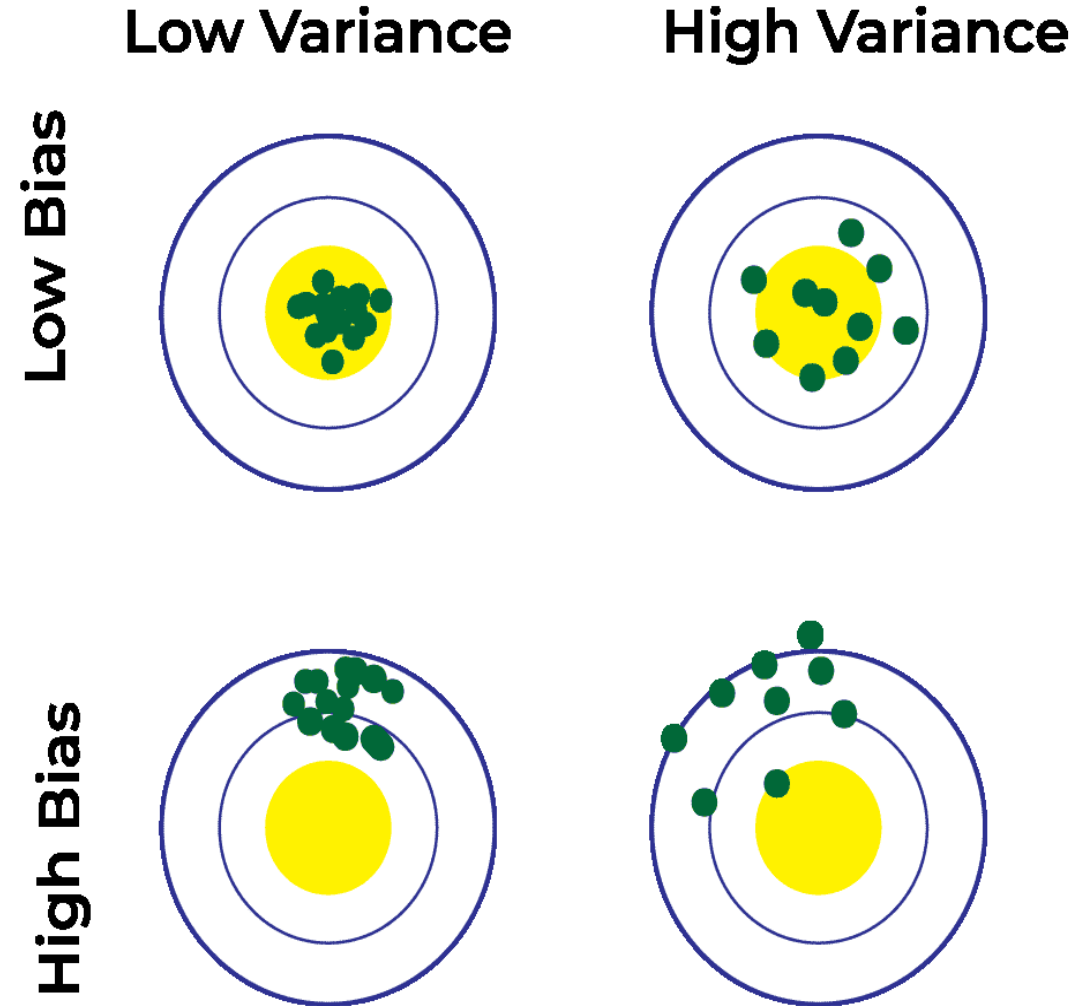


Over-fitting

Regularization Functions

Bias and Variance?

- Bias occurs when a statistical model fails to fit well on real-world data, leading to High Bias, where the model fails to learn patterns and performs poorly.
- Variance refers to the error value in predictions made using new data, while high variance occurs when the model learns from existing data noise.
- High Bias, Low Variance: A model that has high bias and low variance is considered to be underfitting.
- High Variance, Low Bias: A model that has high variance and low bias is considered to be overfitting.
- High-Bias, High-Variance: High bias and variance models struggle to capture underlying patterns and are sensitive to training data changes, resulting in unreliable and inconsistent predictions.
- Low Bias, Low Variance: A model with low bias and low variance can capture data patterns and handle variations in training data.



Source:

<https://www.geeksforgeeks.org/regularization-in-machine-learning/>

Regularization Functions-Benefits

- Regularization improves model generalization by reducing overfitting. Regularized models learn underlying patterns, while overfit models memorize noise in training data.
- Regularization techniques such as L1 (Lasso) regularization simplifies models and improves interpretability by reducing coefficients of less important features to zero.
- Regularization improves model performance by preventing excessive weighting of outliers or irrelevant features.
- Regularization makes models stable across different subsets of the data. It reduces the sensitivity of model outputs to minor changes in the training set.
- Regularization prevents models from becoming overly complex, which is especially important when dealing with limited data or noisy environments.
- Regularization can help handle multicollinearity (high correlation between features) by reducing the magnitudes of correlated coefficients.
- Regularization introduces hyperparameters (e.g., α or λ) that control the strength of regularization. This allows fine-tuning models to achieve the right balance between bias and variance.
- Regularization promotes consistent model performance across different datasets. It reduces the risk of dramatic performance changes when encountering new data.

Regularization Functions

Lasso Regression

A regression model which uses the **L1 Regularization** technique is called **LASSO(Least Absolute Shrinkage and Selection Operator)** regression. **Lasso Regression** adds the “*absolute value of magnitude*” of the coefficient as a penalty term to the loss function(L). Lasso regression also helps us achieve feature selection by penalizing the weights to approximately equal to zero if that feature does not serve any purpose in the model.

$$\text{Cost} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{i=1}^m |w_i|$$

where,

- m – Number of Features
- n – Number of Examples
- y_i – Actual Target Value
- $y_i(\text{hat})$ – Predicted Target Value

Regularization Functions

Ridge Regression

A regression model that uses the **L2 regularization** technique is called **Ridge regression**. Ridge regression adds the “*squared magnitude*” of the coefficient as a penalty term to the loss function(L).

$$\text{Cost} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{i=1}^m w_i^2$$

Elastic Net Regression

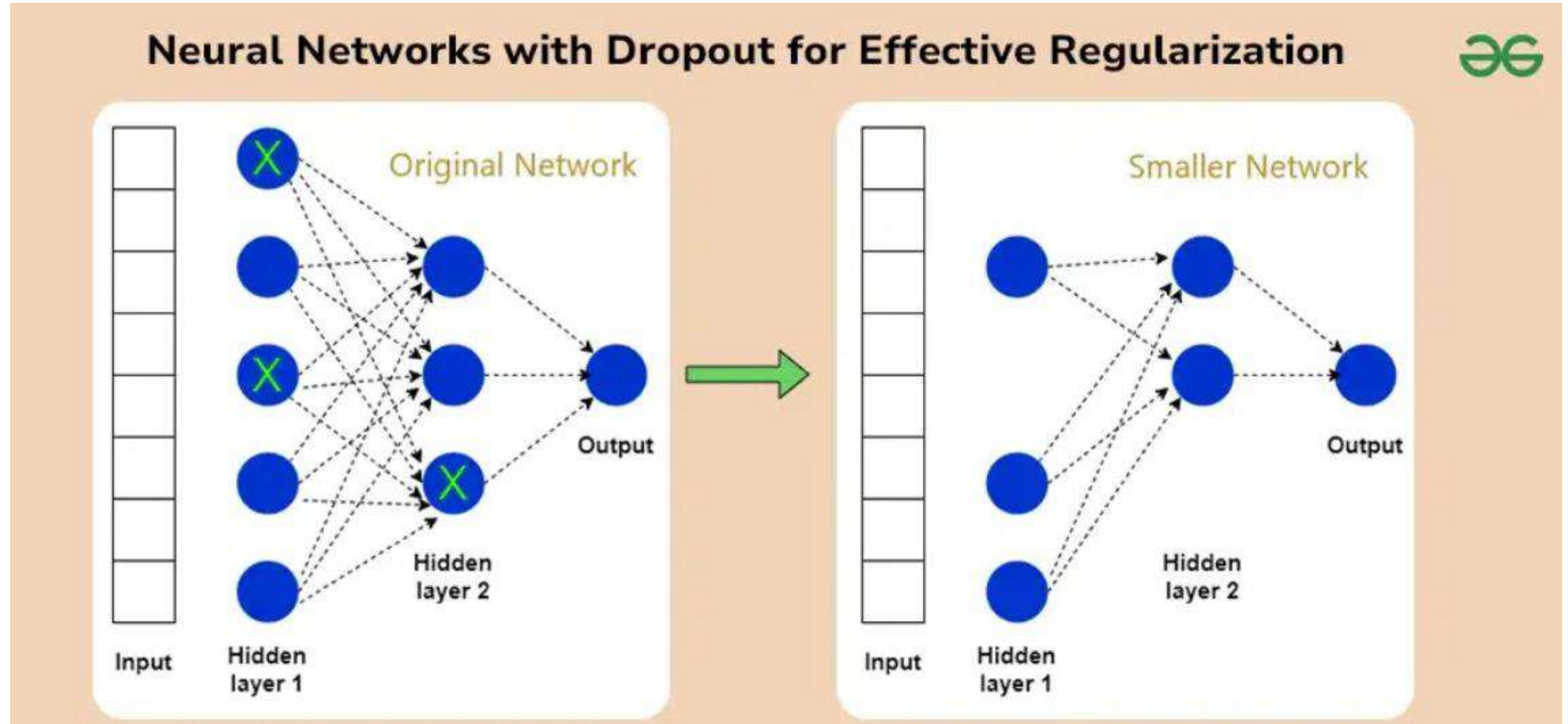
This model is a combination of L1 as well as L2 regularization. That implies that we add the absolute norm of the weights as well as the squared measure of the weights. With the help of an extra hyperparameter that controls the ratio of the L1 and L2 regularization.

$$\text{Cost} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \left((1 - \alpha) \sum_{i=1}^m |w_i| + \alpha \sum_{i=1}^m w_i^2 \right)$$

Regularization Functions

Dropout:

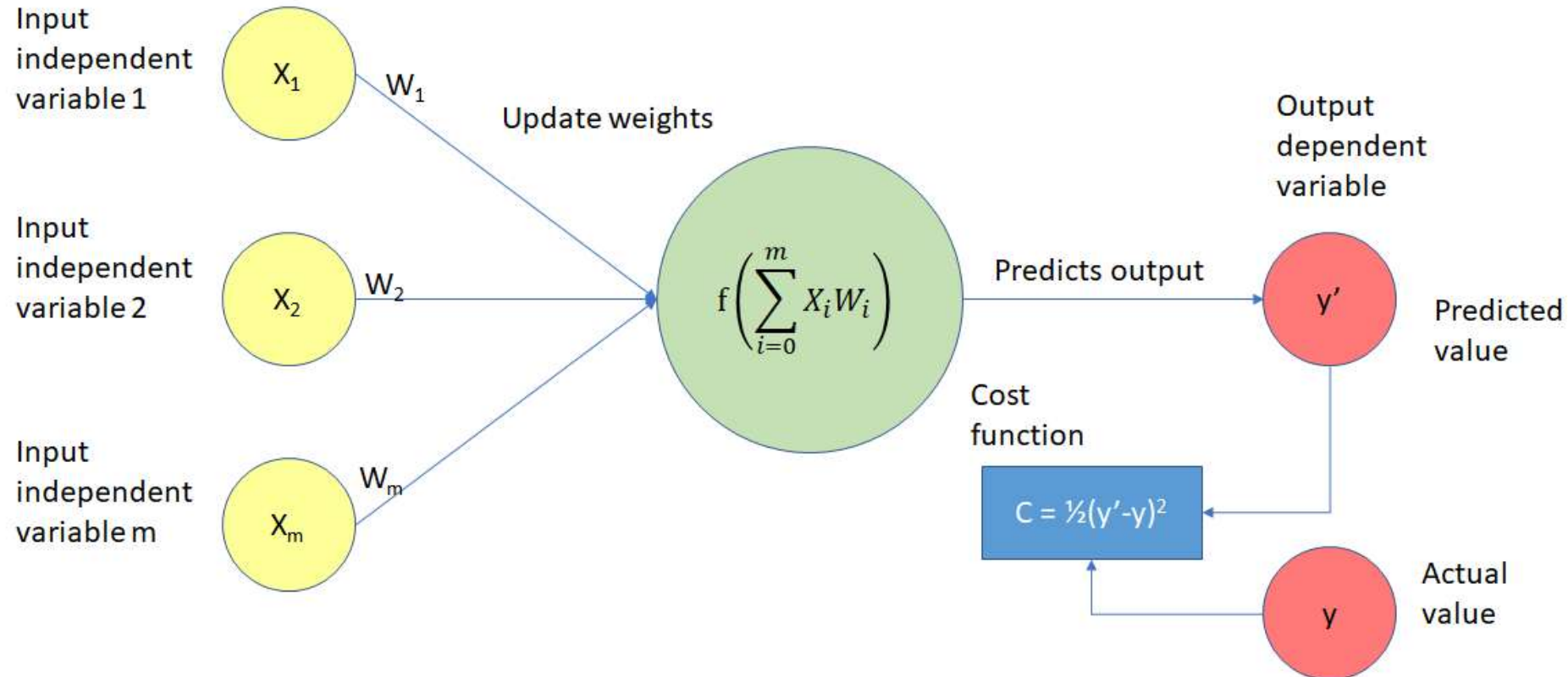
- A technique where random units (neurons) are "dropped" during training, meaning their output is temporarily set to zero. This helps prevent overfitting by reducing the dependency on particular neurons.



Cost Functions

In neural networks, the **cost function** is typically synonymous with the **loss function**, though sometimes "cost" can refer to an aggregate loss over an entire dataset. It quantifies the performance of the network during training.

Cost Function: Often, the cost function is the mean of the loss function values across all data points (e.g., Mean Squared Error for regression or Cross-Entropy Loss for classification).

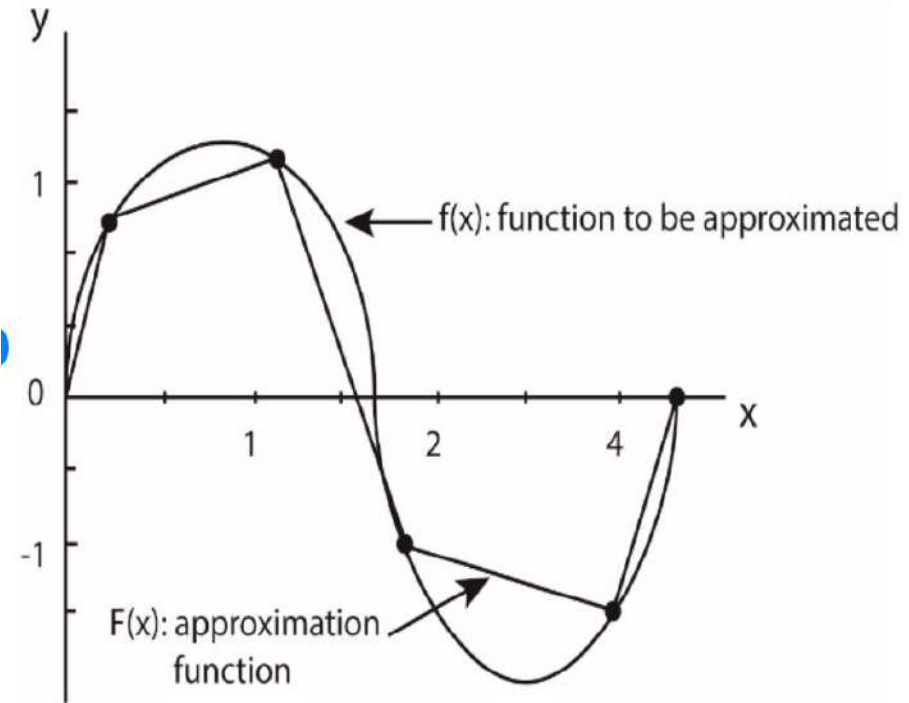


Function Approximation

- Function Approximation in Reinforcement Learning is a crucial concept in reinforcement learning, enabling algorithms to generalize from limited experience to broader states and actions, especially in complex environments.
- Reinforcement learning aims to maximize cumulative reward over time by estimating value functions or approximating policies.
- Function approximation compactly represents state or action spaces, addressing practical problems with large spaces.

Some key points highlighting its significance:

- **Handling Complexity:** In many real-world problems, the state and action spaces are too vast to enumerate or store explicitly. Function approximation allows RL algorithms to represent value functions or policies compactly using parameterized functions.
- **Generalization:** Function approximation enables RL agents to generalize from limited experience to unseen states and actions. This is crucial for robust performance in environments where exhaustive exploration is impractical.



Efficiency: By approximating value functions or policies, RL algorithms can operate efficiently even in high-dimensional spaces. This efficiency is essential for scaling RL to complex tasks such as robotic control or game playing.

Function Approximation

Module-1; Topic-2: Functions in Neural networks

Types of Function Approximation in Reinforcement learning:

1. Linear Function Approximation:

Linear function approximators use a linear combination of features to represent value functions or policies. If $\phi(s)$ represents the feature vector for state s , the value function $V(s)$ can be approximated as:

$$V(s) \approx \theta^T \phi(s)$$

where θ is a vector of weights. Linear approximators are simple and efficient but may lack the capacity to represent complex functions.

2. Non-linear Function Approximation

Non-linear methods, particularly those based on neural networks, have gained prominence due to their ability to capture complex patterns. [Deep Q-Networks \(DQNs\)](#) and [Actor-Critic methods](#) are prime examples. A [neural network](#) with weights θ approximates the value function or policy as:

$$V(s) \approx f(s; \theta)$$

These methods, while powerful, require careful tuning and substantial computational resources.

Function Approximation

Module-1; Topic-2: Functions in Neural networks

3. Basis Function Methods

Basis functions transform the state space into a higher-dimensional space where the function approximation becomes easier. Examples include Radial Basis Functions (RBF) and tile coding.

4. Kernel Methods

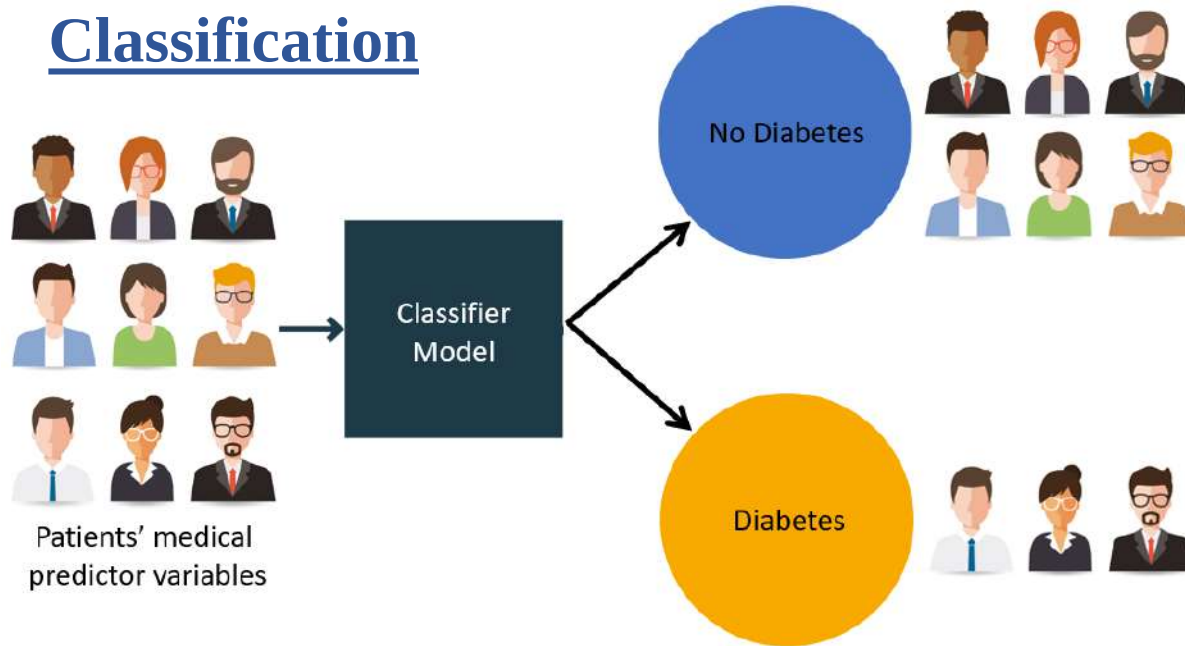
Kernel-based methods, such as Gaussian processes, provide a non-parametric approach to function approximation, offering flexibility and robustness at the cost of computational scalability.

Key Concepts in Function Approximation for Reinforcement Learning

1. **Features:** These are characteristics extracted from the agent's state that represent relevant information for making decisions. Choosing informative features is crucial for accurate value estimation.
2. **Learning Algorithm:** This algorithm updates the parameters of the chosen function to minimize the difference between the estimated value and the actual value experienced by the agent (temporal-difference learning). Common algorithms include linear regression, gradient descent variants, or policy gradient methods depending on the function class.
3. **Function Class:** This refers to the type of function used for approximation. Common choices include linear functions, neural networks, decision trees, or a combination of these. The complexity of the function class should be balanced with the available data and computational resources.

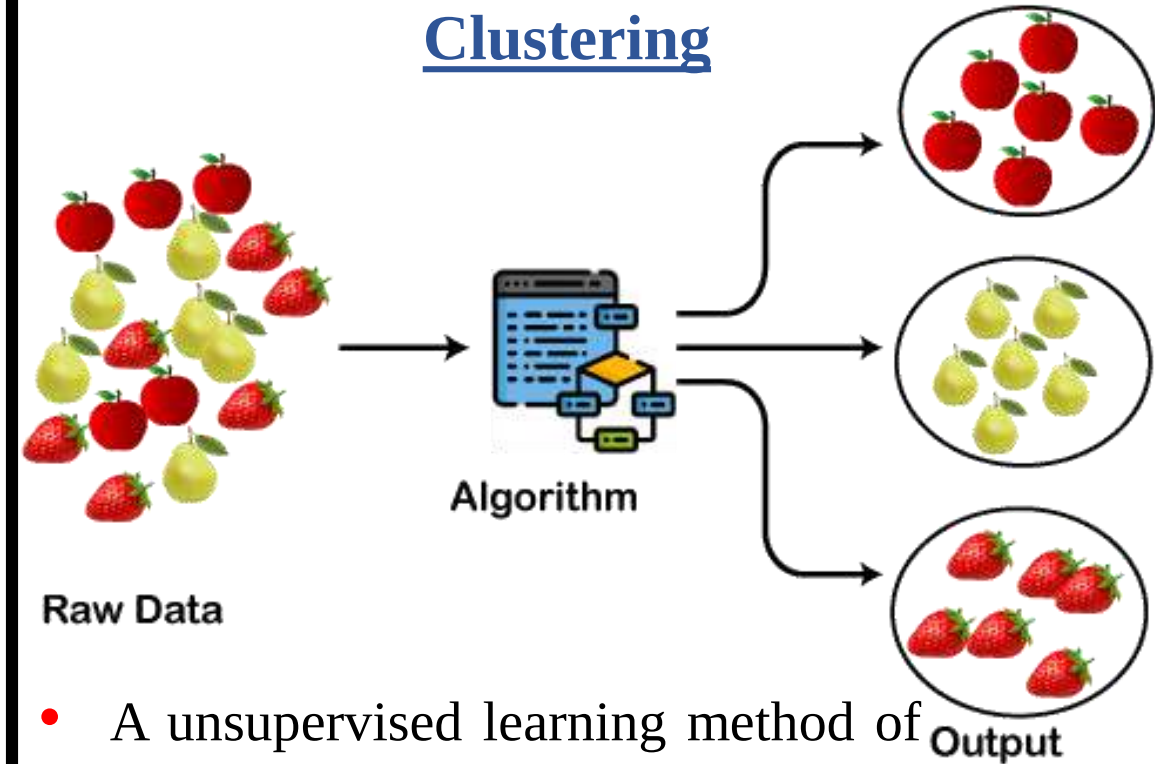
Classification and Clustering problems

Classification



- The Classification algorithm is a **Supervised Learning technique** that is used to **identify the category of new observations on the basis of training data**.
- In Classification, a program learns from the given dataset or observations and then classifies new observation into a number of classes or groups- Segregate vast quantities of data into discrete values, i.e. :distinct, like 0/1, True/False, or a pre-defined output label class.

Clustering



- A unsupervised learning method of identifying similar groups of data in a dataset is called clustering.
- The objects with the possible similarities remain in a group that has less or no similarities with another group

Classification and Clustering problems

Classification Vs. Clustering

Module-1; Topic-3: Classification and Clustering problems

Aspect	Classification with Neural Networks	Clustering with Neural Networks
Supervision	Supervised (requires labeled data)	Unsupervised (no labeled data required)
Output	Predicts a class label for each input	Groups data points into clusters based on similarity
Architecture	Feedforward NN, CNN, RNN, Transformers	Autoencoders, Self-Organizing Maps, Variational Autoencoders, GANs
Loss Function	Cross-Entropy Loss, Binary Cross-Entropy	Contrastive Loss, Triplet Loss, Reconstruction Loss
Example Applications	Image classification, speech recognition, sentiment analysis	Customer segmentation, document clustering, anomaly detection
Evaluation Metrics	Accuracy, Precision, Recall, F1-Score, Confusion Matrix	Silhouette Score, Davies-Bouldin Index, Inertia, ARI
Activation Function	Softmax (multi-class), Sigmoid (binary)	Typically no output activation; clustering performed in latent space

Classification problems

Module-1; Topic-3: Classification and Clustering problems

1. **Overfitting:** Neural networks have a high capacity to fit complex models, and with insufficient data or excessive training, they can easily memorize the training data and fail to generalize to new data.

- **Solution:** Regularization techniques like dropout, L2 regularization, early stopping, or data augmentation can help prevent overfitting.
- **Underfitting:** If the model is too simple or has insufficient capacity (too few layers/neurons), it may fail to learn important patterns in the data.
- **Solution:** Increase the model complexity by adding more layers, neurons, or using more advanced architectures.

2. Class Imbalance:

- In many real-world classification problems, certain classes are underrepresented (e.g., rare diseases, fraud detection).
Problem: The model might become biased toward the majority class, leading to poor performance on the minority class.
- **Solution:** Use weighted loss functions to penalize misclassifications of minority classes more heavily.
- Apply oversampling or undersampling techniques to balance the dataset.
- Use data augmentation for the minority class.
- Synthetic data generation techniques (e.g., SMOTE).

3. **Data Quality and Preprocessing Problem:** Neural networks are sensitive to noisy, missing, or poorly processed data.

- **Solution:** Clean and preprocess data (e.g., normalization, standardization, handling missing values). Use robust methods for dealing with noisy or incomplete data (e.g., data imputation, noise filtering).

Classification problems

4. Hyperparameter Tuning: Neural networks have many hyperparameters that need to be tuned, such as the learning rate, batch size, number of layers, number of neurons per layer, activation functions, and dropout rates.

- **Problem:** Finding the optimal set of hyperparameters can be time-consuming and computationally expensive.
- **Solution:** Use Grid Search or Random Search for hyperparameter optimization.
- **Bayesian Optimization** can help automate the search for the best hyperparameters.

5. Training Time and Computational Cost: Deep neural networks, particularly with large datasets, require significant computational resources, leading to long training times.

- **Problem:** Large models and datasets can be computationally expensive to train, requiring specialized hardware (GPUs, TPUs).
- **Solution:** Use transfer learning or pretrained models to reduce training time. Optimize model architectures (e.g., pruning, quantization). Use more efficient algorithms and parallel processing techniques.

6. Vanishing/Exploding Gradients: During backpropagation, the gradients of certain layers can either become too small (vanishing) or too large (exploding), leading to issues in training.

Problem: Vanishing gradients make it difficult to train deep networks, especially with activation functions like **sigmoid** or **tanh**.

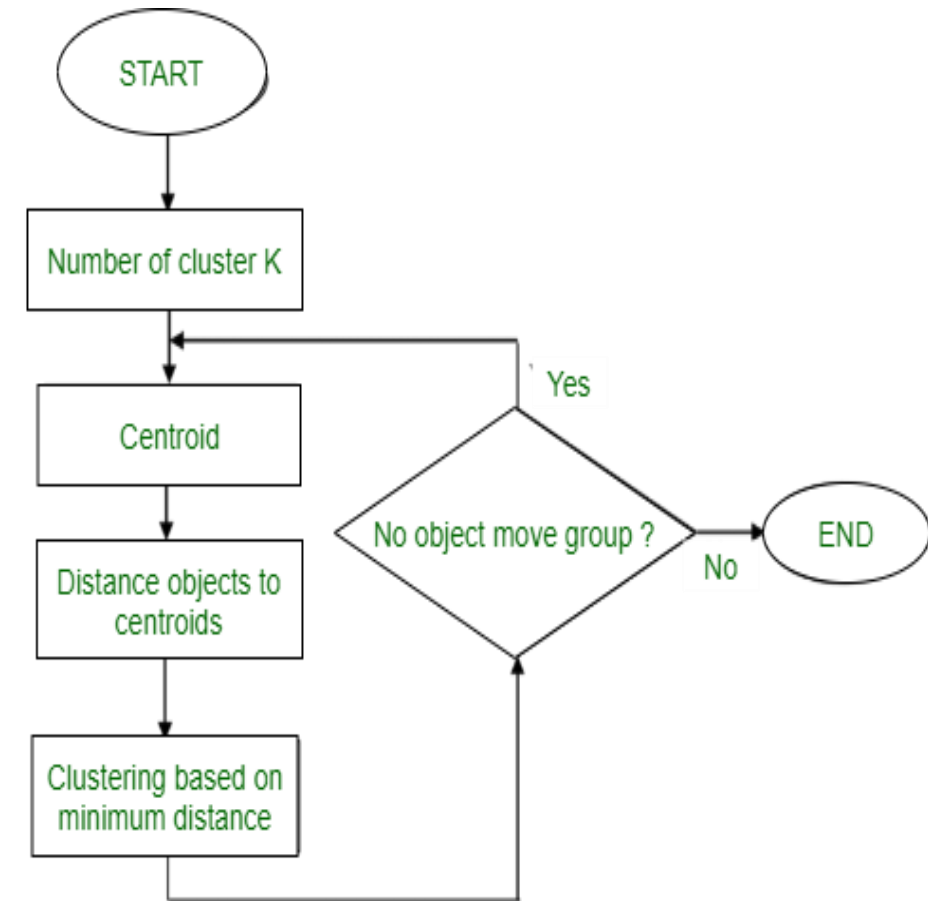
Solution: Use activation functions like **ReLU** or **Leaky ReLU** that help mitigate this issue.

Gradient clipping can help manage exploding gradients.

Batch normalization helps stabilize training by normalizing layer inputs.

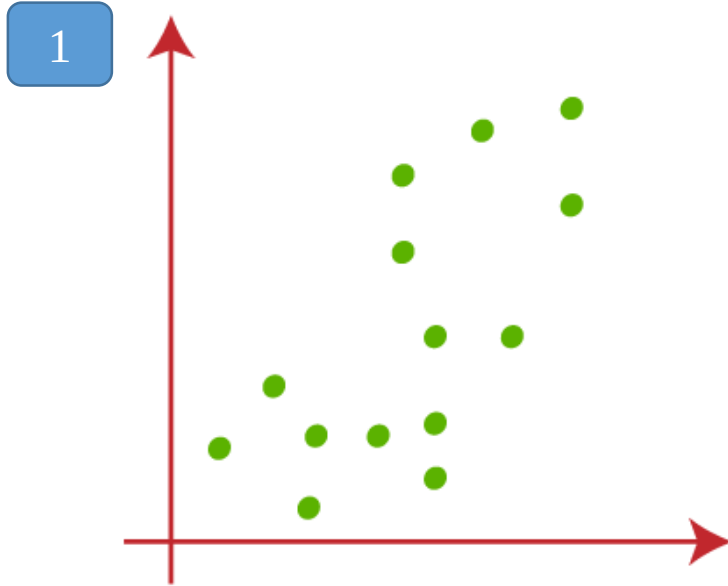
K-Means Clustering Analysis

- First, we need to choose k , the number of clusters that we want to find.
- Then, the algorithm will select randomly the centroids of each cluster.
- It will be assigned each datapoint to the closest centroid (using euclidean distance).
- It will be computed the cluster inertia.
- The new centroids will be calculated as the mean of the points that belong to the centroid of the previous step. In other words, by calculating the minimum quadratic error of the datapoints to the center of each cluster, moving the center towards that point
- Back to step 3.

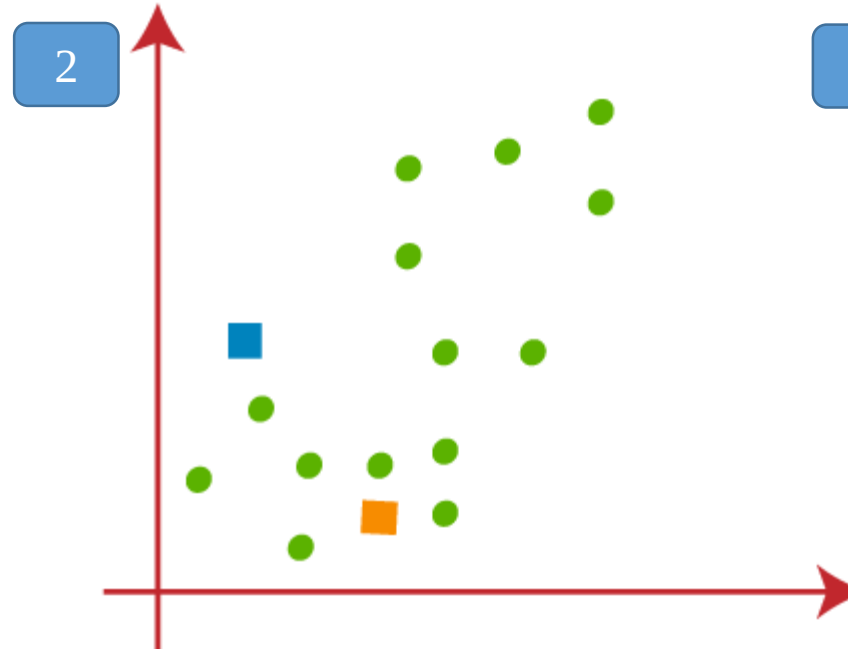


K-Means Clustering Analysis

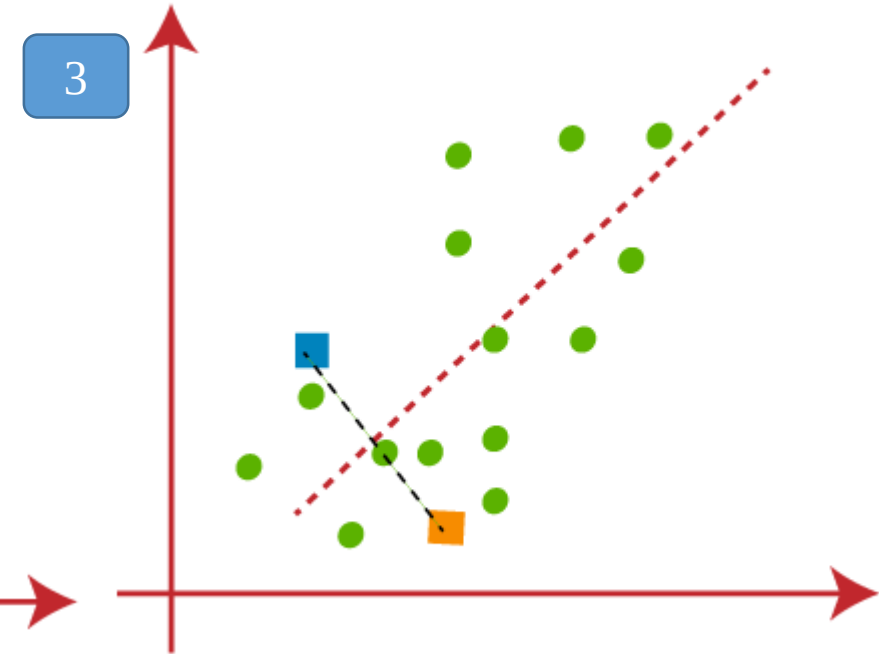
Module-1; Topic-3: Classification and Clustering problems



Choose $K=2$, to identify the dataset and to put them into different clusters. It means to group these datasets into two different clusters.



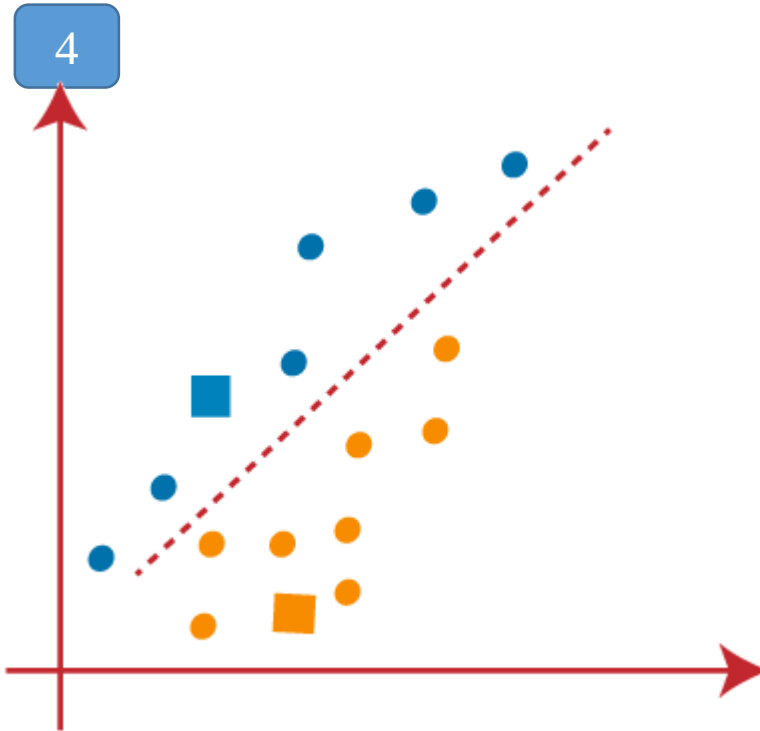
Choose some random k points or centroid to form the cluster. These points can be either the points from the dataset or any other point. So, here we are selecting the below two points as k points, which are not the part of our dataset.



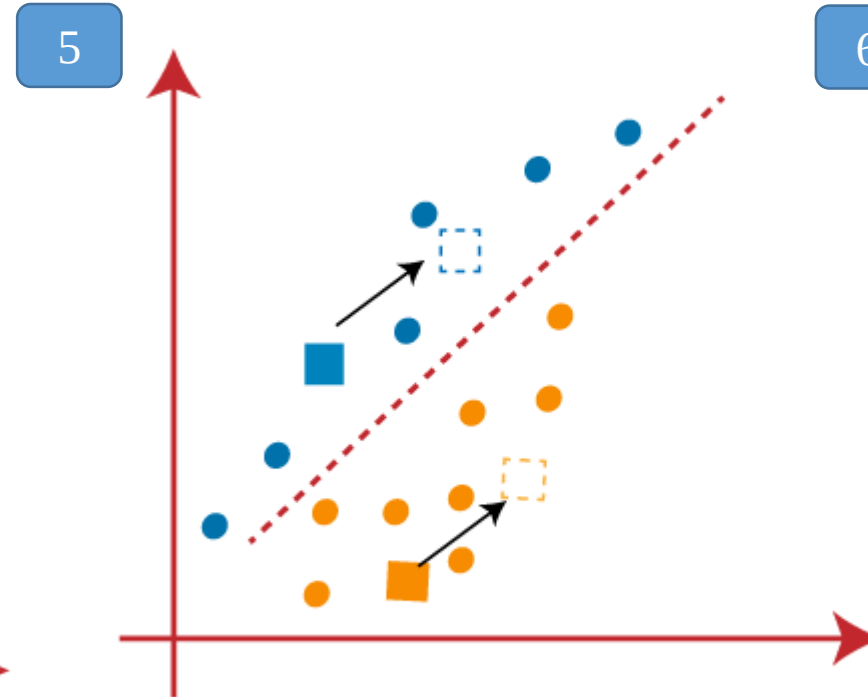
Assign each data point of the scatter plot to its closest K -point or centroid. Calculate the distance between two points. So, we will draw a median between both the centroids.

K-Means Clustering Analysis

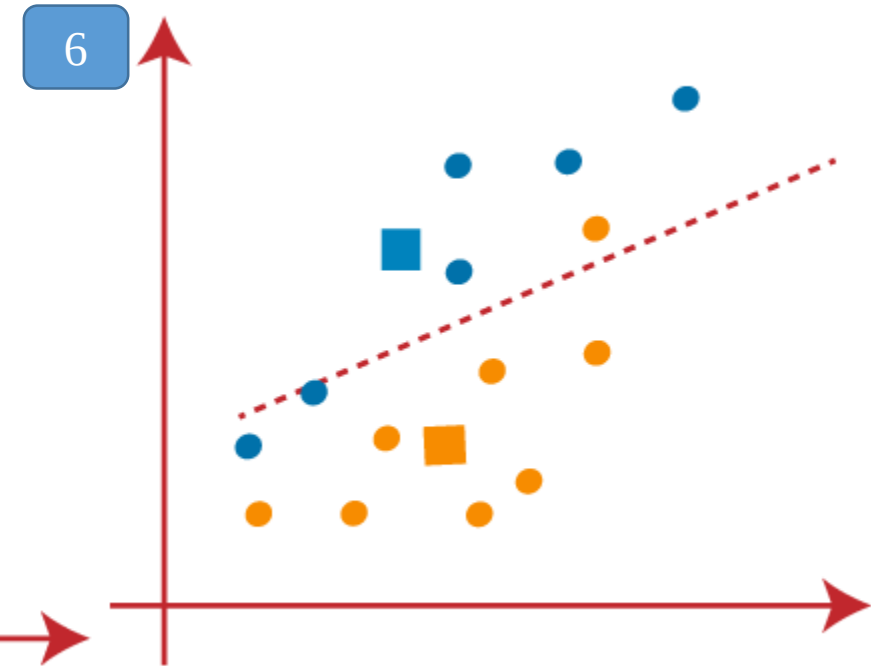
Module-1; Topic-3: Classification and Clustering problems



Points left side of the line is near to the K1 or blue centroid, and points to the right of the line are close to the yellow centroid



To find the closest cluster, Repeat the process by choosing a new centroid. To choose the new centroids, we will compute the center of gravity of these centroids.

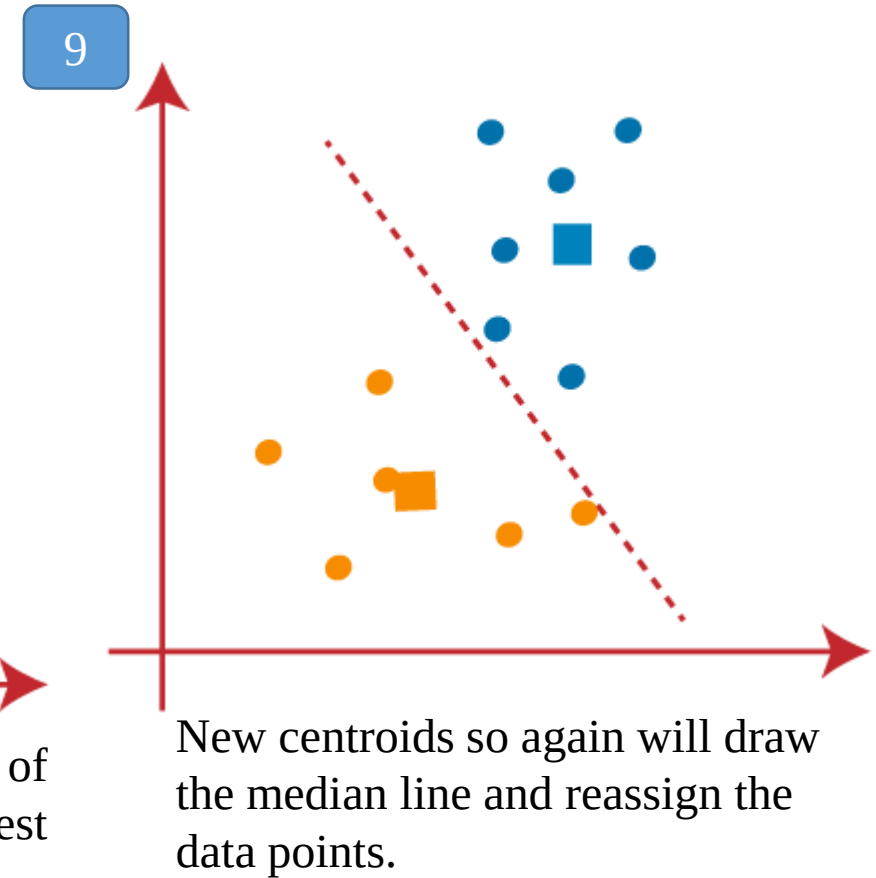
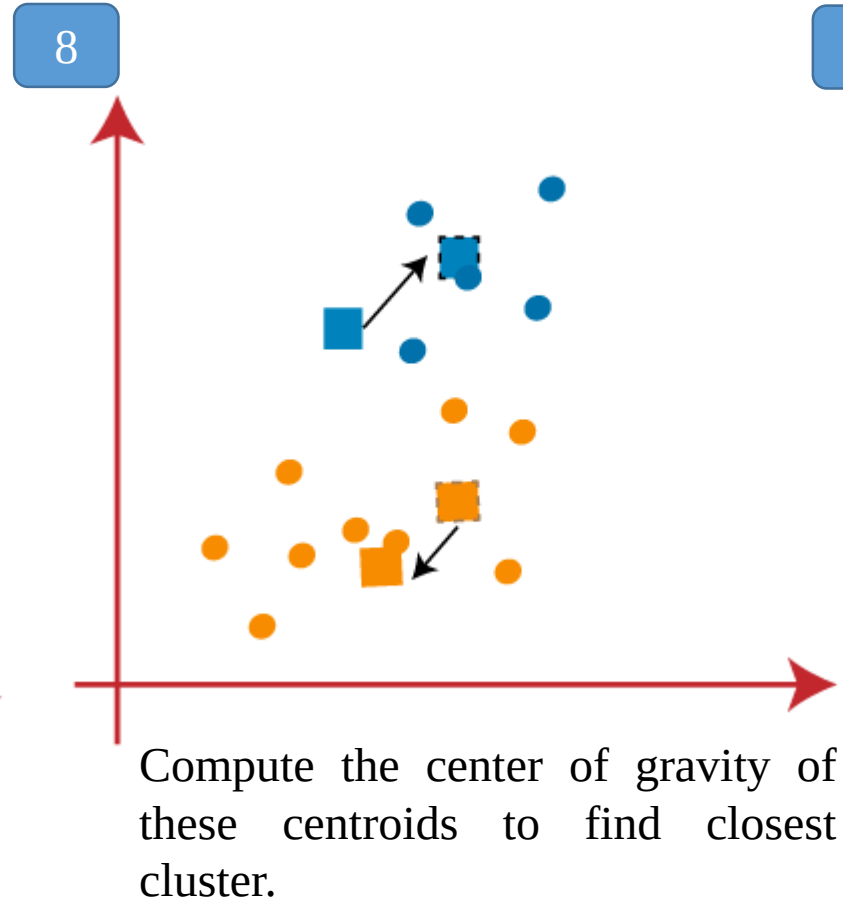
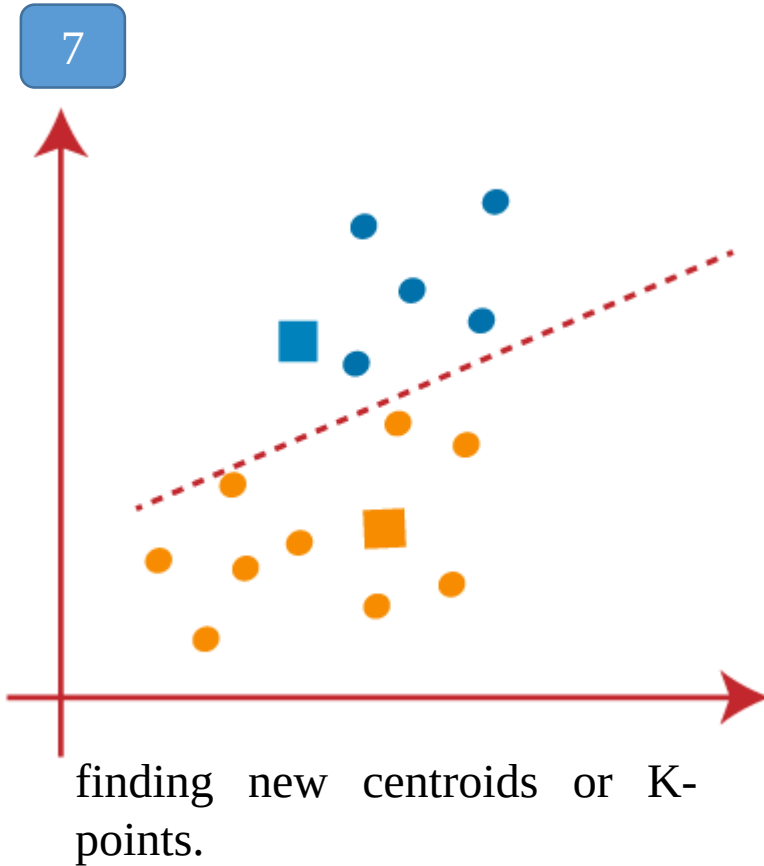


Reassign each datapoint to the new centroid.

- One yellow point is on the left side of the line, and two blue points are right to the line. So, these three points will be assigned to new centroids.

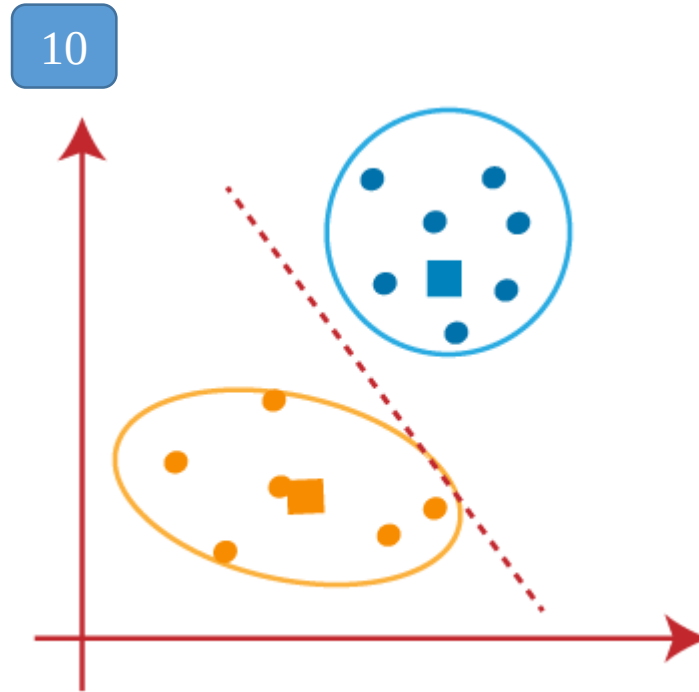
K-Means Clustering Analysis

Module-1; Topic-3: Classification and Clustering problems

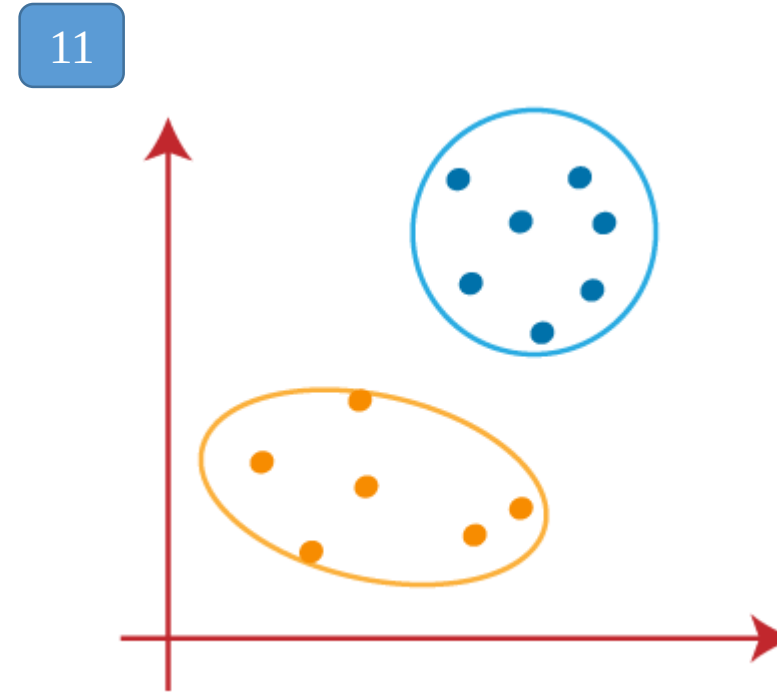


K-Means Clustering Analysis

Module-1; Topic-3: Classification and Clustering problems



There are no dissimilar data points on either side of the line, which means our model is formed.



Remove the assumed centroids, and the two final clusters will be as shown in the below image:

Kmeans Solved Example

Module-1; Topic-3: Classification and Clustering problems

Cluster the following eight points (with (x, y) representing locations) into three clusters:

A1(2, 10), A2(2, 5), A3(8, 4), A4(5, 8), A5(7, 5), A6(6, 4), A7(1, 2), A8(4, 9).

Initial cluster centers are: A1(2, 10), A4(5, 8) and A7(1, 2).

Iteration-01: Distance of each point from each of the center of the three clusters.

The distance function between two points a = (x1, y1) and b = (x2, y2) using **Manhattan Distance** is defined as-

$$P(a, b) = |x2 - x1| + |y2 - y1|$$

For Cluster-01: We have only one point A1(2, 10) in Cluster-01. So, cluster center remains the same.

For Cluster-02: Center of Cluster-02
= $((8 + 5 + 7 + 6 + 4)/5, (4 + 8 + 5 + 4 + 9)/5)$
= (6, 6)

For Cluster-03: Center of Cluster-03
= $((2 + 1)/2, (5 + 2)/2)$
= (1.5, 3.5)

C1={1}, C2={3,4,5,6,8}, C3={2,7}

Given Points	Distance from center (2, 10) of Cluster-01	Distance from center (5, 8) of Cluster-02	Distance from center (1, 2) of Cluster-03	Point belongs to Cluster
A1(2, 10)	0	5	9	C1
A2(2, 5)	5	6	4	C3
A3(8, 4)	12	7	9	C2
A4(5, 8)	5	0	10	C2
A5(7, 5)	10	5	9	C2
A6(6, 4)	10	5	7	C2
A7(1, 2)	9	10	0	C3
A8(4, 9)	3	2	10	C2

Kmeans Solved Example

Module-1; Topic-3: Classification and Clustering problems

Iteration-02: Distance of each point from each of the center of the three clusters using the given distance function.

Re-compute the new cluster clusters by the new cluster center is computed by taking mean of all the points contained in that cluster.

For Cluster-01: Center of Cluster-01

$$= ((2 + 4)/2, (10 + 9)/2)$$

$$= (3, 9.5)$$

For Cluster-02: Center of Cluster-02

$$= ((8 + 5 + 7 + 6)/4, (4 + 8 + 5 + 4)/4)$$

$$= (6.5, 5.25)$$

For Cluster-03: Center of Cluster-03

$$= ((2 + 1)/2, (5 + 2)/2)$$

$$= (1.5, 3.5)$$

C1={1,8}, C2={3,4,5,6}, C3={2,7}

Given Points	Distance from center (2, 10) of Cluster-01	Distance from center (6, 6) of Cluster-02	Distance from center (1.5, 3.5) of Cluster-03	Point belongs to Cluster
A1(2, 10)	0	8	7	C1
A2(2, 5)	5	5	2	C3
A3(8, 4)	12	4	7	C2
A4(5, 8)	5	3	8	C2
A5(7, 5)	10	2	7	C2
A6(6, 4)	10	2	5	C2
A7(1, 2)	9	9	2	C3
A8(4, 9)	3	5	8	C1

Kmeans Solved Example#2

Module-1; Topic-3: Classification and Clustering problems

Divide the given sample data into two clusters using Euclidean Distance

Height	Weight
185	72
170	56
168	60
179	68
182	72
188	77
180	71
180	70
183	84
180	88
180	67
177	76

$$\text{Euclidean Distance : (ED)} = \sqrt{(X_0 - X_c)^2 + (Y_0 - Y_c)^2}$$

Iteration-01:

For Cluster-01: We have only one point A1(185, 72) in Cluster-01. So, cluster center remains the same.

For Cluster-02:
Center of Cluster-02
= $((170 + 168)/2, (56 + 60)/2)$
= (169, 58)

Given Points	Distance from Center of (185,72) Cluster-1	Distance from Center of (170, 56) Cluster-2	Point Cluster belongs to
A1(185,72)	0	21.93	C1
A2(170,56)	21.93	0	C2
A3(168,60)	20.80	4.47	C2
A4(179,68)	7.21	15	C1
A5(182,72)	3	20	C1
A6(188,77)	5.83	27.65	C1
A7(180,71)	5.09	18.02	C1
A8(180,70)	5.38	17.20	C1
A9(183,84)	12.16	30.87	C1
A10(180,88)	16.76	33.52	C1
A11(180,67)	7.07	14.86	C1
A12(177,76)	8.94	21.18	C1

C1={1,4,5,6,7,8,9,10,11,12}, C2={2,3}

Kmeans Solved Example#2

Module-1; Topic-3: Classification and Clustering problems

Divide the given sample data into two clusters using Euclidean Distance

Height	Weight
185	72
170	56
168	60
179	68
182	72
188	77
180	71
180	70
183	84
180	88
180	67
177	76

$$\text{Euclidean Distance : } \sqrt{(X_0 - X_c)^2 + (Y_0 - Y_c)^2}$$

(ED)

Iteration-02:

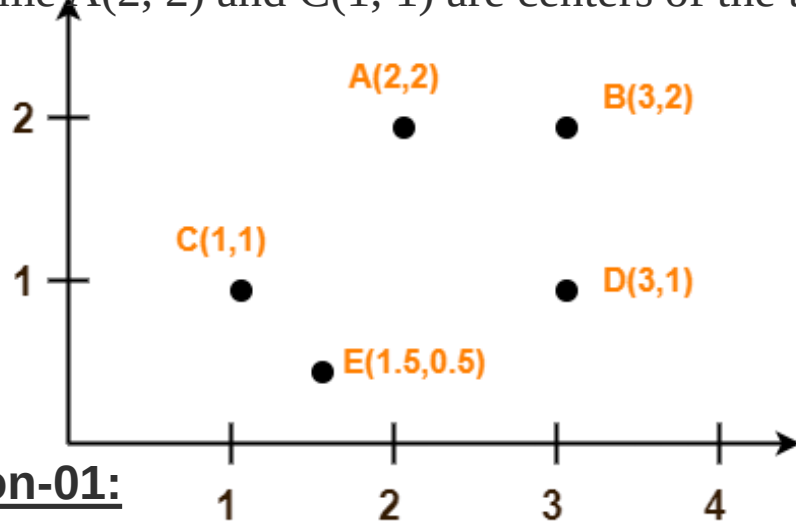
Given Points	Distance from Center of (185,72) Cluster-1	Distance from Center of (169, 58) Cluster-2	Point Cluster belongs to
A1(185,72)	0	21.26	C1
A2(170,56)	21.93	2.23	C2
A3(168,60)	20.80	2.23	C2
A4(179,68)	7.21	14.14	C1
A5(182,72)	3	19.10	C1
A6(188,77)	5.83	26.87	C1
A7(180,71)	5.09	17.02	C1
A8(180,70)	5.38	16.27	C1
A9(183,84)	12.16	29.52	C1
A10(180,88)	16.76	31.95	C1
A11(180,67)	7.07	14.21	C1
A12 (177,76)	8.94	19.69	C1

C1={1,4,5,6,7,8,9,10,11,12}, C2={2,3}

Kmeans Solved Example#3

Use K-Means Algorithm to create two clusters

Assume A(2, 2) and C(1, 1) are centers of the two clusters.



Iteration-01:

Given Points	Distance from Center of (2,2) Cluster-1	Distance from Center of (1, 1) Cluster-2	Point Cluster belongs to
A(2,2)	0	1.41	C1
B(3,2)	1	2.23	C1
C(1,1)	1.41	0	C2
D(3,1)	1.41	2	C1
E(1.5,0.5)	1.58	0.707	C2

$$\text{Euclidean Distance : } \sqrt{(X_0 - X_c)^2 + (Y_0 - Y_c)^2} \\ \text{(ED)}$$

C1={A,B,D}, C2={C,E}

For Cluster-01: We have only one point A(2, 2) in Cluster-01. So, cluster center remains the same.

For Cluster-02: Center of Cluster-02
 $= ((1 + 1.5) / 2, (1 + 0.5) / 2)$
 $= (1.25, 0.75)$

Iteration-02:

Given Points	Distance from Center of (2,2) Cluster-1	Distance from Center of (1.25,0.75) Cluster-2	Point Cluster belongs to
A(2,2)	0	1.45	C1
B(3,2)	1	2.15	C1
C(1,1)	1.41	0.35	C2
D(3,1)	1.41	1.67	C1
E(1.5,0.5)	1.58	0.35	C2

C1={A,B,D}, C2={C,E}

Clustering Problems

Module-1; Topic-3: Classification and Clustering problems

1. Lack of Labeled Data Problem: Clustering is an unsupervised learning task, which means there are no labels available to guide the network in learning.

- Solution: Use semi-supervised learning where a small amount of labeled data is available.
 - Self-supervised learning can be explored to generate labels from the data itself (e.g., using clustering results as pseudo-labels).
 - Apply techniques like autoencoders to learn a useful representation of the data before applying clustering.

2. Determining the Number of Clusters Problem: Many clustering algorithms (e.g., K-means) require the number of clusters (K) to be specified beforehand, which is not always known.

- Solution: Use unsupervised methods to estimate the number of clusters, such as the elbow method, silhouette score, or gap statistics.

3. Choice of Neural Network Architecture

- **Problem:** Selecting the right architecture for clustering can be challenging. Different types of data (e.g., images, time series, or text) require different neural network designs.
- **Solution:** Use autoencoders for unsupervised feature extraction and clustering in latent space.
 - Self-Organizing Maps (SOMs) and Variational Autoencoders (VAEs) can be used for more complex clustering tasks.
 - Employ Deep Clustering algorithms that can jointly learn representations and perform clustering tasks.

Clustering Problems

Module-1; Topic-3: Classification and Clustering problems

4. Scalability Problem: Neural network-based clustering, particularly deep learning models like autoencoders, may struggle with very large datasets due to computational limits and memory constraints.

- **Solution:** Use mini-batch processing to reduce memory usage. Apply parallelized versions of clustering algorithms (e.g., distributed K-means).

5. Training Stability Problem: Unsupervised neural networks (e.g., autoencoders or clustering networks) can sometimes have stability issues during training, especially with large datasets or high-dimensional input.

- Solution: Use Batch Normalization or Layer Normalization to stabilize training.
- Implement gradient clipping to avoid exploding gradients.
- Regularize the model with L2 regularization or dropout to prevent overfitting.

6. Evaluation and Validation Problem: Evaluating clustering performance is more challenging without ground truth labels. Traditional classification metrics (accuracy, precision, recall) don't apply directly.

Solution: Use internal validation metrics like silhouette score, Davies-Bouldin Index, or inertia to evaluate the quality of clusters.

• **For Classification**, challenges often center around issues like overfitting, class imbalance, computational cost, and interpretability. Advanced techniques such as transfer learning, regularization, and hyperparameter tuning can mitigate these issues.

• **For Clustering**, the main difficulties arise from a lack of labels, choosing the right architecture, training stability, and scalability. Unsupervised learning methods, autoencoders, and deep clustering techniques can help address these challenges, although they still require careful attention to data quality and model selection.

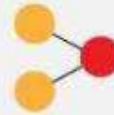
Both tasks require careful data preprocessing, model tuning, and sometimes novel architectures or hybrid methods to overcome these challenges effectively.

Types of Deep Learning Networks

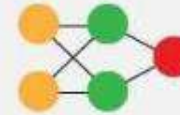
- ▶ Feed Forward Neural Network
- ▶ Recurrent Neural Network
- ▶ Convolutional Neural Network
- ▶ Restricted Boltzmann Machine
- ▶ Autoencoders

Neural Networks

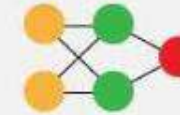
Perceptron (P)



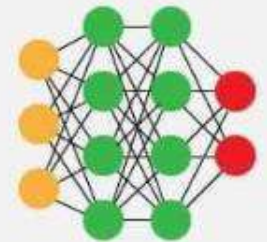
Feed Forward (FF)



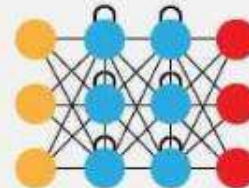
Radial Basis Network (RBF)



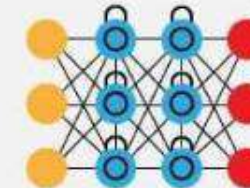
Deep Feed Forward (DFF)



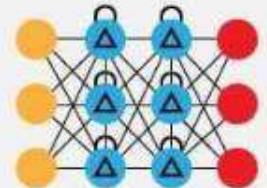
Recurrent Neural Network (RNN)



Long / Short Term Memory (LSTM)



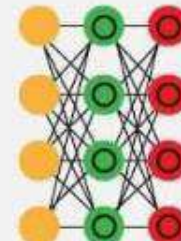
Gated Recurrent (GRU)



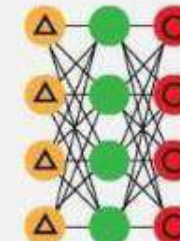
Auto Encoder (AE)



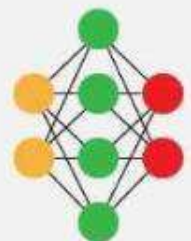
Variational AE (VAE)



Denoising AE(DAE)



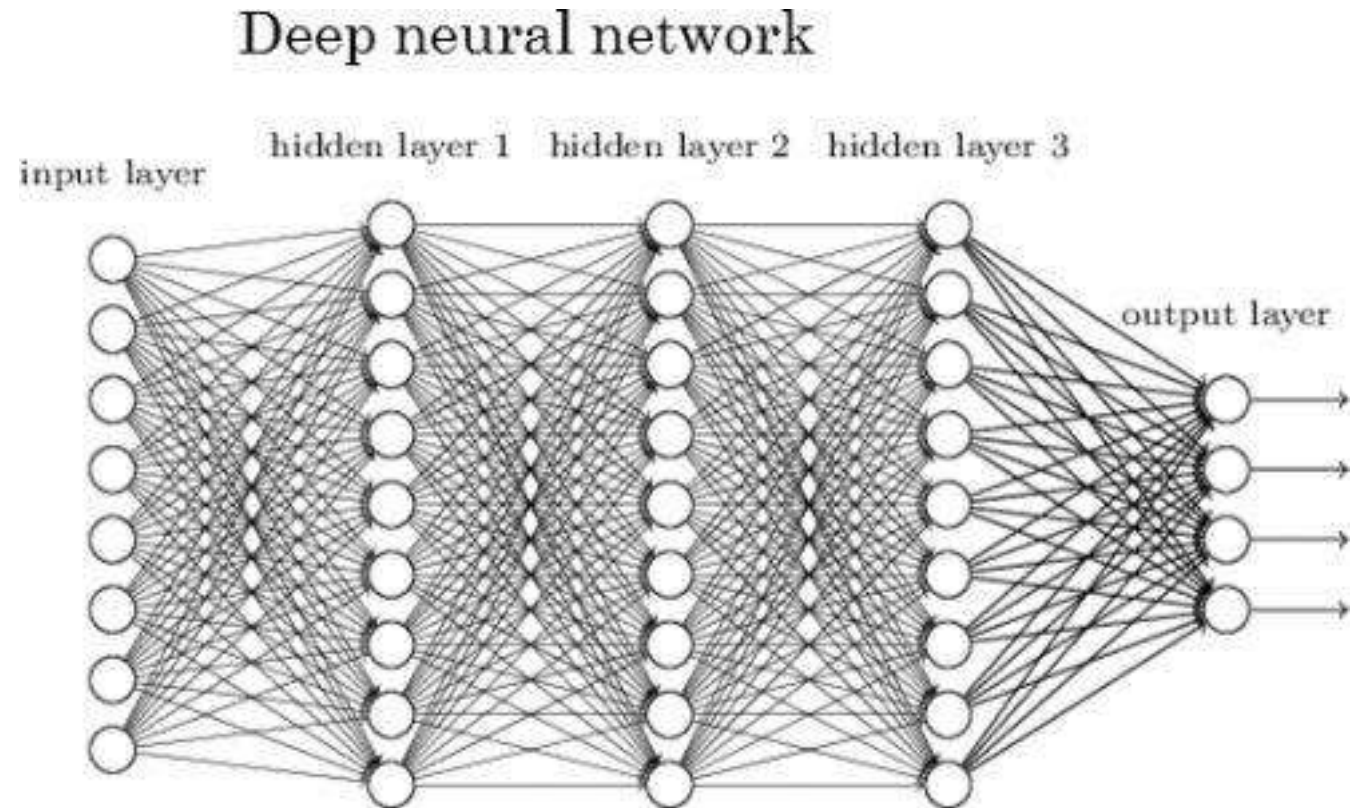
Sparse AE (SAE)



Deep Learning Networks-Types

Deep Neural Networks

It is a neural network that incorporates the complexity of a certain level, which means several numbers of hidden layers are encompassed in between the input and output layers. They are highly proficient on model and process non-linear associations.



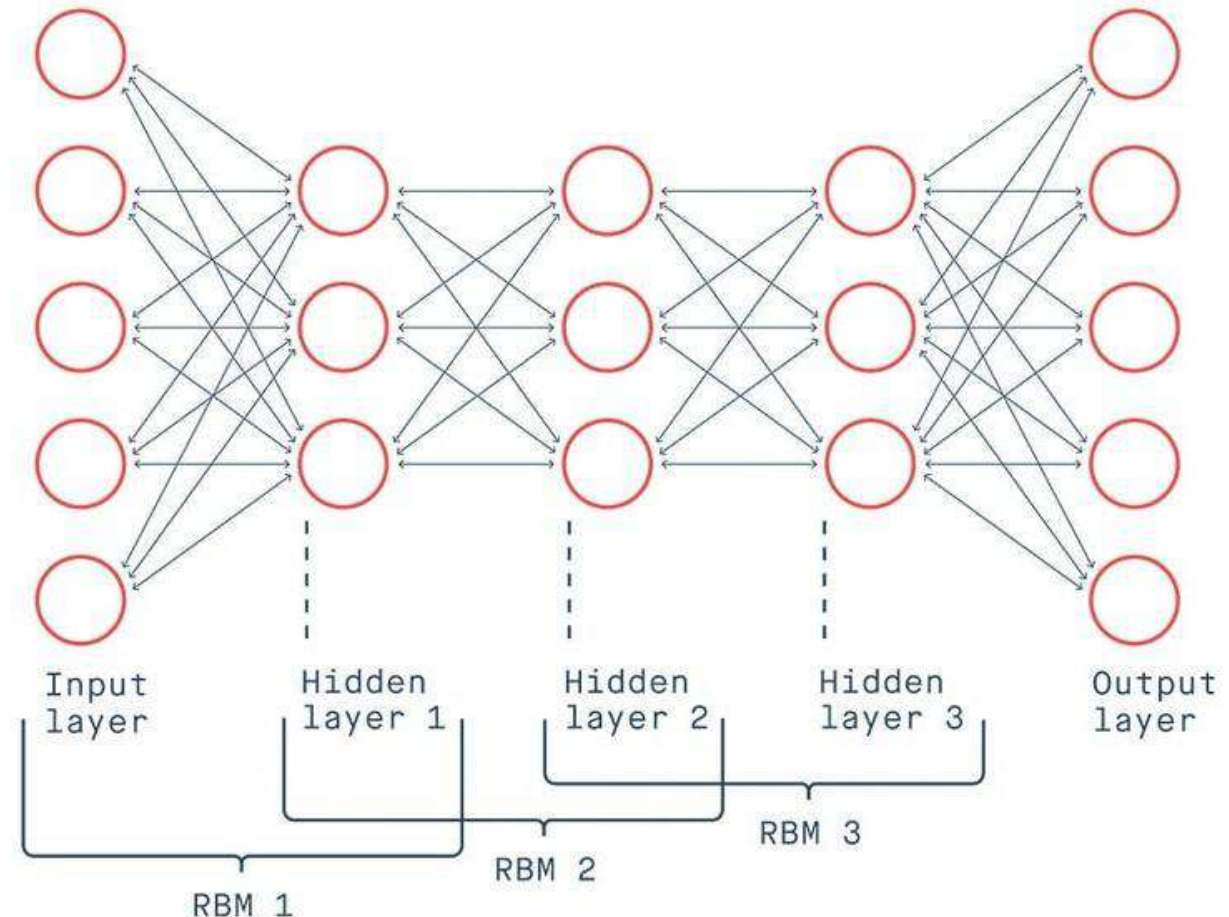
Deep Learning Networks-Types

Deep Belief Networks

A deep belief network is a class of Deep Neural Network that comprises of multi-layer belief networks.

Steps to perform DBN:

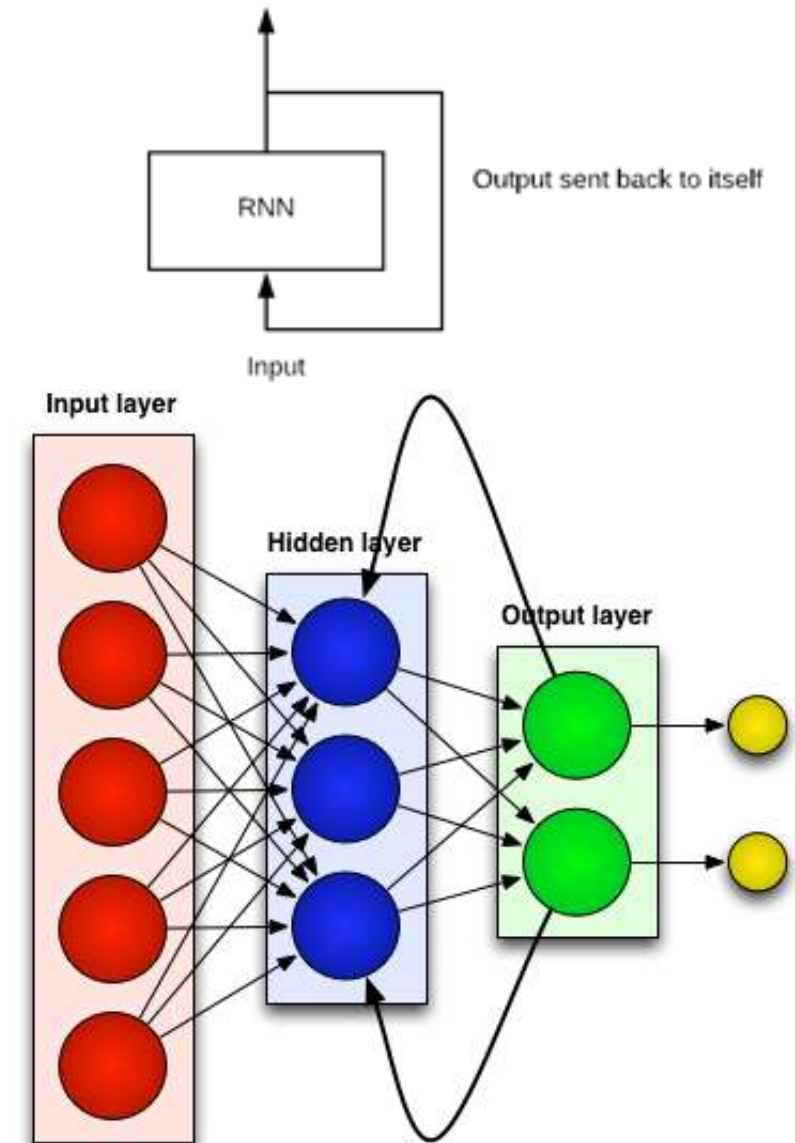
- With the help of the Contrastive Divergence algorithm, a layer of features is learned from perceptible units.
- Next, the formerly trained features are treated as visible units, which perform learning of features.
- Lastly, when the learning of the final hidden layer is accomplished, then the whole DBN is trained.



Deep Learning Networks-Types

Recurrent Neural Networks

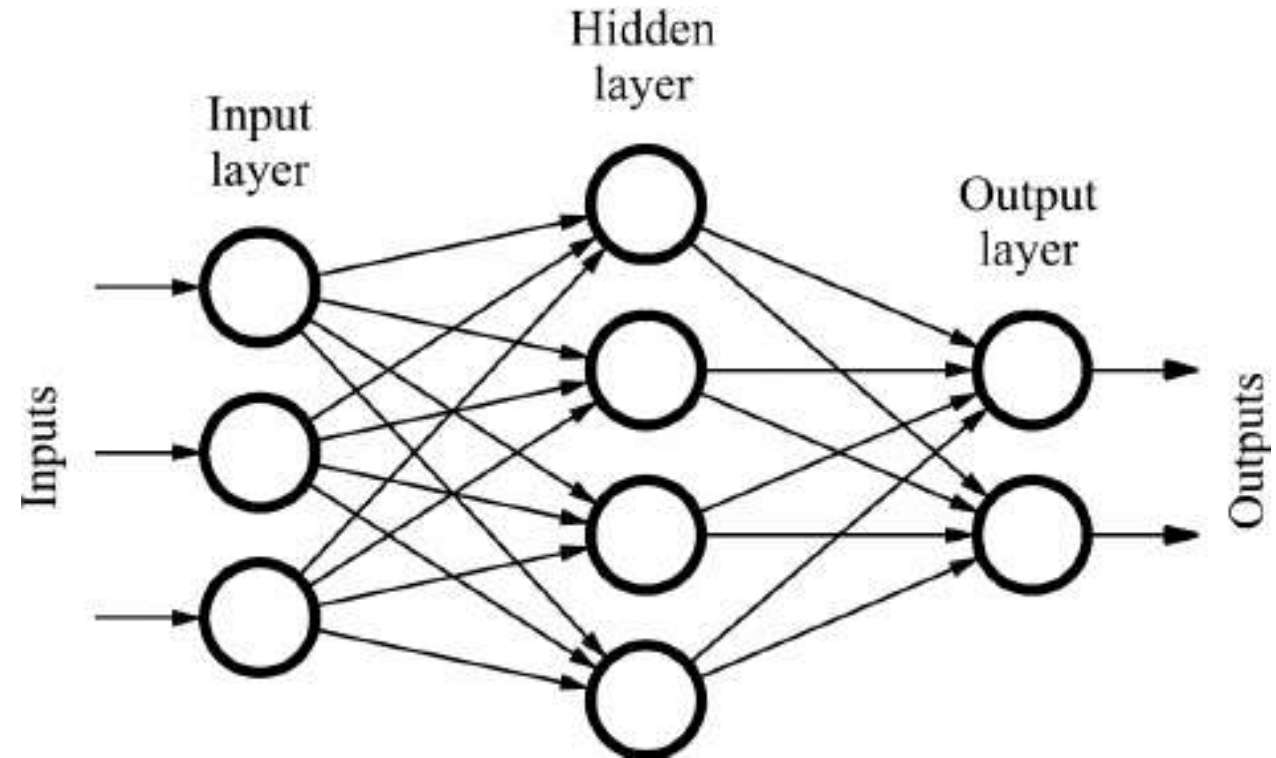
- RNN is a multi-layered neural network that can store information in context nodes, allowing it to learn data sequences and output a number or another sequence. In simple words, it is an Artificial neural networks whose connections between neurons include loops. RNNs are well suited for processing sequences of inputs.
- It permits parallel as well as sequential computation, and it is exactly similar to that of the human brain (large feedback network of connected neurons).
- Since they are capable enough to reminisce all of the imperative things related to the input they have received, so they are more precise.



Deep Learning Networks-Types

Feed Forward Neural Network

- The simplest type of artificial neural network. With this type of architecture, information flows in only one direction, forward.
- It means, the information's flows starts at the input layer, goes to the "hidden" layers, and end at the output layer. The network does not have a loop. Information stops at the output layers.

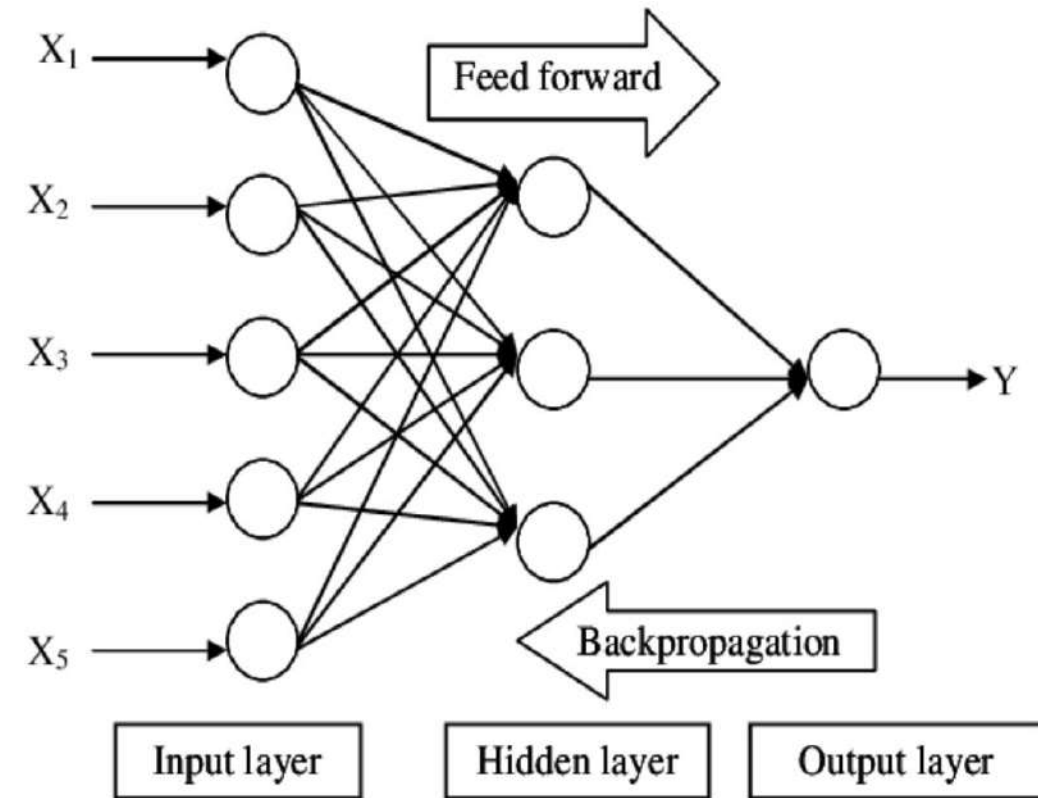


Common Architectural Principles of Deep Network

- Common architectural principles of deep neural networks (DNNs) refer to the design patterns, strategies, and guidelines that help build and optimize deep learning models.
- These principles are essential to ensure the network can learn effectively from data and generalize well.

1. Layered Architecture Depth: Deep networks are characterized by multiple layers (hence the term "deep"). These layers consist of different types of units or neurons that transform input data into increasingly abstract representations.

- **Hierarchical Learning:** Each successive layer captures increasingly complex patterns. For example, in image processing, lower layers might detect edges, while higher layers detect objects or faces.



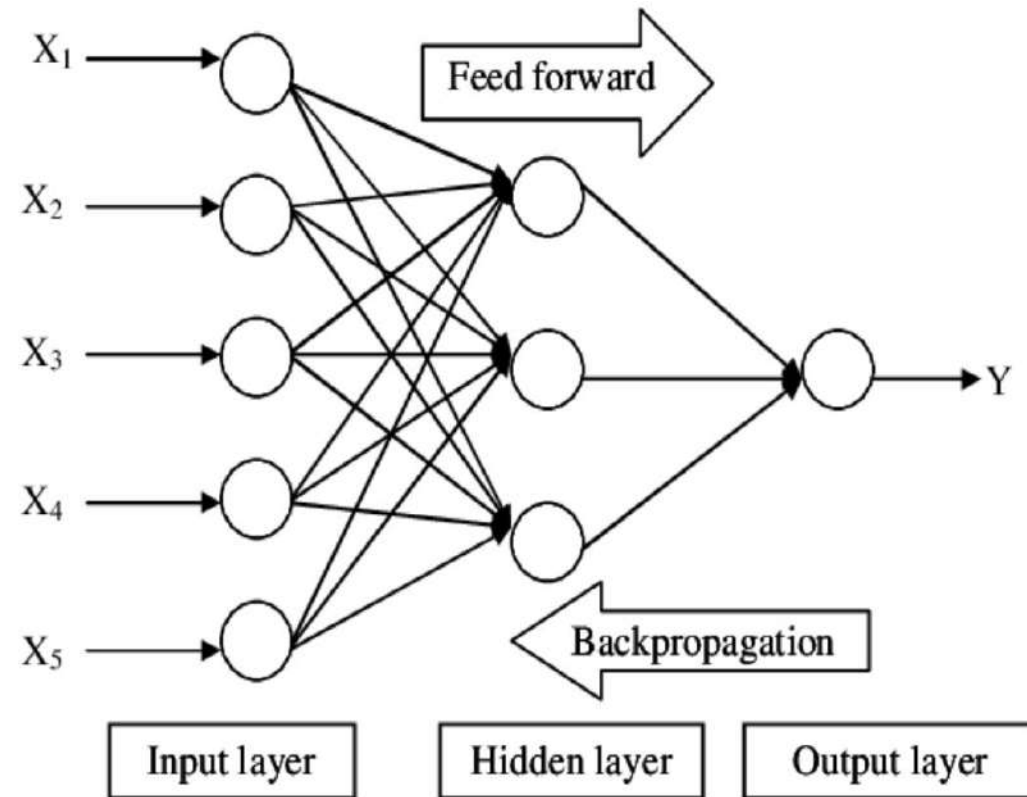
Common Architectural Principles of Deep Network

2. Feedforward and Backpropagation: In a typical deep network, data flows from the input layer through hidden layers to the output layer in a forward direction.

- **Backpropagation:** During training, the model adjusts its weights based on the error between the predicted output and the true output using backpropagation, which is the mechanism for updating the network's parameters.

3. Activation Functions-Non-linearity: Activation functions introduce non-linearity into the network. Without non-linear functions, even deep networks would behave like a single-layer perceptron. Popular activation functions include ReLU (Rectified Linear Unit), Sigmoid, and Tanh.

- **Activation Functions by Layer Type:** ReLU: Often used in hidden layers, offering efficient gradient propagation and sparsity.



Common Architectural Principles of Deep Network

4. Convolutional Layers (For CNNs)

Convolution: In Convolutional Neural Networks (CNNs), convolutional layers are designed to automatically learn spatial hierarchies of features in data like images. Filters or kernels are applied to local regions of input data, capturing spatial patterns such as edges, textures, and shapes.

Pooling: Pooling layers (like max pooling) help reduce the spatial dimensions of the input, which reduces computational load and helps prevent overfitting.

5. Regularization Techniques

Dropout: Dropout is a technique where randomly selected neurons are ignored during training, which helps in reducing overfitting by making the network more robust.

L2 Regularization (Weight Decay): This involves penalizing large weights, helping the network to generalize better by reducing overfitting.

Batch Normalization: It normalizes the inputs to each layer, which helps in speeding up the training and stabilizing learning by reducing internal covariate shifts.

6. Residual Connections (Skip Connections)

Residual Networks (ResNets): These architectures allow gradients to flow through deeper networks more effectively by introducing skip connections that bypass one or more layers.

Skip Connections: These connections help to solve the vanishing gradient problem, enabling training of much deeper networks.

Common Architectural Principles of Deep Network

7. Recurrent Connections (For RNNs)

Recurrent Neural Networks (RNNs): These networks have loops in their architecture, allowing information to persist. They are particularly suited for sequential data, like time series or text.

Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRUs): These are specialized types of RNNs designed to handle long-term dependencies and mitigate the vanishing gradient problem in standard RNNs.

8. Attention Mechanisms

Self-attention: Attention mechanisms allow the network to focus on different parts of the input sequence, helping the model to weigh the importance of different elements of the input more effectively. This is a critical principle in models like Transformers.

Transformer Architectures: These architectures rely on attention to capture long-range dependencies in sequences without relying on recurrent connections.

9. Optimization Techniques

Gradient Descent Variants: Stochastic Gradient Descent (SGD) and its variants like Adam, RMSprop, and AdaGrad are used to minimize the loss function and update the network weights efficiently during training.

Learning Rate Scheduling: Techniques like learning rate decay or cyclical learning rates adjust the learning rate during training to improve convergence.

Common Architectural Principles of Deep Network

10. Layer Types and Variants

Fully Connected Layers (Dense): These layers connect every neuron in one layer to every neuron in the next layer. They are typically used in the final layers of the network for classification or regression tasks.

Deconvolution Layers: Also called transposed convolutions, these layers are used in tasks like image segmentation and generative models, where the aim is to increase the spatial resolution.

11. Data Augmentation:

Artificially increasing the dataset size: By generating new training data through techniques such as rotation, flipping, scaling, cropping, etc., models can be trained to generalize better and avoid overfitting.

12. Loss Functions

Loss Function Selection: The loss function measures how well the network performs. For regression tasks, Mean Squared Error (MSE) might be used, while for classification, Cross-Entropy Loss is common.

13. Transfer Learning

Pre-training and Fine-tuning: Transfer learning involves using a pre-trained model on a similar task and fine-tuning it for a specific problem. This is particularly useful when labeled data is scarce.

Common Architectural Principles of Deep Network

14. Scalability and Parallelization

- **Model Parallelism:** Dividing the model across multiple GPUs or devices to handle large models and datasets.
- **Data Parallelism:** Distributing the data across multiple devices, where each device processes a batch of data and updates the model parameters accordingly.

15. Scalability and Computational Efficient Architectures: Models like MobileNets and EfficientNet are designed to be both computationally efficient and accurate, making them suitable for resource-constrained environments.

- By adhering to these principles, deep neural networks can be designed to effectively learn from large datasets, generalize well, and optimize computational resources. These principles are foundational for the development of modern deep learning models across various domains such as computer vision, natural language processing, and reinforcement learning.

Challenges in Deep Networks

a. Vanishing/Exploding Gradients:

In deep networks, gradients can become very small (vanishing gradients) or very large (exploding gradients), making it difficult to train the model effectively.

Solution: Use activation functions like **ReLU** that are less prone to vanishing gradients and **gradient clipping** for exploding gradients.

b. Training Time and Computational Cost:

Deep networks, especially with many layers and large datasets, can take a long time to train and require high computational power (GPUs, TPUs).

Solution: Use **pretrained models** and **transfer learning** to reduce training time, or employ **distributed training** across multiple machines.

c. Overfitting:

With more parameters (due to the deep architecture), the model may memorize the training data and fail to generalize to new, unseen data.

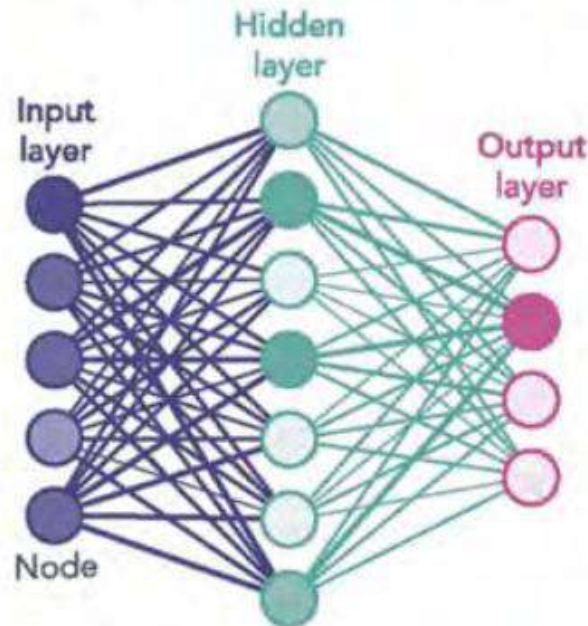
Solution: Use **regularization** techniques, increase training data, or use simpler models.

However, training deep networks requires careful consideration of the architecture, optimization, regularization, and evaluation methods. Despite the challenges, DNNs have become the backbone of modern AI systems due to their flexibility and ability to model complex patterns in high-dimensional data.

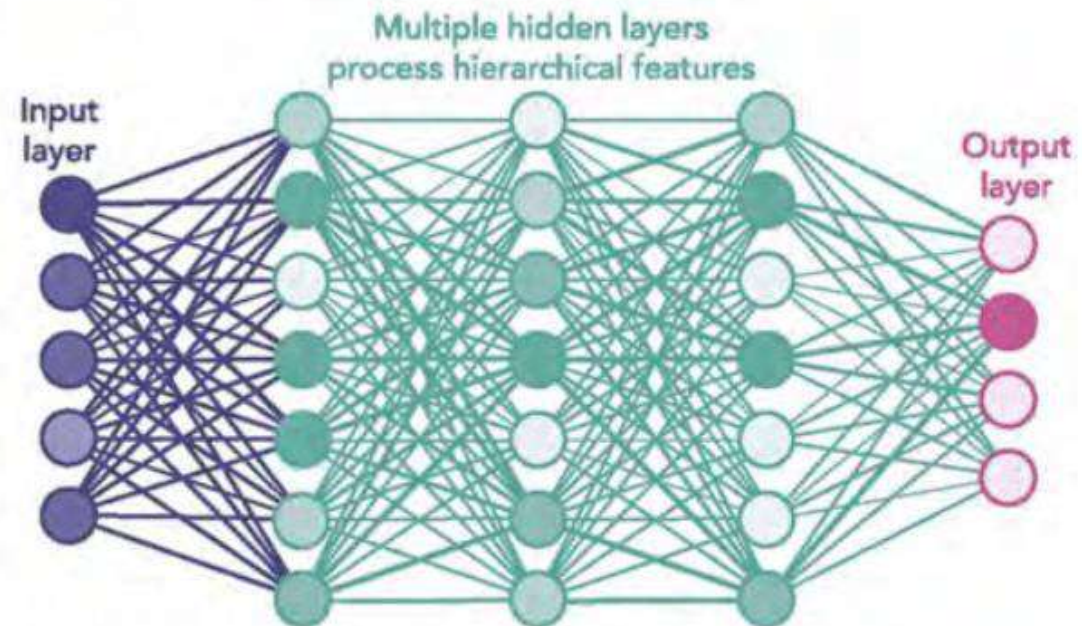
Shallow Neural Networks

- A **shallow neural network** refers to a neural network that typically consists of only **one hidden layer** between the input and output layers.
- While deep neural networks (DNNs) have multiple hidden layers (hence the term "deep"), shallow networks are simpler, with fewer layers and, therefore, fewer parameters.
- Shallow neural networks were the foundation of early neural network research and are still used for simpler tasks or when computational resources are limited.
- However, their performance is often limited compared to deeper networks on complex tasks like image and speech recognition.

SHALLOW NEURAL NETWORK



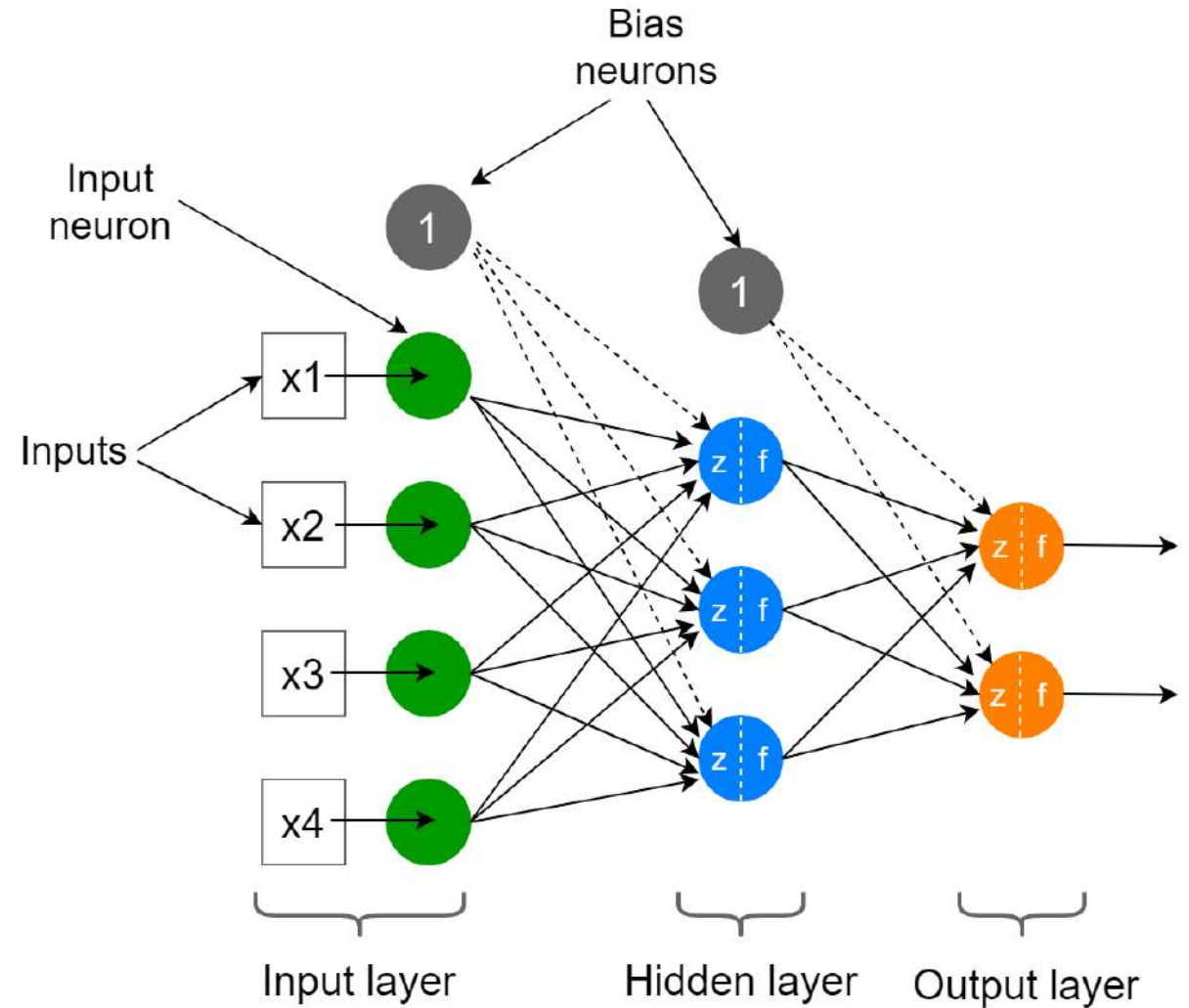
DEEP NEURAL NETWORK



Shallow Neural Networks

A shallow neural network typically has:

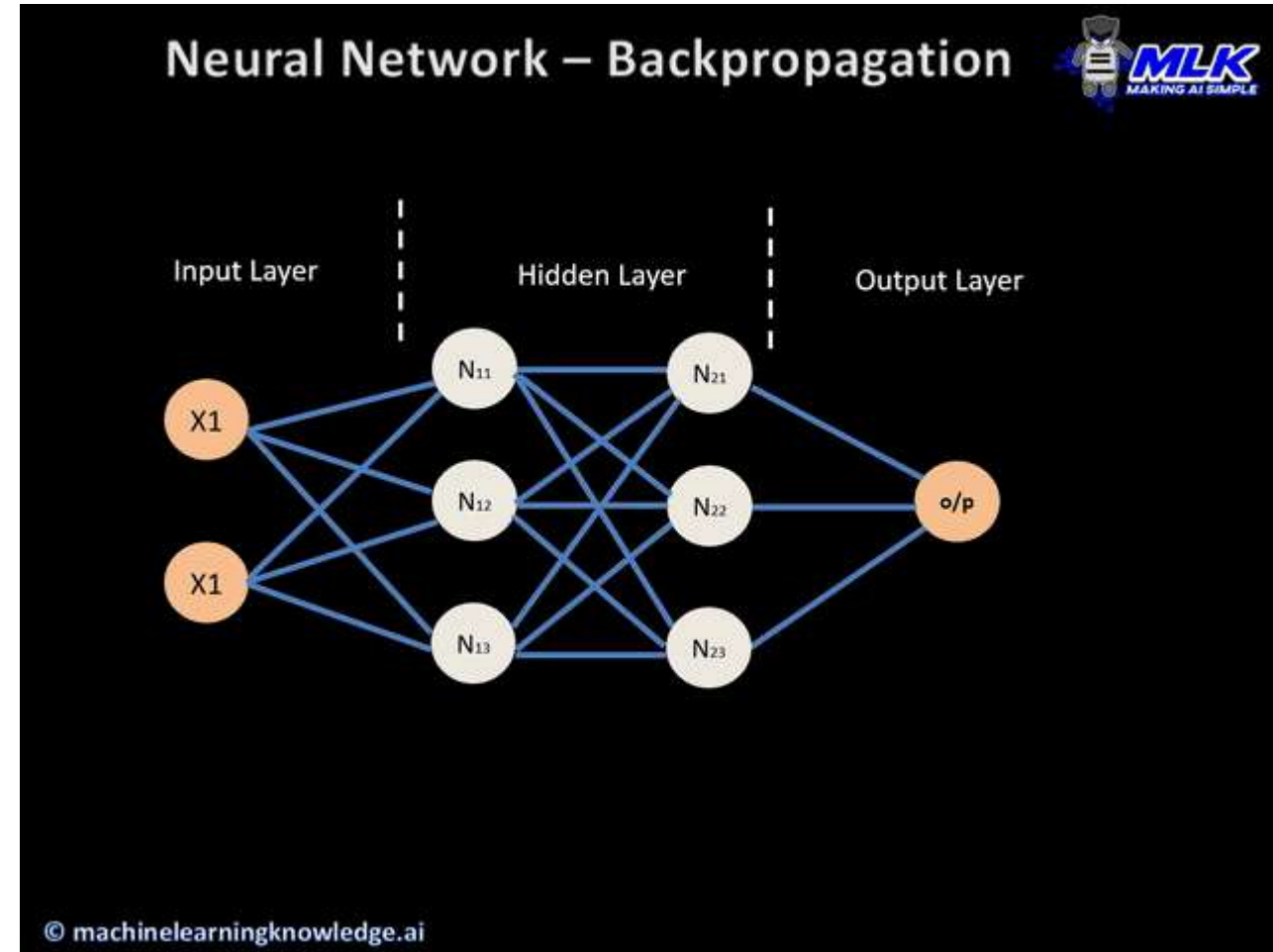
- **Input Layer:** Receives the input data (e.g., features of the data in tabular form, pixel values in an image).
- **Hidden Layer:** The core of the neural network where learning occurs. A shallow network typically has only **one hidden layer**.
- **Output Layer:** Produces the final result, which could be a prediction (regression) or a class label (classification).
- A shallow network's architecture can be represented as:
- **Input Layer → Hidden Layer → Output Layer**
- Each layer consists of **neurons** or **nodes**. The output from each neuron is computed based on a weighted sum of the inputs, followed by the application of an **activation function**.



Shallow Neural Networks

a. Forward Propagation: The process of passing input data through the network to obtain an output involves:

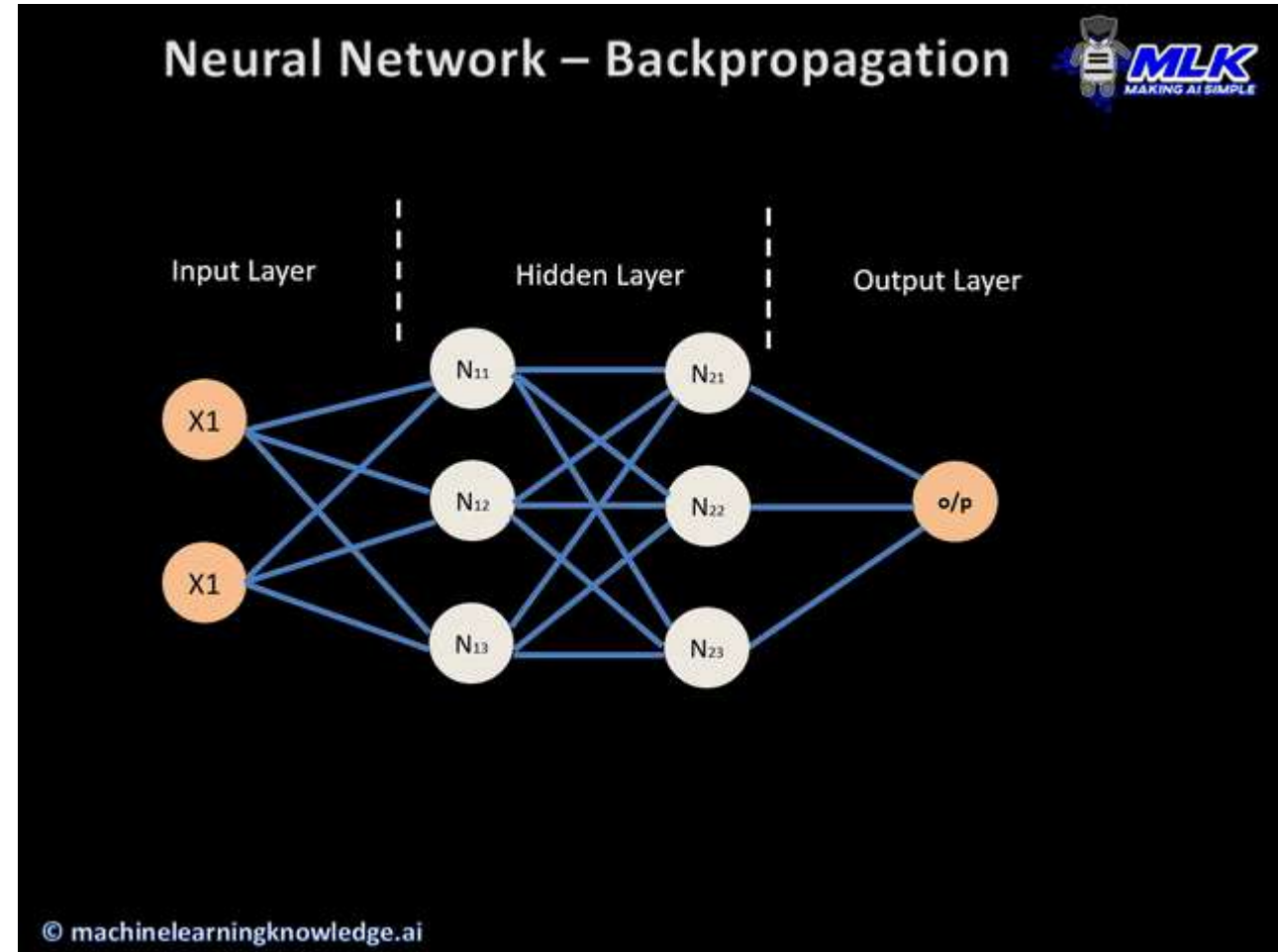
- **Input Layer:** The raw data (features) are fed into the input layer. For example, in a binary classification task, each feature could represent a pixel or a numerical value.
- **Hidden Layer:** The input is passed to the hidden layer, where each neuron performs the following:
 - **Weighted Sum:** Each neuron calculates a weighted sum of the inputs it receives.
 - **Activation Function:** The weighted sum is then passed through an activation function, such as **sigmoid**, **tanh**, or **ReLU**.
- **Output Layer:** The output from the hidden layer is then passed to the output layer, where another weighted sum and activation function (like **softmax** for classification or **linear** for regression) produce the final result.



Shallow Neural Networks

b. Loss Function: The network compares the predicted output with the actual output using a **loss function** (or **cost function**) like **Mean Squared Error** (MSE) for regression or **Cross-Entropy Loss** for classification. The goal is to minimize this loss during training.

c. Backpropagation: **Backpropagation** is the process used to update the weights of the network during training. It calculates the gradient of the loss function with respect to each weight using the **chain rule of calculus** and updates the weights in the direction that minimizes the loss (via **gradient descent** or other optimization techniques).



Shallow Neural Networks

Advantages of Shallow Network:

- **Faster Training:** With fewer layers and parameters, shallow networks are generally faster to train compared to deep networks. This can be beneficial when the dataset is small or computational resources are limited.
- **Lower Computational Complexity:** Since shallow networks have fewer weights and parameters, they are less computationally demanding than deeper networks.
- **Simplicity:** Shallow networks are easier to understand and interpret, making them a good choice for simpler problems or when model interpretability is essential.
- **Less Data Needed:** Shallow networks can perform well when there is limited data, as they are less likely to overfit compared to deep networks.

Limitations of Shallow Neural Networks:

- **Limited Expressive Power:** Shallow networks can only model relatively simple relationships in the data. They may struggle to capture complex patterns that deep networks can learn, especially for tasks like image classification, speech recognition, or natural language processing.
- **Underfitting:** A shallow network with too few hidden neurons may underfit the data, meaning it fails to capture important patterns in the training data, leading to poor performance on both training and test data.
- **Difficulty in Learning Non-Linearities:** A single hidden layer can only approximate relatively simple non-linear functions. To model more complex relationships, multiple hidden layers are often required.

Types of Shallow Neural Networks

Shallow networks can take different forms based on the type of task or data they are handling:

a. Single-Layer Perceptron (SLP): The simplest form of a shallow neural network.

Architecture: Consists of an input layer, a single hidden layer, and an output layer.

Usage: Historically used for binary classification, such as logical functions like AND, OR, XOR.

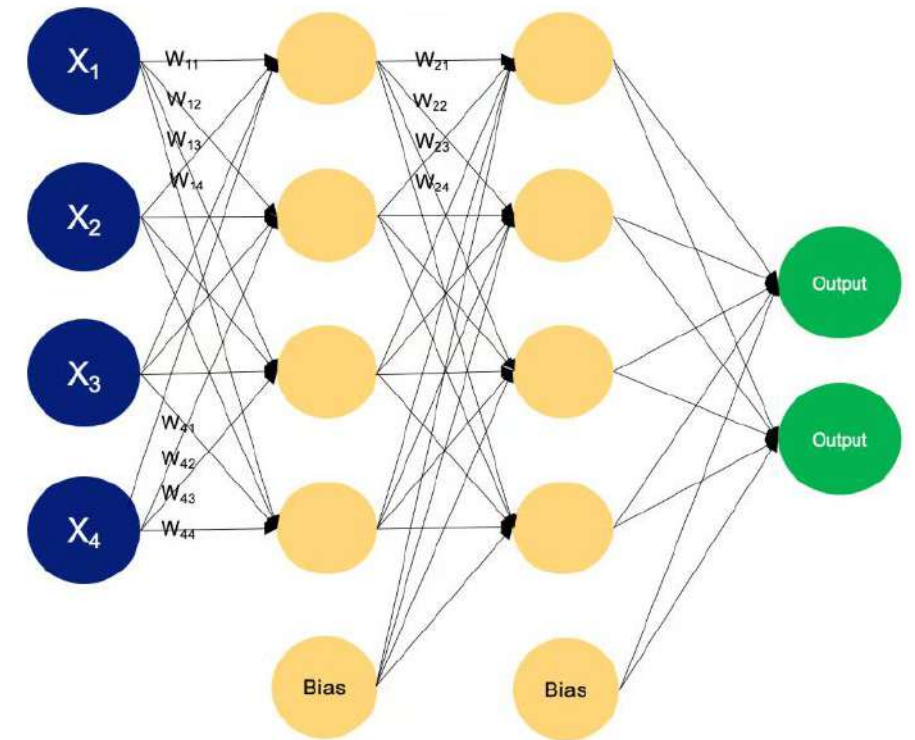
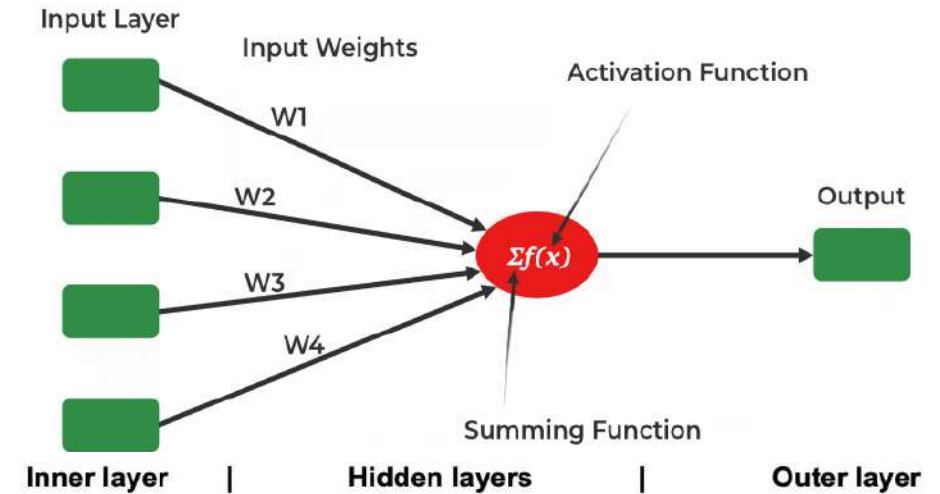
Limitation: The single-layer perceptron cannot solve non-linearly separable problems (e.g., XOR problem).

b. Multilayer Perceptron (MLP): A slightly more complex shallow network that can have one hidden layer with multiple neurons.

Architecture: An input layer, one or more hidden layers, and an output layer.

Activation Function: Typically uses **ReLU**, **sigmoid**, or **tanh** for hidden layers and **softmax** or **sigmoid** for the output layer.

Usage: Used in a wide variety of classification and regression tasks with simple to moderate complexity.



Shallow Neural Networks

Training Shallow Neural Networks: Training a shallow network involves:

- **Forward Propagation:** Pass the inputs through the network to calculate the output.
- **Loss Calculation:** Compare the predicted output to the ground truth using a loss function.
- **Backpropagation:** Compute gradients for each weight and update the weights using an optimization algorithm (e.g., gradient descent).
- **Iteration:** Repeat the forward pass, loss calculation, and backpropagation steps until the loss converges or the model reaches an acceptable level of performance.

Optimization Algorithms:

- **Stochastic Gradient Descent (SGD):** Updates weights after each training example, making it efficient but noisy.
- **Mini-batch Gradient Descent:** A compromise between full-batch and stochastic gradient descent, updating weights after small batches of data.
- **Adam:** An advanced optimization algorithm combining the advantages of both SGD and momentum-based approaches.

Applications:

- **Binary Classification:** Used for tasks like spam detection (spam vs. not spam) or sentiment analysis (positive vs. negative).
- **Simple Regression:** Can be used for predicting continuous values, such as house prices based on certain features (e.g., square footage, number of rooms).
- **Function Approximation:** Basic shallow networks can learn to approximate simple mathematical functions or relationships.
- **Pattern Recognition:** Basic image classification tasks where the relationships between pixels are not highly complex.

Shallow Vs. Deep Neural Networks

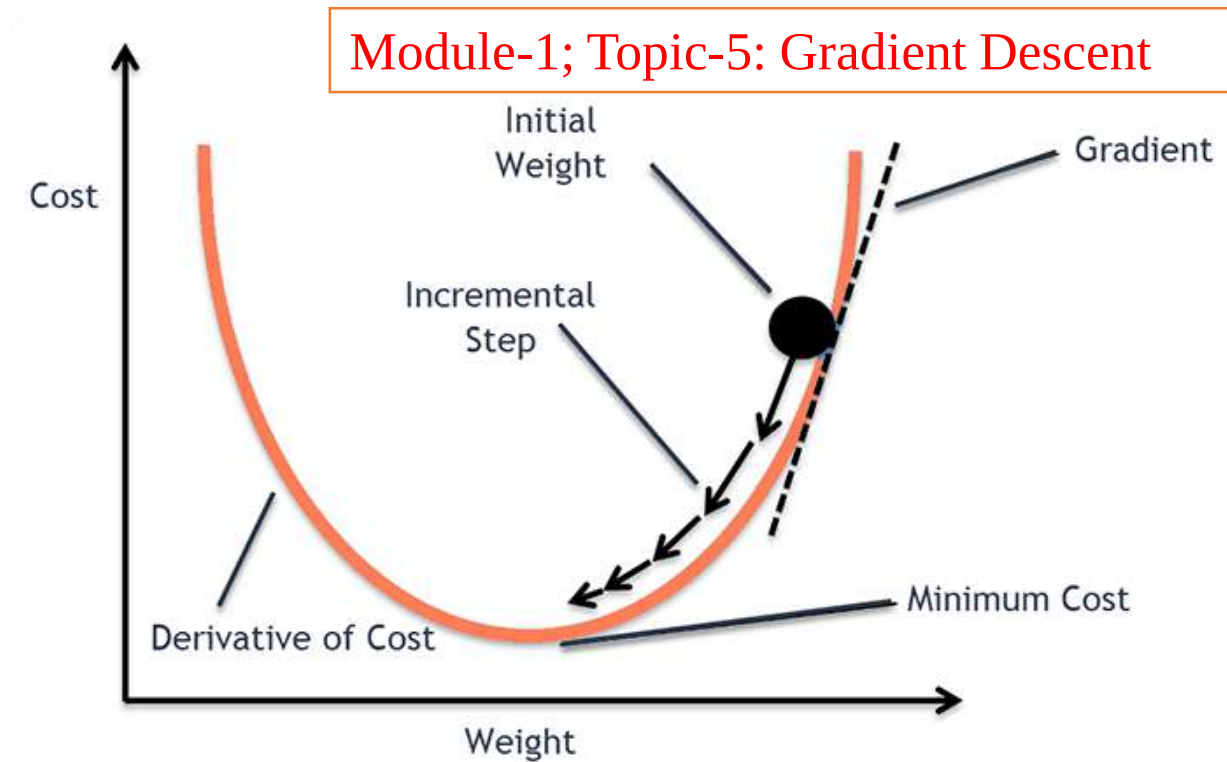
Aspect	Shallow Neural Networks	Deep Neural Networks
Number of Layers	1 hidden layer	Multiple hidden layers (usually 2+ layers)
Model Complexity	Limited; good for simpler tasks	High complexity; good for complex tasks
Training Time	Faster training, fewer parameters to optimize	Slower training, more parameters to optimize
Data Requirement	Works well with small datasets	Requires large datasets for optimal performance
Performance	May underperform on complex tasks	Can perform excellently on complex tasks
Risk of Overfitting	Lower (due to fewer parameters)	Higher (due to many parameters)
Interpretability	Easier to interpret	Harder to interpret ("black-box" nature)
Computation	Lower computational cost	Higher computational cost

Optimizing Functions

- Gradient Descent: first-order, iterative-based optimization algorithm
- Momentum Optimizer: accelerates the stochastic gradient descent in the relevant direction
- Adagrad: adaptively scales learning rate for different dimension

Common Variants of Gradient Descent:

- **Stochastic Gradient Descent (SGD):** Updates weights using a random subset (mini-batch) of the training data, making it faster but more noisy.
- **Mini-Batch Gradient Descent:** A compromise between Batch Gradient Descent (which uses the entire dataset for each update) and SGD, using small batches to reduce computational overhead.
- **Adam (Adaptive Moment Estimation):** Combines ideas from momentum and RMSprop, adjusting the learning rate for each parameter dynamically.
- **RMSprop:** Adapts the learning rate for each parameter by dividing the gradient by a moving average of its recent magnitude.



Variants of gradient descent

1. Batch gradient descent:

- Vanilla gradient descent, is the simplest variant of gradient descent.
- In batch gradient descent, *the entire training dataset is used to compute the gradients of the cost function with respect to the model parameters in each iteration.*
- This can be computationally expensive for large datasets, but it guarantees convergence to a local minimum of the cost function.

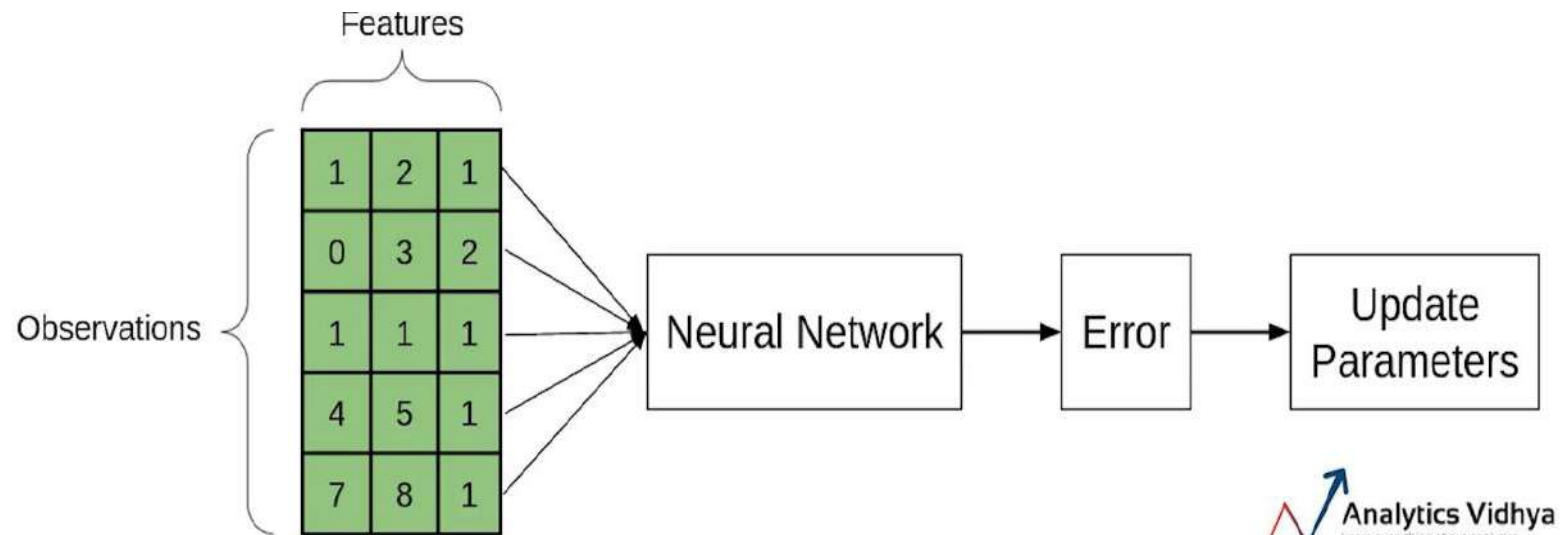
$$\theta = \theta - \alpha \cdot \nabla J(\theta)$$

where:

θ is the parameter vector

α is the learning rate

$\nabla J(\theta)$ is the gradient of the cost function J with respect to θ



Variants of gradient descent

2. Stochastic Gradient Descent (SGD):

- Stochastic gradient descent is a variant of gradient descent that updates the model parameters for each training example in the dataset.
- Unlike batch gradient descent, which uses the entire dataset to compute the gradients, SGD updates the parameters based on a randomly selected training example.
- This can lead to faster convergence because the updates are more frequent and noisy, but it can also result in more oscillations in the cost function due to the randomness of the updates.

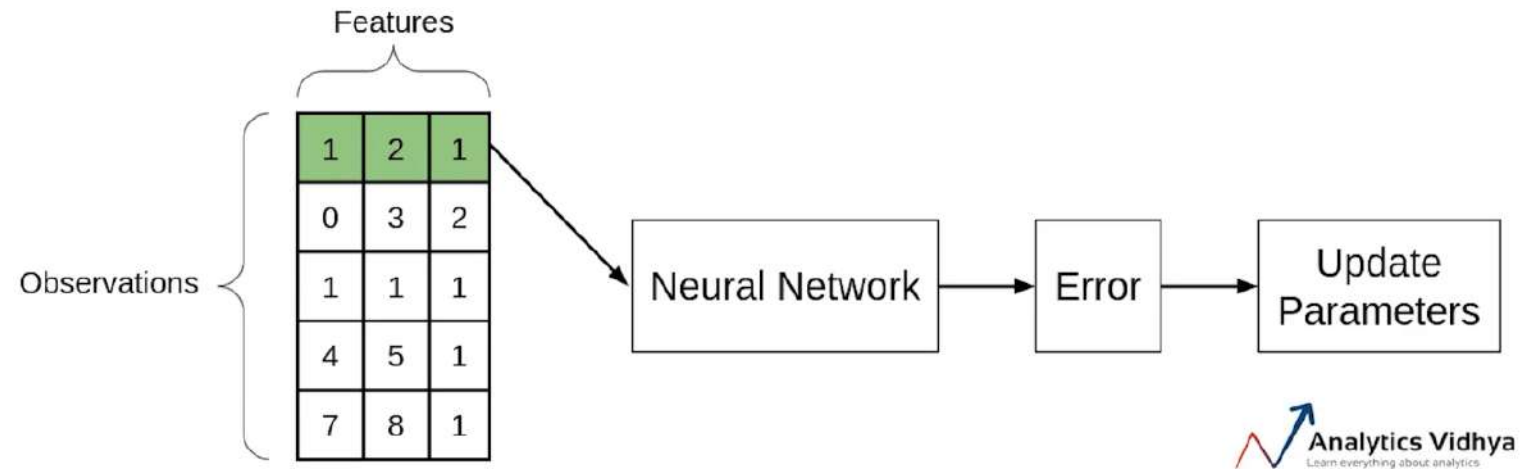
$$\theta = \theta - \alpha \cdot \nabla J_i(\theta)$$

where:

θ is the parameter vector

α is the learning rate

$\nabla J_i(\theta)$ is the gradient of the cost function J with respect to θ computed on a single randomly selected training example i



Variants of gradient descent

3. Mini-batch Gradient Descent

- Mini-batch gradient descent is a compromise between batch gradient descent and stochastic gradient descent.
- In mini-batch gradient descent, the gradients are computed on a small random subset of the training dataset, typically between 10 and 1000 examples, called a **mini-batch**.

$$\theta = \theta - \alpha \cdot \frac{1}{m} \sum_{i=1}^m \nabla J_i(\theta)$$

where:

θ is the parameter vector

α is the learning rate

m is the mini-batch size

$\nabla J_i(\theta)$ is the gradient of the cost function J with respect to θ computed on a mini-batch of size m

- This reduces the computational cost of the algorithm compared to batch gradient descent, while also reducing the variance of the updates compared to SGD.
- Mini-batch gradient descent is widely used in deep learning because it strikes a good balance between convergence speed and stability.

Features		
1	2	1
0	3	2
1	1	1
4	5	1
7	8	1
Observations		

Variants of gradient descent

4. Momentum Gradient

- Momentum gradient descent is a variant of gradient descent that adds a momentum term to the update rule.
- The momentum term accumulates the gradient values over time and dampens the oscillations in the cost function, leading to faster convergence. This is particularly useful in cases where the cost function has a lot of noise or curvature, which can cause traditional gradient descent to get stuck in local minima.

$$v_t = \gamma v_{t-1} + \alpha \cdot \nabla J(\theta_{t-1})$$

$$\theta_t = \theta_{t-1} - v_t$$

where:

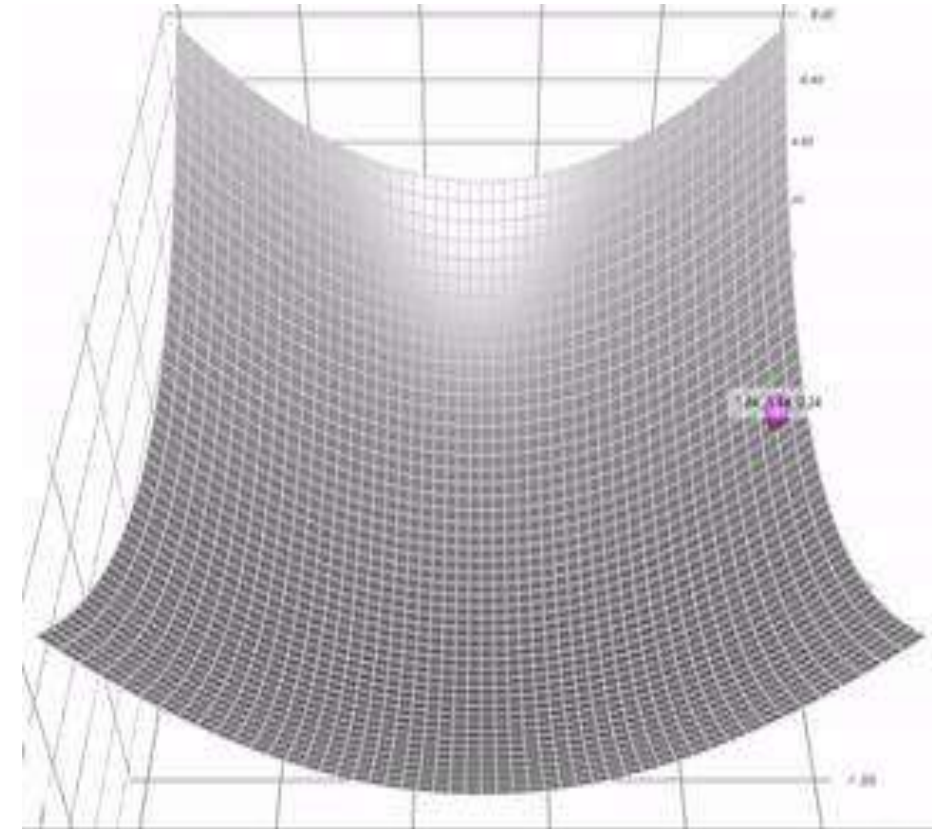
θ_t is the parameter vector at iteration t

α is the learning rate

$\nabla J(\theta_{t-1})$ is the gradient of the cost function J with respect to θ evaluated at θ_{t-1}

v_t is the velocity vector at iteration t

γ is the momentum coefficient, which controls the contribution of the previous velocity vector to the current velocity vector



Variants of gradient descent

5. Adagrad

- Adagrad is a variant of gradient descent that adapts the learning rate for each parameter based on its historical gradient values.
- Parameters with large gradients have their learning rates reduced, while parameters with small gradients have their learning rates increased. This helps to normalize the updates and can be useful in cases where the cost function has a lot of curvature or different scales of gradients.

$$g_t = \nabla J(\theta_t)$$

$$s_t = s_{t-1} + g_t^2$$

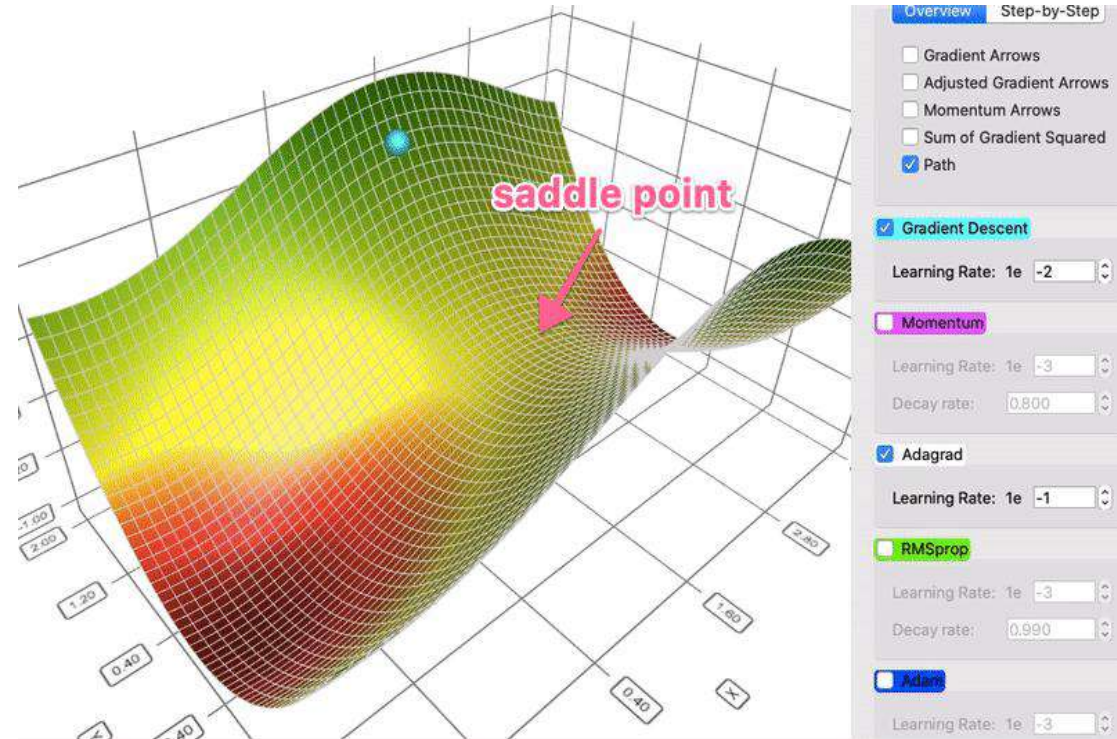
$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{s_t + \epsilon}} \cdot g_t$$

where:

θ_t is the parameter vector at iteration t

α is learning rate

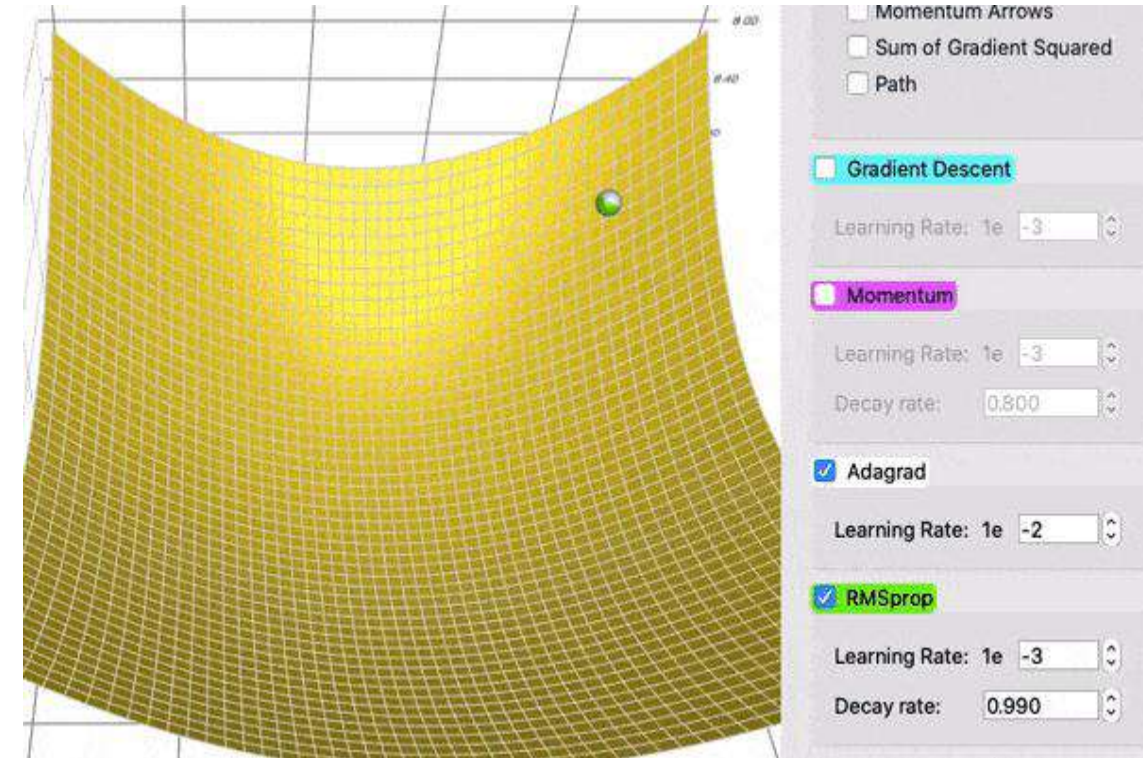
$\nabla J(\theta_t)$ is gradient of the cost function J with respect to θ evaluated at θ_t



Variants of gradient descent

6. RMSProp

- RMSProp is a variant of gradient descent that also *adapts the learning rate for each parameter, but instead of using the historical gradient values, it uses a moving average of the squared gradient values*. This helps to reduce the learning rate for parameters that have large squared gradient values, which can cause the algorithm to oscillate or diverge.



$$\text{sum_of_gradient_squared} = \text{previous_sum_of_gradient_squared} * \text{decay_rate} + \text{gradient}^2 * (1 - \text{decay_rate})$$

$$\text{delta} = -\text{learning_rate} * \text{gradient} / \text{sqrt}(\text{sum_of_gradient_squared})$$

$$\text{theta} += \text{delta}$$

Variants of gradient descent

7. Adam

- Adam, Adaptive Moment Estimation, is a variant of gradient descent that ***combines the ideas of Adagrad and RMSProp***.
- It adapts the learning rate for each parameter based on the historical gradient values and also uses a moving average of the gradient values to compute the momentum term.
- Adam is one of the most widely used optimization algorithms in deep learning because it is efficient, stable, and robust to different types of cost functions and datasets.

$$\text{sum_of_gradient} = \text{previous_sum_of_gradient} * \text{beta1} + \text{gradient} * (1 - \text{beta1})$$

[Momentum]

$$\text{sum_of_gradient_squared} = \text{previous_sum_of_gradient_squared} * \text{beta2} + \text{gradient}^2 * (1 - \text{beta2})$$

[RMSProp]

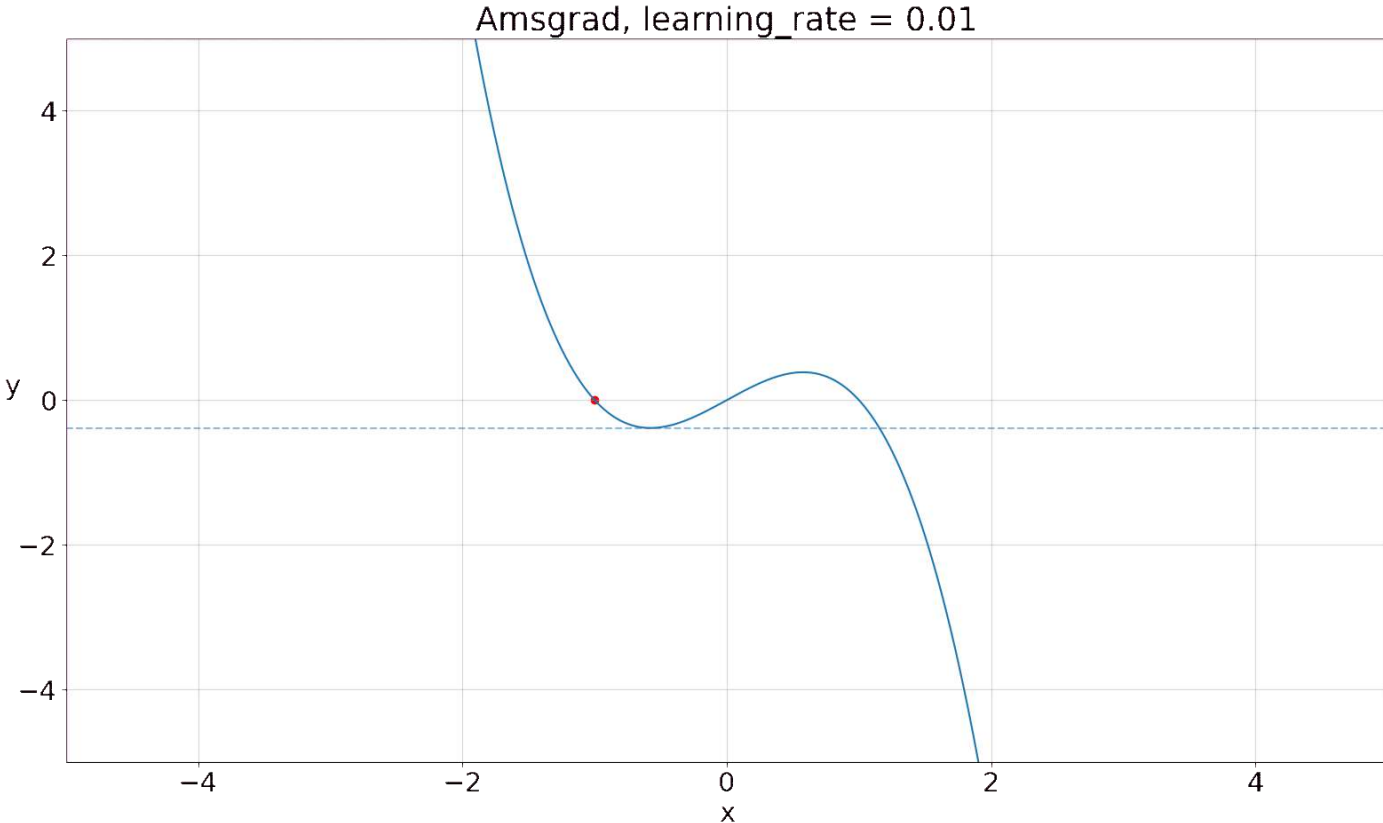
$$\text{delta} = -\text{learning_rate} * \text{sum_of_gradient} / \sqrt{\text{sum_of_gradient_squared}}$$

$$\text{theta} += \text{delta}$$

Variants of gradient descent

8. AMSGrad

- AMSGrad is an extension to the Adam version of gradient descent that attempts to improve the convergence properties of the algorithm, avoiding large abrupt changes in the learning rate for each input variable.



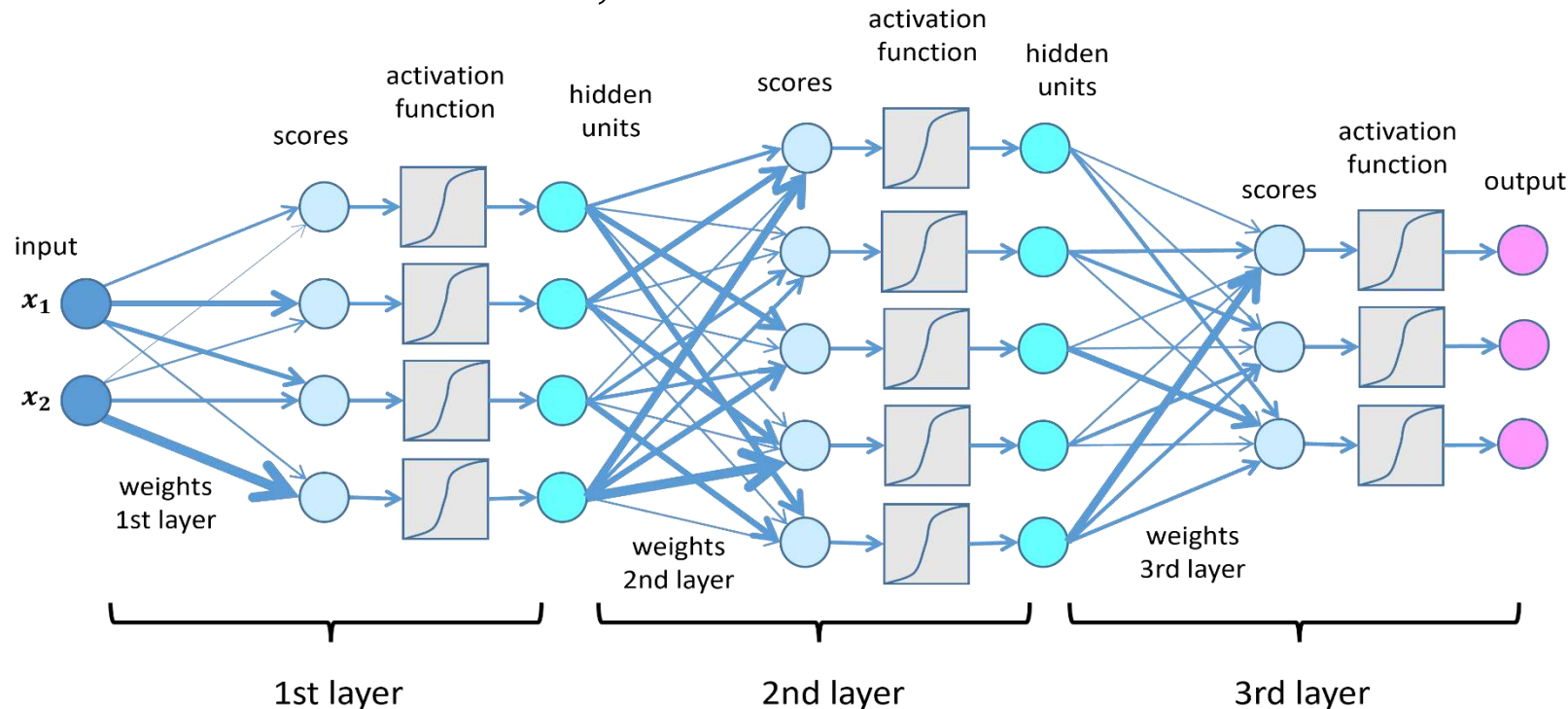
$$update = \widehat{learning_rate} * \frac{moment1}{\sqrt{\widehat{moment2} + epsilon}}$$

Variants of Gradient descent optimization algorithms

Variant	Advantages	Disadvantages
Batch gradient descent	Guaranteed convergence to global optimum	Computationally expensive for large datasets, slow convergence
Stochastic gradient descent	Faster convergence, more efficient for large datasets	High variance, may not converge to global optimum
Mini-batch gradient descent	Balanced convergence speed and computational cost, efficient for large datasets	Choice of mini-batch size can be a challenge
Momentum gradient descent	Faster convergence, less likely to get stuck in local minima	May overshoot and oscillate around the optimum
Adagrad	Adaptive learning rate, efficient for sparse data	Can stop learning too early
RMSProp	Adaptive learning rate, efficient for non-stationary problems	Can stop learning too early, requires tuning of hyperparameters
Adam	Adaptive learning rate, efficient for large datasets and noisy data	Can converge to suboptimal solutions, requires tuning of hyperparameters

What is a Deep Neural Network?

- **Deep Neural Network (DNN):** A neural network with multiple hidden layers between the input and output layers. The term “deep” refers to the number of layers in the network. A DNN typically has:
 - **Input Layer:** Takes in the raw data or features.
 - **Hidden Layers:** Layers of neurons that process the input data through learned weights and activation functions.
 - **Output Layer:** Produces the final output (e.g., class label, regression value).
- **Feedforward Network:** A type of DNN where data flows in one direction—from input to output—without any loops. In contrast to recurrent networks, feedforward networks do not have connections that cycle back.



Key Components of Deep Neural Network

a. Neuron/Node: The basic unit in a neural network.

- **Input:** Each neuron receives input, which is typically a weighted sum of the outputs from previous neurons.
- **Activation:** The weighted sum is passed through an activation function to produce the output.
- **Output:** The result is then forwarded to the next layer.

b. Layers:

- **Input Layer:** The first layer that receives the raw data (e.g., pixel values for an image, numerical features for tabular data).
- **Hidden Layers:** Layers in between the input and output. The network learns complex features and representations through these layers. The deeper the network, the more abstract the features become.
- **Output Layer:** The final layer that produces the network's output. For classification, it could output class probabilities (using **softmax**); for regression, it could output continuous values.

c. Weights and Biases:

- **Weights:** Each connection between two neurons has a weight that determines the strength of the connection.
- **Biases:** Each neuron has a bias term, which helps shift the activation function to the left or right, improving the model's ability to fit the data.

d. Activation Function:

Defines the output of a neuron based on its input. Common activation functions include:

- **Sigmoid:** Maps input to a value between 0 and 1.
- **Tanh (Hyperbolic Tangent):** Maps input to a value between -1 and 1.
- **ReLU (Rectified Linear Unit):** Outputs the input if it is positive; otherwise, it outputs zero. Most widely used in deep learning.
- **Leaky ReLU:** Similar to ReLU, but allows a small negative slope for negative values.
- **Softmax:** Used in the output layer for multi-class classification, converting the output into probabilities.

Training a Deep Neural Network

a. Forward Propagation: The process of passing the input through the network layer by layer to compute the final output. At each layer, the weighted sum of inputs is calculated, and an activation function is applied to determine the output of each neuron.

b. Loss Function (Objective Function): A function that measures the difference between the predicted output and the actual target (ground truth). Common loss functions: **Mean Squared Error (MSE):** Used for regression tasks, **Cross-Entropy Loss:** Used for classification tasks (e.g., categorical cross-entropy for multi-class classification).

c. Backpropagation: **Backpropagation** adjusts the weights and biases based on the error (loss) computed in the output layer, by propagating the error backward through the network. The gradient of the loss function is calculated with respect to each weight using the **chain rule of calculus**, allowing for the adjustment of weights and biases to minimize the error.

d. Gradient Descent: A technique for minimizing the loss function by adjusting the weights. **Gradient descent** updates weights in the direction opposite to the gradient of the loss function.

Variants of gradient descent include:

Batch Gradient Descent: Uses the entire dataset to compute the gradient in each step.

Stochastic Gradient Descent (SGD): Uses one data point at a time, leading to faster but noisier updates.

Mini-batch Gradient Descent: A compromise between batch and stochastic, using small batches of data.

Training a Deep Neural Network

e. Optimization Algorithms:

These improve the gradient descent process by adapting the learning rate and accelerating convergence:

- **Momentum:** Helps accelerate gradients vectors in the right directions, thus leading to faster converging.
- **Adam:** Combines momentum and adaptive learning rates, a popular choice for most deep learning tasks.
- **RMSprop:** Adjusts the learning rate based on the moving average of recent gradients.

f. Regularization Techniques: To prevent overfitting and improve the generalization ability of the network, various regularization techniques are used:

- **Dropout:** Randomly "drops" a certain percentage of neurons during training, forcing the network to learn redundant representations.
- **L2 Regularization (Weight Decay):** Adds a penalty to the loss function for large weights, encouraging smaller weights.
- **Early Stopping:** Stops training when the performance on a validation set starts to deteriorate, preventing overfitting.
- **Data Augmentation:** In tasks like image classification, augmenting the dataset with transformations (rotations, shifts, flips) can help prevent overfitting.

Forward and Back Propagation

- Forward and backward propagation are two essential processes in the training of neural networks. They are responsible for making predictions and adjusting the weights of the model to minimize errors.

Forward Propagation:

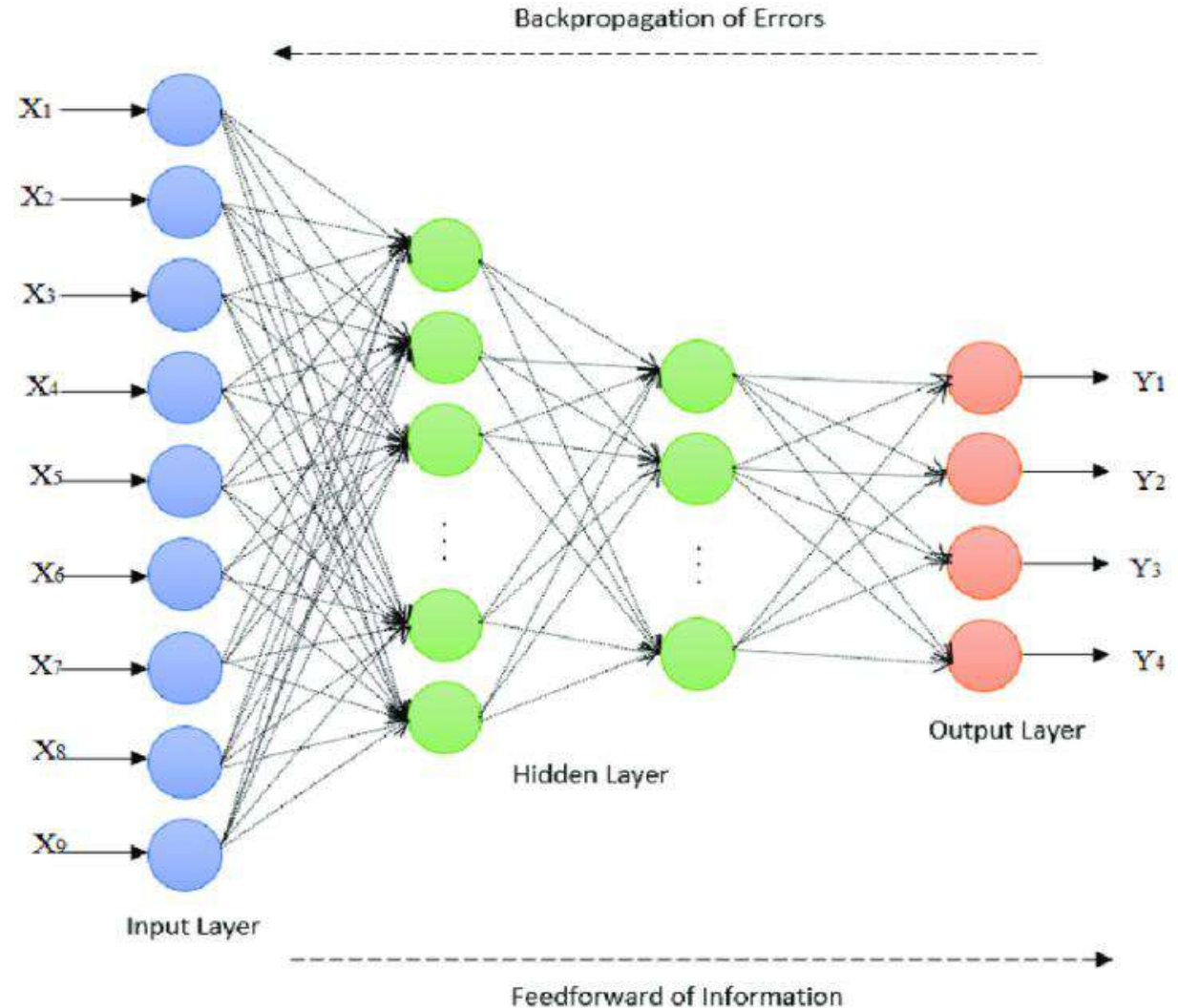
- Involves passing the input through the layers to get the output.
- Each layer computes a weighted sum of the inputs and applies an activation function.
- The output of the last layer is the prediction.

Backward Propagation:

- Involves calculating the loss between the predicted and true values.
- The gradients of the loss with respect to the weights are computed using the chain rule.
- The weights are updated to reduce the loss.

Together, these processes enable the neural network to learn from the data and improve its predictions over time.

Module-1; Topic-6: Forward and Back Propagation



Forward and Back Propagation

- **Forward Propagation:** Forward propagation is the process of passing the input data through the neural network, layer by layer, to obtain the output (predictions). It involves the following steps:
- **Input Layer:** The input data (features) are fed into the neural network. Each feature corresponds to a neuron in the input layer.
- **Weighted Sum:** The input data is multiplied by weights (parameters of the model) associated with each connection between neurons. Each neuron in the hidden layers computes a weighted sum of its inputs.

$$Z = W \cdot X + b$$

Where: W = Weights, X = Input values, b = Bias term

- **Activation Function:** The result of the weighted sum is passed through an activation function, such as ReLU, sigmoid, or tanh. This introduces non-linearity and helps the network to learn complex patterns.
- $A = \text{activation}(Z)$ Where A is the output of the neuron after applying the activation function.
- **Hidden Layers:** The output from the previous layer is passed as input to the next layer. This continues until the final output layer is reached.
- **Output Layer:** The final layer produces the output of the network, which could be a classification label (in the case of classification problems) or a continuous value (for regression).
- In forward propagation, the objective is simply to calculate the predictions of the neural network for given inputs. The output is then compared to the actual target values to compute the error.

Forward and Back Propagation

Backward Propagation: Backward propagation is the process of updating the weights of the network by propagating the error backward from the output layer to the input layer. It is key to minimizing the loss function and improving the model's performance.

1. **Calculate the Loss:** First, the loss (or error) is computed using a loss function (e.g., Mean Squared Error for regression, Cross-Entropy for classification). The loss measures how far the network's predictions are from the actual values.

$$\text{Loss} = \text{loss}(y, \hat{y})$$

Where:

- y = Actual target values
 - $\hat{y}$ = Predicted values from the output layer
2. **Gradient of the Loss Function:** The gradient of the loss with respect to each weight is calculated. This step computes how much each weight in the network contributed to the error.

$$\frac{\partial \text{Loss}}{\partial W}$$

This is done using the chain rule of calculus to propagate the gradient back through the network.

Forward and Back Propagation

3. **Update Weights:** The weights are updated using an optimization algorithm such as **Gradient Descent**. The weights are adjusted in the opposite direction of the gradient to reduce the loss.

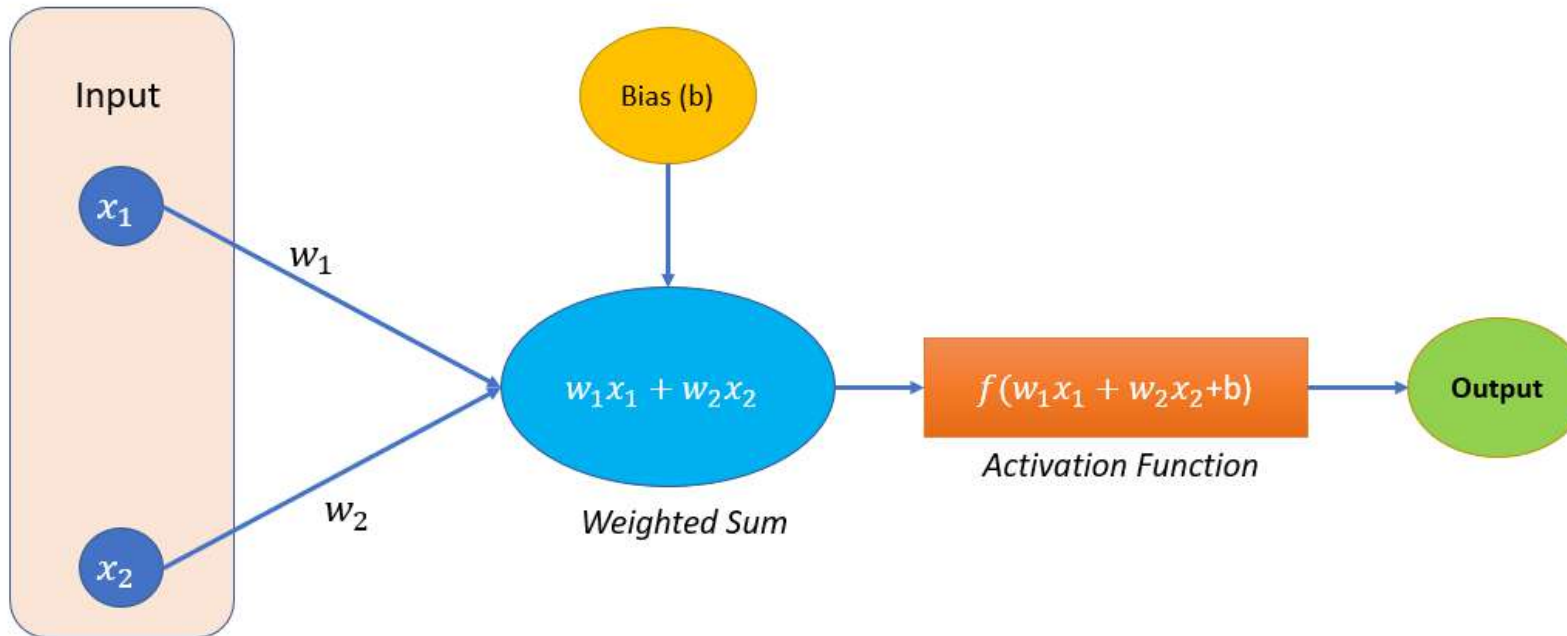
$$W_{\text{new}} = W_{\text{old}} - \alpha \cdot \frac{\partial \text{Loss}}{\partial W}$$

Where:

- α = Learning rate (controls how much the weights are updated)
 - $\frac{\partial \text{Loss}}{\partial W}$ = Gradient of the loss with respect to the weights
4. **Repeat:** The forward and backward propagation steps are repeated for each batch of data. Over time, the weights are adjusted to minimize the overall loss, leading to improved model performance.

Parameters

Parameters are the internal variables of a model that are learned during training. In the context of neural networks, parameters are the values that define the model's structure and behavior. These parameters are adjusted during the training process in order to minimize the loss and improve the model's predictions.



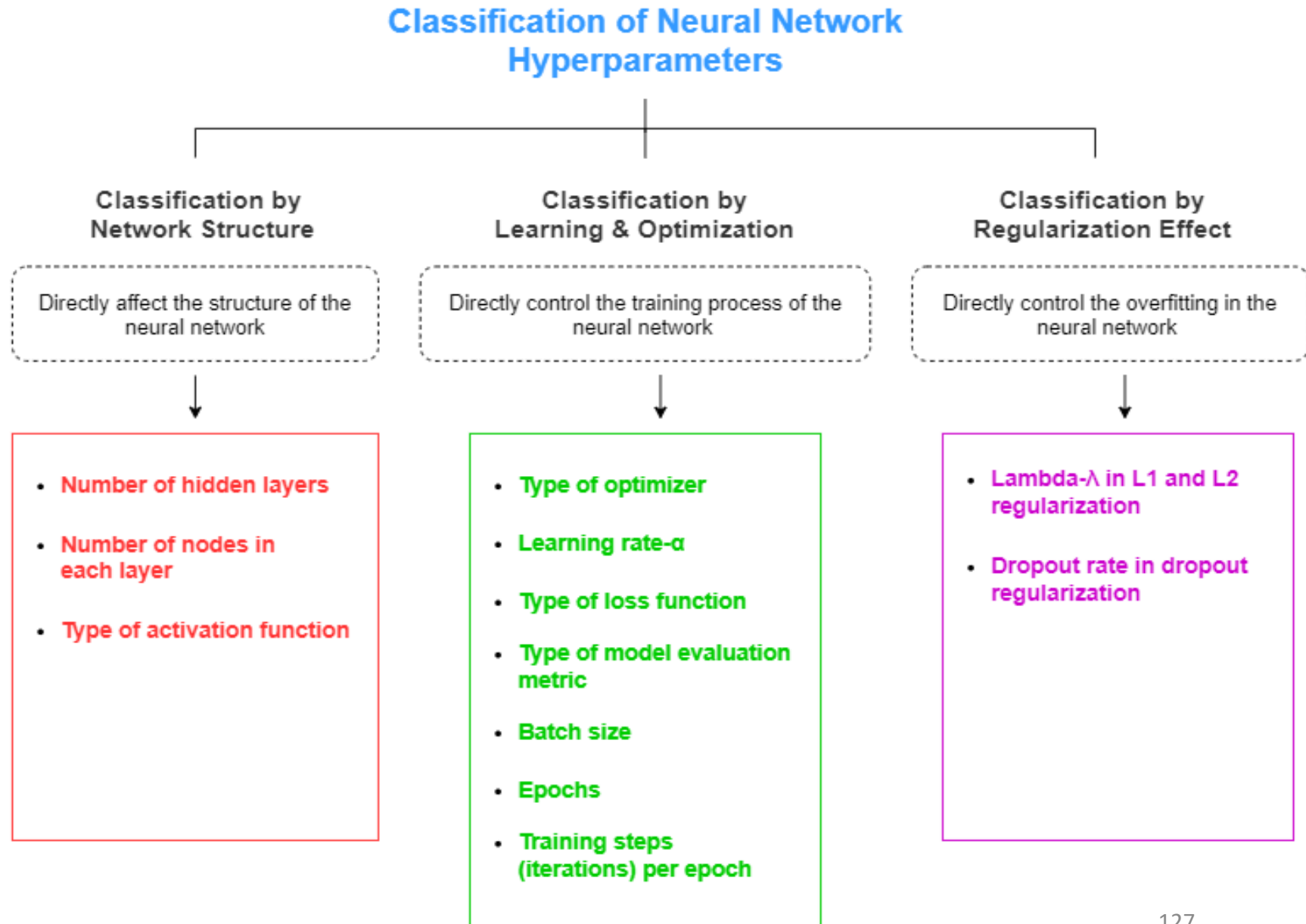
During training, the **parameters (weights and biases)** are adjusted through **backpropagation** and **gradient descent** in order to minimize the loss function.

Examples of parameters in neural networks:

- 1. Weights (W):** These are the values that connect the neurons across layers. Each connection between two neurons has a weight that determines the strength of the signal from one neuron to the next.
- 2. Bias (b):** These are additional parameters that are added to the weighted sum before applying the activation function. Biases help shift the activation function and allow the network to learn patterns that do not pass through the origin.
Bias allows the network to learn a broader range of data patterns, especially when the inputs are zero or very small.

Hyper parameters

- Hyperparameters are the external configuration settings that control the training process.
- Unlike parameters, hyperparameters are not learned from the data; instead, they are set before training and can have a significant impact on the performance of the model.
- These are often tuned manually or via automated processes (like grid search or random search).



Common hyperparameters in neural networks:

- **Learning Rate:** This determines how big a step the optimizer takes when updating the model's weights. A higher learning rate can speed up the convergence but risks overshooting the optimal solution, while a lower learning rate ensures smoother convergence but may take longer.
- **Number of Epochs:** An epoch represents one complete cycle through the entire dataset during training. The number of epochs defines how many times the model will learn from the full training data.
- **Batch Size:** This is the number of training examples utilized in one forward and backward pass. Larger batch sizes often speed up training but might reduce the accuracy of the model. Smaller batch sizes might lead to more accurate models but increase training time.
- **Number of Hidden Layers and Neurons:** In deep neural networks, you need to specify the architecture, i.e., how many hidden layers and how many neurons each hidden layer will contain. More layers and neurons generally allow the network to model more complex functions, but they can also lead to overfitting and increased computational cost.
- **Activation Function:** The choice of activation function, such as ReLU, sigmoid, or tanh, determines how the neurons' output is transformed before passing it to the next layer. It introduces non-linearity into the model, allowing it to learn more complex patterns.

Optimization Algorithm: This defines how the model updates its weights. Common choices include **Stochastic Gradient Descent (SGD)**, **Adam** or **RMSprop** based on how they adjust the learning rate and update weights to minimize the loss.

- **Regularization Parameters: L1 and L2 regularization:** These techniques penalize large weights to prevent overfitting.
- **Dropout Rate:** Dropout is a regularization method where a percentage of neurons are randomly dropped out during training, forcing the network to learn more robust features.
- **Momentum:** In optimization, momentum is a technique that helps accelerate gradients vectors in the right directions, thus leading to faster converging.
- **Weight Initialization:** How weights are initially set before training starts. Examples include **Random initialization** (Assigning random values to weights), **Xavier/Glorot initialization** (Designed to keep the variance of the inputs and outputs across layers in check), **He initialization** (Designed to work well with ReLU activation functions).
- **Choosing Hyperparameters:** Hyperparameters significantly affect the model's performance, but there is no one-size-fits-all set of values. Often, hyperparameter tuning requires experimentation and can be done through:
 - **Grid Search:** Systematically testing a set of hyperparameter values.
 - **Random Search:** Randomly selecting hyperparameters from a predefined range.
 - **Bayesian Optimization:** A probabilistic model that intelligently explores the space of hyperparameters.
 - **Cross-validation:** Evaluating the performance of different hyperparameter settings on a validation set.

Parameters Vs. Hyper parameters

Aspect	Parameters	Hyperparameters
Definition	Learned from data during training	Set before training begins
Role	Define the model's predictions	Control the training process
Examples	Weights, biases	Learning rate, number of layers, batch size
Optimization	Updated during training using backpropagation	Chosen before training and tuned using methods like grid search
Training Process	Directly adjusted through optimization algorithms	Not directly updated during training

Summary

Module-1; Summary

1. Neural Networks Basics: Neural networks (NNs) are a class of machine learning algorithms inspired by the human brain's structure. They are composed of layers of nodes (neurons) connected by weighted links. These models learn to map inputs to outputs by adjusting their internal parameters based on the data they are exposed to. NNs are used in a wide range of applications such as classification, regression, image recognition, natural language processing, and more.

2. Key Functions in Neural Networks:

- Neural networks use activation functions like sigmoid, tanh, ReLU, leaky ReLU, and softmax to introduce non-linearity into their models.
- Loss functions measure the difference between the model's predictions and actual target values, such as MSE for regression and Cross-Entropy Loss for classification.
- Neural networks are powerful function approximators, learning complex mappings between inputs and outputs by adjusting weights during training.

3. Classification and Clustering Problems: Classification: The task of predicting a discrete label (e.g., classifying images as cats or dogs). Neural networks solve classification problems by outputting class probabilities using activation functions like **softmax**.

- **Clustering:** A type of unsupervised learning where the goal is to group similar data points together. Neural networks can be used for clustering tasks through techniques like **Deep Embedded Clustering (DEC)** or **autoencoders**.

4. Deep Networks Basics Deep Neural Networks (DNNs) have multiple hidden layers between the input and output layers, allowing them to learn complex, hierarchical representations of data.

Components: A typical deep network includes:

- **Input Layer:** Receives the raw data (e.g., features or image pixels).
- **Hidden Layers:** Layers that process and learn features. Deeper networks tend to perform better for more complex tasks.
- **Output Layer:** Provides the final output of the network, such as a class label or predicted value.

Advantages: DNNs are highly flexible and can learn from large datasets, solving complex tasks like image recognition and speech understanding. Deep layers allow the model to capture abstract features (e.g., edges, shapes, textures in images).

5. Shallow Neural Networks: Shallow Neural Networks (SNNs) are simpler networks with only one hidden layer.

Pros: Faster to train, fewer parameters, easier to interpret, and computationally cheaper.

Cons: Limited in modeling complex data; may underperform on tasks requiring learning high-level abstractions (e.g., deep learning applications).

A basic **Single-Layer Perceptron (SLP)** is an example of a shallow network. While SLPs can handle linearly separable data, they struggle with more complex problems like the **XOR problem**.

6. Gradient Descent: **Gradient Descent** is the optimization algorithm used to minimize the loss function by adjusting the weights in the network. It involves:

- **Forward Propagation:** Calculating the output of the network given the input.
- **Loss Calculation:** Evaluating how far the predicted output is from the true label.
- **Backward Propagation:** Calculating gradients and updating weights accordingly.

Variants of Gradient Descent:

- **Batch Gradient Descent:** Uses the entire dataset for each update.
- **Stochastic Gradient Descent (SGD):** Uses one data point at a time, which can be faster but more noisy.
- **Mini-Batch Gradient Descent:** A compromise, using a small subset of the dataset for each update.
- **Adam:** A popular optimizer that combines momentum and adaptive learning rates.

7. Deep Neural Networks – Forward and Backpropagation

a. Forward Propagation: In forward propagation, the input data is passed through each layer of the network, undergoing a series of weighted sums followed by activation functions. The output of each layer serves as the input for the subsequent layer, eventually producing a prediction at the output layer.

b. Backpropagation: Backpropagation is used to update the weights of the network. After forward propagation and loss calculation, backpropagation computes the **gradients of the loss** with respect to each weight in the network using the **chain rule** of calculus. The gradients indicate how to change the weights to minimize the loss function. The weights are then updated in the direction that reduces the error.

6. Gradient Descent: **Gradient Descent** is the optimization algorithm used to minimize the loss function by adjusting the weights in the network. It involves:

- **Forward Propagation:** Calculating the output of the network given the input.
- **Loss Calculation:** Evaluating how far the predicted output is from the true label.
- **Backward Propagation:** Calculating gradients and updating weights accordingly.

Variants of Gradient Descent:

- **Batch Gradient Descent:** Uses the entire dataset for each update.
- **Stochastic Gradient Descent (SGD):** Uses one data point at a time, which can be faster but more noisy.
- **Mini-Batch Gradient Descent:** A compromise, using a small subset of the dataset for each update.
- **Adam:** A popular optimizer that combines momentum and adaptive learning rates.

7. Deep Neural Networks – Forward and Backpropagation

a. Forward Propagation: In forward propagation, the input data is passed through each layer of the network, undergoing a series of weighted sums followed by activation functions. The output of each layer serves as the input for the subsequent layer, eventually producing a prediction at the output layer.

b. Backpropagation: Backpropagation is used to update the weights of the network. After forward propagation and loss calculation, backpropagation computes the **gradients of the loss** with respect to each weight in the network using the **chain rule** of calculus. The gradients indicate how to change the weights to minimize the loss function. The weights are then updated in the direction that reduces the error.

8. Parameters are internal variables of a model that are learned directly from the training data. They adjust during the training process to minimize the model's error. For example, in a neural network, the weights and biases are parameters.

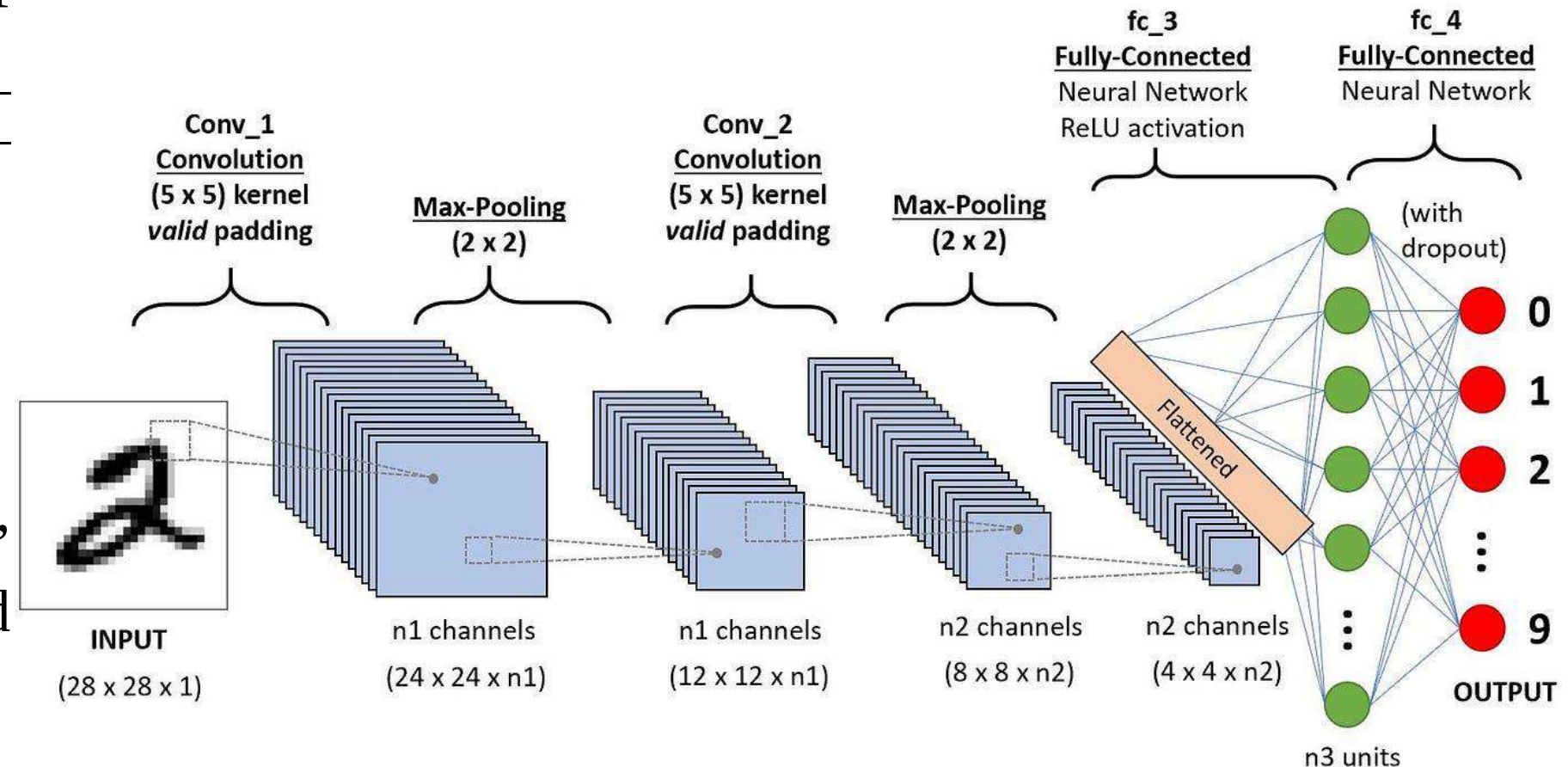
- **Hyperparameters**, on the other hand, are external settings that govern the model's training process. They are set before training and remain fixed throughout. Hyperparameters include values like learning rate, batch size, number of layers, and epochs. These are not learned from the data but are crucial for controlling model performance and training efficiency.
- **parameters** define the model's behavior and are optimized during training, while **hyperparameters** influence the training process and model configuration, but are manually set before training.

9. Neural networks are powerful tools for solving a wide range of problems, from classification to regression, and even more complex tasks like image and speech recognition. Understanding the basics—such as **activation functions**, **loss functions**, **forward/backpropagation**, and the differences between **shallow** and **deep networks**—is essential for building and training effective models.

- **Shallow neural networks** are simpler and computationally efficient but may underperform on complex tasks.
- **Deep neural networks** can learn more complex representations but require more data and computational power.
- The choice of hyperparameters and optimization techniques like **gradient descent** is key to building a successful neural network. Regularization methods, such as **dropout**, and the use of advanced optimizers (e.g., **Adam**) can significantly improve performance.

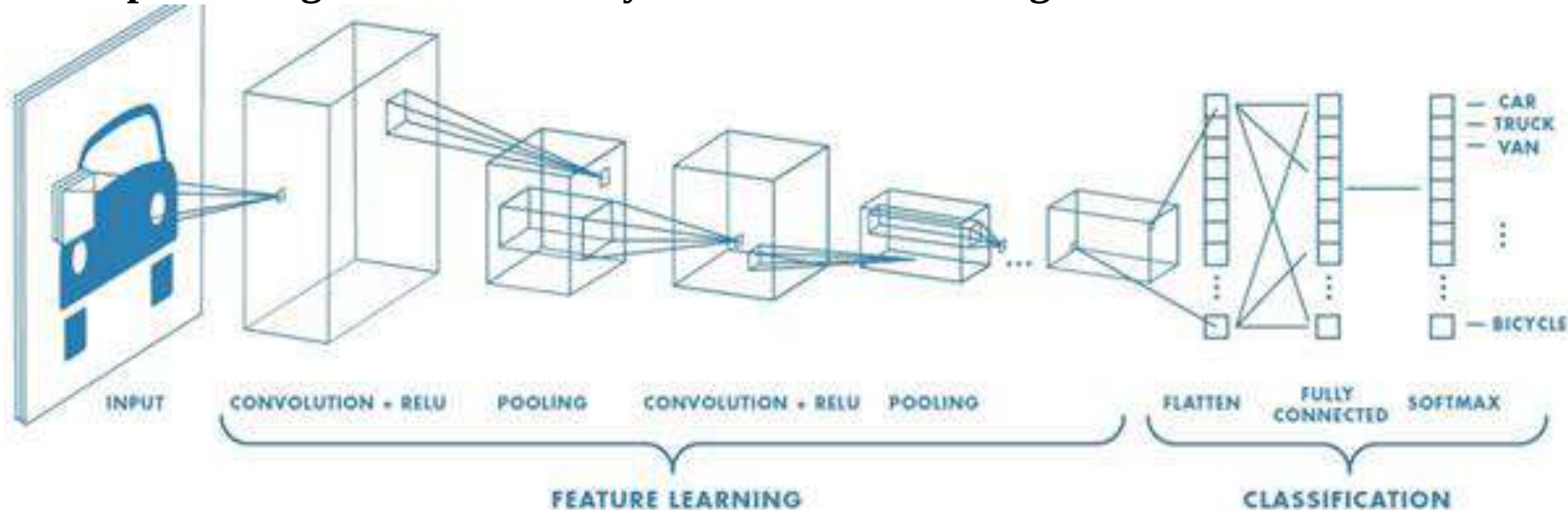
Module-2-Convolution neural networks

- Foundations of Convolutional Neural Networks – CNN operations – Architecture- Simple Convolution Network
- Deep Convolutional Models – ResNet, AlexNet, InceptionNet and others



Introduction to Convolution Neural Networks (CNN)

- A convolutional neural network (**CNN**, or **ConvNet**) is a class of deep learning, **designed for processing structured arrays of data such as images.**

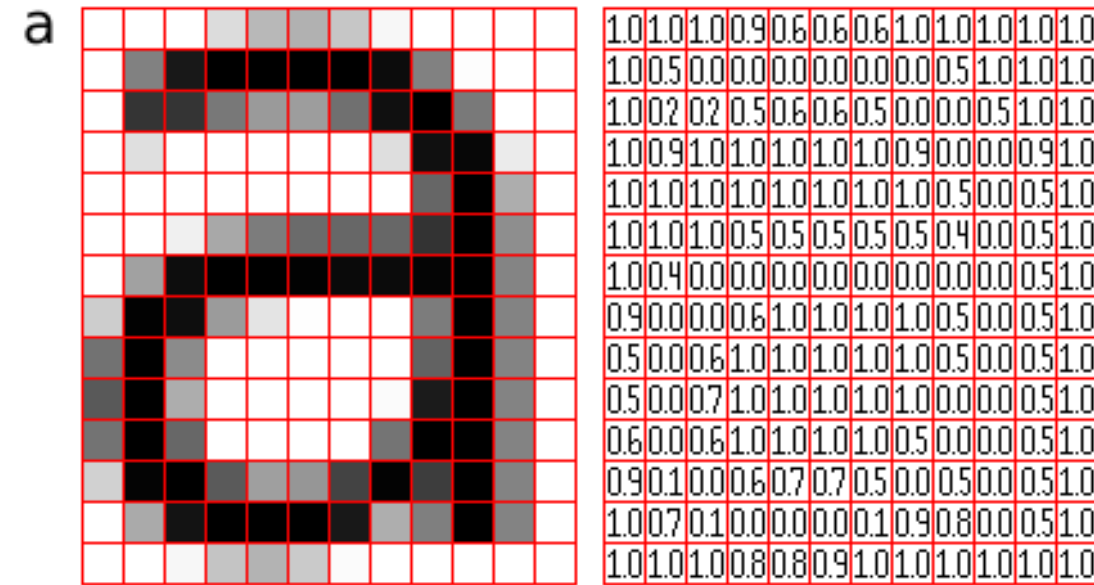


- A feed-forward artificial neural networks that explicitly assumes that the inputs are images, which allows us to encode certain properties into the architecture.
- CNN is a multi-layered neural network with a unique architecture designed to extract increasingly complex features of the data at each layer to determine the output. CNN's are well suited for perceptual tasks.

Convolution Neural Networks (CNN)

Module-2; Topic-1: Foundations of Convolutional Neural Networks – CNN operations – Architecture

- A neural network consisting of multiple convolutional layers used for image processing, classification, segmentation and other correlated data.
- Convolution is a mathematical function of linear operation in which multiplication of two functions create a third function. The third function is expressed as the change in the one function's shape changes others.
- Two images that are represented in the form of two matrices are multiplied to provide an output that **extract information from the image**.
- **Goal:** To reduce the images that would be easier to processing without losing features that are valuable for accurate prediction.



- CNN works for a **labelled Multidimensional array**.
- Instead of preprocessing the **data to derive features like textures, shapes**, CNN takes **image raw pixel data as input** and “Learns” how to **extract features and infer the object**.

Convolution neural networks (CNN)

Module-2; Topic-1: Foundations of Convolutional Neural Networks – CNN operations – Architecture

- A CNN is made of numerous layers such as convolution layers, pooling layers, and fully connected layers.
- Uses a back propagation algorithm to learn spatial hierarchies of data automatically and adaptively.

Advantages

- **Automatically detects the important features without any human supervision.**
- **Computationally efficient with high accuracy**
- **Weight Sharing** –Making use of the same knowledge across all image locations.

Disadvantages

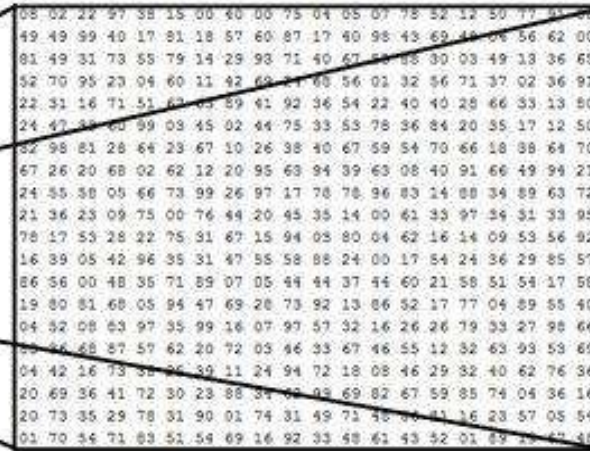
- Requires lot of training data
- Requires sophisticated computational facilities.

Applications

- Image classification
- Object Detection
- Voice Synthesis

Convolution neural networks (CNN)

Module-2; Topic-1: Foundations of Convolutional Neural Networks – CNN operations – Architecture



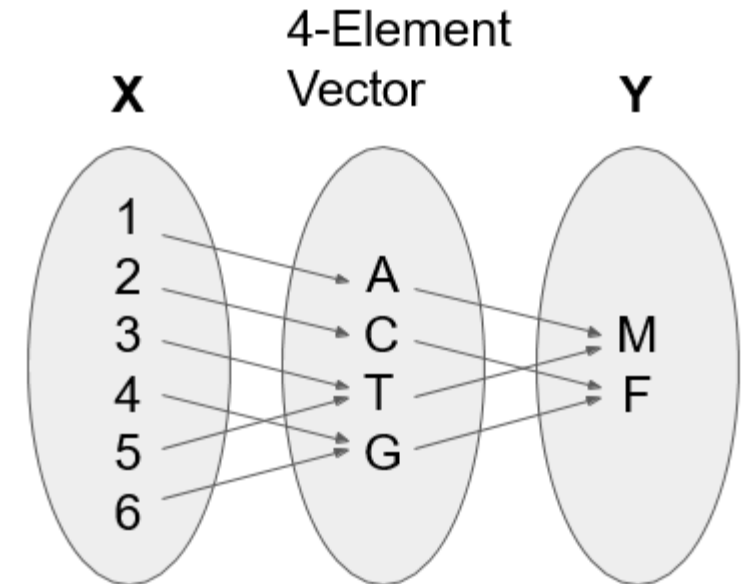
What the computer sees

image classification →
82% cat
15% dog
2% hat
1% mug

256 X 256
Matrix

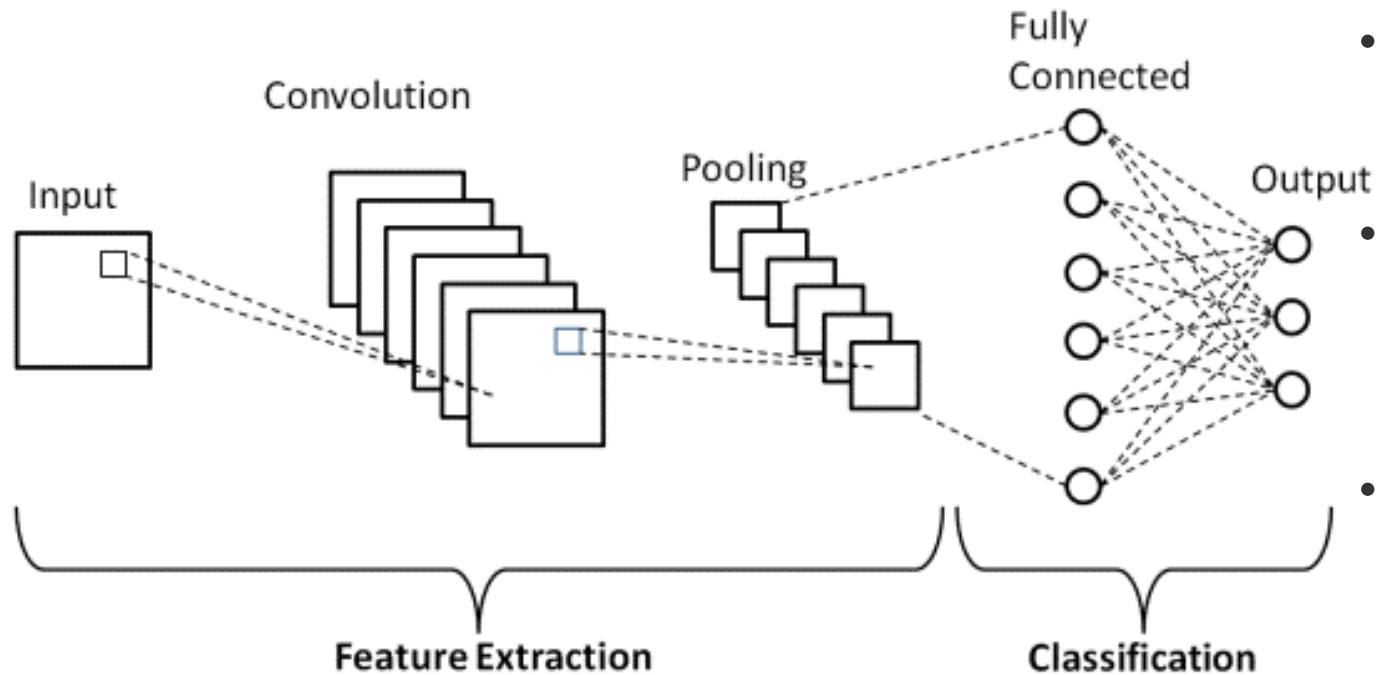
DL model

- As soon as we see an image, Brain starts to categorizing it based on color and shape.
- Machine does a rigorous training of interpreting a unique pattern of array of pixels-includes objects and information about the image.



With deep learning, we are searching for a **surjective** (or **onto**) function **f** from a set **X** to a set **Y**.

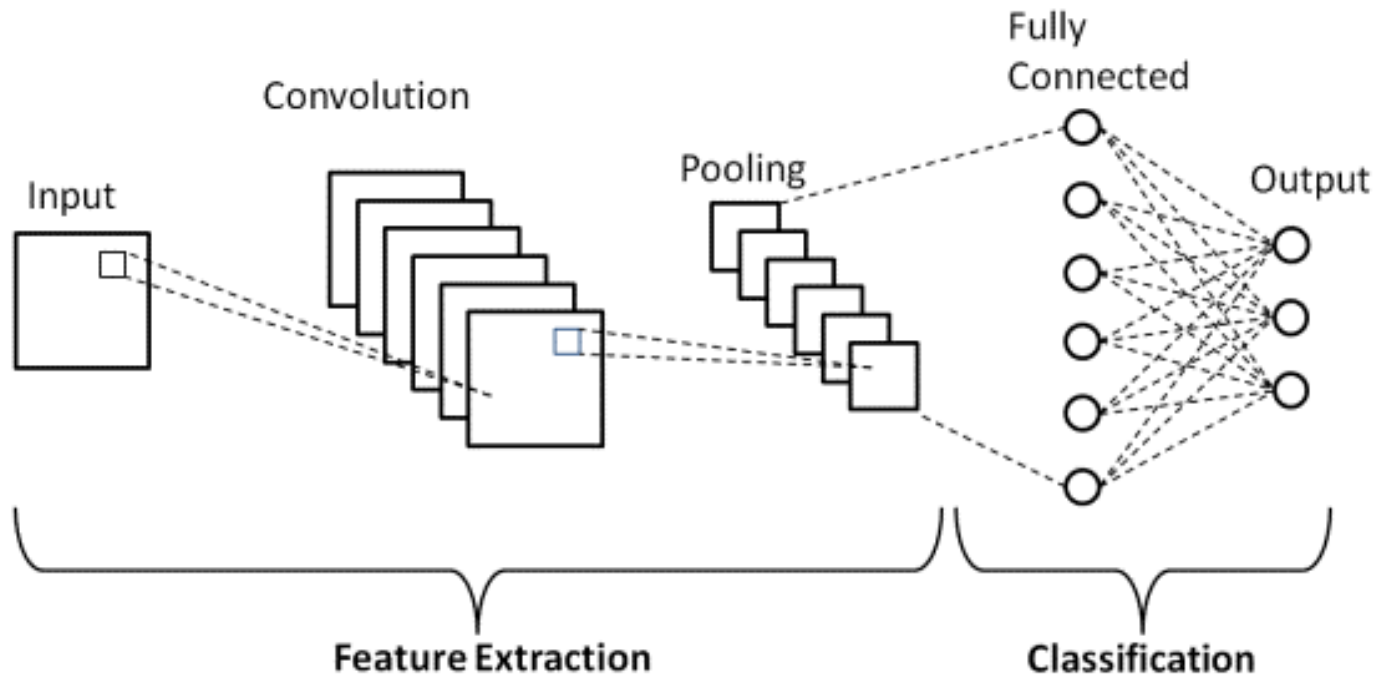
Basic structure of CNN



Feature Map = Input Image x Feature Detector

- The convolution layer in CNN passes the result to the next layer once applying the convolution operation in the input. Convolutional layers in CNN benefit a lot as they ensure **the spatial relationship between the pixels is intact**.
- **Convolution layer:** This layer is the first layer that is used to extract the various features from the input images.
- In this layer, the mathematical operation of convolution is performed between the input image and a filter of a particular size $M \times M$.
- By sliding the filter over the input image, the dot product is taken between the filter and the parts of the input image with respect to the size of the filter ($M \times M$).
- The output is termed as the Feature map which gives us information about the image such as the corners and edges.
- Later, this feature map is fed to other layers to learn several other features of the input image.

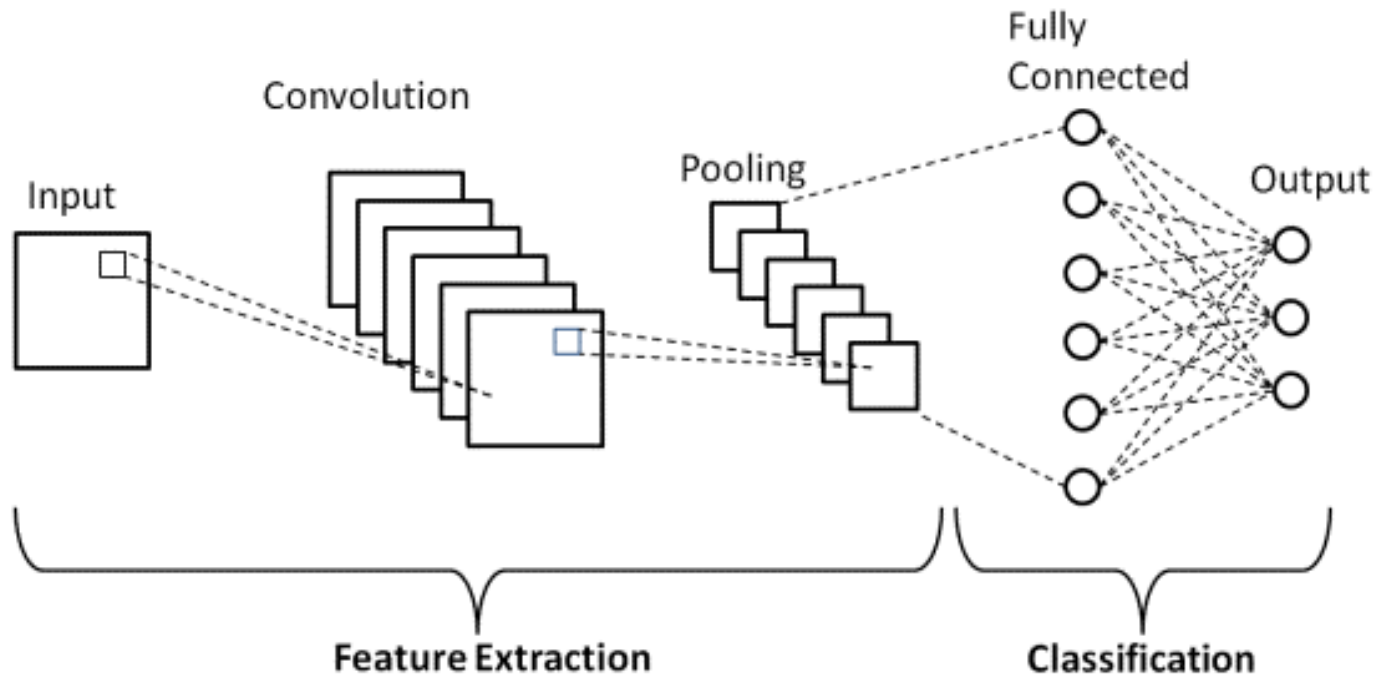
Basic structure of CNN



- In simple terms, activation functions in a CNN model determine whether a neuron should be activated or not.
- It decides whether the input to the work is important or not to predict using mathematical operations.

- **Activation Layer:** Activation layer used to learn and approximate any kind of continuous and complex relationship between variables of the network.
- It adds non-linearity to the network.
- There are several commonly used activation functions such as the ReLU, Softmax, tanH and the Sigmoid functions.
- Each of these functions have a specific usage. For a binary classification CNN model, sigmoid and softmax functions are preferred and for a multi-class classification, generally softmax is used.

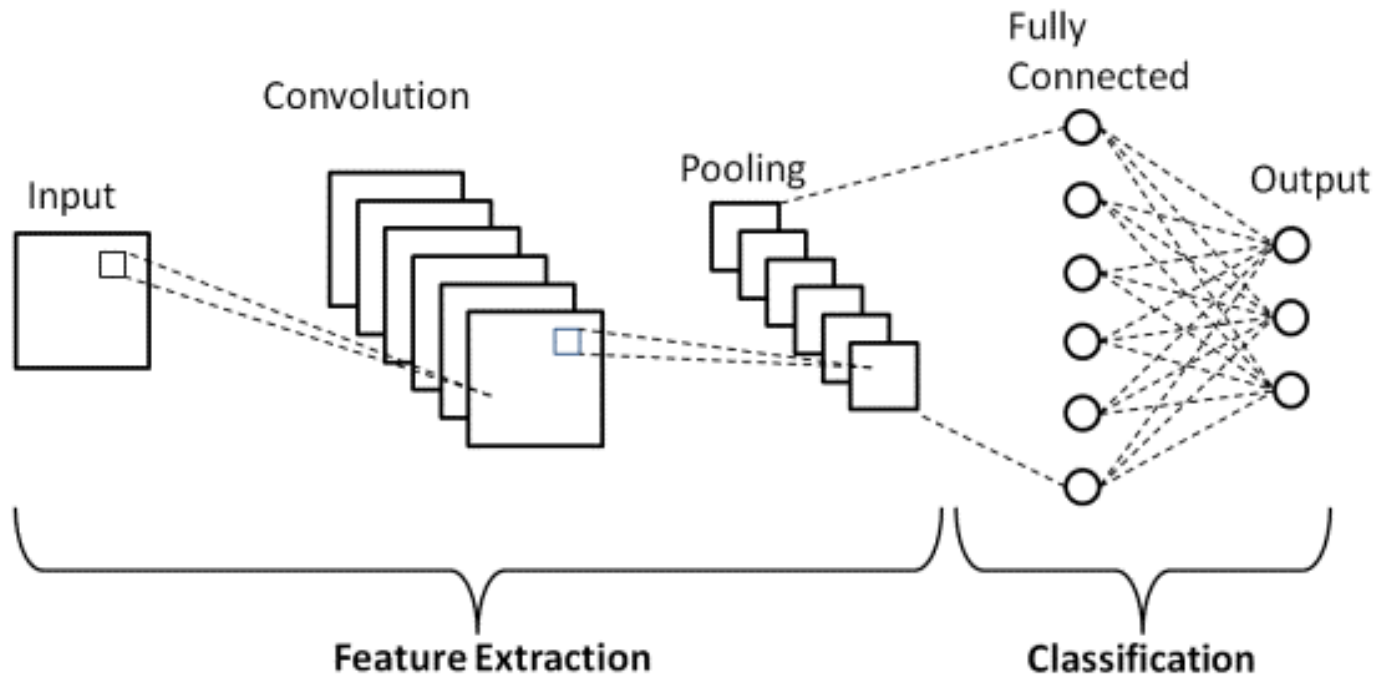
Basic structure of CNN



- In Max Pooling, the largest element is taken from feature map. Average Pooling calculates the average of the elements in a predefined sized Image section. The total sum of the elements in the predefined section is computed in Sum Pooling. The Pooling Layer usually serves as a bridge between the Convolutional Layer and the FC Layer.

- **Pooling Layer:** In most cases, a Convolutional Layer is followed by a Pooling Layer.
- The primary aim of this layer is to decrease the size of the convolved feature map to reduce the computational costs.
- **This is performed by decreasing the connections between layers and independently operates on each feature map.**
- Depending upon method used, there are several types of Pooling operations. It basically summarises the features generated by a convolution layer.
- This CNN model generalises the features extracted by the convolution layer, and helps the networks to recognise the features independently.

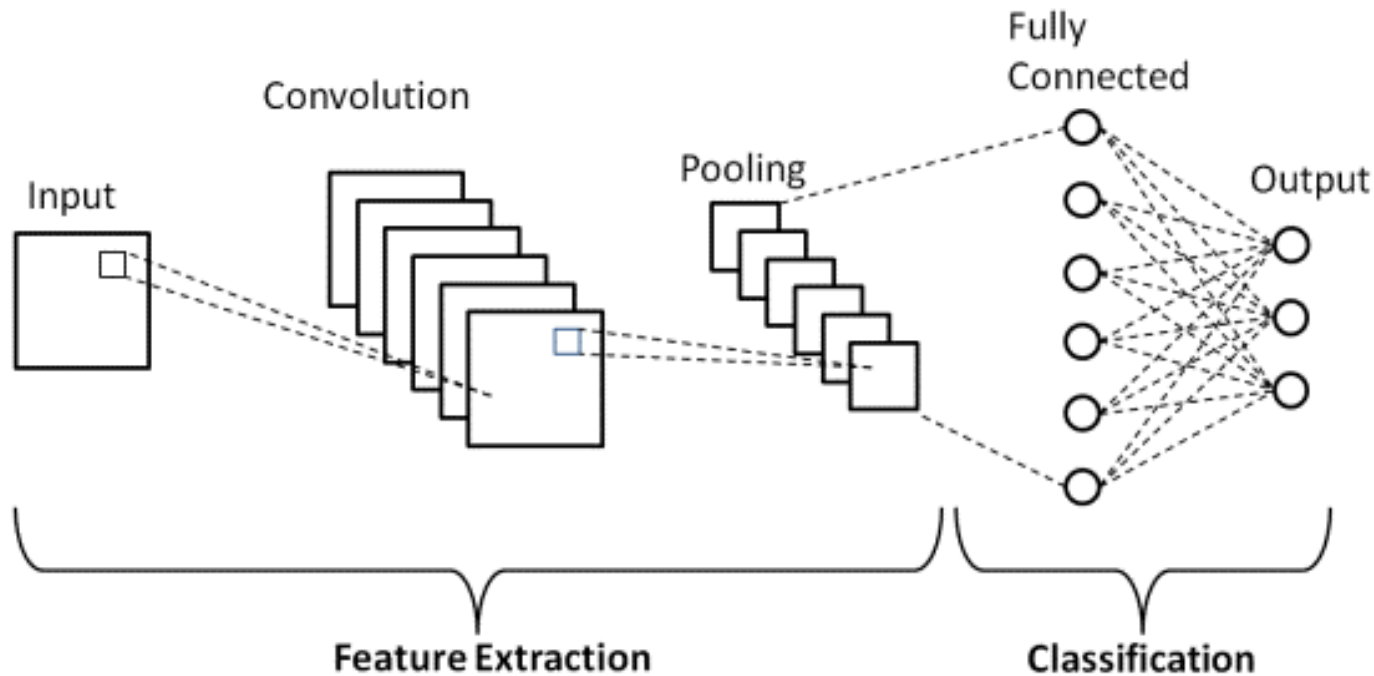
Basic structure of CNN



- Dropout results in improving the performance of a machine learning model as it prevents overfitting by making the network simpler. It drops neurons from the neural networks during training.

- **Drop out layers:** Usually, when all the features are connected to the FC layer, it can cause overfitting in the training dataset.
- Overfitting occurs when a particular model works so well on the training data causing a negative impact in the model's performance when used on a new data.
- To overcome this problem, a dropout layer is utilised wherein a few neurons are dropped from the neural network during training process resulting in reduced size of the model. On passing a dropout of 0.3, 30% of the nodes are dropped out randomly from the neural network.

Basic structure of CNN

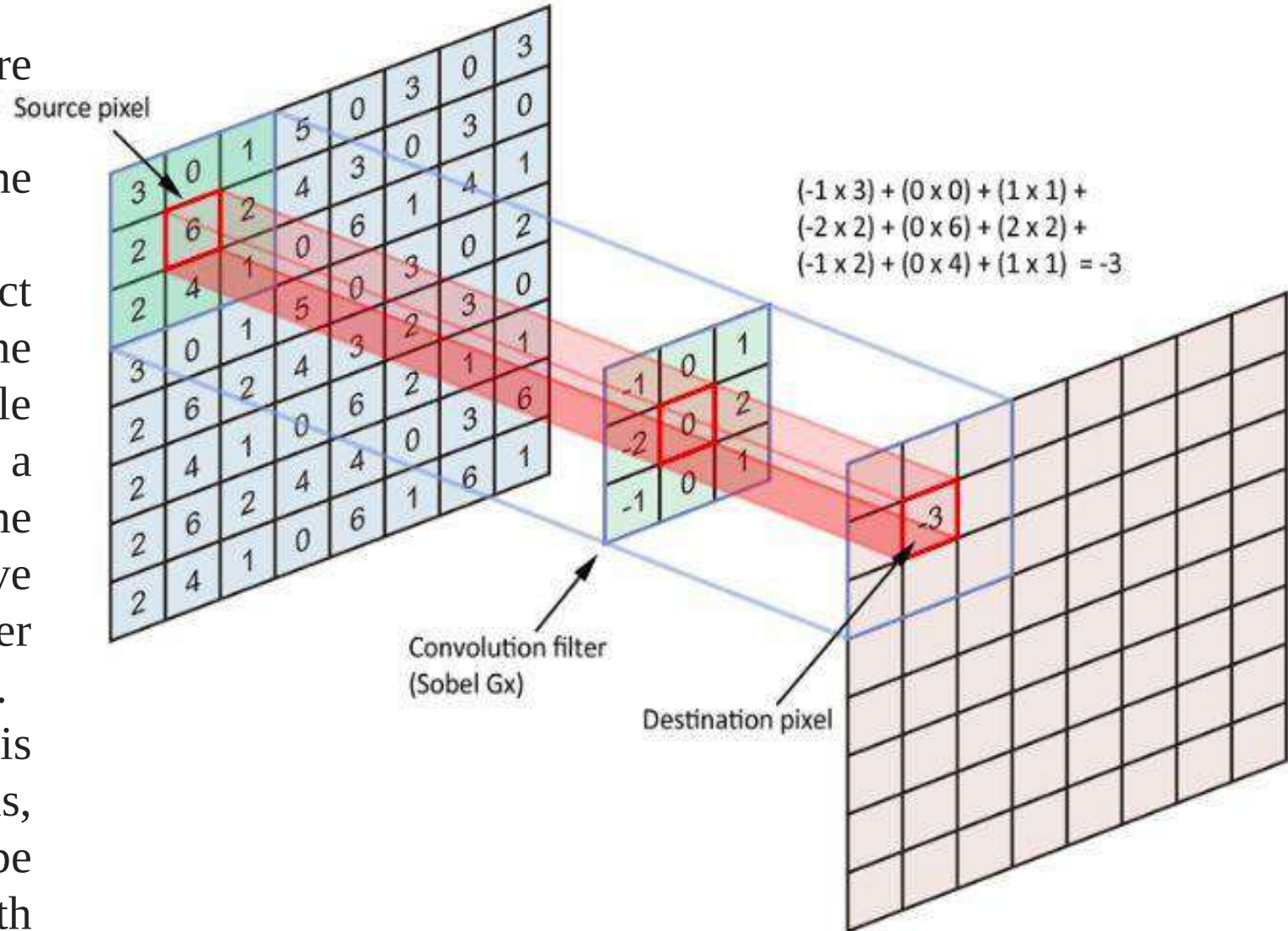


- In this stage, the classification process begins to take place. The reason two layers are connected is that two fully connected layers will perform better than a single connected layer. These layers in CNN reduce the human supervision.

- **Fully connected layer:** The Fully Connected (FC) layer consists of the weights and biases along with the neurons and is used to connect the neurons between two different layers.
- These layers are usually placed before the output layer and form the last few layers of a CNN Architecture.
- In this, the input image from the previous layers are flattened and fed to the FC layer. The flattened vector then undergoes few more FC layers where the mathematical functions operations usually take place.

Convolution in 2D

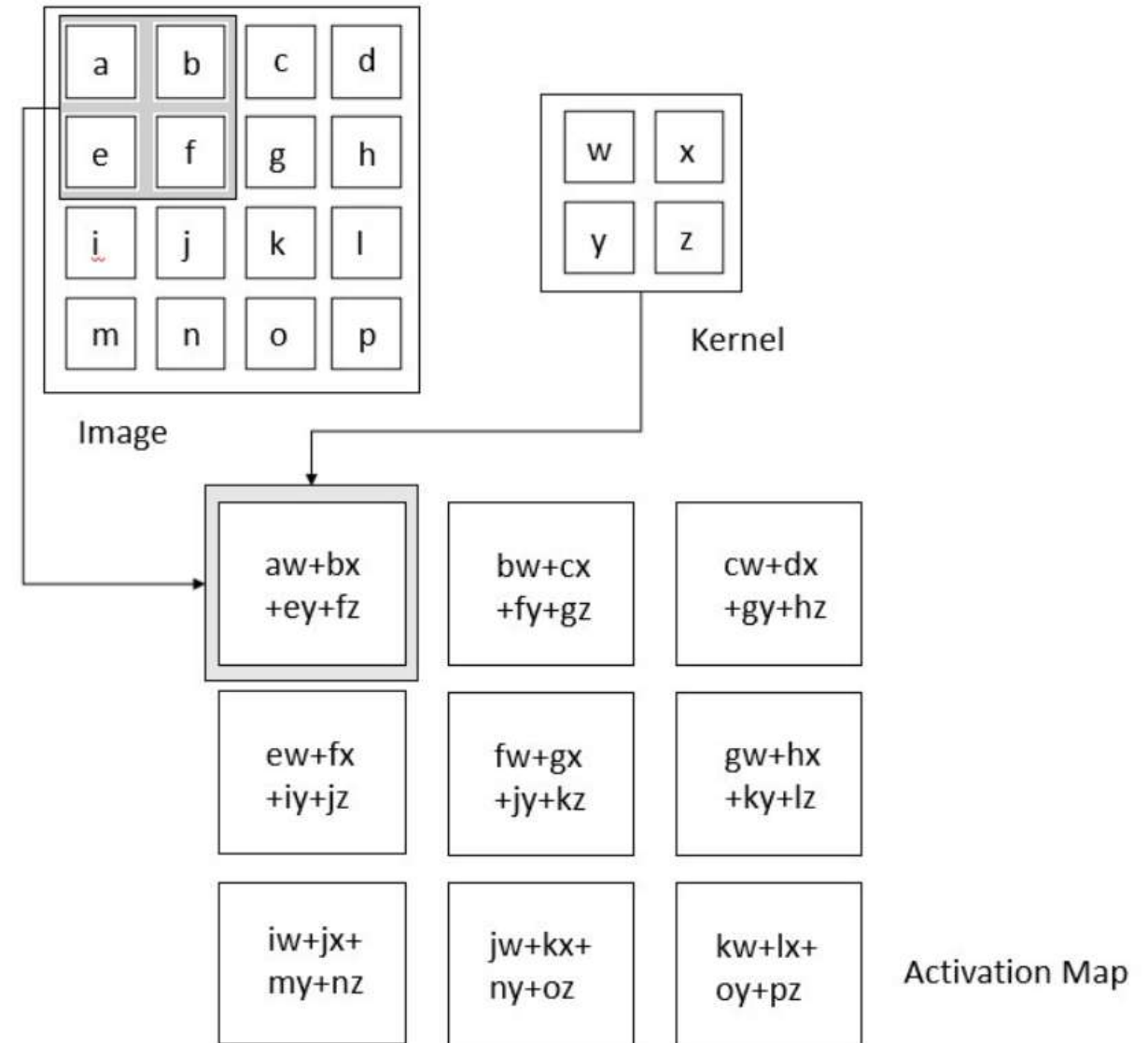
- The convolution layer is the core building block of the CNN.
- It carries the main portion of the network's computational load.
- This layer performs a dot product between two matrices, where one matrix is the set of learnable parameters otherwise known as a kernel, and the other matrix is the restricted portion of the receptive field. The kernel is spatially smaller than an image but is more in-depth.
- This means that, if the image is composed of three (RGB) channels, the kernel height and width will be spatially small, but the depth extends up to all three channels.



(Image Credit: [Applied Deep Learning | Arden Dertat](#))

Convolution Kernel

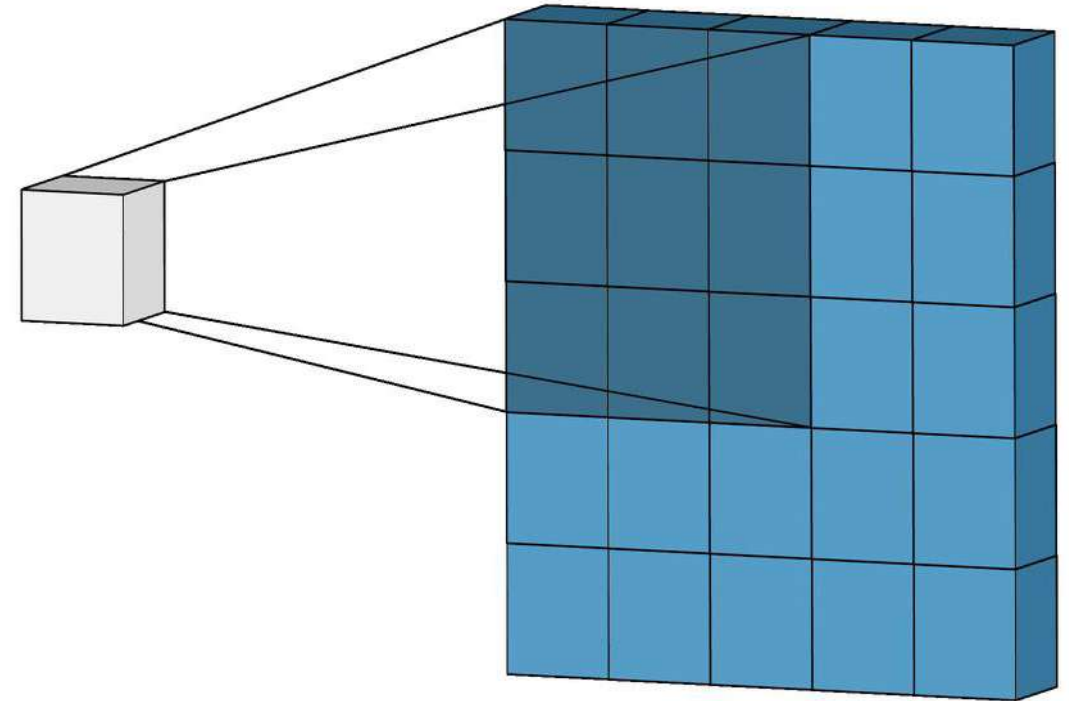
- Trivial neural network layers use matrix multiplication by a matrix of parameters describing the interaction between the input and output unit. This means that every output unit interacts with every input unit. However, convolution neural networks have *sparse interaction*.
- This is achieved by making kernel smaller than the input e.g., an image can have millions or thousands of pixels, but while processing it using kernel we can detect meaningful information that is of tens or hundreds of pixels.
- This means that we need to store fewer parameters that not only reduces the memory requirement of the model but also improves the statistical efficiency of the model.



(Image Credit: <https://towardsdatascience.com/convolutional-neural-networks>)

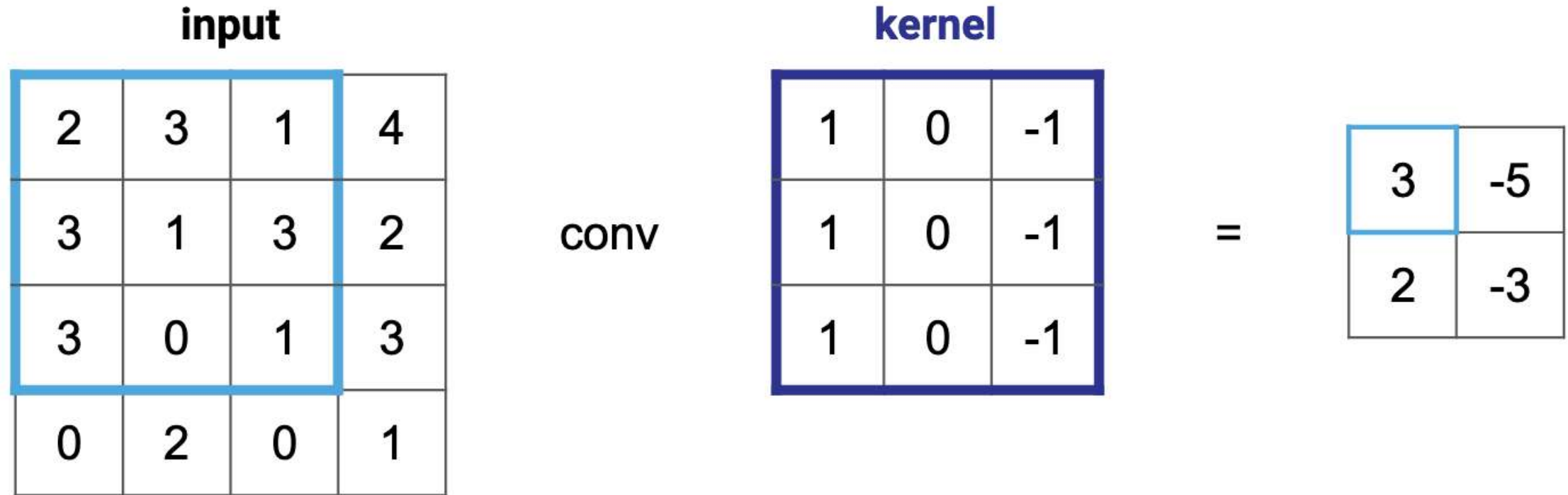
Convolution Kernel

- If computing one feature at a spatial point (x_1, y_1) is useful then it should also be useful at some other spatial point say (x_2, y_2) . It means that for a single two-dimensional slice i.e., for creating one activation map, neurons are constrained to use the same set of weights. In a traditional neural network, each element of the weight matrix is used once and then never revisited, while convolution network has *shared parameters* i.e., for getting output, weights applied to one input are the same as the weight applied elsewhere.
- Due to parameter sharing, the layers of convolution neural network will have a property of *equivariance to translation*. It says that if we changed the input in a way, the output will also get changed in the same way.



(Image Credit: <https://towardsdatascience.com/convolutional-neural-networks>)

Convolution Kernel

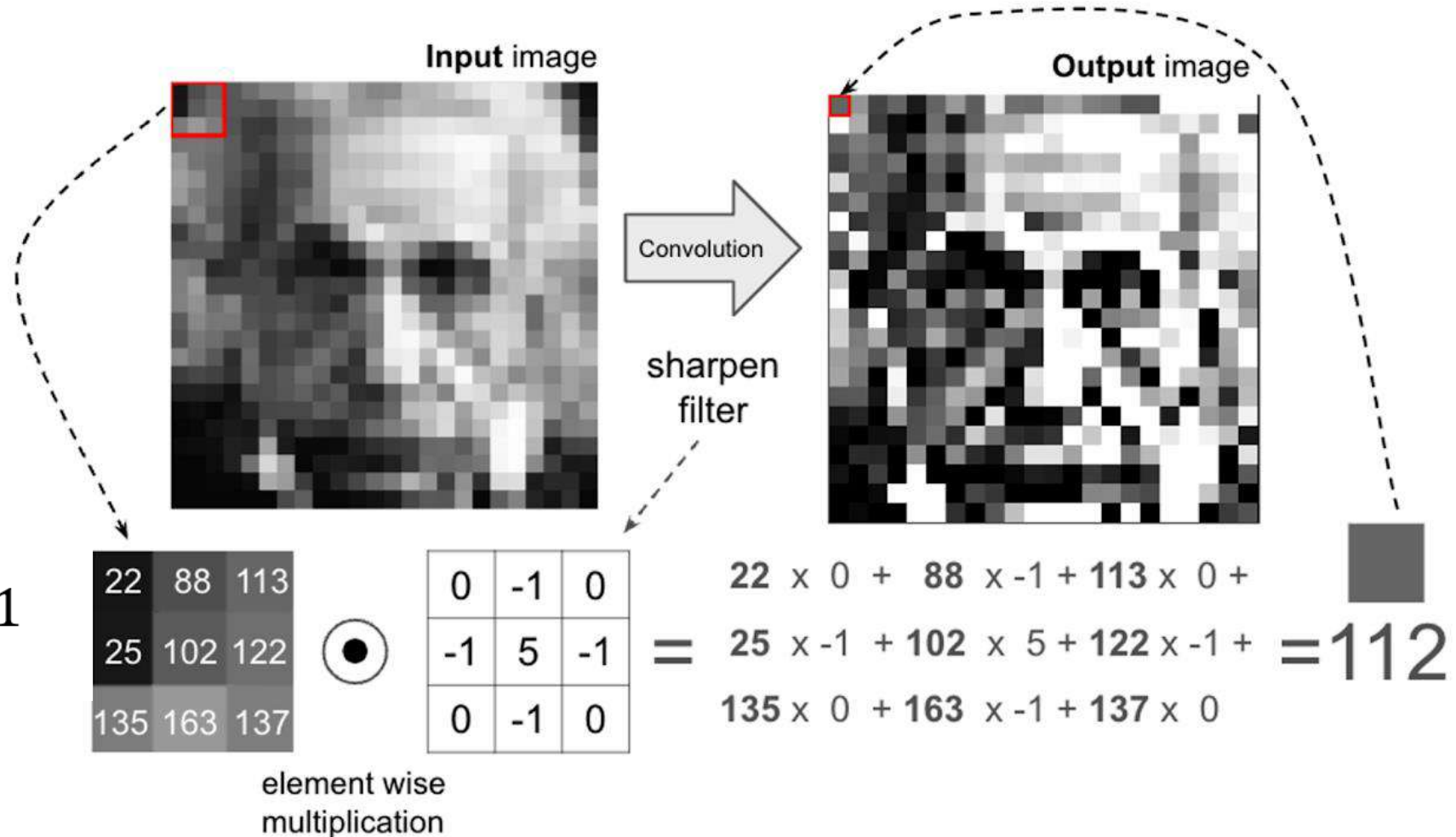


- During the forward pass, the kernel slides across the height and width of the image-producing the image representation of that receptive region.
- This produces a two-dimensional representation of the image known as an activation map that gives the response of the kernel at each spatial position of the image.
- The sliding size of the kernel is called a stride.

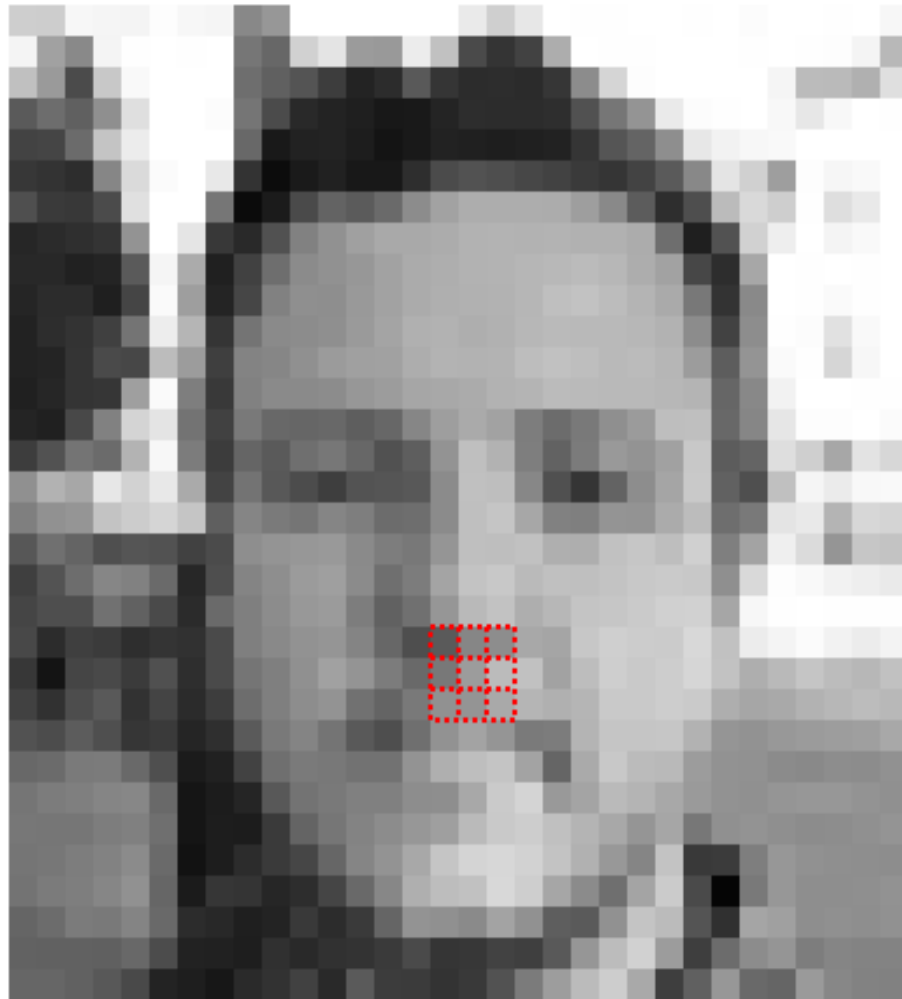
Convolution Kernel

If we have an input of size $W \times W \times D$ and D_{out} number of kernels with a spatial size of F with stride S and amount of padding P , then the size of output volume can be determined by the following formula:

$$W_{out} = \frac{W - F + 2P}{S} + 1$$



Convolution on Image



input image

$$\left(\begin{array}{ccc} 90 & + & 134 & + & 141 \\ \times 0 & & \times -1 & & \times 0 \\ + & 120 & + & 156 & + & 187 \\ & \times -1 & & \times 5 & & \times -1 \\ + & 145 & + & 148 & + & 169 \\ & \times 0 & & \times -1 & & \times 0 \end{array} \right) = 191$$

kernel:

sharpen ▼



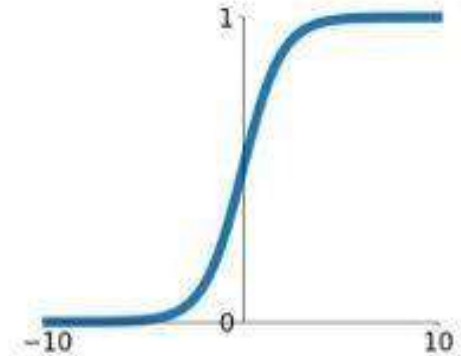
output image

Activation Functions

- In Convolutional Neural Networks (CNNs), activation functions play a crucial role in introducing non-linearity to the model, allowing it to learn complex patterns and decision boundaries.
- **Sigmoid Functions:** The sigmoid function maps any input to a value between 0 and 1. It has an S-shaped curve and is often used in binary classification problems.
 - Advantages: Outputs between 0 and 1: Useful for probabilistic outputs, e.g., in the last layer of a binary classifier.
 - Disadvantages: Vanishing Gradient Problem: For very large or very small inputs, the gradient becomes very small, making training slow. Saturation: For inputs that are too large or too small, the function saturates, leading to slow learning.
- **Tan h Function:** The Tanh function maps input values to a range between -1 and 1. It is similar to the sigmoid but with a different output range.
 - Advantages: Zero-Centered: Unlike sigmoid, which maps values to (0, 1), Tanh outputs values between -1 and 1, which can help with centering data and make training faster.
 - Disadvantages: Vanishing Gradient Problem: Like sigmoid, Tanh also suffers from the vanishing gradient issue for large positive and negative values of input.

Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



tanh

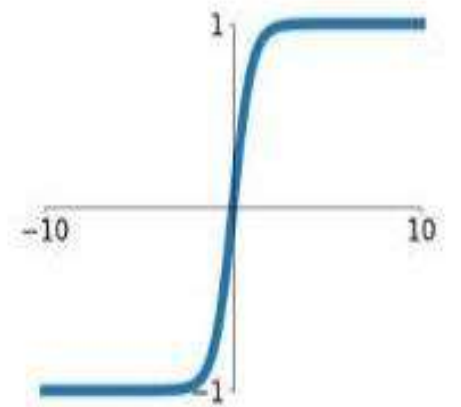
$$\tanh(x)$$


Image Credit: towardsdatascience.com

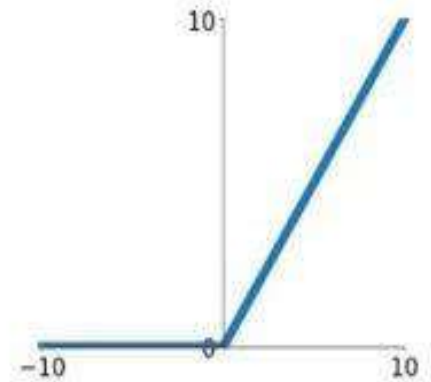
Activation Functions

Rectified Linear Unit: ReLU is one of the most widely used activation functions in CNNs. It outputs the input directly if it's positive; otherwise, it outputs zero.

Advantages:

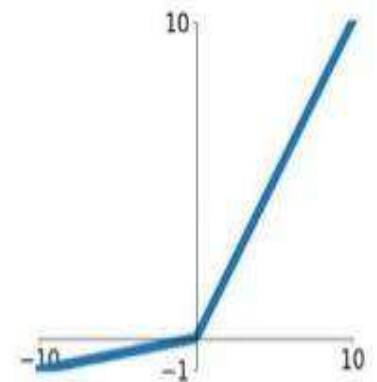
- **Efficient:** It speeds up training and computation by being simple and computationally cheap.
- **Sparsity:** It helps in creating sparse activations, which can improve performance.
- **Mitigates Vanishing Gradient Problem:** Since it doesn't saturate for positive values, it mitigates the vanishing gradient problem in deep networks.
- **Leaky Rectified Linear Unit:** Leaky ReLU is a variant of ReLU that allows a small, non-zero gradient when the input is negative.
- **Advantages:** Fixes Dying ReLU: Since it allows a small negative slope, the "dying ReLU" problem is reduced.
- **Faster convergence:** It often leads to faster convergence than ReLU because of the small slope for negative values.

ReLU
 $\max(0, x)$



Disadvantages: Dying ReLU Problem: Neurons can sometimes "die," meaning they stop learning because they get stuck at zero. This happens when large negative values push the activation to zero, and gradients vanish.

Leaky ReLU
 $\max(0.1x, x)$



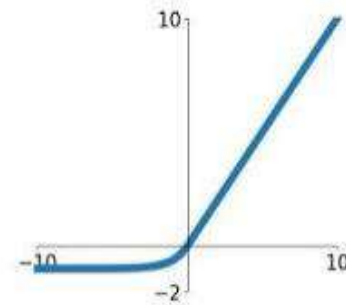
Activation Functions

ELU is similar to ReLU but with an exponential curve for negative values.

- **Advantages: Negative Range:** Helps with negative outputs, reducing bias shift and speeding up learning.
 - **Avoids Vanishing Gradient Problem:** The negative part of the function doesn't saturate as quickly as Tanh or Sigmoid.
- **Disadvantages: Computationally Expensive:** The exponential part for negative values is computationally more expensive than ReLU.

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



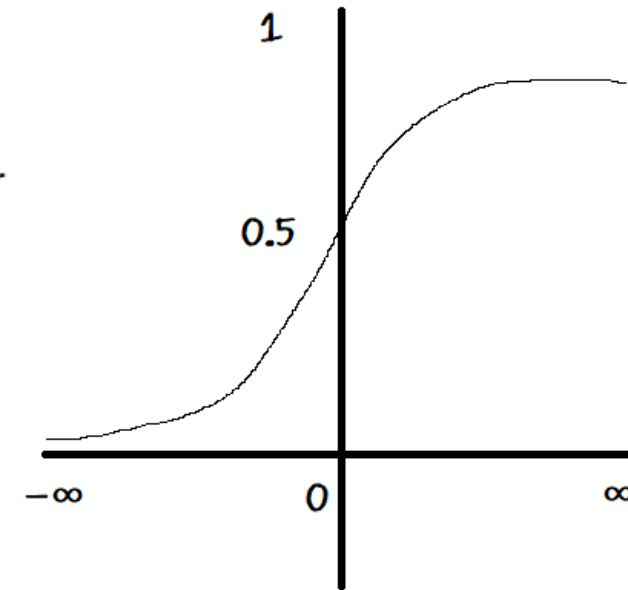
Softmax

Formula: $f(x_i) = \frac{e^{x_i}}{\sum_{j=1}^N e^{x_j}}$

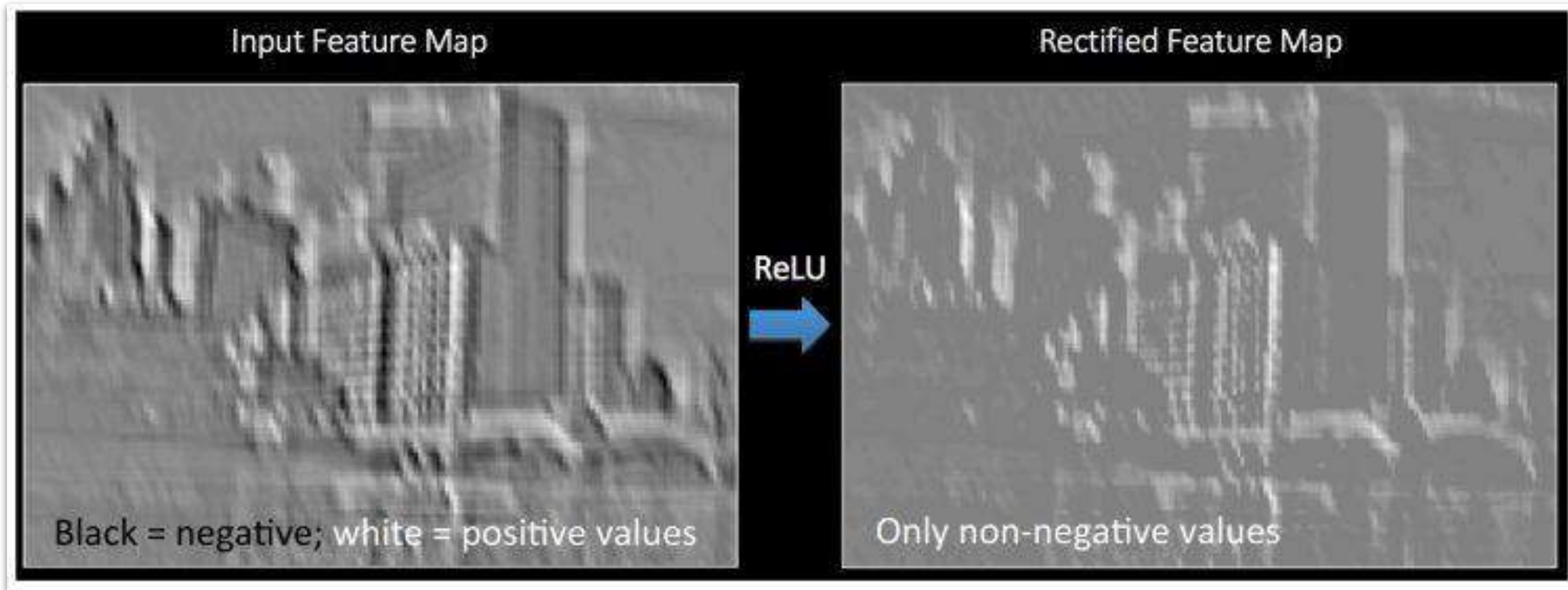
Description: The Softmax function is typically used in the output layer of a neural network for multi-class classification problems. It converts raw scores (logits) into probabilities.

Advantages:

- **Probabilistic Output:** Softmax converts the output into a probability distribution across classes.
- **Normalization:** It normalizes the input values to sum to 1, making them interpretable as probabilities.



Introducing Non-Linearity (ReLU)



Pooling

- **Pooling** is a crucial operation used to reduce the spatial dimensions (width and height) of feature maps while retaining important information.
- This reduction helps to decrease the computational complexity of the network, mitigate overfitting, and make the network invariant to small translations of the input image.

Key Benefits of Pooling:

- **Dimensionality Reduction:** Pooling reduces the number of parameters, which in turn reduces the computational cost.
- **Translation Invariance:** Pooling makes the model invariant to small translations (shifts) in the input image, which is useful for tasks like image classification.
- **Prevents Overfitting:** By reducing the spatial resolution, pooling layers can help prevent the network from overfitting to small details in the training data.

When to Use Pooling?

- **Early Layers:** In the early layers of the network, pooling layers are typically used to reduce the size of the feature maps and focus on the most prominent features in the image.
- **Fully Convolutional Networks:** In fully convolutional networks (FCNs), pooling is often used to make the model invariant to small shifts or distortions in the input.
- **Classification Tasks:** For classification tasks, pooling can be beneficial in reducing the spatial dimensions, making the network less prone to overfitting and helping in faster convergence.

Pooling

Key Concepts in Pooling

- **Stride:** The stride determines how much the pooling window moves during the pooling operation. A stride of 1 means that the window moves one step at a time, while a larger stride results in greater downsampling.
- **Kernel Size:** The size of the window (e.g., 2x2, 3x3) determines how much data is taken into account for each pooling operation.
- **Padding:** Padding is sometimes added around the edges of the input before pooling to ensure that the pooling operation can be applied even at the borders. It can help maintain the spatial size of the input when necessary.

Types of Pooling:

Global Average Pooling (GAP): Global Average Pooling is a special case of average pooling where the entire feature map is reduced to a single value per feature map. This is often used in the final layer of CNNs to reduce the spatial dimensions to 1x1, which can be useful for classification tasks.

•Advantages:

- **Reduces Overfitting:** Since it significantly reduces the number of parameters, it helps prevent overfitting.
- **Simplifies the Model:** It replaces fully connected layers with just one output per feature map, making the model less complex.

$$\text{Output} = \frac{1}{H \times W} \sum_{i=1}^H \sum_{j=1}^W X_{i,j}$$

where $H \times W$ is the size of the feature map (height x width), and $X_{i,j}$ are the feature values.

Pooling

- **Max pooling:** Max pooling works by selecting the maximum value from a set of values in the input feature map within a defined window (usually a 2x2 or 3x3 window) for the strongest features.

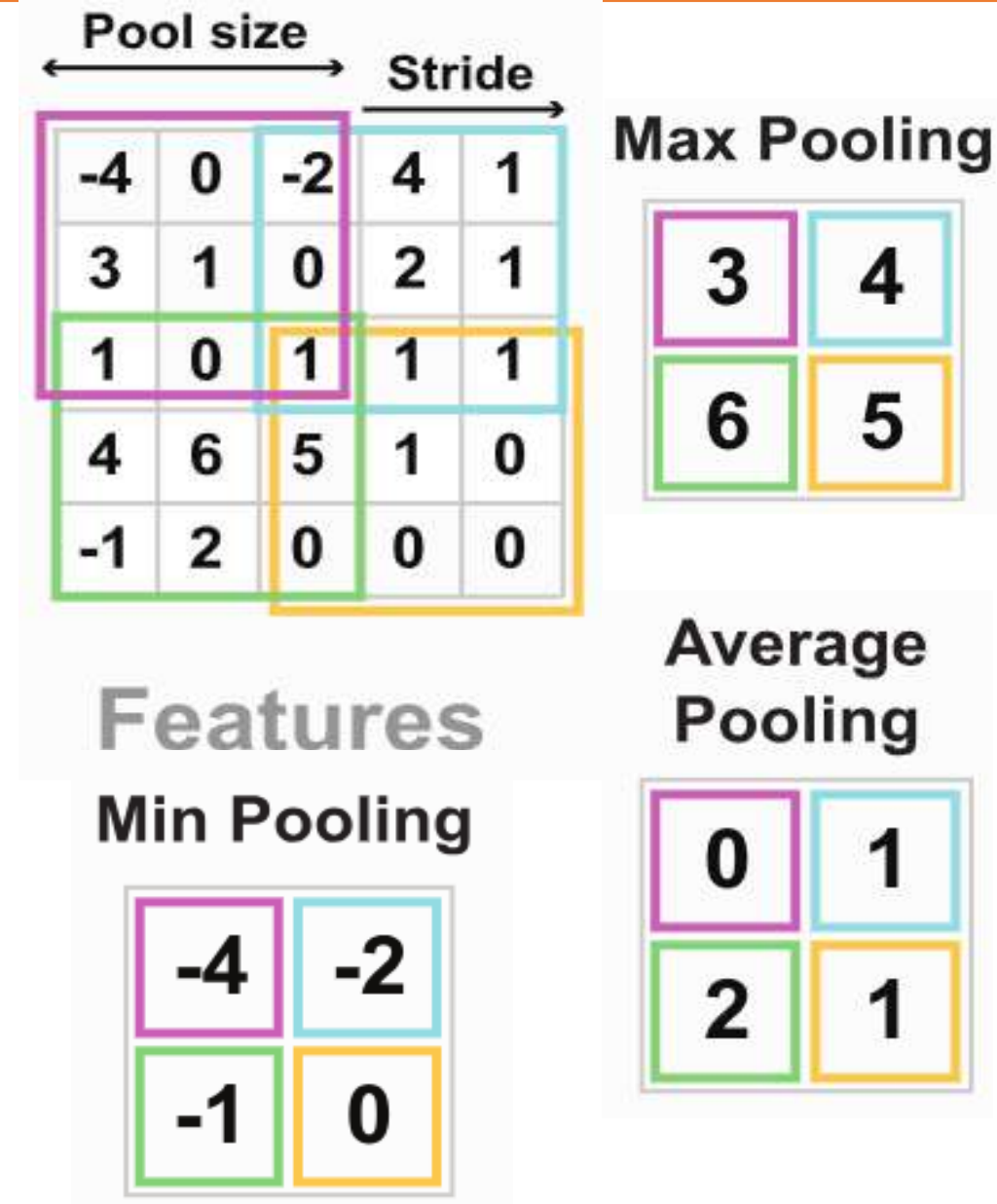
$$\text{Output} = \max(X_1, X_2, X_3, X_4)$$

where X_1, X_2, X_3, X_4 are the values in a 2x2 window.

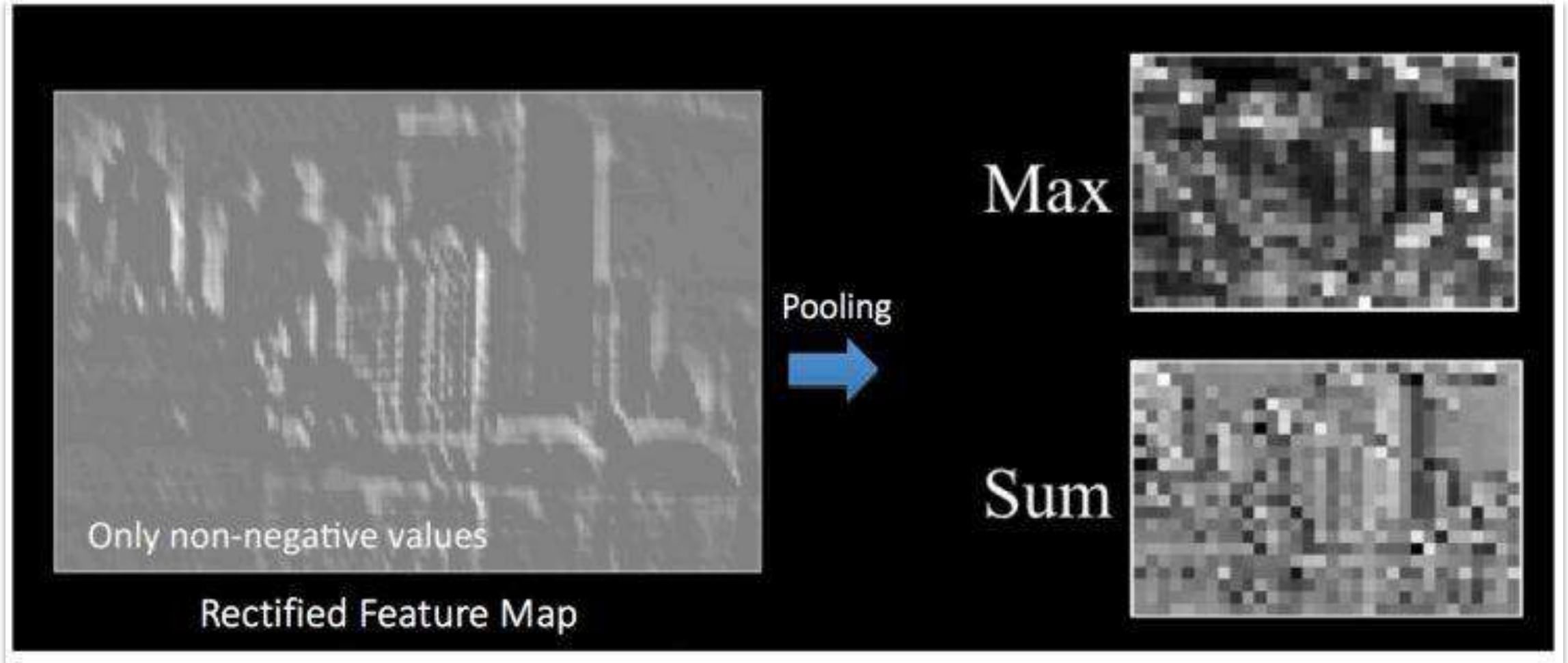
- **Min pooling:** Min pooling works by selecting the minimum value from a set of values in the input feature map within a defined window for retaining a **smoother version of the features**.

- **Average pooling:** Average pooling computes the average of the values in the pooling window for **better for Fine-grained Information**.

$$\text{Output} = \frac{X_1 + X_2 + X_3 + X_4}{4}$$

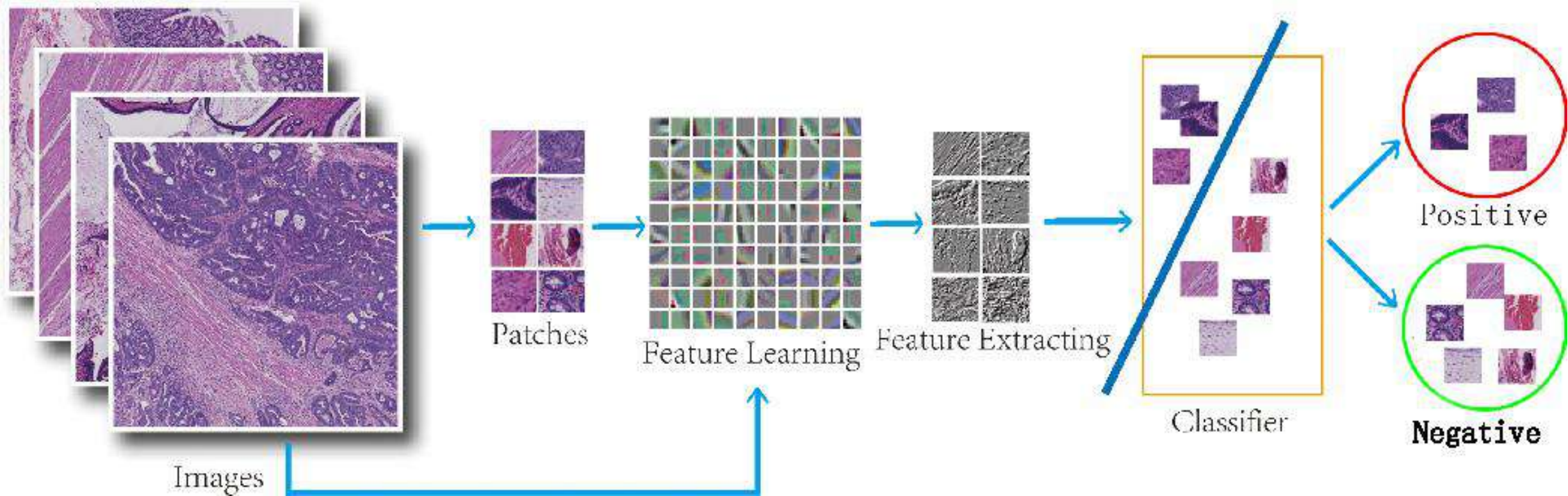


Pooling - Max-Pooling and Sum-Pooling



CNN-Application-Example

Module-2; Topic-1: Foundations of Convolutional Neural Networks – CNN operations – Architecture



- The flow diagram of the algorithms with a minimum of manual annotation and good feature representations.
- The inputs include both cancer images and noncancer images. All images are used to generate patches. In feature learning processing, images/patches are used to downsample receptive fields. Feature learning is implemented by three methods containing full supervised deep learning, unsupervised learning of a single-layer network, and manual features.
- The next step is to extract features for each patches. In classifier processing, we conduct fully supervised classifier (SVM) and weakly supervised classifier (MIL). The overall patch-level classification (cancer vs. noncancer) can be obtained based on the confidences from classifiers. Red represents cancer patches while green represents noncancer patches.

CNN-Solved Numerical

Problem: Apply a 3x3 Convolution on a 5x5 Input Image

Step 1: Define the Input Image and Filter

Consider the following 5x5 input image I :

$$I = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 6 & 7 & 8 & 9 & 10 \\ 11 & 12 & 13 & 14 & 15 \\ 16 & 17 & 18 & 19 & 20 \\ 21 & 22 & 23 & 24 & 25 \end{bmatrix}$$

Let the 3x3 filter K be:

$$K = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

CNN-Solved Numerical

Step 2: Define the Convolution Operation

To apply the filter, we need to slide it over the input image. The filter will move across the image, compute the element-wise multiplication with the region it overlaps, and sum up the results.

In this example, we'll use **valid padding** (no padding, only where the filter fits completely within the image) and a **stride of 1** (the filter moves 1 step at a time).

The resulting output matrix will have dimensions of $(5 - 3 + 1) \times (5 - 3 + 1) = 3 \times 3$.

Therefore, the output matrix will be 3x3.

Step 3: Calculate Each Output Element

For each element in the output matrix, we compute the sum of the element-wise multiplication of the filter and the input region it covers. Let's go through the steps for each position.

Output Element at Position (1, 1):

The filter covers the following region in the input image:

$$\begin{bmatrix} 1 & 2 & 3 \\ 6 & 7 & 8 \\ 11 & 12 & 13 \end{bmatrix}$$

Module-2; Topic-1: Foundations of Convolutional Neural Networks – CNN operations – Architecture

$$I = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 6 & 7 & 8 & 9 & 10 \\ 11 & 12 & 13 & 14 & 15 \\ 16 & 17 & 18 & 19 & 20 \\ 21 & 22 & 23 & 24 & 25 \end{bmatrix}$$

CNN-Solved Numerical

Module-2; Topic-1: Foundations of Convolutional Neural Networks – CNN operations – Architecture

Step 3: Calculate Each Output Element

For each element in the output matrix, we compute the sum of the element-wise multiplication of the filter and the input region it covers. Let's go through the steps for each position.

Output Element at Position (1, 1):

The filter covers the following region in the input image:

$$I = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 6 & 7 & 8 & 9 & 10 \\ 11 & 12 & 13 & 14 & 15 \\ 16 & 17 & 18 & 19 & 20 \\ 21 & 22 & 23 & 24 & 25 \end{bmatrix} \xrightarrow{\text{Filter}} \begin{bmatrix} 1 & 2 & 3 \\ 6 & 7 & 8 \\ 11 & 12 & 13 \end{bmatrix}$$

Now, we perform element-wise multiplication with the filter:

$$(1 \times 1) + (2 \times 0) + (3 \times -1) + (6 \times 1) + (7 \times 0) + (8 \times -1) + (11 \times 1) + (12 \times 0) + (13 \times -1)$$

Calculating:

$$= 1 + 0 - 3 + 6 + 0 - 8 + 11 + 0 - 13 = -6$$

So, the output at position (1, 1) is -6.

$$\begin{bmatrix} (1,1) & (1,2) & (1,3) \\ (2,1) & (2,2) & (2,3) \\ (3,1) & (3,2) & (3,3) \end{bmatrix} \times \begin{bmatrix} K_1 & K_2 & K_3 \\ K_4 & K_5 & K_6 \\ K_7 & K_8 & K_9 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

$$\begin{bmatrix} -6 & - & - \\ - & - & - \\ - & - & - \end{bmatrix}$$

CNN-Solved Numerical

Module-2; Topic-1: Foundations of Convolutional Neural Networks – CNN operations – Architecture

Output Element at Position (1, 2):

$$I = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 6 & 7 & 8 & 9 & 10 \\ 11 & 12 & 13 & 14 & 15 \\ 16 & 17 & 18 & 19 & 20 \\ 21 & 22 & 23 & 24 & 25 \end{bmatrix} \begin{bmatrix} 2 & 3 & 4 \\ 7 & 8 & 9 \\ 12 & 13 & 14 \end{bmatrix}$$

Now, we perform the element-wise multiplication:

$$(2 \times 1) + (3 \times 0) + (4 \times -1) + (7 \times 1) + (8 \times 0) + (9 \times -1) + (12 \times 1) + (13 \times 0) + (14 \times -1)$$

Calculating:

$$= 2 + 0 - 4 + 7 + 0 - 9 + 12 + 0 - 14 = -6$$

So, the output at position (1, 2) is -6.

Output Element at Position (1, 3):

$$I = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 6 & 7 & 8 & 9 & 10 \\ 11 & 12 & 13 & 14 & 15 \\ 16 & 17 & 18 & 19 & 20 \\ 21 & 22 & 23 & 24 & 25 \end{bmatrix} \begin{bmatrix} 3 & 4 & 5 \\ 8 & 9 & 10 \\ 13 & 14 & 15 \end{bmatrix}$$

Now, we perform the element-wise multiplication:

$$(3 \times 1) + (4 \times 0) + (5 \times -1) + (8 \times 1) + (9 \times 0) + (10 \times -1) + (13 \times 1) + (14 \times 0) + (15 \times -1)$$

Calculating:

$$= 3 + 0 - 5 + 8 + 0 - 10 + 13 + 0 - 15 = -6$$

So, the output at position (1, 3) is -6.

$$\begin{bmatrix} -6 & - & - \\ - & - & - \\ - & - & - \end{bmatrix} \rightarrow \begin{bmatrix} -6 & -6 & -6 \\ - & - & - \\ - & - & - \end{bmatrix}$$

CNN-Solved Numerical

Module-2; Topic-1: Foundations of Convolutional Neural Networks – CNN operations – Architecture

Output Element at Position (2, 1):

$$I = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 6 & 7 & 8 & 9 & 10 \\ 11 & 12 & 13 & 14 & 15 \\ 16 & 17 & 18 & 19 & 20 \\ 21 & 22 & 23 & 24 & 25 \end{bmatrix} \quad \begin{bmatrix} 6 & 7 & 8 \\ 11 & 12 & 13 \\ 16 & 17 & 18 \end{bmatrix}$$

Now, we perform the element-wise multiplication:

$$(6 \times 1) + (7 \times 0) + (8 \times -1) + (11 \times 1) + (12 \times 0) + (13 \times -1) + (16 \times 1) + (17 \times 0) + (18 \times -1)$$

Calculating:

$$= 6 + 0 - 8 + 11 + 0 - 13 + 16 + 0 - 18 = -6$$

So, the output at position (2, 1) is -6.

Output Element at Position (2, 2):

$$I = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 6 & 7 & 8 & 9 & 10 \\ 11 & 12 & 13 & 14 & 15 \\ 16 & 17 & 18 & 19 & 20 \\ 21 & 22 & 23 & 24 & 25 \end{bmatrix} \quad \begin{bmatrix} 7 & 8 & 9 \\ 12 & 13 & 14 \\ 17 & 18 & 19 \end{bmatrix}$$

Now, we perform the element-wise multiplication:

$$(7 \times 1) + (8 \times 0) + (9 \times -1) + (12 \times 1) + (13 \times 0) + (14 \times -1) + (17 \times 1) + (18 \times 0) + (19 \times -1)$$

Calculating:

$$= 7 + 0 - 9 + 12 + 0 - 14 + 17 + 0 - 19 = -6$$

So, the output at position (2, 2) is -6.

$$\begin{bmatrix} 6 & -6 & -6 \\ - & - & - \\ - & - & - \end{bmatrix} \rightarrow \begin{bmatrix} -6 & -6 & -6 \\ -6 & -6 & - \\ - & - & - \end{bmatrix}$$

CNN-Solved Numerical

Output Element at Position (2, 3):

$$I = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 6 & 7 & 8 & 9 & 10 \\ 11 & 12 & 13 & 14 & 15 \\ 16 & 17 & 18 & 19 & 20 \\ 21 & 22 & 23 & 24 & 25 \end{bmatrix} \quad \begin{bmatrix} 8 & 9 & 10 \\ 13 & 14 & 15 \\ 18 & 19 & 20 \end{bmatrix}$$

Now, we perform the element-wise multiplication:

$$(8 \times 1) + (9 \times 0) + (10 \times -1) + (13 \times 1) + (14 \times 0) + (15 \times -1) + (18 \times 1) + (19 \times 0) + (20 \times -1)$$

Calculating:

$$= 8 + 0 - 10 + 13 + 0 - 15 + 18 + 0 - 20 = -6$$

So, the output at position (2, 3) is -6.

Step 4: Continue for Remaining Positions

Following the same process, we continue computing the output for the remaining positions (3,1), (3,2), and (3,3).

Final Output:

The resulting 3x3 output matrix is:

$$\text{Output} = \begin{bmatrix} -6 & -6 & -6 \\ -6 & -6 & -6 \\ -6 & -6 & -6 \end{bmatrix}$$

$$\begin{bmatrix} -6 & -6 & -6 \\ -6 & -6 & - \\ - & - & - \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 6 & 7 & 8 & 9 & 10 \\ 11 & 12 & 13 & 14 & 15 \\ 16 & 17 & 18 & 19 & 20 \\ 21 & 22 & 23 & 24 & 25 \end{bmatrix}, \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 6 & 7 & 8 & 9 & 10 \\ 11 & 12 & 13 & 14 & 15 \\ 16 & 17 & 18 & 19 & 20 \\ 21 & 22 & 23 & 24 & 25 \end{bmatrix}, \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 6 & 7 & 8 & 9 & 10 \\ 11 & 12 & 13 & 14 & 15 \\ 16 & 17 & 18 & 19 & 20 \\ 21 & 22 & 23 & 24 & 25 \end{bmatrix}$$

Calculate the number of Parameter in CNN

Module-2; Topic-1: Foundations of Convolutional Neural Networks – CNN operations – Architecture

1. Convolutional Layer Parameters:

For a convolutional layer, the number of parameters is calculated as:

$$\text{Parameters} = (k_w \times k_h \times C_{in} + 1) \times C_{out}$$

where k_w = KernelWidth, k_h = KernelHeight

2. Fully Connected Layer Parameters:

C_{in} = Number of input channels, C_{out} = Number of Filter outputs

For a fully connected (dense) layer, the number of parameters is:

$$\text{Number of parameters} = (N_{in} + 1) \times N_{out}$$

Where:

- N_{in} = number of input neurons to the layer (which is the number of features or the flattened output from previous layers)
- N_{out} = number of output neurons (units) in the layer
- +1 is for the bias term.

Source: <https://www.geeksforgeeks.org/how-to-calculate-the-number-of-parameters-in-cnn/>

Calculate the number of Parameter in CNN

Let's consider a simple CNN with the following architecture:

1. Conv1 Layer:

- Input: 32x32x3 (height, width, depth)
- Filter size: 3x3
- Number of filters: 64

2. Conv2 Layer:

- Input: 30x30x64 (after the first convolution, assuming stride 1 and no padding)
- Filter size: 3x3
- Number of filters: 128

3. Fully Connected Layer (Dense):

- Input: Flattened size of the output from Conv2
- Number of output neurons: 10 (for classification)

Module-2; Topic-1: Foundations of
Convolutional Neural Networks – CNN
operations – Architecture

Calculate the number of Parameter in CNN

Module-2; Topic-1: Foundations of Convolutional Neural Networks – CNN operations – Architecture

Step-by-step calculation:

1. Conv1 Layer Parameters:

- Input channels $C_{in} = 3$
- Filter size $K = 3$ (3x3)
- Output channels $C_{out} = 64$

The number of parameters for Conv1:

$$\text{Conv1 parameters} = (3 \times 3 \times 3 + 1) \times 64 = (27 + 1) \times 64 = 28 \times 64 = 1792$$

2. Conv2 Layer Parameters:

- Input channels $C_{in} = 64$ (from Conv1)
- Filter size $K = 3$ (3x3)
- Output channels $C_{out} = 128$

The number of parameters for Conv2:

$$\text{Conv2 parameters} = (64 \times 3 \times 3 + 1) \times 128 = (576 + 1) \times 128 = 577 \times 128 = 73856$$

3. Fully Connected Layer (Dense) Parameters: After Conv2, the output size is 28x28x128 (assuming stride=1 and no padding). We flatten this to a vector of size $28 \times 28 \times 128 = 100352$.

The number of parameters for the dense layer:

$$\text{Dense parameters} = (100352 + 1) \times 10 = 100353 \times 10 = 1003530$$

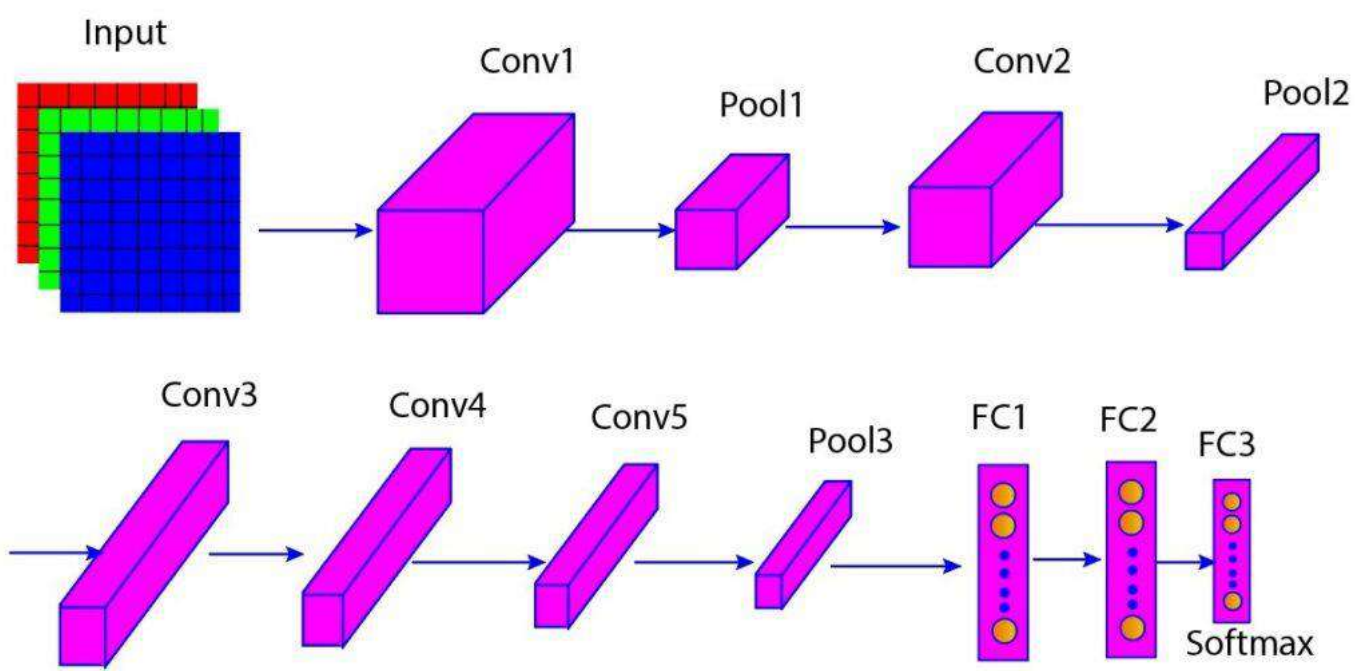
Total Parameters:

Adding all the parameters together:

$$\text{Total parameters} = 1792(\text{Conv1}) + 73856(\text{Conv2}) + 1003530(\text{Dense}) = 1084178$$

Thus, the total number of parameters in this simple CNN is 1,084,178.

AlexNet Architecture



Type	Output Size	Filter/Neuron Count
Convolutional (Conv1)	55x55x96	96 filters, 11x11
Convolutional (Conv2)	27x27x256	256 filters, 5x5
Convolutional (Conv3)	13x13x384	384 filters, 3x3
Convolutional (Conv4)	13x13x384	384 filters, 3x3
Convolutional (Conv5)	13x13x256	256 filters, 3x3
Fully Connected (FC1)	4096	4096 neurons
Fully Connected (FC2)	4096	4096 neurons
Fully Connected (FC3)	1000	1000 neurons

AlexNet is a pioneering deep convolutional neural network (CNN) that made a significant impact on the field of deep learning and computer vision. It was introduced by **Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton** in their paper "**ImageNet Classification with Deep Convolutional Neural Networks**" in 2012. AlexNet was the winner of the **ImageNet Large Scale Visual Recognition Challenge (ILSVRC)** in 2012, achieving a substantial improvement in classification accuracy compared to previous methods.

AlexNet Architecture

Module-2; Topic-2: Deep Convolutional Models – ResNet, AlexNet, InceptionNet and others

Key Contributions of AlexNet

- **Deep Convolutional Neural Network:** AlexNet demonstrated the power of deep CNNs for image classification tasks, helping to popularize deep learning in computer vision.
- **Use of GPUs for Training:** One of the most important aspects of AlexNet's success was the use of Graphics Processing Units (GPUs) to accelerate the training process. This significantly reduced training time and allowed for the use of a deeper, more powerful model.
- **Rectified Linear Unit (ReLU):** AlexNet introduced the use of the **ReLU (Rectified Linear Unit)** activation function in place of traditional activation functions like sigmoid or tanh. ReLU helps the network to learn faster and avoid issues like the vanishing gradient problem.
- **Data Augmentation:** To combat overfitting, AlexNet used **data augmentation** techniques, such as random cropping and horizontal flipping of images, to artificially increase the size of the training dataset.
- **Dropout:** AlexNet employed **dropout** as a regularization technique to prevent overfitting, where randomly selected neurons are "dropped" or ignored during training to force the network to learn more robust features.
- **Local Response Normalization (LRN):** LRN is used in AlexNet to normalize the activations in a way that helps with the generalization of the model. It applies normalization across the feature maps to emphasize more prominent features.

AlexNet Architecture

Module-2; Topic-2: Deep Convolutional Models –
ResNet, AlexNet, InceptionNet and others

Important Techniques Used in AlexNet

- 1.ReLU Activation:** Instead of using traditional activation functions like sigmoid or tanh, AlexNet used **ReLU**, which speeds up training by allowing for faster convergence and reduces the vanishing gradient problem.
- 2.Data Augmentation:** By using data augmentation techniques, such as random cropping and flipping, AlexNet increased the effective size of the training dataset, reducing the chances of overfitting.
- 3.Dropout:** To prevent overfitting in fully connected layers, AlexNet used dropout, which randomly deactivates a subset of neurons during training.
- 4.GPU Acceleration:** One of the reasons for AlexNet's success was the use of **GPUs** for training. This significantly reduced the time required for training the model and allowed the use of a deeper network.
- 5.Local Response Normalization (LRN):** LRN was used in AlexNet to normalize the activations across feature maps, helping the network generalize better.

Impact of AlexNet

- Revolutionizing Deep Learning:** AlexNet demonstrated the potential of deep learning for large-scale image classification tasks and was a major factor in the rise of deep neural networks for various computer vision problems.
- Winning ILSVRC 2012:** AlexNet achieved a significant improvement in the **ImageNet** challenge, outperforming the second-place finisher by a large margin (16% error rate vs. 25%).
- GPU-based Training:** The use of GPUs for training made it possible to train much larger models in less time, making deep learning more feasible for practical applications.
- Foundational for Modern Architectures:** AlexNet inspired the development of other deep learning architectures, such as **VGG**, **GoogLeNet**, **ResNet**, and many others.

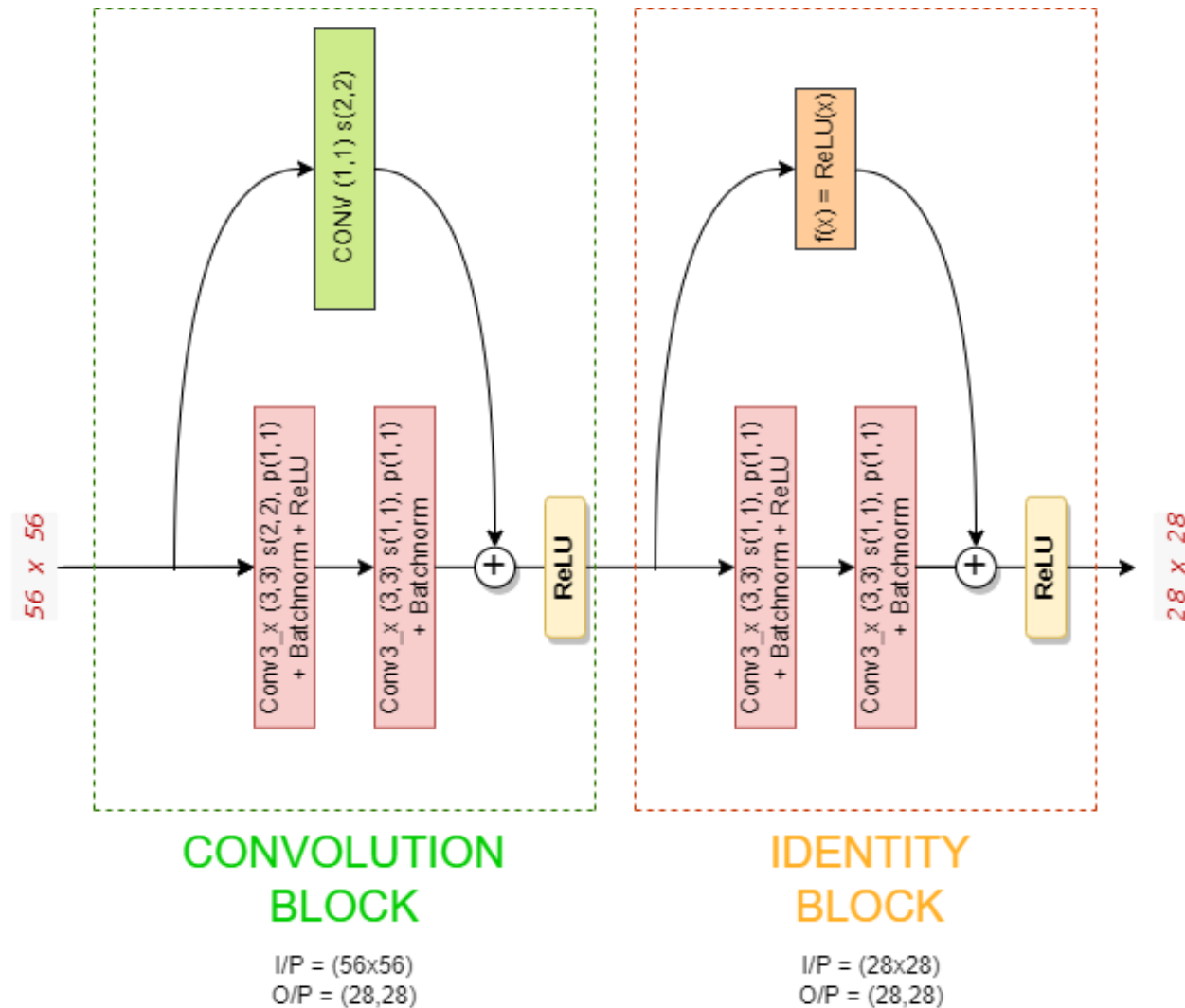
ResNet

ResNet (Residual Networks) is a type of deep neural network architecture that was introduced by **Kaiming He** and his colleagues in the paper "**Deep Residual Learning for Image Recognition**" in 2015.

Why Residual Networks (ResNet) Work

- **Avoiding the Vanishing Gradient Problem:** Residual connections allow gradients to propagate more easily through the network during backpropagation, helping to prevent the vanishing gradient problem. When the gradient can flow through these shortcuts, it helps update weights in deeper layers more effectively.
- **Enabling Deeper Networks:** With ResNet, networks can be made much deeper without the risk of degradation in performance. For example, the original ResNet paper demonstrated networks with **152 layers**, which was much deeper than previous state-of-the-art networks (e.g., VGG-16 or VGG-19, which had only 16-19 layers).
- **Improved Training:** The introduction of residual connections helps to stabilize and speed up the training process, as the network has more efficient gradient propagation.

Module-2; Topic-2: Deep Convolutional Models – ResNet, AlexNet, InceptionNet and others

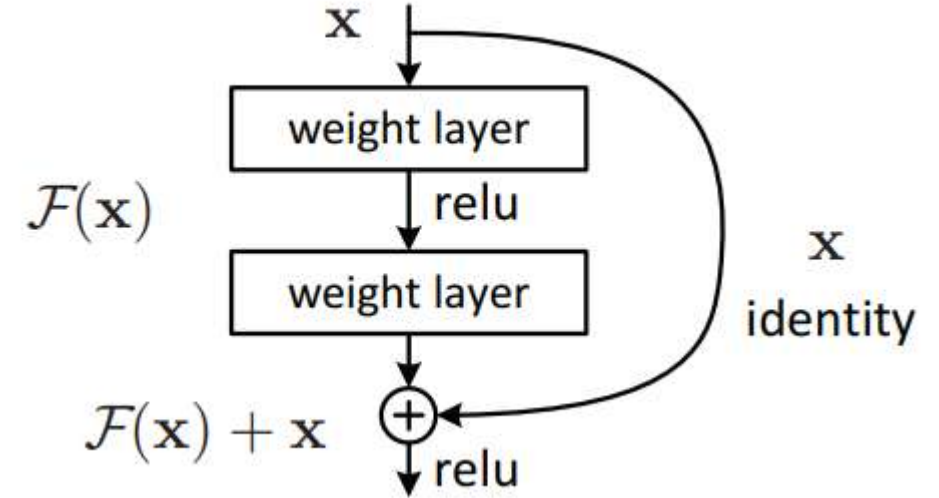


ResNet

Key Concept: Residual Connections

- The core idea behind ResNet is the introduction of **residual connections** or **skip connections**.
- These connections allow the model to bypass certain layers and directly propagate information forward from earlier layers to deeper layers.
- The purpose of these connections is to help the model avoid the problem of training deep networks effectively.
- In traditional deep networks, as the depth increases, the gradients can become very small (vanish) during backpropagation, which can make learning difficult.
- ResNet addresses the issue of learning output by introducing shortcuts that directly connect input to output, bypassing intermediate layers, allowing the network to learn residuals.

Module-2; Topic-2: Deep Convolutional Models – ResNet, AlexNet, InceptionNet and others



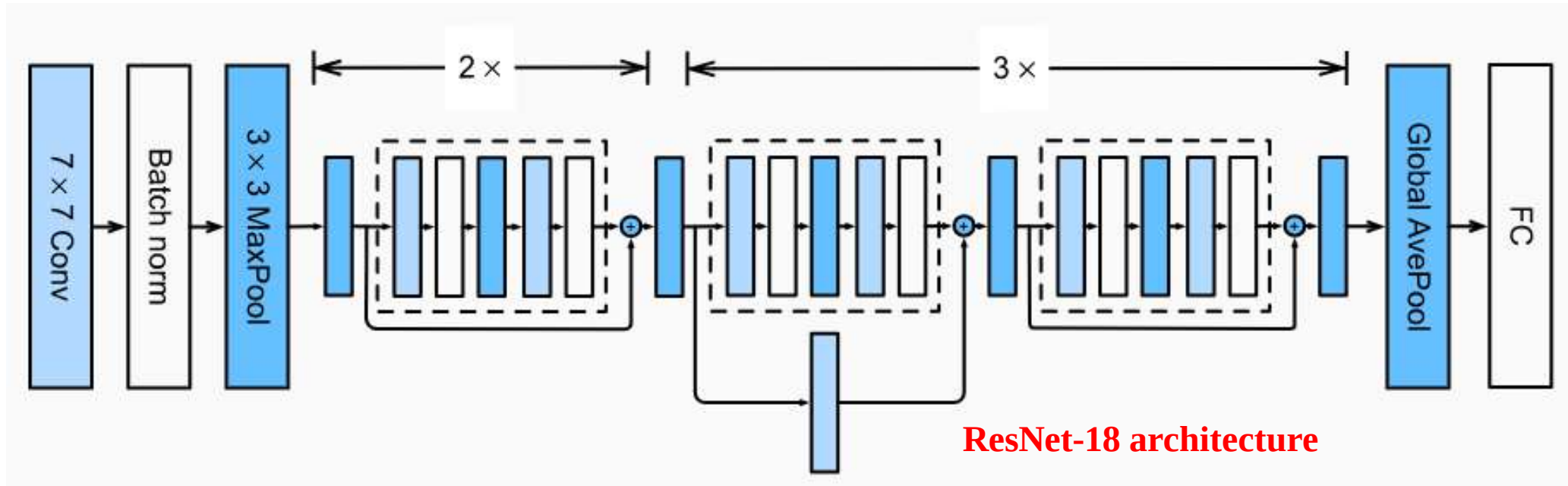
$$y = F(x, \{W_i\}) + x$$

Where:

- x is the input to the residual block.
- $F(x, \{W_i\})$ represents the output of the learned transformation (weights W_i) within the block.
- y is the output of the residual block, which is the sum of the learned transformation $F(x, \{W_i\})$ and the original input x (the skip connection).
- The skip connection adds the input x directly to the output.

ResNet- architecture

Module-2; Topic-2: Deep Convolutional Models – ResNet, AlexNet, InceptionNet and others



The ResNet architecture typically consists of the following elements:

- **Residual blocks:** These are the building blocks that use skip connections, as described above.
- **Convolutional layers:** Like traditional convolutional networks, ResNet uses convolutional layers to extract features from input images.
- **Batch normalization:** This normalizes the activations in each layer to stabilize training.
- **Activation functions (e.g., ReLU):** These introduce non-linearity in the network.
- **Fully connected layers:** At the end, the network typically has one or more fully connected layers to make the final prediction (e.g., classification).

ResNet- architecture

Module-2; Topic-2: Deep Convolutional Models – ResNet, AlexNet, InceptionNet and others

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				

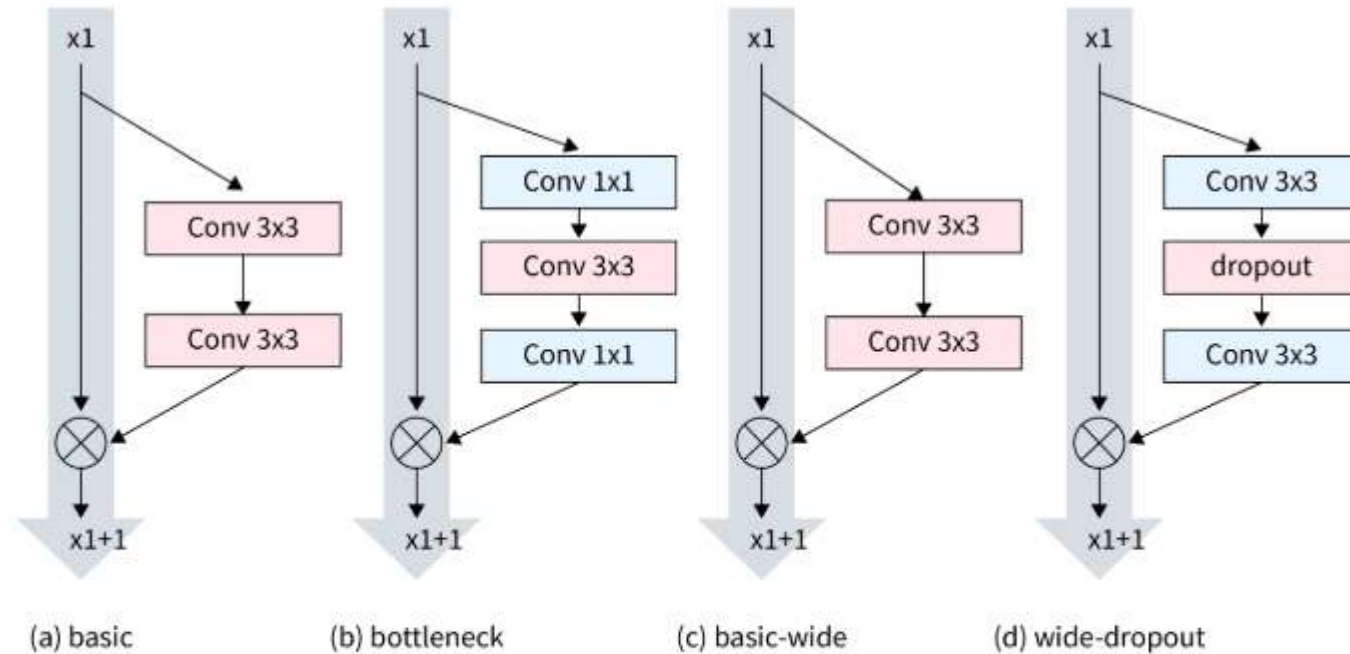
ResNet Variants: There are several popular variants of ResNet, based on the depth of the network:

- **ResNet-18:** A ResNet model with 18 layers.
- **ResNet-34:** A ResNet model with 34 layers.
- **ResNet-50:** A deeper ResNet model with 50 layers, often used in practice. It incorporates **bottleneck layers** to reduce the computational cost of deeper networks.
- **ResNet-101:** A 101-layer deep ResNet.
- **ResNet-152:** A very deep ResNet with 152 layers, typically used for challenging tasks in computer vision.

ResNet- architecture

- **Bottleneck Architecture (in deeper networks)**
- For deeper ResNet models like ResNet-50, ResNet-101, and ResNet-152, the **bottleneck architecture** is used.
- A bottleneck block reduces the computational cost by using 1×1 convolutions for dimensionality reduction before applying 3×3 convolutions.
- This helps in speeding up training without compromising performance.
- A typical bottleneck block has the following structure:
- A 1×1 convolution reduces the number of input channels.
- A 3×3 convolution processes the data.
- A 1×1 convolution restores the original number of channels.

Module-2; Topic-2: Deep Convolutional Models – ResNet, AlexNet, InceptionNet and others



SCALER
Topics

ResNet- architecture

Module-2; Topic-2: Deep Convolutional Models – ResNet, AlexNet, InceptionNet and others

Advantages of ResNet

- **Deeper Networks:** ResNet enables the training of much deeper networks (e.g., 152 layers), which was previously challenging due to the degradation problem in traditional networks.
- **Faster Training:** The residual connections help gradients propagate more efficiently during backpropagation, speeding up convergence.
- **Better Performance:** ResNet architectures achieve state-of-the-art results in tasks like image classification, object detection, and segmentation. For example, ResNet models have won several computer vision challenges, including the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) in 2015.
- **Versatility:** ResNet architectures can be adapted for a variety of tasks, such as classification, segmentation (using **ResNet with fully convolutional layers**), and more.

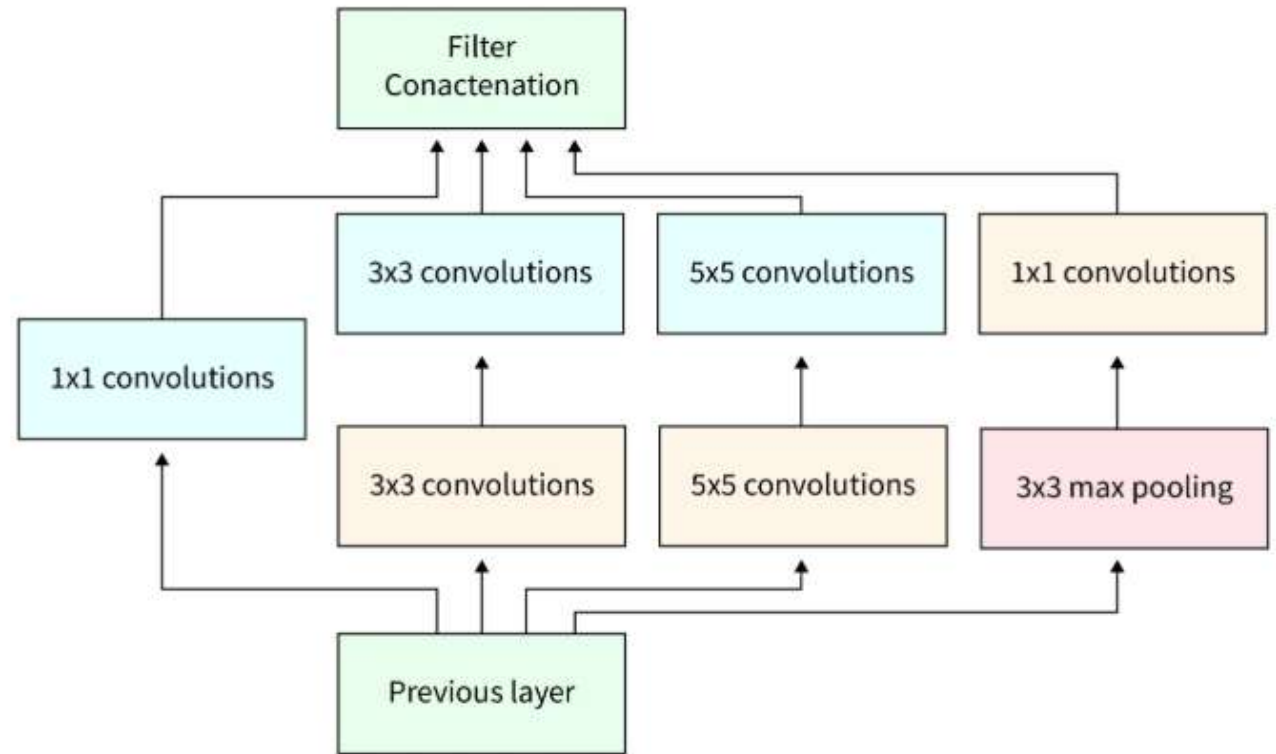
Applications of ResNet: ResNet has been widely used in various computer vision tasks, including:

- **Image Classification:** Recognizing objects or classes within images.
- **Object Detection:** Identifying and localizing objects in images.
- **Image Segmentation:** Classifying every pixel of an image into a specific category (semantic segmentation).
- **Transfer Learning:** Pretrained ResNet models can be used for feature extraction in other applications, such as medical image analysis or satellite imagery.

InceptionNet Architecture

- **InceptionNet**, also known as **GoogLeNet**, is a deep convolutional neural network (CNN) architecture developed by **Google** for the **ImageNet Large Scale Visual Recognition Challenge (ILSVRC)** in 2014. It was introduced by **Szegedy et al.** in the paper "**Going Deeper with Convolutions**".
- InceptionNet is a convolutional neural network (CNN) designed for image classification tasks and developed for the **ImageNet Large Scale Visual Recognition Challenge**.
- InceptionNet is known for using **inception modules**, blocks of layers designed to learn a combination of local and global features from the input data.
- These modules are composed of **smaller convolutional and pooling layers**, which are combined to allow the network to learn spatial and temporal features from the input data.

Module-2; Topic-2: Deep Convolutional Models – ResNet, AlexNet, InceptionNet and others



- The key innovation of InceptionNet lies in its **Inception module**, which enables the network to extract features at multiple scales using different types of convolutions in parallel.
- This structure allows the network to perform better without increasing the computational cost drastically, as it optimizes the balance between network depth and computational efficiency.

InceptionNet Architecture

Module-2; Topic-2: Deep Convolutional Models – ResNet, AlexNet, InceptionNet and others

Key Features of InceptionNet

1.Inception Modules: The Inception module employs multiple filters of varying sizes in parallel to capture features from different spatial scales, allowing for simultaneous feature extraction. Specifically, an **Inception module** consists of: 1×1 convolutions for dimensionality reduction, 3×3 convolutions for capturing medium-range patterns, 5×5 convolutions for capturing long-range features, Max pooling layers to capture features at different scales, and All of these operations are applied in parallel, and their results are concatenated along the depth dimension (the number of channels).

2.Dimensionality Reduction with 1×1 Convolutions: The Inception module employs **1×1 convolutions** as a method for **dimensionality reduction**. This helps reduce the number of parameters and computational cost, making the model more efficient without sacrificing performance. The 1×1 convolution serves as a bottleneck layer that reduces the depth of feature maps before applying more computationally expensive convolutions (such as 3×3 or 5×5).

3.Efficient Use of Computational Resources: The Inception module allows the network to capture different levels of features using multiple convolution filters of varying sizes (1×1 , 3×3 , 5×5) and pooling operations. This parallel processing approach enables the model to use computational resources more efficiently, as each layer focuses on different spatial patterns and scales.

4.Global Average Pooling (GAP): Instead of using fully connected layers at the end of the network (which increases the number of parameters), InceptionNet uses **Global Average Pooling (GAP)**. GAP replaces the fully connected layers with a global average of each feature map, significantly reducing the number of parameters and preventing overfitting. The output of GAP is then passed through a softmax activation function to obtain the final classification.

5.Auxiliary Classifiers: To improve the gradient flow during training, **auxiliary classifiers** are added at intermediate layers, which help in improving convergence by providing additional gradients to the network during backpropagation. The predictions from these auxiliary classifiers are combined with the final output prediction.

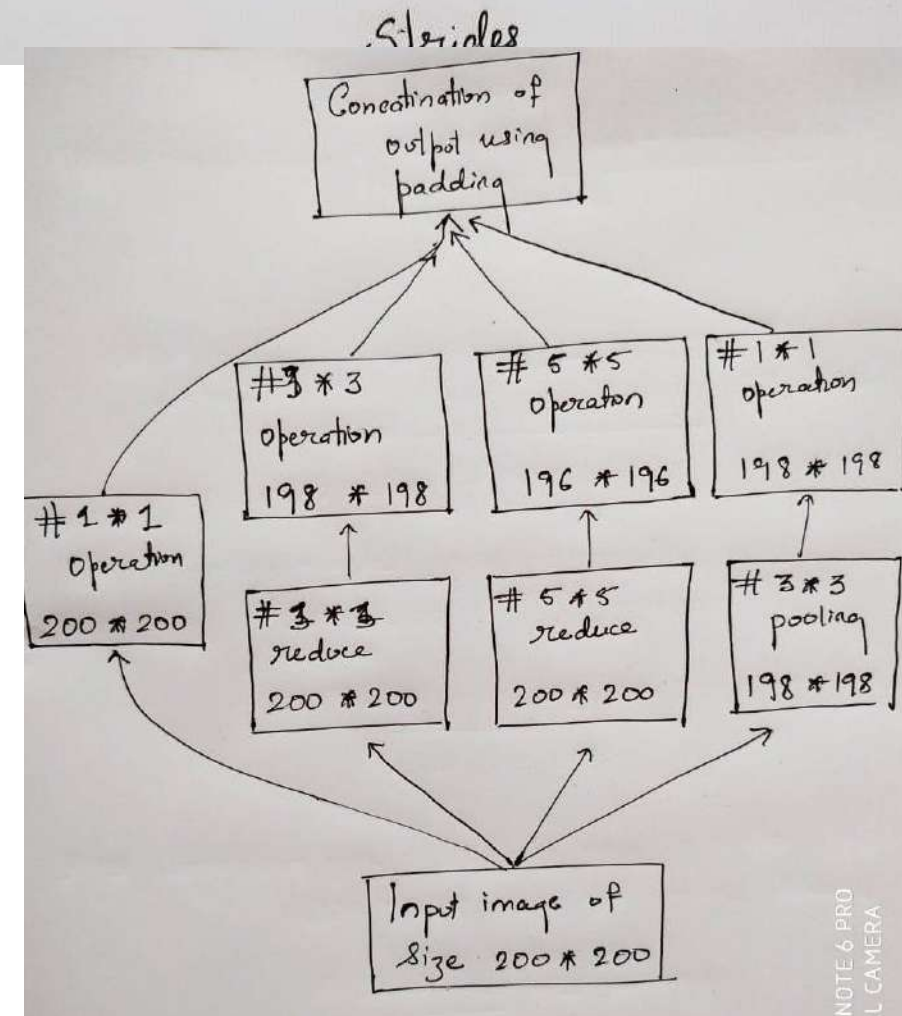
InceptionNet Architecture

Module-2; Topic-2: Deep Convolutional Models – ResNet, AlexNet, InceptionNet and others

The **GoogLeNet** (Inception v1) architecture consists of **22 layers** deep and follows a modular design based on the Inception modules.

- **Input Layer:** Input image size is **224x224x3** (height × width × channels).
- **Initial Convolution Layer:** First, a standard convolutional layer with 7x7 kernels is applied with a stride of 2, followed by a **max-pooling layer**.
- **Inception Modules:** The core of the architecture consists of several Inception modules stacked together. Each Inception module has multiple branches, each using different kernel sizes (1×1, 3×3, 5×5) and pooling layers (max pooling).
- **Reduction Layers:** Between some of the Inception modules, **reduction layers** are added to reduce the spatial size of the feature maps and control the model's complexity.
- **Global Average Pooling (GAP):** After the last Inception module, instead of flattening the output, **GAP** is applied to each feature map, reducing it to a single value (the average of all pixels in the map).
- **Softmax Layer:** The final output is passed through a **softmax** activation function to classify the image into one of the predefined categories (e.g., 1000 classes in ImageNet).

$$\text{Output image Shape} = \frac{\text{Input image Shape} - \text{Kernel Size} + 1}{\text{Stride}}$$



InceptionNet Architecture

Module-2; Topic-2: Deep Convolutional Models – ResNet, AlexNet, InceptionNet and others

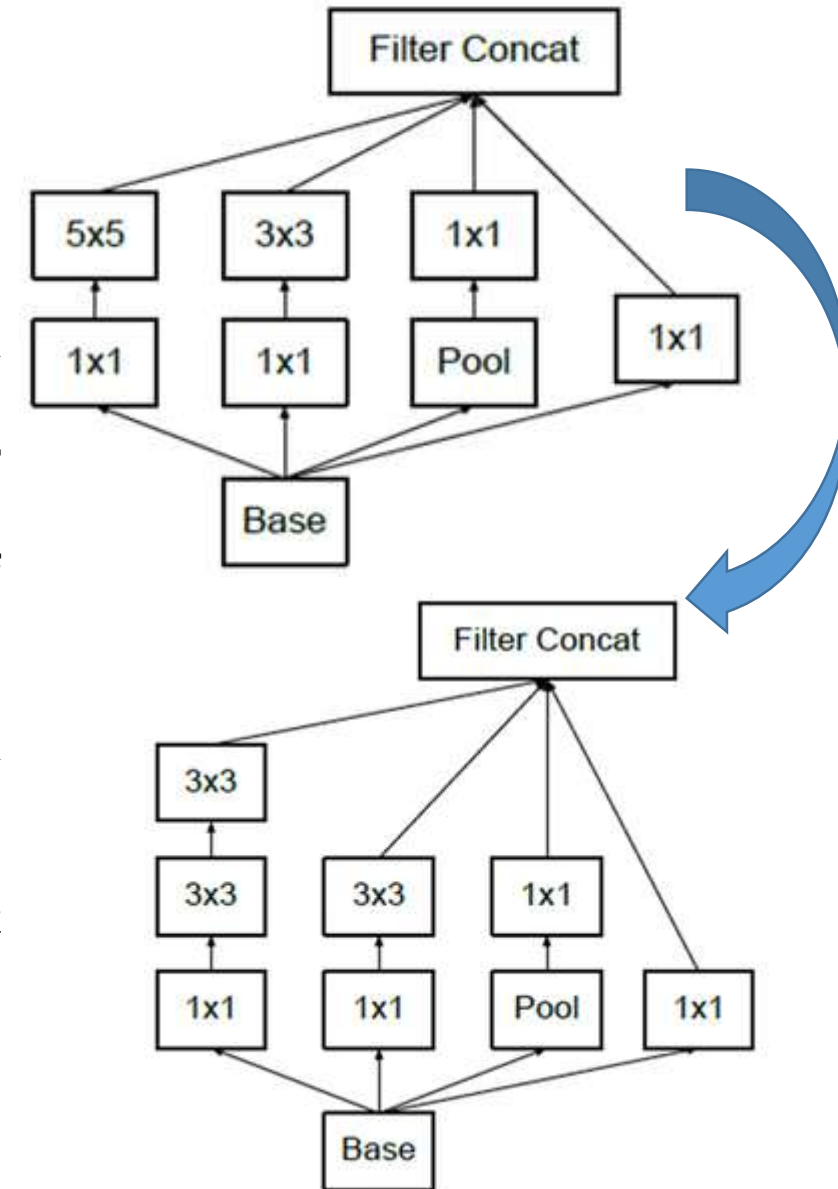
Inception v2 and Inception v3: The two most notable versions are **Inception v2** and **Inception v3**, which introduced optimizations to improve the network's performance and efficiency.

Inception v2

- **Batch Normalization:** Inception v2 introduced **batch normalization** to stabilize training and improve convergence. Batch normalization helps the network learn faster by normalizing the activations and gradients across each mini-batch.
- **Factorization:** Inception v2 optimized the model by factorizing larger convolutions into smaller ones. For example, instead of using a 5×5 convolution, two 3×3 convolutions are used in sequence. This reduces the number of parameters and computational complexity.

Inception v3

- **Factorized Convolutions:** Inception v3 further optimized the factorization approach, using techniques like **3×3 convolution factorization** and **1×1 convolutions** for dimensionality reduction.
- **Rational Functions:** Inception v3 introduced **rational functions** for efficient approximation of certain functions, which helped improve the performance while reducing computational cost.
- **Auxiliary Classifiers:** While Inception v2 dropped auxiliary classifiers, Inception v3 retained them to improve gradient flow during training.



InceptionNet Architecture

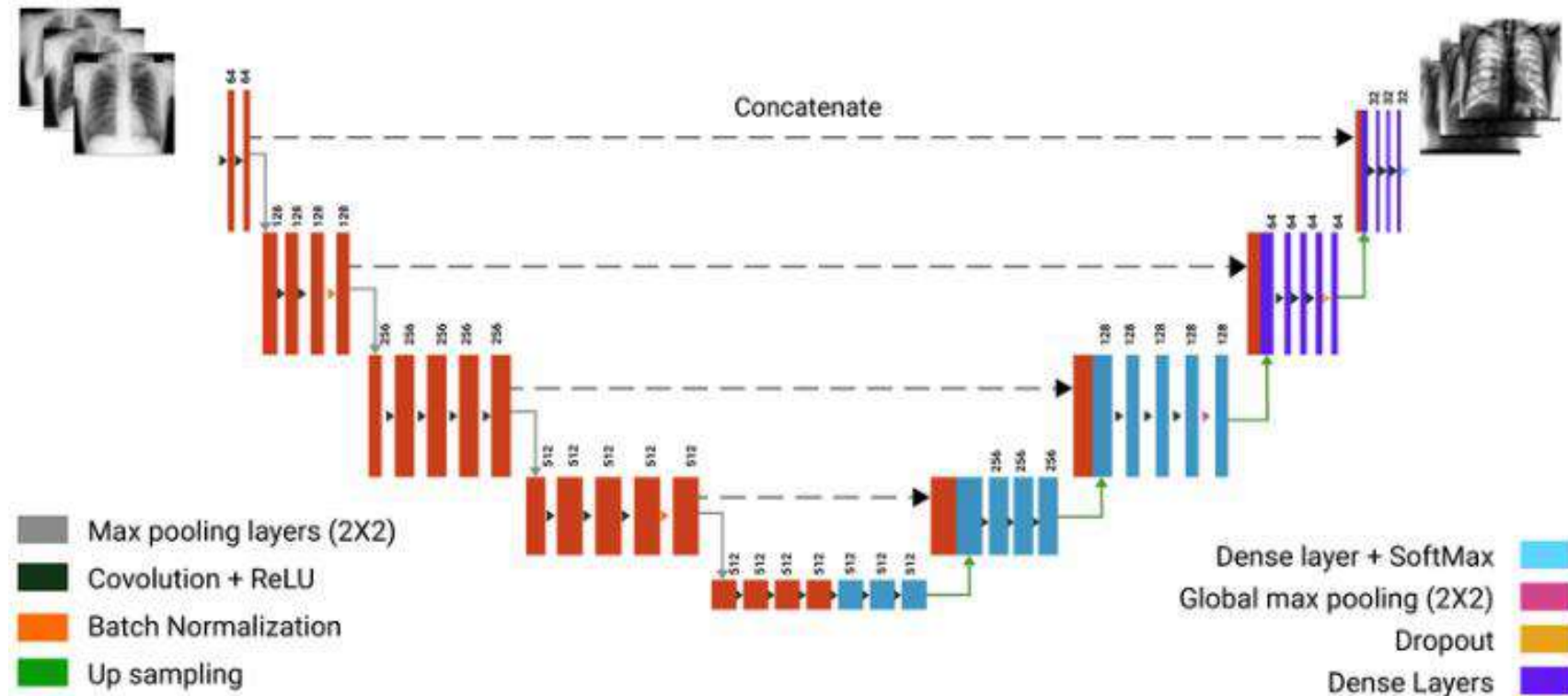
Advantages of InceptionNet

- **Multi-scale Feature Extraction:** The Inception module allows the model to extract features at multiple scales (using different kernel sizes) simultaneously, which makes it robust to different types of visual patterns.
- **Computational Efficiency:** The use of **1×1 convolutions** and **dimensionality reduction** techniques significantly reduces the number of parameters and computation compared to traditional deep networks. This makes InceptionNet more efficient and faster to train.
- **Depth and Flexibility:** InceptionNet can be very deep and flexible, allowing it to scale up to more complex architectures (e.g., Inception v3) while maintaining computational efficiency.
- **Global Average Pooling:** By using GAP instead of fully connected layers, InceptionNet reduces the risk of overfitting and reduces the number of parameters, making the model more generalizable.
- **Improved Training with Auxiliary Classifiers:** The inclusion of auxiliary classifiers helps improve training by providing additional gradients for deeper layers, speeding up convergence.
- **Applications of InceptionNet:** InceptionNet, particularly Inception v3, has been widely used in a variety of computer vision tasks, such as:
 - **Image classification:** Recognizing objects and classes within images (e.g., ImageNet challenge).
 - **Object detection:** Identifying and localizing objects in images.
 - **Semantic segmentation:** Classifying each pixel of an image into a specific category.
 - **Feature extraction:** Using pre-trained InceptionNet models for transfer learning in other domains like medical imaging, satellite image analysis, etc.

Other Deep Network

1. VGG (Visual Geometry Group)

- **VGG16** and **VGG19** are well-known convolutional neural networks introduced by the **Visual Geometry Group** at Oxford University. **VGG16** has 16 layers (13 convolutional layers and 3 fully connected layers), while **VGG19** has 19 layers.
- The architecture uses **3×3 convolutions** with a **stride of 1** and **2×2 max pooling** layers with a stride of 2.
- It is very simple and deep but suffers from having a large number of parameters (due to fully connected layers), which can make it computationally expensive.



Key Characteristics:

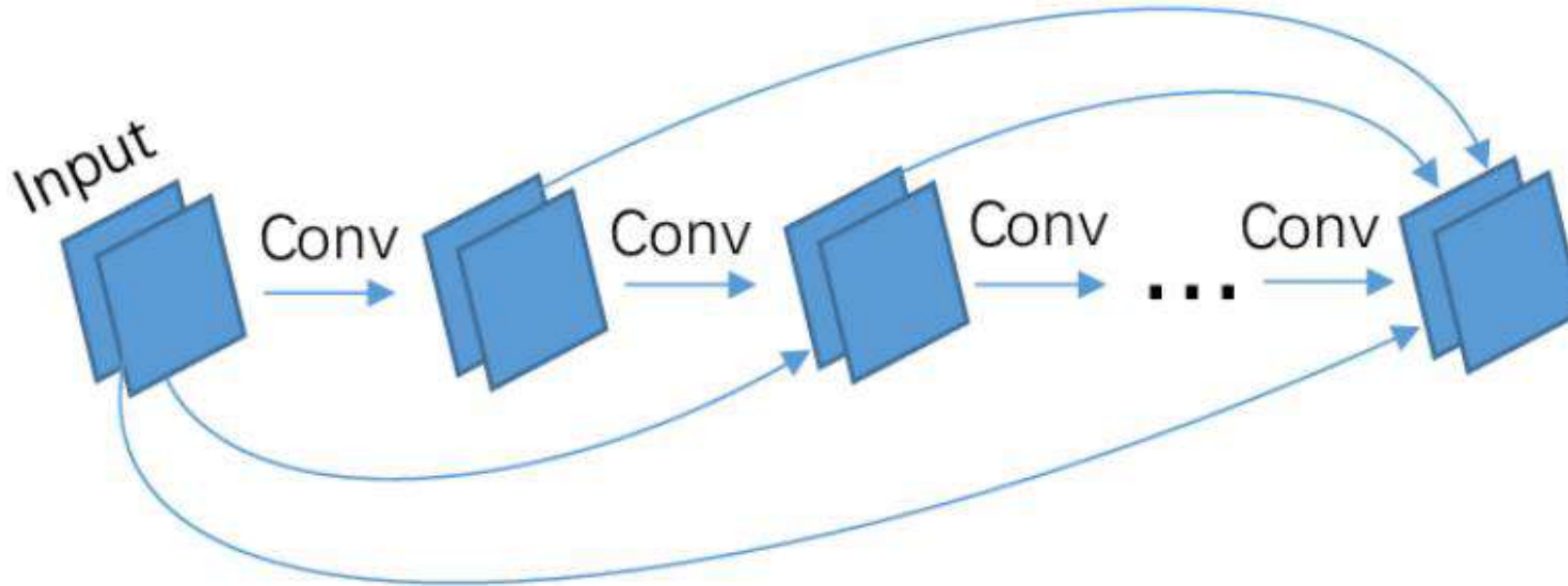
- **Deep architecture:** Deep network with a focus on smaller filters (3×3) stacking them to create deeper. VGG16 and VGG19 are two famous versions with 16 and 19 layers, respectively.
- **Fully connected layers:** After the convolutional and pooling layers, VGGNet has fully connected layers for classification tasks.
- **Max pooling** layers to reduce dimensionality.
- Used as a backbone network for many transfer learning tasks.

Module-2; Topic-2: Deep Convolutional Models – ResNet, AlexNet, InceptionNet and others

Other Deep Network

2. DenseNet (Densely Connected Convolutional Networks)

- DenseNet improves upon **ResNet** by connecting each layer to every other layer in a **feedforward manner**.
- Instead of using residual connections, each layer receives input from all previous layers, resulting in **dense connections**. This improves the flow of information and gradients throughout the network.
- DenseNet requires fewer parameters than traditional CNNs while maintaining performance.



Key Characteristics:

Dense blocks with direct connections between layers.

- Enhances feature reuse and reduces the number of parameters.
- Improves gradient flow and mitigates the vanishing gradient problem.

Other Deep Network

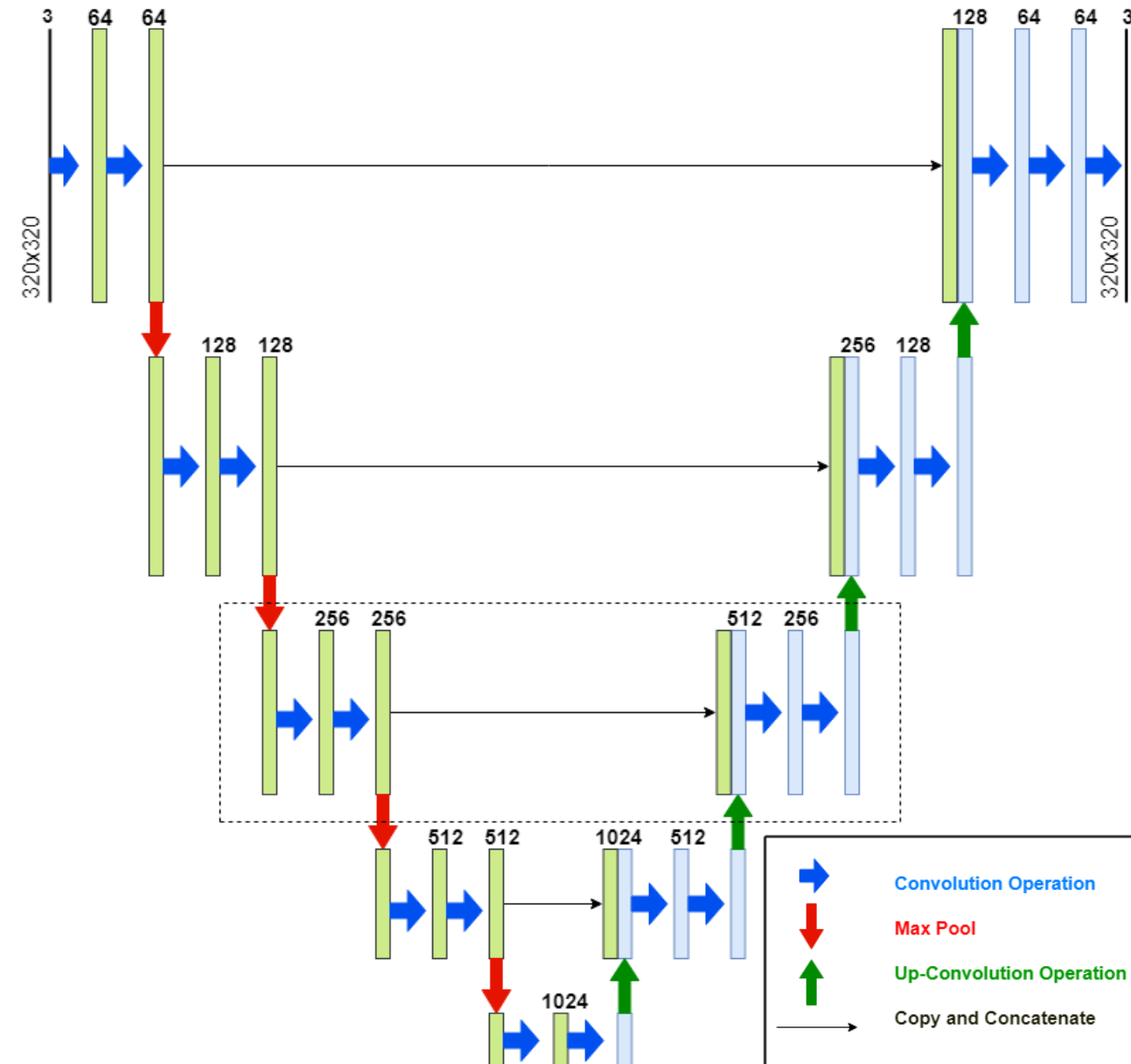
3. U-Net

- U-Net is a popular architecture used primarily for **image segmentation** tasks, especially in medical imaging.
- It uses an **encoder-decoder architecture**, where the encoder captures the context (downsampling), and the decoder refines the output (upsampling).
- The key innovation is the use of **skip connections** between corresponding layers in the encoder and decoder, which help preserve spatial information that might be lost during downsampling.

Key Characteristics:

- **Encoder-decoder architecture** with skip connections.
- Effective for **semantic segmentation** and **pixel-level predictions**.
- Widely used in medical imaging tasks like tumor detection or organ segmentation.

Module-2; Topic-2: Deep Convolutional Models – ResNet, AlexNet, InceptionNet and others



Other Deep Network

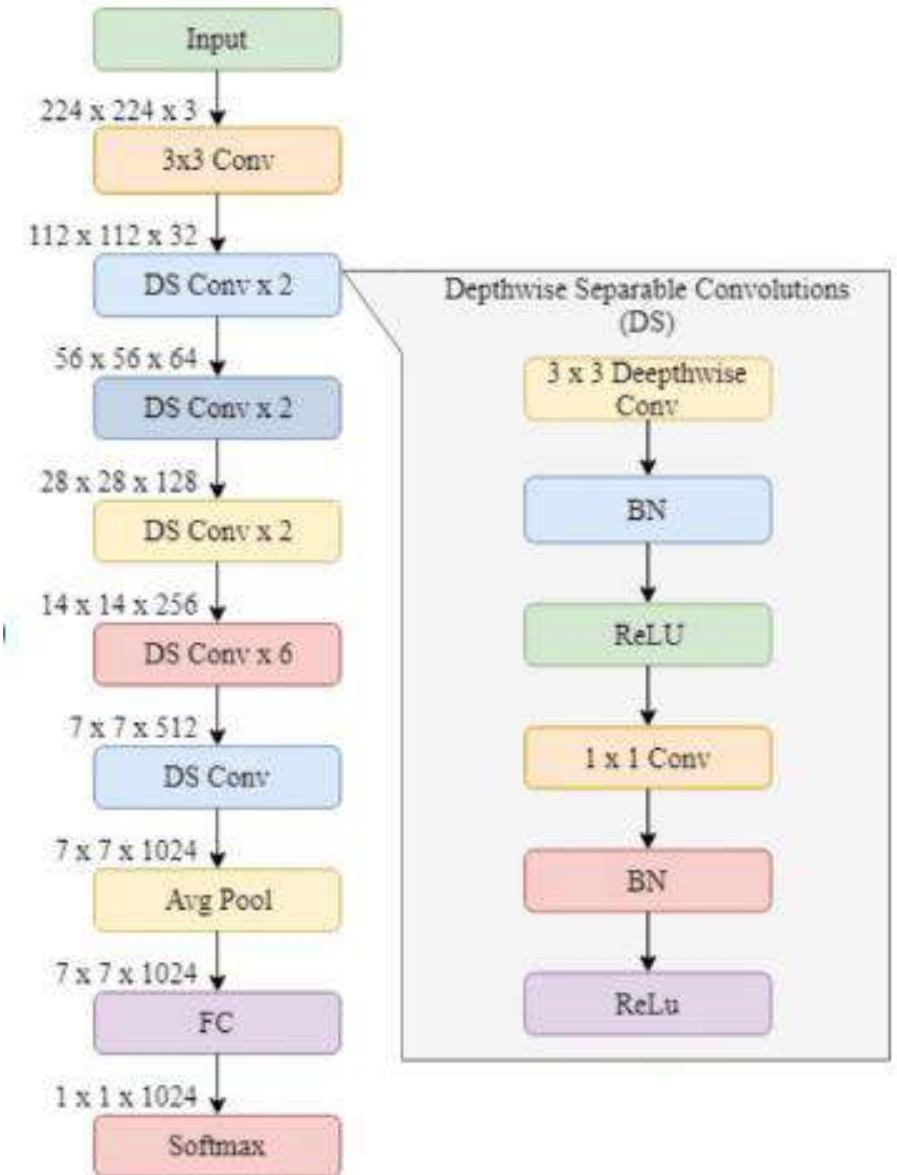
Module-2; Topic-2: Deep Convolutional Models – ResNet, AlexNet, InceptionNet and others

4. MobileNet

- MobileNet is a lightweight CNN architecture designed for **mobile and embedded devices**.
- It uses **depthwise separable convolutions**, which split a standard convolution into two steps:
 - **Depthwise convolution:** Each input channel is convolved with its own set of filters.
 - **Pointwise convolution:** A 1×1 convolution to combine the outputs.
- This greatly reduces the number of parameters and computations, making MobileNet efficient for devices with limited computational resources.

Key Characteristics:

- Uses **depthwise separable convolutions**.
- Highly efficient and computationally inexpensive.



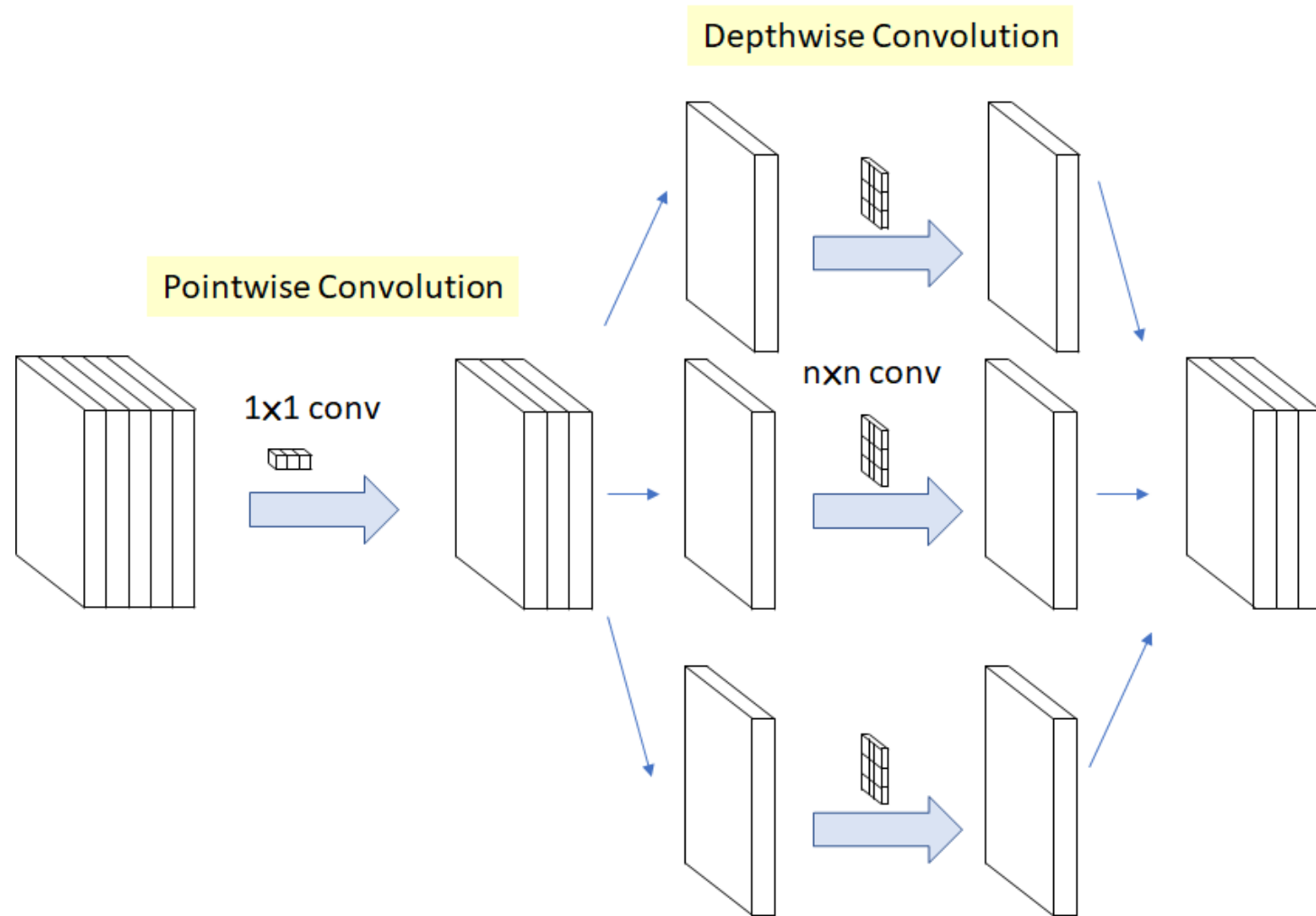
Other Deep Network

5. Xception

- **Xception** is an extension of the **Inception** architecture, where the regular convolutions in the Inception modules are replaced with **depthwise separable convolutions**.
- The idea behind **Xception** is that replacing standard convolutions with depthwise separable convolutions can achieve better performance while being more efficient.

Key Characteristics:

- Based on **depthwise separable convolutions**.
- Achieves similar performance to Inception models but with more efficiency.
- Good performance in large-scale image classification tasks.



Other Deep Network

6. YOLO (You Only Look Once)

YOLO is a popular deep learning model for **real-time object detection**.

Unlike traditional object detection models that treat the problem as a sliding window or region proposal task, YOLO predicts bounding boxes and class probabilities in a single pass (hence "You Only Look Once").

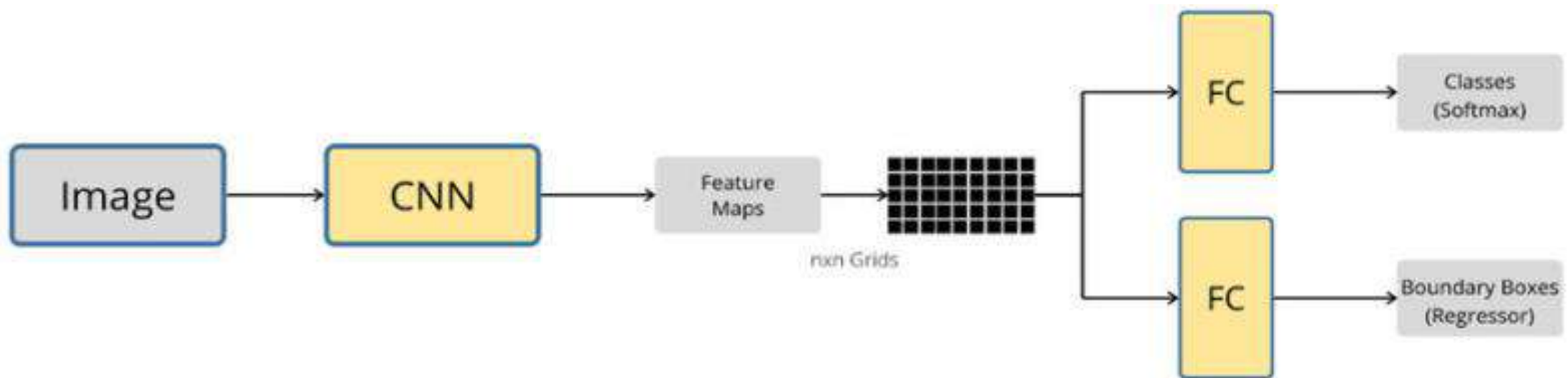
It divides the image into a grid and makes predictions for each grid cell, making it much faster than other models.

Key Characteristics:

Real-time **object detection**.

Divides the image into a grid for simultaneous detection.

Fast and efficient, suitable for real-time applications.

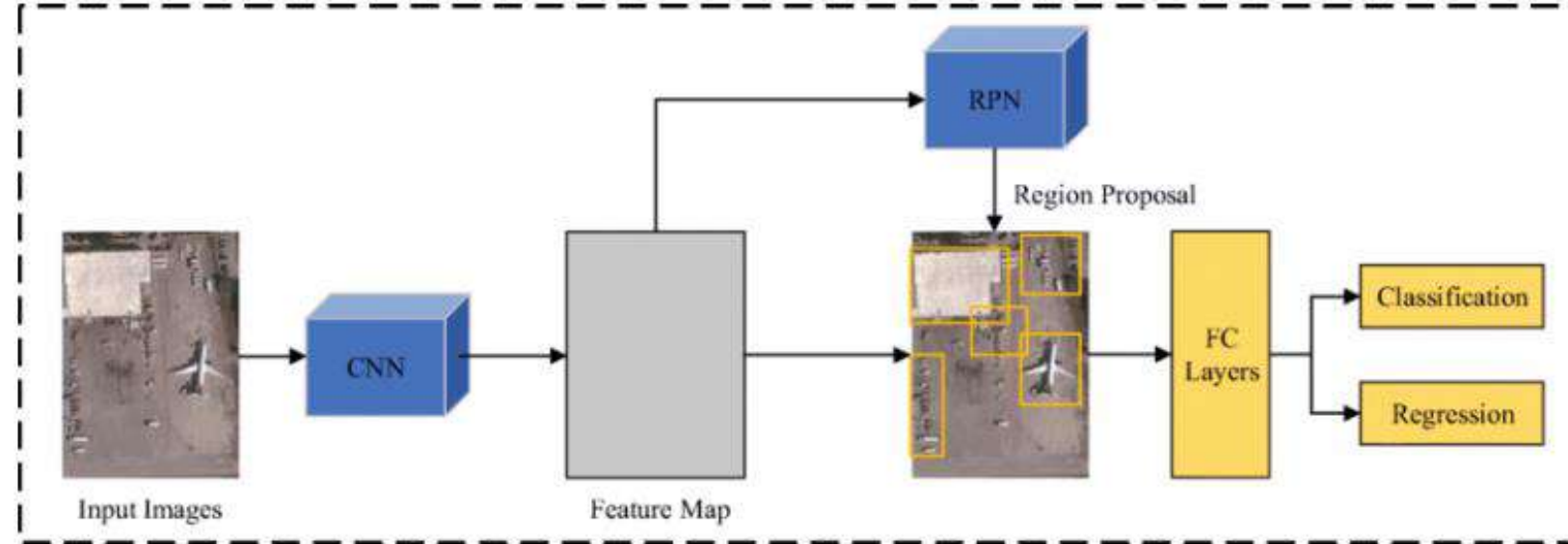


Other Deep Network

7. Faster R-CNN

- **Faster R-CNN** is a combination of two deep learning models: **Region Proposal Networks (RPN)** and a standard **Fast R-CNN** detector.
- The **RPN** generates proposals for regions in an image that are likely to contain objects, and then Fast R-CNN refines these proposals to classify the objects and localize them with bounding boxes.

Module-2; Topic-2: Deep Convolutional Models – ResNet, AlexNet, InceptionNet and others



Faster R-CNN is much faster than traditional methods because the region proposals are generated by a CNN.

Key Characteristics:

- Combines **Region Proposal Networks (RPN)** with Fast R-CNN.
- Highly effective for **object detection** tasks.
- More accurate and faster than earlier object detection models.

Other Deep Network

8. Transformer Networks

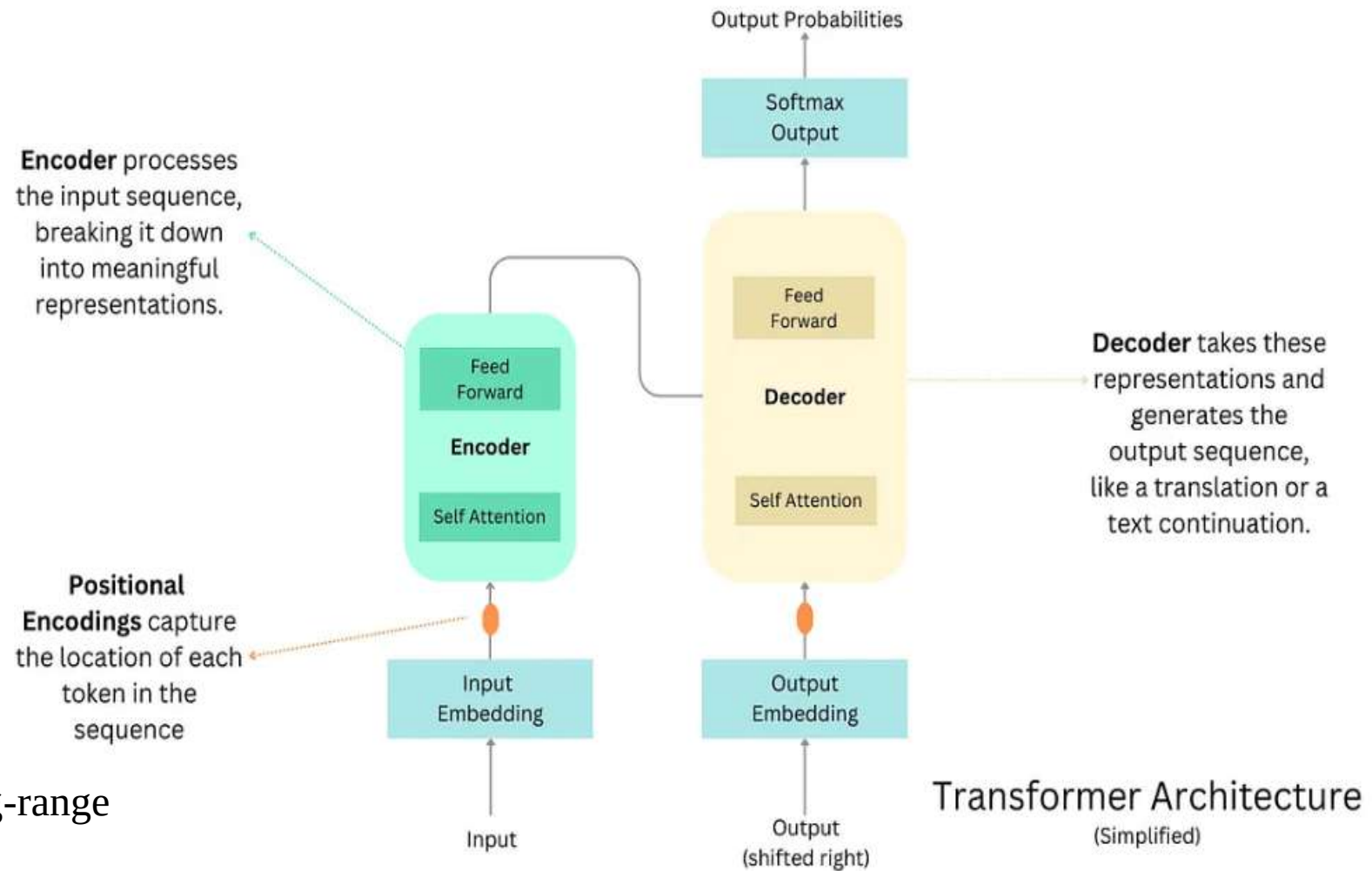
Initially developed for **NLP tasks**, **transformers** have become a key architecture for many deep learning tasks beyond just language.

Transformers are based on the **self-attention mechanism**, allowing models to weigh the importance of different parts of the input sequence when making predictions.

Transformers have led to the creation of several models like **BERT**, **GPT**, **T5**, and **Vision Transformers (ViT)**

Key Characteristics:

- **Self-attention** mechanism to capture long-range dependencies.
- Highly parallelizable (great for training on large datasets).
- State-of-the-art in NLP and starting to perform well in vision (Vision Transformers).



Other Deep Networks

9. BERT (Bidirectional Encoder Representations from Transformers)

BERT is a transformer-based model for **NLP** tasks such as question answering and text classification. It uses a bidirectional attention mechanism, allowing it to consider both the left and right context of a word simultaneously (compared to traditional left-to-right models).

BERT is pre-trained on vast amounts of text data and fine-tuned for specific tasks.

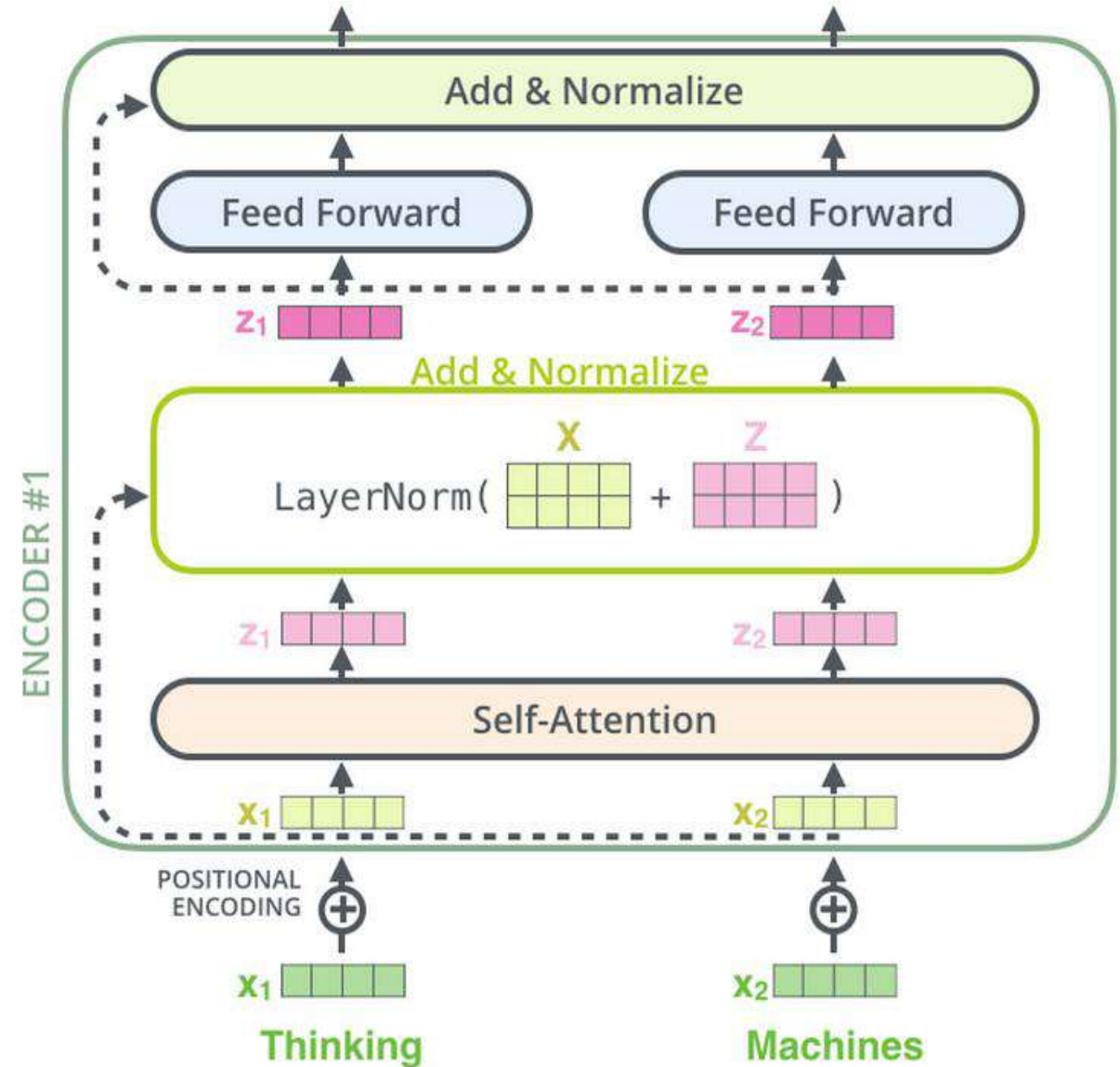
Key Characteristics:

Bidirectional transformer model for understanding context in text.

Pretrained on massive corpora and fine-tuned for specific tasks.

Very effective for tasks like **question answering**, **text classification**, and **named entity recognition**.

Module-2; Topic-2: Deep Convolutional Models – ResNet, AlexNet, InceptionNet and others



Other Deep Networks

10. GAN (Generative Adversarial Networks)

GANs are a type of deep learning model used for **generative tasks**, such as generating realistic images, text, or music.

A GAN consists of two networks: a **generator** and a **discriminator**. The generator creates fake data (e.g., images), and the discriminator tries to distinguish between real and fake data. Both networks are trained simultaneously in a competitive setting.

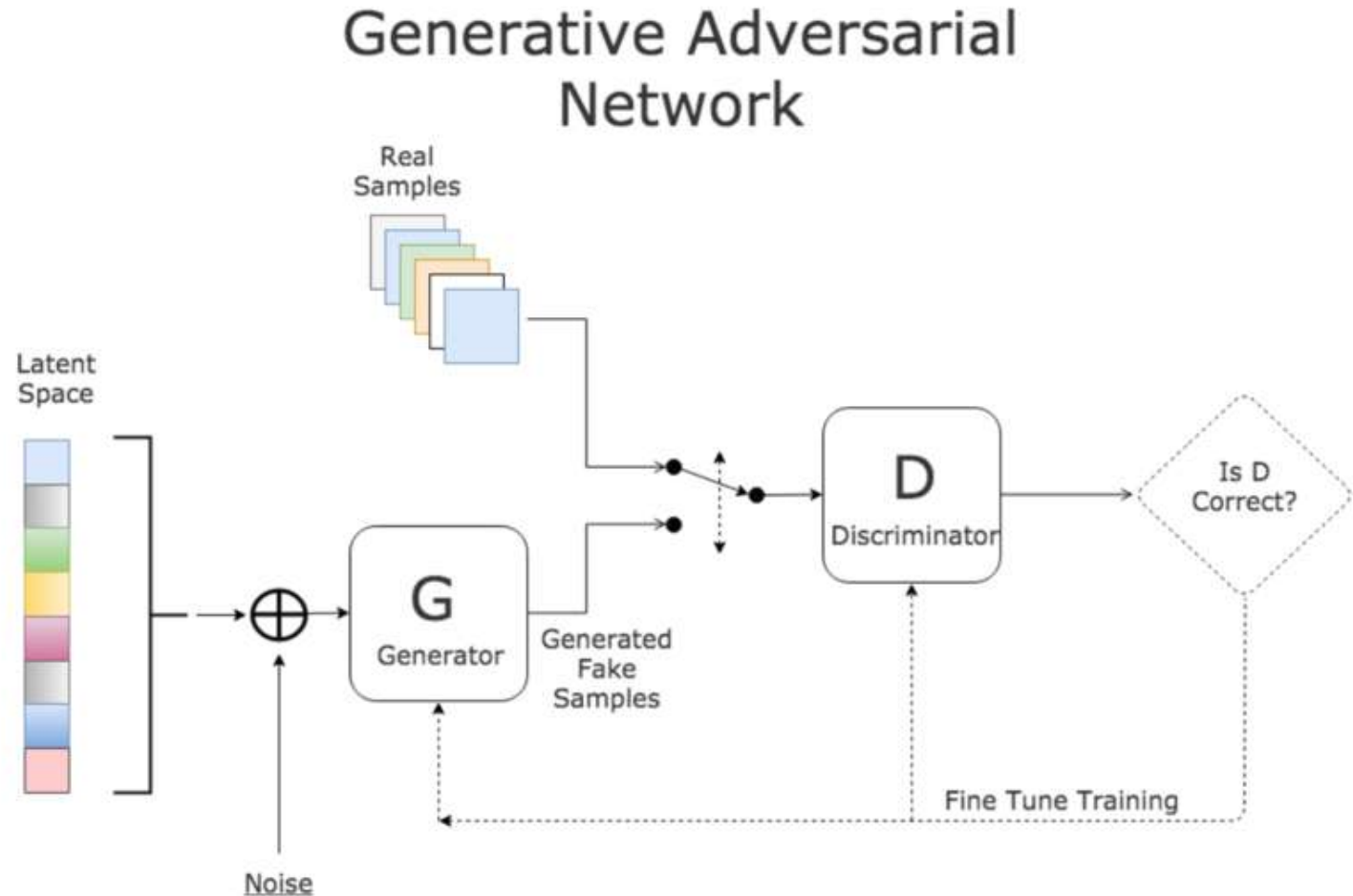
Key Characteristics:

Generative model for creating new data.

Two components: **generator** and **discriminator**.

Widely used for image generation, **style transfer**, and **super-resolution** tasks.

Module-2; Topic-2: Deep Convolutional Models – ResNet, AlexNet, InceptionNet and others



Foundations of Convolutional Neural Networks (CNNs): Convolutional Neural Networks (CNNs) are a class of deep neural networks commonly used in computer vision tasks, including image classification, object detection, and segmentation. They are designed to automatically and adaptively learn spatial hierarchies of features from input images, making them especially effective in processing grid-like data, such as images.

CNN Operations

CNNs perform several operations to process and learn from images. These operations enable the network to capture important patterns and structures at multiple scales. Here are the key operations in CNNs:

1.Convolution: The core operation in CNNs, convolution involves applying filters (also called kernels) to an image to extract feature maps. These filters slide over the image (with a certain stride) and compute the dot product between the filter and the receptive field of the image at each position. Convolution helps capture local spatial patterns, such as edges, textures, and corners.

2.Activation (ReLU): After each convolution operation, the output is passed through a non-linear activation function, usually **ReLU (Rectified Linear Unit)**, which introduces non-linearity and helps the network learn more complex patterns.

3.Pooling: Pooling (usually **max pooling**) is used to downsample feature maps and reduce spatial dimensions. This helps to decrease computational complexity and control overfitting. Max pooling selects the maximum value in a specific region, helping retain important features.

Summary

Module-2; Summary



Fully Connected Layers: After the convolutional and pooling layers, the feature maps are often flattened into a 1D vector and passed through **fully connected layers** to perform classification or regression tasks. These layers connect every neuron to every other neuron in the next layer.

Softmax: In classification tasks, the final layer often uses a **softmax** function to output probabilities for different classes.

Simple Convolution Network Architecture

A **simple convolutional network** typically includes the following layers:

- **Input Layer:** This is the raw input image, often resized to a specific dimension (e.g., $224 \times 224 \times 3$ for RGB images).
- **Convolutional Layer:** A set of filters is applied to the input image to detect low-level features such as edges, textures, and simple patterns.
- **Activation Layer (ReLU):** After the convolution operation, the output is passed through an activation function (ReLU) to introduce non-linearity.
- **Pooling Layer:** Max pooling or average pooling is used to reduce the spatial dimensions of the feature maps, improving computational efficiency and reducing the risk of overfitting.
- **Fully Connected Layer:** The feature maps are flattened, and the resulting vector is passed through fully connected layers to make predictions (e.g., classification).
- **Output Layer:** The output layer typically consists of a softmax activation for multi-class classification, which converts the network's raw outputs into probabilities.